libloki

Généré par Doxygen 1.18.0

1 Index des sujets 1

1.1 Sujets . . . 1

2 Index des structures de données 3

2.1 Structures de données . . . 3

3 Index des fichiers 5

3.1 Liste des fichiers . . . 5

4 Documentation des sujets 7

4.1 Définitions communes . . . 7

4.1.1 Description détaillée . . . 7

4.1.2 Support de structures de données avec polymorphisme. . . . 7

4.1.2.1 Description détaillée . . . 8

4.1.2.2 Documentation des macros . . . 9

4.1.3 Macros utiles. . . . 13

4.1.3.1 Description détaillée . . . 14

4.1.3.2 Documentation des macros . . . 14

4.1.4 Instrumentation des fonctions malloc et free . . . 15

4.1.4.1 Description détaillée . . . 16

4.1.4.2 Documentation des fonctions . . . 16

4.1.4.3 Documentation des variables . . . 18

4.1.5 Support des fonctions intrinsics ARM NEON . . . 18

4.1.5.1 Description détaillée . . . 19

4.1.5.2 Documentation des fonctions . . . 19

4.2 Échange de données entre modules d'acquisition, de traitement et d'enregistrement . . . 23

4.2.1 Description détaillée . . . 23

4.2.2 Exemple de consommateur . . . 23

4.2.2.1 Description détaillée . . . 24

4.2.2.2 Documentation des définitions de type . . . 25

4.2.2.3 Documentation des fonctions . . . 25

4.2.3 Structure de données FIFO . . . 27

4.2.3.1 Description détaillée . . . 28

4.2.3.2 Documentation des définitions de type . . . 28

4.2.3.3 Documentation des fonctions . . . 29

4.2.4 Type paquets d'échantillons . . . 32

4.2.4.1 Description détaillée . . . 32

4.2.4.2 Documentation des fonctions . . . 33

4.2.5 Types et fonctions de gestion du temps . . . 33

4.2.5.1 Description détaillée . . . 34

4.2.5.2 Documentation des macros . . . 34

4.2.5.3 Documentation des définitions de type . . . 35

4.2.5.4 Documentation des fonctions . . . 35

4.3 Acquisition . . . 37
4.3.1 Description détaillée . . . 37
4.3.2 Acquisition de données provenant des capteurs auxiliaires . . . 37
4.3.2.1 Description détaillée . . . 40
4.3.2.2 Documentation des macros . . . 40
4.3.2.3 Documentation des définitions de type . . . 40
4.3.2.4 Documentation des fonctions . . . 40
4.3.3 Pilotage des composants modulateur et démodulateur . . . 52
4.3.3.1 Description détaillée . . . 60
4.3.3.2 Documentation des macros . . . 60
4.3.3.3 Documentation des définitions de type . . . 63
4.3.3.4 Documentation des fonctions . . . 63
4.4 Enregistrement et transmission de données . . . 102
4.4.1 Description détaillée . . . 103
4.4.2 Interface pour la mise en forme de séries temporelles . . . 103
4.4.2.1 Description détaillée . . . 104
4.4.2.2 Documentation des définitions de type . . . 104
4.4.2.3 Documentation des fonctions . . . 104
4.4.3 Enregistrement de séries temporelles au format miniSEED . . . 107
4.4.3.1 Description détaillée . . . 108
4.4.3.2 Documentation des macros . . . 109
4.4.3.3 Documentation des définitions de type . . . 109
4.4.3.4 Documentation des fonctions . . . 110
4.4.4 Enregistrement de séries temporelles brutes . . . 112
4.4.4.1 Description détaillée . . . 113
4.4.4.2 Documentation des définitions de type . . . 113
4.4.4.3 Documentation des fonctions . . . 114
4.4.5 Enregistrement de séries temporelles . . . 115
4.4.5.1 Description détaillée . . . 117
4.4.5.2 Documentation des macros . . . 117
4.4.5.3 Documentation des définitions de type . . . 118
4.4.5.4 Documentation des fonctions . . . 118
4.4.5.5 Documentation des variables . . . 123
4.4.6 Serveur de données sur socket Unix . . . 123
4.4.6.1 Description détaillée . . . 124
4.4.6.2 Documentation des macros . . . 125
4.4.6.3 Documentation des définitions de type . . . 125
4.4.6.4 Documentation des fonctions . . . 125
4.5 Calcul de déplacement . . . 128
4.5.1 Description détaillée . . . 129
4.5.2 Calcul de déplacement . . . 129
4.5.2.1 Description détaillée . . . 130

4.5.2.2 Documentation des définitions de type . . . 130
4.5.2.3 Documentation des fonctions . . . 130
4.5.3 Interface d'un filtre adaptatif pour estimer les paramètres d'une ellipse . . . 133
4.5.3.1 Description détaillée . . . 134
4.5.3.2 Documentation des définitions de type . . . 135
4.5.3.3 Documentation des fonctions . . . 135
4.5.4 Représentation et manipulation d'ellipses . . . 139
4.5.4.1 Description détaillée . . . 140
4.5.4.2 Documentation des définitions de type . . . 140
4.5.4.3 Documentation des fonctions . . . 141
4.5.5 Filtre FIR . . . 142
4.5.5.1 Description détaillée . . . 144
4.5.5.2 Documentation des définitions de type . . . 144
4.5.5.3 Documentation des fonctions . . . 144
4.5.6 Filtre de Kalman . . . 148
4.5.6.1 Description détaillée . . . 149
4.5.6.2 Documentation des définitions de type . . . 149
4.5.6.3 Documentation des fonctions . . . 149
4.5.7 Filtre NLMS . . . 151
4.5.7.1 Description détaillée . . . 152
4.5.7.2 Documentation des macros . . . 152
4.5.7.3 Documentation des définitions de type . . . 152
4.5.7.4 Documentation des fonctions . . . 153
4.5.8 Filtre RLS . . . 154
4.5.8.1 Description détaillée . . . 155
4.5.8.2 Documentation des définitions de type . . . 156
4.5.8.3 Documentation des fonctions . . . 156
4.5.9 Valeurs par défaut des paramètres de calcul de déplacement . . . 158
4.5.9.1 Description détaillée . . . 159
4.5.9.2 Documentation des macros . . . 160
4.5.10 Gestion des acquisitions et du calcul de déplacement . . . 163
4.5.10.1 Description détaillée . . . 167
4.5.10.2 Documentation des définitions de type . . . 167
4.5.10.3 Documentation des fonctions . . . 167
4.5.10.4 Documentation des variables . . . 184

5 Documentation des structures de données **187**

5.1 Référence de la structure AD8370ControlWord::[struct].__unnamed0__ . . . 187
5.1.1 Description détaillée . . . 187
5.2 Référence de la structure float32x4_t::[struct].__unnamed0__ . . . 187
5.2.1 Description détaillée . . . 188
5.3 Référence de l'union LokiEnvSensorsSample::[union].__unnamed0__ . . . 188

5.3.1 Description détaillée . . . 188
5.4 Référence de l'union LokiModemSample::[union].__unnamed0__ . . . 188
5.4.1 Description détaillée . . . 189
5.5 Référence de l'union LokiMSeedInputSample::[union].__unnamed0__ . . . 189
5.5.1 Description détaillée . . . 189
5.6 Référence de l'union LokiModemSample::[union].__unnamed1__ . . . 189
5.6.1 Description détaillée . . . 190
5.7 Référence de l'union AD8370ControlWord . . . 190
5.7.1 Description détaillée . . . 190
5.7.2 Documentation des champs . . . 191
5.7.2.1 raw . . . 191
5.8 Référence de la structure float32x2_t . . . 191
5.8.1 Description détaillée . . . 191
5.8.2 Documentation des champs . . . 191
5.8.2.1 val . . . 191
5.9 Référence de l'union float32x4_t . . . 192
5.9.1 Description détaillée . . . 192
5.9.2 Documentation des champs . . . 192
5.9.2.1 val . . . 192
5.10 Référence de la structure LokiConsumer . . . 192
5.10.1 Description détaillée . . . 193
5.10.2 Documentation des champs . . . 193
5.10.2.1 cid . . . 193
5.10.2.2 in . . . 193
5.10.2.3 stop_request . . . 193
5.10.2.4 thread . . . 194
5.11 Référence de la structure LokiDisplacement . . . 194
5.11.1 Description détaillée . . . 195
5.11.2 Documentation des champs . . . 195
5.11.2.1 alpha_prev . . . 195
5.11.2.2 alpha_ref . . . 195
5.11.2.3 ellipse . . . 195
5.11.2.4 estimator . . . 196
5.11.2.5 estimator_cnt . . . 196
5.11.2.6 estimator_throttle . . . 196
5.11.2.7 filter . . . 196
5.11.2.8 iir_a . . . 196
5.11.2.9 iir_b . . . 196
5.11.2.10 phi_filt . . . 197
5.11.2.11 phi_prev . . . 197
5.11.2.12 phi_ref . . . 197
5.11.2.13 phi_turns . . . 197

5.11.2.14 scale . . . 197
5.12 Référence de la structure LokiEllipse . . . 197
5.12.1 Description détaillée . . . 198
5.12.2 Documentation des champs . . . 198
5.12.2.1 A . . . 198
5.12.2.2 alpha . . . 198
5.12.2.3 B . . . 198
5.12.2.4 xc . . . 199
5.12.2.5 yc . . . 199
5.13 Référence de la structure LokiEllipseEstimator . . . 199
5.13.1 Description détaillée . . . 199
5.13.2 Documentation des champs . . . 199
5.13.2.1 impl . . . 199
5.13.2.2 instance . . . 200
5.14 Référence de la structure LokiEnvSensors . . . 200
5.14.1 Description détaillée . . . 201
5.14.2 Documentation des champs . . . 201
5.14.2.1 bmp390_t_fine . . . 201
5.14.2.2 channel_mask . . . 201
5.14.2.3 channel_output_width . . . 202
5.14.2.4 channels . . . 202
5.14.2.5 clk_frequency . . . 202
5.14.2.6 cycles_per_sample_max . . . 202
5.14.2.7 data_mask . . . 202
5.14.2.8 data_sign_mask . . . 202
5.14.2.9 data_width . . . 203
5.14.2.10 file . . . 203
5.14.2.11 fileno . . . 203
5.14.2.12 out . . . 203
5.14.2.13 raw . . . 203
5.14.2.14 scales . . . 203
5.14.2.15 stop_request . . . 204
5.14.2.16 stream_channels . . . 204
5.14.2.17 thread . . . 204
5.15 Référence de la structure LokiEnvSensorsSample . . . 204
5.15.1 Description détaillée . . . 205
5.15.2 Documentation des champs . . . 205
5.15.2.1 channel . . . 205
5.16 Référence de la structure LokiFIFO . . . 205
5.16.1 Description détaillée . . . 206
5.16.2 Documentation des champs . . . 206
5.16.2.1 abort . . . 206

5.16.2.2 all_read . . . 206
5.16.2.3 consumer_records . . . 206
5.16.2.4 item_size . . . 206
5.16.2.5 mutex . . . 207
5.16.2.6 producer_record . . . 207
5.16.2.7 pulled . . . 207
5.16.2.8 pushed . . . 207
5.17 Référence de la structure LokiFIFORecord . . . 207
5.17.1 Description détaillée . . . 208
5.17.2 Documentation des champs . . . 208
5.17.2.1 item . . . 208
5.17.2.2 next . . . 208
5.17.2.3 status . . . 208
5.18 Référence de la structure LokiFIRFilter . . . 209
5.18.1 Description détaillée . . . 209
5.18.2 Documentation des champs . . . 210
5.18.2.1 acc . . . 210
5.18.2.2 coeff_ptr . . . 210
5.18.2.3 coeffs . . . 210
5.18.2.4 coeffs_end . . . 210
5.18.2.5 decimation . . . 210
5.18.2.6 is_clone . . . 210
5.18.2.7 length . . . 211
5.18.2.8 phase_length . . . 211
5.18.2.9 phase_ptr . . . 211
5.18.2.10 sample_index . . . 211
5.18.2.11 samples . . . 211
5.18.2.12 samples_end . . . 211
5.19 Référence de la structure LokiFormatter . . . 212
5.19.1 Description détaillée . . . 212
5.19.2 Documentation des champs . . . 212
5.19.2.1 impl . . . 212
5.19.2.2 instance . . . 212
5.20 Référence de la structure LokiKalman . . . 212
5.20.1 Description détaillée . . . 213
5.20.2 Documentation des champs . . . 213
5.20.2.1 P . . . 213
5.20.2.2 rv . . . 213
5.20.2.3 rw . . . 213
5.20.2.4 state . . . 214
5.21 Référence de la structure LokiModem . . . 214
5.21.1 Description détaillée . . . 217

5.21.2 Documentation des champs . . . 217
5.21.2.1 clk_frequency . . . 217
5.21.2.2 demod_channel_mask . . . 218
5.21.2.3 demod_channel_output_width . . . 218
5.21.2.4 demod_channels . . . 218
5.21.2.5 demod_cycles_per_sample_max . . . 218
5.21.2.6 demod_cycles_per_sample_ref . . . 218
5.21.2.7 demod_fir1_decimation . . . 218
5.21.2.8 demod_fir1_length . . . 219
5.21.2.9 demod_fir2_bandwidth . . . 219
5.21.2.10 demod_fir2_decimation . . . 219
5.21.2.11 demod_fir2_length . . . 219
5.21.2.12 demod_mixerless . . . 219
5.21.2.13 demod_osc_wave_frequency_min . . . 219
5.21.2.14 demod_power_fractional_width . . . 220
5.21.2.15 demod_power_integer_width . . . 220
5.21.2.16 demod_power_max . . . 220
5.21.2.17 demod_sample_frequency_ref . . . 220
5.21.2.18 demod_xy_fractional_width . . . 220
5.21.2.19 demod_xy_integer_width . . . 220
5.21.2.20 demod_xy_mask . . . 221
5.21.2.21 demod_xy_max . . . 221
5.21.2.22 demod_xy_sign_mask . . . 221
5.21.2.23 demod_xy_width . . . 221
5.21.2.24 file . . . 221
5.21.2.25 fileno . . . 221
5.21.2.26 mod_cycles_per_sample_max . . . 222
5.21.2.27 mod_cycles_per_sample_ref . . . 222
5.21.2.28 mod_dc_value_max . . . 222
5.21.2.29 mod_dc_value_res . . . 222
5.21.2.30 mod_dc_value_width . . . 222
5.21.2.31 mod_gain_fractional_width . . . 222
5.21.2.32 mod_gain_integer_width . . . 223
5.21.2.33 mod_gain_res . . . 223
5.21.2.34 mod_offset_res . . . 223
5.21.2.35 mod_osc1_wave_frequency_min . . . 223
5.21.2.36 mod_osc2_wave_frequency_min . . . 223
5.21.2.37 mod_pga_gain_max . . . 223
5.21.2.38 mod_pga_gain_width . . . 224
5.21.2.39 mod_phase_amplitude_max . . . 224
5.21.2.40 mod_phase_fractional_width . . . 224
5.21.2.41 mod_phase_integer_width . . . 224

5.21.2.42 mod_sample_frequency_ref . . . 224
5.21.2.43 out . . . 224
5.21.2.44 raw . . . 225
5.21.2.45 sim_channel . . . 225
5.21.2.46 sim_disp_phase . . . 225
5.21.2.47 sim_osc1_phase . . . 225
5.21.2.48 simulate . . . 225
5.21.2.49 stop_request . . . 225
5.21.2.50 stream_channels . . . 226
5.21.2.51 thread . . . 226
5.22 Référence de la structure LokiModemSample . . . 226
5.22.1 Description détaillée . . . 226
5.22.2 Documentation des champs . . . 227
5.22.2.1 channel . . . 227
5.23 Référence de la structure LokiMSeed . . . 227
5.23.1 Description détaillée . . . 227
5.23.2 Documentation des champs . . . 227
5.23.2.1 buffer . . . 227
5.23.2.2 config . . . 228
5.23.2.3 msr . . . 228
5.24 Référence de la structure LokiMSeedConfig . . . 228
5.24.1 Description détaillée . . . 229
5.24.2 Documentation des champs . . . 229
5.24.2.1 channel_names . . . 229
5.24.2.2 location . . . 229
5.24.2.3 network . . . 229
5.24.2.4 raw . . . 229
5.24.2.5 separate_channels . . . 229
5.24.2.6 station . . . 230
5.24.2.7 steim2 . . . 230
5.25 Référence de la structure LokiMSeedInputSample . . . 230
5.25.1 Description détaillée . . . 230
5.25.2 Documentation des champs . . . 231
5.25.2.1 channel . . . 231
5.26 Référence de l'union LokiMSeedOutputSample . . . 231
5.26.1 Description détaillée . . . 231
5.26.2 Documentation des champs . . . 231
5.26.2.1 f . . . 231
5.26.2.2 i . . . 232
5.27 Référence de la structure LokiNLMS . . . 232
5.27.1 Description détaillée . . . 232
5.27.2 Documentation des champs . . . 232

5.27.2.1 alpha . . . 232
5.27.2.2 mu . . . 233
5.27.2.3 state . . . 233
5.27.2.4 weights . . . 233
5.28 Référence de la structure LokiPacket . . . 233
5.28.1 Description détaillée . . . 233
5.28.2 Documentation des champs . . . 234
5.28.2.1 buffer . . . 234
5.28.2.2 timestamp . . . 234
5.29 Référence de la structure LokiRaw . . . 234
5.29.1 Description détaillée . . . 234
5.29.2 Documentation des champs . . . 234
5.29.2.1 out . . . 234
5.30 Référence de la structure LokiRecorder . . . 235
5.30.1 Description détaillée . . . 236
5.30.2 Documentation des champs . . . 236
5.30.2.1 channel_count . . . 236
5.30.2.2 channels . . . 236
5.30.2.3 cid . . . 236
5.30.2.4 filename . . . 237
5.30.2.5 filetype . . . 237
5.30.2.6 formatter . . . 237
5.30.2.7 in . . . 237
5.30.2.8 is_recording . . . 237
5.30.2.9 max_channels . . . 237
5.30.2.10 multifile . . . 238
5.30.2.11 mutex . . . 238
5.30.2.12 recording_done . . . 238
5.30.2.13 sample_count . . . 238
5.30.2.14 sample_count_per_file . . . 238
5.30.2.15 sample_size . . . 238
5.30.2.16 sampling_frequency . . . 239
5.30.2.17 stop_request . . . 239
5.30.2.18 thread . . . 239
5.31 Référence de la structure LokiRLS . . . 239
5.31.1 Description détaillée . . . 240
5.31.2 Documentation des champs . . . 240
5.31.2.1 lambda . . . 240
5.31.2.2 lambda_inv . . . 240
5.31.2.3 P . . . 240
5.31.2.4 state . . . 240
5.32 Référence de la structure LokiSocketServer . . . 240

5.32.1 Description détaillée . . . 241
5.32.2 Documentation des champs . . . 242
5.32.2.1 cid . . . 242
5.32.2.2 data . . . 242
5.32.2.3 data_size . . . 242
5.32.2.4 filename . . . 242
5.32.2.5 has_data . . . 242
5.32.2.6 has_data_cond . . . 242
5.32.2.7 in . . . 243
5.32.2.8 is_streaming . . . 243
5.32.2.9 mutex . . . 243
5.32.2.10 receiver_thread . . . 243
5.32.2.11 server_thread . . . 243
5.32.2.12 stop_request . . . 243
5.33 Référence de la structure LokiSystem . . . 244
5.33.1 Description détaillée . . . 245
5.33.2 Documentation des champs . . . 245
5.33.2.1 channels . . . 245
5.33.2.2 cid . . . 245
5.33.2.3 config . . . 245
5.33.2.4 disp . . . 245
5.33.2.5 has_own_modem . . . 245
5.33.2.6 in . . . 246
5.33.2.7 is_streaming . . . 246
5.33.2.8 modem . . . 246
5.33.2.9 mutex . . . 246
5.33.2.10 out . . . 246
5.33.2.11 stop_request . . . 246
5.33.2.12 thread . . . 247
5.34 Référence de la structure LokiSystemConfig . . . 247
5.34.1 Description détaillée . . . 248
5.34.2 Documentation des champs . . . 248
5.34.2.1 cavity_index . . . 248
5.34.2.2 distance_unit . . . 248
5.34.2.3 estimator_impl . . . 248
5.34.2.4 estimator_initial_radius . . . 249
5.34.2.5 estimator_params . . . 249
5.34.2.6 estimator_throttle . . . 249
5.34.2.7 filter_attenuation . . . 249
5.34.2.8 filter_bandwidth . . . 249
5.34.2.9 filter_decimation . . . 249
5.34.2.10 filter_length . . . 250

5.34.2.11 reflections . . . 250
5.34.2.12 simulate . . . 250
5.34.2.13 skip . . . 250
5.34.2.14 time_constants . . . 250
5.34.2.15 wavelength . . . 250
5.35 Référence de la structure LokiSystemSample . . . 251
5.35.1 Description détaillée . . . 251
5.35.2 Documentation des champs . . . 251
5.35.2.1 channel . . . 251
5.35.2.2 value . . . 251
5.35.2.3 x_norm . . . 252
5.35.2.4 y_norm . . . 252

6 Documentation des fichiers **253**

6.1 Référence du fichier libloki-class.h . . . 253
6.1.1 Description détaillée . . . 254
6.2 libloki-class.h . . . 254
6.3 Référence du fichier libloki-common.h . . . 255
6.3.1 Description détaillée . . . 255
6.4 libloki-common.h . . . 255
6.5 Référence du fichier libloki-consumer.c . . . 256
6.5.1 Description détaillée . . . 256
6.5.2 Documentation des macros . . . 257
6.5.2.1 _GNU_SOURCE . . . 257
6.6 libloki-consumer.c . . . 257
6.7 Référence du fichier libloki-consumer.h . . . 258
6.7.1 Description détaillée . . . 258
6.8 libloki-consumer.h . . . 259
6.9 Référence du fichier libloki-displacement.c . . . 259
6.9.1 Description détaillée . . . 260
6.10 libloki-displacement.c . . . 260
6.11 Référence du fichier libloki-displacement.h . . . 262
6.11.1 Description détaillée . . . 263
6.12 libloki-displacement.h . . . 263
6.13 Référence du fichier libloki-ellipse-estimator.c . . . 263
6.13.1 Description détaillée . . . 264
6.14 libloki-ellipse-estimator.c . . . 264
6.15 Référence du fichier libloki-ellipse-estimator.h . . . 265
6.15.1 Description détaillée . . . 266
6.16 libloki-ellipse-estimator.h . . . 266
6.17 Référence du fichier libloki-ellipse.c . . . 267
6.17.1 Description détaillée . . . 267

6.18 libloki-ellipse.c . . . 268
6.19 Référence du fichier libloki-ellipse.h . . . 269
6.19.1 Description détaillée . . . 269
6.20 libloki-ellipse.h . . . 270
6.21 Référence du fichier libloki-env-sensors.c . . . 270
6.21.1 Description détaillée . . . 273
6.21.2 Documentation des macros . . . 273
6.21.2.1 _GNU_SOURCE . . . 273
6.22 libloki-env-sensors.c . . . 273
6.23 Référence du fichier libloki-env-sensors.h . . . 279
6.23.1 Description détaillée . . . 280
6.24 libloki-env-sensors.h . . . 281
6.25 Référence du fichier libloki-fifo.c . . . 281
6.25.1 Description détaillée . . . 282
6.26 libloki-fifo.c . . . 283
6.27 Référence du fichier libloki-fifo.h . . . 285
6.27.1 Description détaillée . . . 286
6.28 libloki-fifo.h . . . 286
6.29 Référence du fichier libloki-fir.c . . . 286
6.29.1 Description détaillée . . . 287
6.30 libloki-fir.c . . . 287
6.31 Référence du fichier libloki-fir.h . . . 291
6.31.1 Description détaillée . . . 292
6.32 libloki-fir.h . . . 292
6.33 Référence du fichier libloki-formatter.c . . . 292
6.33.1 Description détaillée . . . 293
6.34 libloki-formatter.c . . . 293
6.35 Référence du fichier libloki-formatter.h . . . 294
6.35.1 Description détaillée . . . 295
6.36 libloki-formatter.h . . . 295
6.37 Référence du fichier libloki-hooks.c . . . 296
6.37.1 Description détaillée . . . 297
6.37.2 Documentation des macros . . . 297
6.37.2.1 _GNU_SOURCE . . . 297
6.38 libloki-hooks.c . . . 297
6.39 Référence du fichier libloki-hooks.h . . . 298
6.39.1 Description détaillée . . . 298
6.40 libloki-hooks.h . . . 299
6.41 Référence du fichier libloki-kalman.c . . . 299
6.41.1 Description détaillée . . . 300
6.42 libloki-kalman.c . . . 300
6.43 Référence du fichier libloki-kalman.h . . . 302

6.43.1 Description détaillée . . . 302
6.44 libloki-kalman.h . . . 303
6.45 Référence du fichier libloki-modem.c . . . 303
6.45.1 Description détaillée . . . 311
6.45.2 Documentation des macros . . . 311
6.45.2.1 _GNU_SOURCE . . . 311
6.46 libloki-modem.c . . . 311
6.47 Référence du fichier libloki-modem.h . . . 324
6.47.1 Description détaillée . . . 330
6.48 libloki-modem.h . . . 330
6.49 Référence du fichier libloki-mseed.c . . . 332
6.49.1 Description détaillée . . . 334
6.50 libloki-mseed.c . . . 334
6.51 Référence du fichier libloki-mseed.h . . . 337
6.51.1 Description détaillée . . . 337
6.52 libloki-mseed.h . . . 338
6.53 Référence du fichier libloki-neon.h . . . 338
6.53.1 Description détaillée . . . 339
6.54 libloki-neon.h . . . 339
6.55 Référence du fichier libloki-nlms.c . . . 340
6.55.1 Description détaillée . . . 341
6.56 libloki-nlms.c . . . 341
6.57 Référence du fichier libloki-nlms.h . . . 342
6.57.1 Description détaillée . . . 343
6.58 libloki-nlms.h . . . 343
6.59 Référence du fichier libloki-packet.h . . . 344
6.59.1 Description détaillée . . . 344
6.60 libloki-packet.h . . . 345
6.61 Référence du fichier libloki-raw.c . . . 345
6.61.1 Description détaillée . . . 346
6.62 libloki-raw.c . . . 346
6.63 Référence du fichier libloki-raw.h . . . 347
6.63.1 Description détaillée . . . 348
6.64 libloki-raw.h . . . 348
6.65 Référence du fichier libloki-recorder.c . . . 348
6.65.1 Description détaillée . . . 350
6.65.2 Documentation des macros . . . 350
6.65.2.1 _GNU_SOURCE . . . 350
6.66 libloki-recorder.c . . . 351
6.67 Référence du fichier libloki-recorder.h . . . 354
6.67.1 Description détaillée . . . 354
6.68 libloki-recorder.h . . . 355

6.69 Référence du fichier libloki-rls.c . . . 355
6.69.1 Description détaillée . . . 356
6.70 libloki-rls.c . . . 356
6.71 Référence du fichier libloki-rls.h . . . 358
6.71.1 Description détaillée . . . 358
6.72 libloki-rls.h . . . 359
6.73 Référence du fichier libloki-socket-server.c . . . 359
6.73.1 Description détaillée . . . 360
6.73.2 Documentation des macros . . . 360
6.73.2.1 _GNU_SOURCE . . . 360
6.74 libloki-socket-server.c . . . 361
6.75 Référence du fichier libloki-socket-server.h . . . 363
6.75.1 Description détaillée . . . 364
6.76 libloki-socket-server.h . . . 364
6.77 Référence du fichier libloki-system-defaults.h . . . 365
6.77.1 Description détaillée . . . 366
6.78 libloki-system-defaults.h . . . 366
6.79 Référence du fichier libloki-system.c . . . 367
6.79.1 Description détaillée . . . 371
6.79.2 Documentation des macros . . . 371
6.79.2.1 _GNU_SOURCE . . . 371
6.80 libloki-system.c . . . 371
6.81 Référence du fichier libloki-system.h . . . 378
6.81.1 Description détaillée . . . 381
6.82 libloki-system.h . . . 381
6.83 Référence du fichier libloki-time.h . . . 383
6.83.1 Description détaillée . . . 384
6.84 libloki-time.h . . . 384
6.85 Référence du fichier libloki.h . . . 385
6.85.1 Description détaillée . . . 385
6.86 libloki.h . . . 385
6.87 util.py . . . 386

Chapitre 1

Index des sujets

1.1 Sujets

Liste de tous les sujets avec une brève description :

Définitions communes . . . 7
Support de structures de données avec polymorphisme. . . . 7
Macros utiles. . . . 13
Instrumentation des fonctions malloc et free . . . 15
Support des fonctions intrinsics ARM NEON . . . 18
Échange de données entre modules d'acquisition, de traitement et d'enregistrement . . . 23
Exemple de consommateur . . . 23
Structure de données FIFO . . . 27
Type paquets d'échantillons . . . 32
Types et fonctions de gestion du temps . . . 33
Acquisition . . . 37
Acquisition de données provenant des capteurs auxiliaires . . . 37
Pilotage des composants modulateur et démodulateur . . . 52
Enregistrement et transmission de données . . . 102
Interface pour la mise en forme de séries temporelles . . . 103
Enregistrement de séries temporelles au format miniSEED . . . 107
Enregistrement de séries temporelles brutes . . . 112
Enregistrement de séries temporelles . . . 115
Serveur de données sur socket Unix . . . 123
Calcul de déplacement . . . 128
Calcul de déplacement . . . 129
Interface d'un filtre adaptatif pour estimer les paramètres d'une ellipse . . . 133
Représentation et manipulation d'ellipses . . . 139
Filtre FIR . . . 142
Filtre de Kalman . . . 148
Filtre NLMS . . . 151
Filtre RLS . . . 154
Valeurs par défaut des paramètres de calcul de déplacement . . . 158
Gestion des acquisitions et du calcul de déplacement . . . 163

Chapitre 2

Index des structures de données

2.1 Structures de données

Liste des structures de données avec une brève description :

AD8370ControlWord::[struct].__unnamed0__
: Les champs du mot de contrôle . . . 187

float32x4_t::[struct].__unnamed0__
: Décomposition d'un vecteur en deux moitiés . . . 187

LokiEnvSensorsSample::[union].__unnamed0__
: La valeur de l'échantillon . . . 188

LokiModemSample::[union].__unnamed0__
: La composante X de l'échantillon . . . 188

LokiMSeedInputSample::[union].__unnamed0__
: La valeur de l'échantillon . . . 189

LokiModemSample::[union].__unnamed1__
: La composante Y de l'échantillon . . . 189

AD8370ControlWord
: Représentation du mot de contrôle du gain pour l'amplificateur AD8370 . . . 190

float32x2_t
: Vecteur de deux nombres flottants 32 bits . . . 191

float32x4_t
: Vecteur de quatre nombres flottants 32 bits . . . 192

LokiConsumer
: Définition d'un consommateur . . . 192

LokiDisplacement
: Représentation d'un calculateur de déplacement . . . 194

LokiEllipse
: Structure de données représentant une ellipse . . . 197

LokiEllipseEstimator
: Représentation d'un filtre d'estimation des paramètres d'une ellipse . . . 199

LokiEnvSensors
: Représentation de l'interface d'acquisition des capteurs auxiliaires . . . 200

LokiEnvSensorsSample
: Représentation d'un échantillon en provenance des capteurs auxiliaires . . . 204

LokiFIFO
: Représentation d'une FIFO . . . 205

LokiFIFORecord
: Représentation d'un enregistrement dans une FIFO . . . 207

LokiFIRFilter
: Représentation d'un filtre FIR . . . 209

LokiFormatter
Représentation d'un formateur de séries temporelles . . . 212
LokiKalman
Représentation d'un filtre de Kalman . . . 212
LokiModem
Représentation des périphériques modulateur et démodulateur . . . 214
LokiModemSample
Données retournées par le démodulateur pour un échantillon . . . 226
LokiMSeed
Représentation d'un formateur de séries temporelles miniSEED . . . 227
LokiMSeedConfig
Configuration d'un formateur de séries temporelles miniSEED . . . 228
LokiMSeedInputSample
Type des échantillons attendus par LokiMSeed_work . . . 230
LokiMSeedOutputSample
Type des échantillons passés à la bibliothèque `libmseed` . . . 231
LokiNLMS
Représentattion d'un filtre NLMS . . . 232
LokiPacket
Structure d'un paquet d'échantillons échangés entre tâches concurrentes . . . 233
LokiRaw
Représentation d'un formateur de données brutes . . . 234
LokiRecorder
Représentation d'un enregistreur de séries temporelles . . . 235
LokiRLS
Représentation d'un filtre RLS . . . 239
LokiSocketServer
Représentation d'un serveur de données . . . 240
LokiSystem
Représentation d'un système d'acquisition et de traitement . . . 244
LokiSystemConfig
Les paramètres de configuration d'un système d'acquisition et de traitement . . . 247
LokiSystemSample
Représentation d'un échantillon en sortie d'une instance de LokiSystem . . . 251

Chapitre 3

Index des fichiers

3.1 Liste des fichiers

Liste de tous les fichiers documentés avec une brève description :

libloki-class.h . . . 253
libloki-common.h . . . 255
libloki-consumer.c . . . 256
libloki-consumer.h . . . 258
libloki-displacement.c . . . 259
libloki-displacement.h . . . 262
libloki-ellipse-estimator.c . . . 263
libloki-ellipse-estimator.h . . . 265
libloki-ellipse.c . . . 267
libloki-ellipse.h . . . 269
libloki-env-sensors.c . . . 270
libloki-env-sensors.h . . . 279
libloki-fifo.c . . . 281
libloki-fifo.h . . . 285
libloki-fir.c . . . 286
libloki-fir.h . . . 291
libloki-formatter.c . . . 292
libloki-formatter.h . . . 294
libloki-hooks.c . . . 296
libloki-hooks.h . . . 298
libloki-kalman.c . . . 299
libloki-kalman.h . . . 302
libloki-modem.c . . . 303
libloki-modem.h . . . 324
libloki-mseed.c . . . 332
libloki-mseed.h . . . 337
libloki-neon.h . . . 338
libloki-nlms.c . . . 340
libloki-nlms.h . . . 342
libloki-packet.h . . . 344
libloki-raw.c . . . 345
libloki-raw.h . . . 347
libloki-recorder.c . . . 348
libloki-recorder.h . . . 354
libloki-rls.c . . . 355

libloki-rls.h 358
libloki-socket-server.c 359
libloki-socket-server.h 363
libloki-system-defaults.h 365
libloki-system.c 367
libloki-system.h 378
libloki-time.h 383
libloki.h 385
util.py 386

Chapitre 4

Documentation des sujets

4.1 Définitions communes

Graphe de collaboration de Définitions communes:

Sujets

- Support de structures de données avec polymorphisme.

  Ce module fournit des macros facilitant la définition d'interfaces et de classes.

- Macros utiles.
- Instrumentation des fonctions malloc et free

  Ce module peut être utilisées en phase de mise au point pour identifier l'origine de fuites mémoire ou de double désallocation.

- Support des fonctions intrinsics ARM NEON

  Sur les architectures ARM qui supportent les instructions SIMD NEON, ce fichier inclut directement l'en-tête `arm_neon.h` *officiel.*

4.1.1 Description détaillée

4.1.2 Support de structures de données avec polymorphisme.

Ce module fournit des macros facilitant la définition d'interfaces et de classes.

Graphe de collaboration de Support de structures de données avec polymorphisme.:

Fichiers

- fichier libloki-class.h

Macros

— #define LokiClass_interface_name(prefix)

Construit un nom d'interface pour le préfixe donné.

— #define LokiClass_class_name(prefix)

Construit un nom de classe pour le préfixe donné.

— #define LokiClass_function_name(prefix, name)

Construit un nom de fonction pour une méthode d'une classe.

— #define LokiClass_interface(prefix, methods)

Déclare une interface.

— #define LokiClass_attribute(decl)

Dans une interface, déclare un attribut.

— #define LokiClass_method(prefix, name)

Dans une interface, déclare une méthode.

— #define LokiClass_declare_implementation(superclass, prefix)

Déclare une classe qui implémente une interface donnée.

— #define LokiClass_implementation(superclass, prefix, impls)

Définit le contenu d'une classe qui implémente une interface donnée.

— #define LokiClass_set_attribute(name, value)

Dans une implémentation, affecte une valeur à un attribut.

— #define LokiClass_define_method(prefix, name)

Dans une implémentation, associe une fonction à une méthode.

4.1.2.1 Description détaillée

Ce module fournit des macros facilitant la définition d'interfaces et de classes.

- Une interface est un type `struct` qui déclare des attributs (des champs de données) et des méthodes (des pointeurs sur des fonctions).
- Une implémentation est une variable de ce type `struct`, dans laquelle on a affecté des valeurs aux attributs, et des fonctions aux méthodes.

On ne fournit pas de support particulier pour créer des instances de ces classes.

Exemple :

```
// Déclarer une interface Config.
LokiClass_interface(Config,
    LokiClass_attribute(char *category)
    LokiClass_method(Config, new)
```

```
);

// Le type des instances de Config.
typedef struct {
    void *instance;
    // Les pointeurs vers les méthodes seront copiés ici pour éviter un niveau
    // d'indirection supplémentaire et réduire la durée d'un appel de méthode.
    ConfigInterface impl;
} Config;

// Un constructeur de Config, qui délègue à une implémentation.
Config *Config_new(const ConfigInterface *impl) {
    printf("Création d'une configuration dans la catégorie : %s.\n", impl->category);
    Config *self = calloc(1, sizeof(Config));
    self->instance = impl->new();
    memcpy(&self->impl, impl, sizeof(ConfigInterface));
    return self;
}

// Déclarer une classe UserConfig qui implémente Config.
// Cette ligne se place généralement dans un fichier .h
LokiClass_declare_implementation(Config, UserConfig)

// Le type des instances de UserConfig.
typedef struct {
    ...
} UserConfig;

// Le constructeur de UserConfig, associé à la méthode Config.new.
UserConfig *UserConfig_new() {
    UserConfig *self = calloc(1, sizeof(UserConfig));
    ...
    return self;
}

// Définir l'implémentation de UserConfig.
LokiClass_implementation(Config, UserConfig,
    LokiClass_set_attribute(category, "User")
    LokiClass_define_method(UserConfig, new)
);

int main(void) {
    Config *cfg = Config_new(UserConfig);
    ...
}
```

4.1.2.2 Documentation des macros

4.1.2.2.1 LokiClass_attribute

```
#define LokiClass_attribute(
                decl)
```

Valeur :

```
decl;
```

Dans une interface, déclare un attribut.

Un attribut est un champ qui sera affecté dans la structure qui décrit une classe (voir LokiClass_implementation).

Paramètres

decl	Une déclaration de champ, sans `;` terminal.

Définition à la ligne 124 du fichier libloki-class.h.

4.1.2.2.2 LokiClass_class_name

```
#define LokiClass_class_name(
              prefix)
```

Valeur :

```
CAT(prefix, Class)
```

Construit un nom de classe pour le préfixe donné.

Paramètres

prefix	Le nom de base de la classe.

Exemple :

```
LokiClass_class_name(UserConfig) // --> UserConfigClass
```

Définition à la ligne 93 du fichier libloki-class.h.

4.1.2.2.3 LokiClass_declare_implementation

```
#define LokiClass_declare_implementation(
              superclass,
              prefix)
```

Valeur :

```
extern LokiClass_interface_name(superclass) LokiClass_class_name(prefix)
```

Déclare une classe qui implémente une interface donnée.

Paramètres

superclass	Le nom de base de l'interface.
prefix	Le nom de base de la classe.

Définition à la ligne 140 du fichier libloki-class.h.

4.1.2.2.4 LokiClass_define_method

```
#define LokiClass_define_method(
              prefix,
              name)
```

Valeur :

```
.name = (typeof(LokiClass_class_name(prefix).name))LokiClass_function_name(prefix, name),
```

Dans une implémentation, associe une fonction à une méthode.

Paramètres

prefix	Le nom de base de la classe.
name	Le nom de la méthode.

Définition à la ligne 167 du fichier libloki-class.h.

4.1.2.2.5 LokiClass_function_name

```
#define LokiClass_function_name(
              prefix,
              name)
```

Valeur :

```
CAT3(prefix, _, name)
```

Construit un nom de fonction pour une méthode d'une classe.

Paramètres

prefix	Le nom de base de l'interface ou de la classe.
name	Le nom de la méthode.

Exemple :

```
LokiClass_function_name(Config, new) // --> Config_new
```

Définition à la ligne 105 du fichier libloki-class.h.

4.1.2.2.6 LokiClass_implementation

```
#define LokiClass_implementation(
              superclass,
              prefix,
              impls)
```

Valeur :

```
LokiClass_interface_name(superclass) LokiClass_class_name(prefix) = { \
    impls                                                             \
}
```

Définit le contenu d'une classe qui implémente une interface donnée.

Paramètres

superclass	Le nom de base de l'interface.
prefix	Le nom de base de la classe.
impls	Les affectations d'attributs et de méthodes qui implémentent l'interface.

Définition à la ligne 149 du fichier libloki-class.h.

4.1.2.2.7 LokiClass_interface

```
#define LokiClass_interface(
              prefix,
              methods)
```

Valeur :

```
typedef struct {                        \
    methods                             \
} LokiClass_interface_name(prefix)
```

Déclare une interface.

Paramètres

prefix	Le nom de base de l'interface.
methods	Les attributs ou méthodes déclarées dans cette interface.

Définition à la ligne 112 du fichier libloki-class.h.

4.1.2.2.8 LokiClass_interface_name

```
#define LokiClass_interface_name(
              prefix)
```

Valeur :

```
CAT(prefix, Interface)
```

Construit un nom d'interface pour le préfixe donné.

Paramètres

prefix	Le nom de base de l'interface.

Exemple :

```
LokiClass_interface_name(Config) // --> ConfigInterface
```

Définition à la ligne 82 du fichier libloki-class.h.

4.1.2.2.9 LokiClass_method

```
#define LokiClass_method(
              prefix,
              name)
```

Valeur :

```
typeof(LokiClass_function_name(prefix, name)) *name;
```

Dans une interface, déclare une méthode.

Paramètres

prefix	Le nom de base de l'interface.
name	Le nom de la méthode.

Définition à la ligne 132 du fichier libloki-class.h.

4.1.2.2.10 LokiClass_set_attribute

```
#define LokiClass_set_attribute(
              name,
              value)
```

Valeur :

```
.name = value,
```

Dans une implémentation, affecte une valeur à un attribut.

Paramètres

name	Le nom de l'attribut.
value	La valeur de l'attribut.

Définition à la ligne 159 du fichier libloki-class.h.

4.1.3 Macros utiles.

Graphe de collaboration de Macros utiles.:

Fichiers

— fichier libloki-common.h

Macros

— #define LEN(a)

Retourner le nombre d'éléments d'un tableau.

— #define CAT(a, b)

Concaténer deux identifiants.

— #define CAT3(a, b, c)

Concaténer deux identifiants en insérant un séparateur.

4.1.3.1 Description détaillée

4.1.3.2 Documentation des macros

4.1.3.2.1 CAT

```
#define CAT(
              a,
              b)
```

Valeur :

```
a##b
```

Concaténer deux identifiants.

Paramètres

a	Le préfixe.
b	Le suffixe.

Renvoie

Un identifiant construit à partir de a et b.

Définition à la ligne 23 du fichier libloki-common.h.

4.1.3.2.2 CAT3

```
#define CAT3(
              a,
              b,
              c)
```

Valeur :

```
a##b##c
```

Concaténer deux identifiants en insérant un séparateur.

Paramètres

a	Le préfixe.
b	Le séparateur.
c	Le suffixe.

Renvoie

Un identifiant construit à partir de a, b et c.

Définition à la ligne 32 du fichier libloki-common.h.

4.1.3.2.3 LEN

```
#define LEN(
              a)
```

Valeur :

```
(sizeof(a) / sizeof(a[0]))
```

Retourner le nombre d'éléments d'un tableau.

Paramètres

a	Un tableau alloué statiquement.

Renvoie

Le nombre d'éléments dans a.

Définition à la ligne 15 du fichier libloki-common.h.

4.1.4 Instrumentation des fonctions malloc et free

Ce module peut être utilisées en phase de mise au point pour identifier l'origine de fuites mémoire ou de double désallocation.

Graphe de collaboration de Instrumentation des fonctions malloc et free:

Fichiers

- fichier libloki-hooks.c
- fichier libloki-hooks.h

Fonctions

- static void * LokiHooks_malloc (size_t size, const void *caller)

  Une version instrumentée du hook de la fonction `malloc`.

- static void LokiHooks_free (void *ptr, const void *caller)

  Une version instrumentée du hook de la fonction `free`.

- static void LokiHooks_setup (void)

  Affecter les hooks des fonctions `malloc` et `free`.

- static void LokiHooks_save (void)

  Sauvegarder les hooks des fonctions `malloc` et `free`.

- void LokiHooks_restore (void)

  Restaurer les hooks d'origine pour l'allocation/désallocation de mémoire.

- void LokiHooks_init (void)

  Initialiser l'utilisation des hooks instrumentés pour l'allocation/désallocation de mémoire.

Variables

— static void ∗(∗) LokiHooks_saved_malloc (size_t, const void ∗)

Pointeur vers le hook qui exécute la fonction `malloc` d'origine.

— static void(∗) LokiHooks_saved_free (void ∗, const void ∗)

Pointeur vers le hook qui exécute la fonction `free` d'origine.

4.1.4.1 Description détaillée

Ce module peut être utilisées en phase de mise au point pour identifier l'origine de fuites mémoire ou de double désallocation.

4.1.4.2 Documentation des fonctions

4.1.4.2.1 LokiHooks_free()

```
void LokiHooks_free (
            void * ptr,
            const void * caller)  [static]
```

Une version instrumentée du hook de la fonction `free`.

Cette fonction libère une région de la mémoire en utilisant la fonction `free` d'origine, puis elle affiche des informations sur la sortie d'erreur.

Paramètres

ptr	Un pointeur vers la région de mémoire à libérer.
caller	Un pointeur vers la fonction appelante.

Définition à la ligne 80 du fichier libloki-hooks.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.4.2.2 LokiHooks_init()

```
void LokiHooks_init (
            void )
```

Initialiser l'utilisation des hooks instrumentés pour l'allocation/désallocation de mémoire.

Définition à la ligne 39 du fichier libloki-hooks.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.4.2.3 LokiHooks_malloc()

```
void * LokiHooks_malloc (
            size_t size,
            const void * caller)  [static]
```

Une version instrumentée du hook de la fonction `malloc`.

Cette fonction alloue une région de la mémoire en utilisant la fonction `malloc` d'origine, puis elle affiche des informations sur la sortie d'erreur.

Paramètres

size	La taille de la région de mémoire à allouer.
caller	Un pointeur vers la fonction appelante.

Renvoie

Un pointeur vers la région de mémoire allouée.

Définition à la ligne 56 du fichier libloki-hooks.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.4.2.4 LokiHooks_restore()

```
void LokiHooks_restore (
            void )
```

Restaurer les hooks d'origine pour l'allocation/désallocation de mémoire.

Définition à la ligne 34 du fichier libloki-hooks.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.4.2.5 LokiHooks_save()

```
void LokiHooks_save (
            void )  [static]
```

Sauvegarder les hooks des fonctions `malloc` et `free`.

Définition à la ligne 29 du fichier libloki-hooks.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.4.2.6 LokiHooks_setup()

```
void LokiHooks_setup (
            void )  [static]
```

Affecter les hooks des fonctions `malloc` et `free`.

Définition à la ligne 23 du fichier libloki-hooks.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.4.3 Documentation des variables

4.1.4.3.1 LokiHooks_saved_free

```
void(*) LokiHooks_saved_free(void *, const void *) (
            void * ,
            const void * )  [static]
```

Pointeur vers le hook qui exécute la fonction `free` d'origine.

Définition à la ligne 16 du fichier libloki-hooks.c.

4.1.4.3.2 LokiHooks_saved_malloc

```
void *(*) LokiHooks_saved_malloc(size_t, const void *) (
            size_t ,
            const void * )  [static]
```

Pointeur vers le hook qui exécute la fonction `malloc` d'origine.

Définition à la ligne 13 du fichier libloki-hooks.c.

4.1.5 Support des fonctions intrinsics ARM NEON

Sur les architectures ARM qui supportent les instructions SIMD NEON, ce fichier inclut directement l'en-tête `arm_neon.h` officiel.

Graphe de collaboration de Support des fonctions intrinsics ARM NEON:

Fichiers

- fichier libloki-neon.h

Structures de données

- struct float32x2_t

  Vecteur de deux nombres flottants 32 bits.

- union float32x4_t

  Vecteur de quatre nombres flottants 32 bits.

- struct float32x4_t::[struct].__unnamed0__

  Décomposition d'un vecteur en deux moitiés.

Fonctions

- static float32x2_t vadd_f32 (float32x2_t a, float32x2_t b)

  Additionner deux vecteurs.

- static float vget_lane_f32 (float32x2_t a, size_t i)

  Lire un élément d'un vecteur.

- static float32x4_t vmovq_n_f32 (float a)

  Construire un vecteur.

- static float32x4_t vld1q_f32 (const float *a)

  Lire un vecteur en mémoire.

- static float32x2_t vget_low_f32 (float32x4_t a)

  Lire la première moitié d'un vecteur.

- static float32x2_t vget_high_f32 (float32x4_t a)

  Lire la seconde moitié d'un vecteur.

- static float32x4_t vld1q_lane_f32 (const float *a, float32x4_t b, size_t i)

  Remplacer un élément d'un vecteur avec une valeur lue en mémoire.

- static float32x4_t vmulq_f32 (float32x4_t a, float32x4_t b)

  Multiplier deux vecteurs terme à terme.

- static float32x4_t vmlaq_f32 (float32x4_t a, float32x4_t b, float32x4_t c)

  Calculer le résultat d'une multiplication-accumulation sur des vecteurs.

4.1.5.1 Description détaillée

Sur les architectures ARM qui supportent les instructions SIMD NEON, ce fichier inclut directement l'en-tête `arm_neon.h` officiel.

Pour les autres architectures, ce fichier fournit des implémentations en C standard de quelques fonctions intrinsics pour les types flottants 32 bits. Cela permet, par exemple, de tester sur PC une fonction destinée à un processeur ARM.

4.1.5.2 Documentation des fonctions

4.1.5.2.1 vadd_f32()

```
float32x2_t vadd_f32 (
            float32x2_t a,
            float32x2_t b)  [inline], [static]
```

Additionner deux vecteurs.

Paramètres

a	Le premier opérande.
b	Le second opérande.

Renvoie

Un vecteur obtenu en additionnant `a` et `b` terme à terme.

Définition à la ligne 50 du fichier libloki-neon.h.

4.1.5.2.2 vget_high_f32()

```
float32x2_t vget_high_f32 (
            float32x4_t a)  [inline], [static]
```

Lire la seconde moitié d'un vecteur.

Paramètres

a	Le vecteur à décomposer.

Renvoie

Un vecteur contenant la seconde moitié de `a`.

Définition à la ligne 100 du fichier libloki-neon.h.

4.1.5.2.3 vget_lane_f32()

```
float vget_lane_f32 (
            float32x2_t a,
            size_t i)  [inline], [static]
```

Lire un élément d'un vecteur.

Paramètres

a	Le vecteur à inspecter.
i	L'indice de l'élément à lire.

Renvoie

L'élément d'indice `i` dans `a`.

Définition à la ligne 62 du fichier libloki-neon.h.

4.1.5.2.4 vget_low_f32()

```
float32x2_t vget_low_f32 (
            float32x4_t a)  [inline], [static]
```

Lire la première moitié d'un vecteur.

Paramètres

a	Le vecteur à décomposer.

Renvoie

Un vecteur contenant la première moitié de `a`.

Définition à la ligne 91 du fichier libloki-neon.h.

4.1.5.2.5 vld1q_f32()

```
float32x4_t vld1q_f32 (
            const float * a)  [inline], [static]
```

Lire un vecteur en mémoire.

Paramètres

a	Un pointeur vers la région à lire.

Renvoie

Un vecteur des éléments pointés par `a`.

Définition à la ligne 82 du fichier libloki-neon.h.

4.1.5.2.6 vld1q_lane_f32()

```
float32x4_t vld1q_lane_f32 (
            const float * a,
            float32x4_t b,
            size_t i)  [inline], [static]
```

Remplacer un élément d'un vecteur avec une valeur lue en mémoire.

Paramètres

a	Un pointeur vers la valeur à lire.
b	Le vecteur à modifier.
i	L'indice de l'élément à remplacer.

Renvoie

Une copie de `b` dont un élément a été modifié.

Définition à la ligne 111 du fichier libloki-neon.h.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.1.5.2.7 vmlaq_f32()

```
float32x4_t vmlaq_f32 (
             float32x4_t a,
             float32x4_t b,
             float32x4_t c)  [inline], [static]
```

Calculer le résultat d'une multiplication-accumulation sur des vecteurs.

Paramètres

a	La valeur courante de l'accumulateur.
b	Le premier opérande de la multiplication.
c	Le second opérande de la multiplication.

Renvoie

La nouvelle valeur de l'accumulateur.

Définition à la ligne 136 du fichier libloki-neon.h.

4.1.5.2.8 vmovq_n_f32()

```
float32x4_t vmovq_n_f32 (
             float a)  [inline], [static]
```

Construire un vecteur.

Paramètres

a	La valeur des éléments.

Renvoie

Un vecteur dont tous les éléments sont égaux à `a`.

Définition à la ligne 71 du fichier libloki-neon.h.

4.1.5.2.9 vmulq_f32()

```
float32x4_t vmulq_f32 (
             float32x4_t a,
             float32x4_t b)  [inline], [static]
```

Multiplier deux vecteurs terme à terme.

Paramètres

a	Le premier opérande.
b	Le second opérande.

Renvoie

Un vecteur obtenu en calculant le produit de `a` et `b` terme à terme.

Définition à la ligne 122 du fichier libloki-neon.h.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2 Échange de données entre modules d'acquisition, de traitement et d'enregistrement

Graphe de collaboration de Échange de données entre modules d'acquisition, de traitement et d'enregistrement:

Sujets

— Exemple de consommateur

Ce module fournit une implémentation de référence d'un consommateur qui lit les données dans une FIFO et qui ne les utilise pas.

— Structure de données FIFO

File d'attente multi-consommateur pour communication entre threads.

— Type paquets d'échantillons

Type de données pour la communication de paquets d'échantillons datés entre threads.

— Types et fonctions de gestion du temps

Définition de types timestamp et intervalle de temps, et de fonctions de conversion d'unités de temps.

4.2.1 Description détaillée

4.2.2 Exemple de consommateur

Ce module fournit une implémentation de référence d'un consommateur qui lit les données dans une FIFO et qui ne les utilise pas.

Graphe de collaboration de Exemple de consommateur:

Fichiers

— fichier libloki-consumer.c

— fichier libloki-consumer.h

Structures de données

— struct LokiConsumer

Définition d'un consommateur.

Définitions de type

— typedef struct LokiConsumer LokiConsumer

Le type consommateur.

Fonctions

— LokiConsumer * LokiConsumer_new (void)

Construire un consommateur.

— static void * LokiConsumer_work (void *ptr)

Lire les données de la FIFO.

— int LokiConsumer_start (LokiConsumer *self, LokiFIFO *in, size_t cid)

Démarrer un consommateur.

— void LokiConsumer_stop (LokiConsumer *self)

Arrêter un consommateur.

— void LokiConsumer_del (LokiConsumer *self)

Détruire un consommateur.

4.2.2.1 Description détaillée

Ce module fournit une implémentation de référence d'un consommateur qui lit les données dans une FIFO et qui ne les utilise pas.

Il peut être utilisé comme *bouchon* en sortie d'une FIFO dont les données ne sont pas utilisées.

Il peut aussi servir d'exemple de référence pour construire d'autres consommateurs

4.2.2.2 Documentation des définitions de type

4.2.2.2.1 LokiConsumer

```
typedef struct LokiConsumer LokiConsumer
```

Le type consommateur.

Définition à la ligne 22 du fichier libloki-consumer.h.

4.2.2.3 Documentation des fonctions

4.2.2.3.1 LokiConsumer_del()

```
void LokiConsumer_del (
            LokiConsumer * self)
```

Détruire un consommateur.

Désallouer la mémoire allouée par LokiConsumer_new.

Paramètres

self	Le consommateur à détruire.

Définition à la ligne 76 du fichier libloki-consumer.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.2.3.2 LokiConsumer_new()

```
LokiConsumer * LokiConsumer_new (
            void )
```

Construire un consommateur.

Renvoie

Un nouveau consommateur.

Définition à la ligne 31 du fichier libloki-consumer.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.2.3.3 LokiConsumer_start()

```
int LokiConsumer_start (
            LokiConsumer * self,
            LokiFIFO * in,
            size_t cid)
```

Démarrer un consommateur.

Cette fonction est non bloquante. Elle crée un nouveau thread qui consomme les données d'une FIFO.

Paramètres

self	Le consommateur à démarrer.
in	La FIFO à lire.
cid	Le numéro de consommateur pour la FIFO.

Renvoie

Le code d'erreur retourné par `pthread_create`.

Définition à la ligne 60 du fichier libloki-consumer.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.2.3.4 LokiConsumer_stop()

```
void LokiConsumer_stop (
            LokiConsumer * self)
```

Arrêter un consommateur.

Cette fonction arrête le thread démarré par LokiConsumer_start pour le même consommateur.

Paramètres

self	Le consommateur à arrêter.

Définition à la ligne 67 du fichier libloki-consumer.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.2.3.5 LokiConsumer_work()

```
void * LokiConsumer_work (
            void * ptr)  [static]
```

Lire les données de la FIFO.

Cette fonction est exécutée dans un thread démarré par LokiConsumer_start. Elle consomme les données de la FIFO jusqu'à la demande d'arrêt par LokiConsumer_stop.

Paramètres

ptr	Le consommateur courant.

Définition à la ligne 47 du fichier libloki-consumer.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3 Structure de données FIFO

File d'attente multi-consommateur pour communication entre threads.

Graphe de collaboration de Structure de données FIFO:

Fichiers

- fichier libloki-fifo.c
- fichier libloki-fifo.h

Structures de données

- struct LokiFIFO

  Représentation d'une FIFO.

- struct LokiFIFORecord

  Représentation d'un enregistrement dans une FIFO.

Définitions de type

- typedef struct LokiFIFORecord LokiFIFORecord

  Représentation d'un enregistrement dans une FIFO.

- typedef struct LokiFIFO LokiFIFO

  Représentation d'une FIFO.

Fonctions

- LokiFIFO * LokiFIFO_new (size_t item_size, size_t length, size_t consumers)

  Créer une FIFO.

- void LokiFIFO_del (LokiFIFO *self)

  Détruire une FIFO.

- size_t LokiFIFO_get_item_size (const LokiFIFO *self)

  Obtenir la taille d'un élément de la FIFO, en octets.

- void * LokiFIFO_get_producer_item (LokiFIFO *self)

  Obtenir une référence vers l'élément courant en écriture.

- void LokiFIFO_push (LokiFIFO *self)

  Valider l'élément courant en écriture.

- void * LokiFIFO_get_consumer_item (LokiFIFO *self, size_t cid)

  Obtenir une référence vers l'élément courant en lecture.

- void LokiFIFO_pull (LokiFIFO *self, size_t cid)

  Libérer l'élément courant en lecture.

- void LokiFIFO_abort (LokiFIFO *self)

  Terminer les attentes en cours.

4.2.3.1 Description détaillée

File d'attente multi-consommateur pour communication entre threads.

4.2.3.2 Documentation des définitions de type

4.2.3.2.1 LokiFIFO

```
typedef struct LokiFIFO LokiFIFO
```

Représentation d'une FIFO.

Définition à la ligne 20 du fichier libloki-fifo.h.

4.2.3.2.2 LokiFIFORecord

```
typedef struct LokiFIFORecord LokiFIFORecord
```

Représentation d'un enregistrement dans une FIFO.

Définition à la ligne 11 du fichier libloki-fifo.c.

4.2.3.3 Documentation des fonctions

4.2.3.3.1 LokiFIFO_abort()

```
void LokiFIFO_abort (
            LokiFIFO * self)
```

Terminer les attentes en cours.

Paramètres

self	La FIFO à interrompre.

Définition à la ligne 180 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.2 LokiFIFO_del()

```
void LokiFIFO_del (
            LokiFIFO * self)
```

Détruire une FIFO.

Paramètres

self	La FIFO à détruire.

Définition à la ligne 95 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.3 LokiFIFO_get_consumer_item()

```
void * LokiFIFO_get_consumer_item (
            LokiFIFO * self,
            size_t cid)
```

Obtenir une référence vers l'élément courant en lecture.

Cette fonction retourne un pointeur sur un élément disponible en lecture dans la FIFO. Le consommateur peut traiter cet élément et appeler LokiFIFO_pull lorsque le traitement est terminé.

Cette fonction bloque le producteur tant que la FIFO est vide.

Paramètres

self	La FIFO à interroger.
cid	L'identifiant du consommateur courant.

Renvoie

Une référence vers un élément disponible en lecture.

Définition à la ligne 142 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.4 LokiFIFO_get_item_size()

```
size_t LokiFIFO_get_item_size (
            const LokiFIFO * self)
```

Obtenir la taille d'un élément de la FIFO, en octets.

Paramètres

self	La FIFO à interroger.

Renvoie

La taille d'un élément, en octets.

Définition à la ligne 102 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.5 LokiFIFO_get_producer_item()

```
void * LokiFIFO_get_producer_item (
            LokiFIFO * self)
```

Obtenir une référence vers l'élément courant en écriture.

Cette fonction retourne un pointeur sur un élément disponible en écriture dans la FIFO. Le producteur peut modifier cet élément et appeler LokiFIFO_push pour valider les modifications.

Cette fonction bloque le producteur tant que la FIFO est pleine.

Paramètres

self	La FIFO à interroger.

Renvoie

Une référence vers un élément disponible en écriture.

Définition à la ligne 106 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.6 LokiFIFO_new()

```
LokiFIFO * LokiFIFO_new (
             size_t item_size,
             size_t length,
             size_t consumers)
```

Créer une FIFO.

Cette fonction alloue l'espace nécessaire pour enregistrer des buffers de données.

Paramètres

item_size	La taille d'un élément, en octets.
length	Le nombre d'éléments.
consumers	Le nombre de consommateurs.

Renvoie

Une nouvelle FIFO.

Définition à la ligne 60 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.7 LokiFIFO_pull()

```
void LokiFIFO_pull (
             LokiFIFO * self,
             size_t cid)
```

Libérer l'élément courant en lecture.

Avant d'appeler cette fonction, le producteur doit obtenir une référence vers l'élément à lire en appelant LokiFIFO↩ _get_consumer_item. Cette fonction décrémente le niveau courant de la FIFO.

Cette fonction bloque le consommateur jusqu'à ce que l'élément courant ait été consommé par tous les consommateurs.

Paramètres

self	La FIFO à modifier.
cid	L'identifiant du consommateur courant.

Définition à la ligne 160 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.3.3.8 LokiFIFO_push()

```
void LokiFIFO_push (
            LokiFIFO * self)
```

Valider l'élément courant en écriture.

Avant d'appeler cette fonction, le producteur doit obtenir une référence vers l'élément à écrire en appelant Loki↩ FIFO_get_producer_item. Cette fonction incrémente le niveau courant de la FIFO.

Paramètres

self	La FIFO à modifier.

Définition à la ligne 123 du fichier libloki-fifo.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.4 Type paquets d'échantillons

Type de données pour la communication de paquets d'échantillons datés entre threads.

Graphe de collaboration de Type paquets d'échantillons:

Fichiers

— fichier libloki-packet.h

Structures de données

— struct LokiPacket

Structure d'un paquet d'échantillons échangés entre tâches concurrentes.

Fonctions

— static size_t LokiPacket_get_fifo_item_size (size_t packet_length, size_t sample_size)

Calculer la taille d'un paquet de données.

— static size_t LokiPacket_get_length (LokiFIFO ∗fifo, size_t sample_size)

Lire la longueur des paquets supportés par une FIFO.

4.2.4.1 Description détaillée

Type de données pour la communication de paquets d'échantillons datés entre threads.

4.2.4.2 Documentation des fonctions

4.2.4.2.1 LokiPacket_get_fifo_item_size()

```
size_t LokiPacket_get_fifo_item_size (
            size_t packet_length,
            size_t sample_size)  [inline], [static]
```

Calculer la taille d'un paquet de données.

Le résultat inclut la taille des métadonnées contenues dans le paquet.

Paramètres

packet_length	Le nombre d'échantillons contenus dans le paquet.
sample_size	La taille d'un échantillon, en octets.

Définition à la ligne 31 du fichier libloki-packet.h.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.2.4.2.2 LokiPacket_get_length()

```
size_t LokiPacket_get_length (
            LokiFIFO * fifo,
            size_t sample_size)  [inline], [static]
```

Lire la longueur des paquets supportés par une FIFO.

Paramètres

fifo	La FIFO à interroger.
sample_size	La taille d'un échantillon, en octets.

Renvoie

Le nombre d'échantillons par paquet dans cette FIFO.

Définition à la ligne 41 du fichier libloki-packet.h.

Voici le graphe d'appel pour cette fonction :

4.2.5 Types et fonctions de gestion du temps

Définition de types *timestamp* et intervalle de temps, et de fonctions de conversion d'unités de temps.

Graphe de collaboration de Types et fonctions de gestion du temps:

Fichiers

— fichier libloki-time.h

Macros

— #define TIMESTAMP_SEC 1000000000

Le nombre d'unités du type LokiTimestamp dans une seconde.

— #define TIMESTAMP_MS 1000000

Le nombre d'unités du type LokiTimestamp dans une milliseconde.

Définitions de type

— typedef uint64_t LokiTimestamp

Représentation d'une date, en nanosecondes par rapport à une date de référence.

— typedef int64_t LokiTimeDelta

Représentation d'un intervalle de temps, en nanosecondes.

Fonctions

— static LokiTimestamp LokiTime_to_timestamp (struct timespec ts)

Convertir une date en timestamp.

— static time_t LokiTime_seconds (LokiTimestamp timestamp)

Calculer le nombre total de secondes dans un timestamp.

— static time_t LokiTime_milliseconds (LokiTimestamp timestamp)

Calculer le nombre de millisecondes depuis la dernière seconde dans un timestamp.

— static LokiTimestamp LokiTime_update (LokiTimestamp ref, size_t index, float frequency)

Calculer le timestamp d'un échantillon dans un signal.

4.2.5.1 Description détaillée

Définition de types *timestamp* et intervalle de temps, et de fonctions de conversion d'unités de temps.

4.2.5.2 Documentation des macros

4.2.5.2.1 TIMESTAMP_MS

```
#define TIMESTAMP_MS 1000000
```

Le nombre d'unités du type LokiTimestamp dans une milliseconde.

Définition à la ligne 26 du fichier libloki-time.h.

4.2.5.2.2 TIMESTAMP_SEC

```
#define TIMESTAMP_SEC 1000000000
```

Le nombre d'unités du type LokiTimestamp dans une seconde.

Définition à la ligne 23 du fichier libloki-time.h.

4.2.5.3 Documentation des définitions de type

4.2.5.3.1 LokiTimeDelta

```
typedef int64_t LokiTimeDelta
```

Représentation d'un intervalle de temps, en nanosecondes.

Définition à la ligne 20 du fichier libloki-time.h.

4.2.5.3.2 LokiTimestamp

```
typedef uint64_t LokiTimestamp
```

Représentation d'une date, en nanosecondes par rapport à une date de référence.

Définition à la ligne 17 du fichier libloki-time.h.

4.2.5.4 Documentation des fonctions

4.2.5.4.1 LokiTime_milliseconds()

```
time_t LokiTime_milliseconds (
            LokiTimestamp timestamp)  [inline], [static]
```

Calculer le nombre de millisecondes depuis la dernière seconde dans un timestamp.

Paramètres

timestamp	Le timestamp à convertir.

Renvoie

Le nombre de millisecondes, arrondi à l'entier inférieur.

Définition à la ligne 51 du fichier libloki-time.h.

4.2.5.4.2 LokiTime_seconds()

```
time_t LokiTime_seconds (
            LokiTimestamp timestamp)  [inline], [static]
```

Calculer le nombre total de secondes dans un timestamp.

Paramètres

timestamp	Le timestamp à convertir.

Renvoie

Le nombre de secondes, arrondi à l'entier inférieur.

Définition à la ligne 42 du fichier libloki-time.h.

4.2.5.4.3 LokiTime_to_timestamp()

```
LokiTimestamp LokiTime_to_timestamp (
            struct timespec ts)  [inline], [static]
```

Convertir une date en timestamp.

Paramètres

ts	La date à convertir.

Renvoie

Le timestamp correspondant.

Définition à la ligne 33 du fichier libloki-time.h.

4.2.5.4.4 LokiTime_update()

```
LokiTimestamp LokiTime_update (
            LokiTimestamp ref,
            size_t index,
            float frequency)  [inline], [static]
```

Calculer le timestamp d'un échantillon dans un signal.

Paramètres

ref	Le timestamp de l'échantillon d'indice 0.
index	L'indice de l'échantillon courant.
frequency	La fréquence d'échantillonnage, en Hz.

Renvoie

Le timestamp de l'échantillon courant.

Définition à la ligne 62 du fichier libloki-time.h.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3 Acquisition

Graphe de collaboration de Acquisition:

Sujets

— Acquisition de données provenant des capteurs auxiliaires

Ce module fournit une API en espace utilisateur pour accéder aux composants de gestion des capteurs d'environnement interfacés avec le FPGA.

— Pilotage des composants modulateur et démodulateur

Ce module fournit une API en espace utilisateur pour accéder aux composants modulateur et démodulateur du FPGA.

4.3.1 Description détaillée

4.3.2 Acquisition de données provenant des capteurs auxiliaires

Ce module fournit une API en espace utilisateur pour accéder aux composants de gestion des capteurs d'environnement interfacés avec le FPGA.

Graphe de collaboration de Acquisition de données provenant des capteurs auxiliaires:

Fichiers

— fichier libloki-env-sensors.c

— fichier libloki-env-sensors.h

Structures de données

— struct LokiEnvSensors

Représentation de l'interface d'acquisition des capteurs auxiliaires.

— struct LokiEnvSensorsSample

Représentation d'un échantillon en provenance des capteurs auxiliaires.

— union LokiEnvSensorsSample::[union].__unnamed0__

La valeur de l'échantillon.

Macros

— #define USE_MMAP

Utiliser la fonction `mmap` pour accéder aux données DMA.

Définitions de type

— typedef uint32_t LokiDMASample

Le type de donnée brut d'un échantillon passé par transfert DMA.

— typedef struct LokiEnvSensors LokiEnvSensors

Représentation des capteurs auxiliaires.

Fonctions

— static uint32_t LokiEnvSensors_get_reg (const LokiEnvSensors ∗self, LokiEnvSensorsRegister reg)

Lire un registre du circuit d'acquisition.

— static void LokiEnvSensors_set_reg (const LokiEnvSensors ∗self, LokiEnvSensorsRegister reg, uint32_t value)

Affecter un registre du circuit d'acquisition.

— static void LokiEnvSensors_dma_start (const LokiEnvSensors ∗self)

Demander au circuit d'acquisition de démarrer les transferts DMA.

— static void LokiEnvSensors_dma_stop (const LokiEnvSensors ∗self)

Demander au circuit d'acquisition d'arrêter les transferts DMA.

— static bool LokiEnvSensors_dma_is_streaming (const LokiEnvSensors ∗self)

Lire l'état des transferts DMA.

— static void LokiEnvSensors_dma_get_start_time (const LokiEnvSensors ∗self, struct timespec ∗res)

Lire la date de démarrage des transferts DMA.

— static void LokiEnvSensors_dma_get_packet_time (const LokiEnvSensors ∗self, struct timespec ∗res)

Lire la date du dernier transfert DMA.

— static int32_t LokiEnvSensors_bmp390_compensate_temperature (LokiEnvSensors ∗self, uint32_t data)

— static int32_t LokiEnvSensors_bmp390_compensate_pressure (const LokiEnvSensors ∗self, uint32_t data)

— LokiEnvSensors ∗ LokiEnvSensors_new (void)

Construire un module d'acquisition pour les capteurs auxiliaires.

— void LokiEnvSensors_del (LokiEnvSensors ∗self)

Détruire un module d'acquisition pour les capteurs auxiliaires.

— void LokiEnvSensors_restart (const LokiEnvSensors ∗self)

Redémarrer le module d'acquisition pour les capteurs auxiliaires.

— uint32_t LokiEnvSensors_get_clk_frequency (const LokiEnvSensors ∗self)

Lire la fréquence d'horloge du circuit d'acquisition.

— size_t LokiEnvSensors_get_channels (const LokiEnvSensors ∗self)

Lire le nombre de voies du circuit d'acquisition.

— float LokiEnvSensors_get_sampling_frequency (const LokiEnvSensors ∗self)

Lire la fréquence d'échantillonnage du circuit d'acquisition.

— void LokiEnvSensors_set_sampling_frequency (const LokiEnvSensors ∗self, float freq)

Affecter une fréquence d'échantillonnage au circuit d'acquisition.

— uint32_t LokiEnvSensors_get_packet_length_max (const LokiEnvSensors ∗self)

Lire la longueur maximale des paquets DMA.

— uint32_t LokiEnvSensors_get_packet_length (const LokiEnvSensors ∗self)

Lire la longueur courante d'un paquet DMA.

— float LokiEnvSensors_get_scaling_factor (const LokiEnvSensors ∗self, size_t channel)

Lire le facteur de mise à l'échelle des échantillons d'une voie.

— void LokiEnvSensors_set_scaling_factor (LokiEnvSensors ∗self, size_t channel, float scale)

Affecter un facteur de mise à l'échelle des échantillons d'une voie.

— float LokiEnvSensors_get_last_data (const LokiEnvSensors ∗self, size_t channel)

Lire la valeur du dernier échantillon acquis sur une voie.

— static uint32_t LokiEnvSensors_get_packet_index (const LokiEnvSensors ∗self, LokiDMASample raw_item)

Lire l'indice d'un paquet reçu.

— static void LokiEnvSensors_set_packet_item (const LokiEnvSensors ∗self, LokiEnvSensorsSample ∗item, LokiDMASample raw_item)

Convertir un échantillon issu du DMA.

— static void ∗ LokiEnvSensors_work (void ∗ptr)

Traiter un tableau d'échantillons issu d'un transfert DMA.

— int LokiEnvSensors_start_streaming (LokiEnvSensors ∗self, int32_t channels, bool raw, LokiFIFO ∗out)

Démarrer l'acquisition.

— void LokiEnvSensors_stop_streaming (LokiEnvSensors ∗self)

Arrêter l'acquisition.

— void LokiEnvSensors_force_stop (const LokiEnvSensors ∗self)

Arrêter l'acquisition immédiatement.

— bool LokiEnvSensors_is_streaming (const LokiEnvSensors ∗self)

Lire l'état de l'acquisition.

4.3.2.1 Description détaillée

Ce module fournit une API en espace utilisateur pour accéder aux composants de gestion des capteurs d'environnement interfacés avec le FPGA.

Il s'appuie sur un module noyau Linux à travers les interfaces IOCTL et MMAP. Ce module noyau implémente les mécanismes de plus bas niveau pour l'accès au matériel (registres, DMA).

4.3.2.2 Documentation des macros

4.3.2.2.1 USE_MMAP

```
#define USE_MMAP
```

Utiliser la fonction `mmap` pour accéder aux données DMA.

Commenter cette ligne pour utiliser `fread`.

Définition à la ligne 415 du fichier libloki-env-sensors.c.

4.3.2.3 Documentation des définitions de type

4.3.2.3.1 LokiDMASample

```
typedef uint32_t LokiDMASample
```

Le type de donnée brut d'un échantillon passé par transfert DMA.

Définition à la ligne 63 du fichier libloki-env-sensors.c.

4.3.2.3.2 LokiEnvSensors

```
typedef struct LokiEnvSensors LokiEnvSensors
```

Représentation des capteurs auxiliaires.

Définition à la ligne 23 du fichier libloki-env-sensors.h.

4.3.2.4 Documentation des fonctions

4.3.2.4.1 LokiEnvSensors_bmp390_compensate_pressure()

```
int32_t LokiEnvSensors_bmp390_compensate_pressure (
            const LokiEnvSensors * self,
            uint32_t data)  [static]
```

Définition à la ligne 193 du fichier libloki-env-sensors.c.

4.3.2.4.2 LokiEnvSensors_bmp390_compensate_temperature()

```
int32_t LokiEnvSensors_bmp390_compensate_temperature (
            LokiEnvSensors * self,
            uint32_t data)  [static]
```

Définition à la ligne 174 du fichier libloki-env-sensors.c.

4.3.2.4.3 LokiEnvSensors_del()

```
void LokiEnvSensors_del (
            LokiEnvSensors * self)
```

Détruire un module d'acquisition pour les capteurs auxiliaires.

Paramètres

self	Le module à détruire.

Définition à la ligne 290 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.4 LokiEnvSensors_dma_get_packet_time()

```
void LokiEnvSensors_dma_get_packet_time (
            const LokiEnvSensors * self,
            struct timespec * res)  [inline], [static]
```

Lire la date du dernier transfert DMA.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.
res	Un pointeur vers le résultat.

Définition à la ligne 166 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.5 LokiEnvSensors_dma_get_start_time()

```
void LokiEnvSensors_dma_get_start_time (
            const LokiEnvSensors * self,
            struct timespec * res)  [inline], [static]
```

Lire la date de démarrage des transferts DMA.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.
res	Un pointeur vers le résultat.

Définition à la ligne 152 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.6 LokiEnvSensors_dma_is_streaming()

```
bool LokiEnvSensors_dma_is_streaming (
            const LokiEnvSensors * self)  [inline], [static]
```

Lire l'état des transferts DMA.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.

Définition à la ligne 135 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.7 LokiEnvSensors_dma_start()

```
void LokiEnvSensors_dma_start (
            const LokiEnvSensors * self)  [inline], [static]
```

Demander au circuit d'acquisition de démarrer les transferts DMA.

Cette fonction utilise la fonction `ioctl` pour agir sur le circuit d'acquisition à travers le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.

Définition à la ligne 108 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.8 LokiEnvSensors_dma_stop()

```
void LokiEnvSensors_dma_stop (
            const LokiEnvSensors * self)  [inline], [static]
```

Demander au circuit d'acquisition d'arrêter les transferts DMA.

Cette fonction utilise la fonction `ioctl` pour agir sur le circuit d'acquisition à travers le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.

Définition à la ligne 122 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.9 LokiEnvSensors_force_stop()

```
void LokiEnvSensors_force_stop (
            const LokiEnvSensors * self)
```

Arrêter l'acquisition immédiatement.

Interrompt l'exécution du thread lancé par la fonction LokiEnvSensors_start_streaming. Cette fonction force l'arrêt des transferts DMA sans attendre la fin d'un paquet.

Paramètres

self	Le module d'acquisition à arrêter.

Définition à la ligne 589 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.10 LokiEnvSensors_get_channels()

```
size_t LokiEnvSensors_get_channels (
            const LokiEnvSensors * self)
```

Lire le nombre de voies du circuit d'acquisition.

Paramètres

self	Le module d'acquisition à interroger.

Renvoie

Le nombre de voies.

Définition à la ligne 316 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.11 LokiEnvSensors_get_clk_frequency()

```
uint32_t LokiEnvSensors_get_clk_frequency (
            const LokiEnvSensors * self)
```

Lire la fréquence d'horloge du circuit d'acquisition.

Paramètres

self	Le module d'acquisition à interroger.

Renvoie

La fréquence d'horloge, en Hz.

Définition à la ligne 312 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.12 LokiEnvSensors_get_last_data()

```
float LokiEnvSensors_get_last_data (
            const LokiEnvSensors * self,
            size_t channel)
```

Lire la valeur du dernier échantillon acquis sur une voie.

Paramètres

self	Le module d'acquisition à interroger.
channel	Le numéro de la voie.

Renvoie

La valeur du dernier échantillon.

Définition à la ligne 344 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.13 LokiEnvSensors_get_packet_index()

```
uint32_t LokiEnvSensors_get_packet_index (
            const LokiEnvSensors * self,
            LokiDMASample raw_item)  [inline], [static]
```

Lire l'indice d'un paquet reçu.

Cette fonction extrait d'un échantillon brut le champ contenant l'indice du paquet.

Paramètres

self	Le module d'acquisition courant.
raw_item	Un échantillon obtenu lors d'un transfert DMA.

Renvoie

L'indice du paquet contenant cet échantillon.

Définition à la ligne 368 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.14 LokiEnvSensors_get_packet_length()

```
uint32_t LokiEnvSensors_get_packet_length (
            const LokiEnvSensors * self)
```

Lire la longueur courante d'un paquet DMA.

Paramètres

self	Le module d'acquisition à interroger.

Renvoie

Le nombre d'échantillons par paquet.

Définition à la ligne 332 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.15 LokiEnvSensors_get_packet_length_max()

```
uint32_t LokiEnvSensors_get_packet_length_max (
            const LokiEnvSensors * self)
```

Lire la longueur maximale des paquets DMA.

Paramètres

self	Le module d'acquisition à interroger.

Renvoie

Le nombre maximal d'échantillons par paquet.

Définition à la ligne 328 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.16 LokiEnvSensors_get_reg()

```
uint32_t LokiEnvSensors_get_reg (
            const LokiEnvSensors * self,
            LokiEnvSensorsRegister reg)  [inline], [static]
```

Lire un registre du circuit d'acquisition.

Cette fonction utilise la fonction `ioctl` pour interroger le circuit d'acquisition à travers le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.
reg	Le numéro du registre à lire.

Renvoie

La valeur du registre.

Définition à la ligne 74 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.17 LokiEnvSensors_get_sampling_frequency()

```
float LokiEnvSensors_get_sampling_frequency (
            const LokiEnvSensors * self)
```

Lire la fréquence d'échantillonnage du circuit d'acquisition.

Paramètres

self	Le module d'acquisition à interroger.

Renvoie

La fréquence d'échantillonnage, en Hz.

Définition à la ligne 320 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.18 LokiEnvSensors_get_scaling_factor()

```
float LokiEnvSensors_get_scaling_factor (
            const LokiEnvSensors * self,
            size_t channel)
```

Lire le facteur de mise à l'échelle des échantillons d'une voie.

Ce facteur permet de convertir la valeur numérique brute issue du capteur en une valeur dans une unité de mesure de notre choix.

Paramètres

self	Le module d'acquisition à modifier.
channel	Le numéro de la voie.

Renvoie

Le facteur d'échelle appliqué à cette voie.

Définition à la ligne 336 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.19 LokiEnvSensors_is_streaming()

```
bool LokiEnvSensors_is_streaming (
            const LokiEnvSensors * self)
```

Lire l'état de l'acquisition.

Paramètres

self	Le module d'acquisition à interroger.

Renvoie

true si l'acquisition est en cours.

Définition à la ligne 593 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.20 LokiEnvSensors_new()

```
LokiEnvSensors * LokiEnvSensors_new (
            void )
```

Construire un module d'acquisition pour les capteurs auxiliaires.

Renvoie

Une instance de LokiEnvSensors.

Définition à la ligne 235 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.21 LokiEnvSensors_restart()

```
void LokiEnvSensors_restart (
            const LokiEnvSensors * self)
```

Redémarrer le module d'acquisition pour les capteurs auxiliaires.

Cette fonction déclenche une réinitialisation des compteurs d'échantillons du module d'acquisition dans le FPGA.

Paramètres

self	Le module à redémarrer.

Définition à la ligne 306 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.22 LokiEnvSensors_set_packet_item()

```
void LokiEnvSensors_set_packet_item (
            const LokiEnvSensors * self,
            LokiEnvSensorsSample * item,
            LokiDMASample raw_item)  [static]
```

Convertir un échantillon issu du DMA.

Cette fonction extrait les champs contenant le numéro de voie et la valeur de l'échantillon. Elle convertit la valeur en lui appliquant un facteur d'échelle.

Paramètres

self	Le module d'acquisition courant.
item	Un pointeur vers une structure qui contiendra le résultat.
raw_item	Un échantillon obtenu lors d'un transfert DMA.

Définition à la ligne 382 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.23 LokiEnvSensors_set_reg()

```
void LokiEnvSensors_set_reg (
            const LokiEnvSensors * self,
            LokiEnvSensorsRegister reg,
            uint32_t value)  [inline], [static]
```

Affecter un registre du circuit d'acquisition.

Cette fonction utilise la fonction `ioctl` pour agir sur le circuit d'acquisition à travers le module noyau `loki-env-sensors`.

Paramètres

self	Le module d'acquisition courant.
reg	Le numéro du registre à modifier.
value	La nouvelle valeur du registre.

Définition à la ligne 93 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.24 LokiEnvSensors_set_sampling_frequency()

```
void LokiEnvSensors_set_sampling_frequency (
            const LokiEnvSensors * self,
            float freq)
```

Affecter une fréquence d'échantillonnage au circuit d'acquisition.

Paramètres

self	Le module d'acquisition à interroger.
freq	La fréquence d'échantillonnage, en Hz.

Définition à la ligne 324 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.25 LokiEnvSensors_set_scaling_factor()

```
void LokiEnvSensors_set_scaling_factor (
            LokiEnvSensors * self,
            size_t channel,
            float scale)
```

Affecter un facteur de mise à l'échelle des échantillons d'une voie.

Ce facteur permet de convertir la valeur numérique brute issue du capteur en une valeur dans une unité de mesure de notre choix.

Paramètres

self	Le module d'acquisition à modifier.
channel	Le numéro de la voie.
scale	Le facteur d'échelle à appliquer à cette voie.

Définition à la ligne 340 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.26 LokiEnvSensors_start_streaming()

```
int LokiEnvSensors_start_streaming (
            LokiEnvSensors * self,
            int32_t channels,
            bool raw,
            LokiFIFO * out)
```

Démarrer l'acquisition.

Cette fonction est non bloquante. Elle crée un nouveau thread pour la réception des données en provenance du pilote des capteurs.

Paramètres

self	Le module d'acquisition à démarrer.
channels	Un masque indiquant les voies à traiter, -1 pour toutes les voies.
raw	Faut-il conserver les valeurs d'échantillons en virgule fixe ?
out	La FIFO où ranger les échantillons mis à l'échelle.

Renvoie

Le code d'erreur retourné par `pthread_create`.

Définition à la ligne 569 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.27 LokiEnvSensors_stop_streaming()

```
void LokiEnvSensors_stop_streaming (
            LokiEnvSensors * self)
```

Arrêter l'acquisition.

Interrompt l'exécution du thread lancé par la fonction LokiEnvSensors_start_streaming. L'arrêt est effectif à la fin du paquet de données en cours de réception.

Paramètres

self	Le module d'acquisition à arrêter.

Définition à la ligne 580 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.2.4.28 LokiEnvSensors_work()

```
void * LokiEnvSensors_work (
            void * ptr)  [static]
```

Traiter un tableau d'échantillons issu d'un transfert DMA.

Cette fonction est exécutée dans un thread démarré par LokiEnvSensors_start_streaming. Elle se termine lorsqu'une demande d'arrêt est émise par LokiEnvSensors_stop_streaming.

Si la constante `USE_MMAP` est définie, elle s'appuie sur la fonction `mmap` pour se synchroniser avec le pilote du circuit modem et accéder au tampon de données de chaque transfert en mode *zero-copy*. Si la constante `USE_MMAP` n'est pas définie, la fonction `fread` est utilisée.

Cette fonction remplit la FIFO de sortie avec des paquets d'échantillons convertis par la fonction LokiEnvSensors_set_packet_item.

Paramètres

ptr	Le module d'acquisition courant.

Renvoie

Rien.

Définition à la ligne 433 du fichier libloki-env-sensors.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3 Pilotage des composants modulateur et démodulateur

Ce module fournit une API en espace utilisateur pour accéder aux composants modulateur et démodulateur du FPGA.

Graphe de collaboration de Pilotage des composants modulateur et démodulateur:

Fichiers

- fichier libloki-modem.c
- fichier libloki-modem.h

Structures de données

- struct LokiModem

  Représentation des périphériques modulateur et démodulateur.

- union AD8370ControlWord

  Représentation du mot de contrôle du gain pour l'amplificateur AD8370.

- struct AD8370ControlWord::[struct].__unnamed0__

  Les champs du mot de contrôle.

- struct LokiModemSample

  Données retournées par le démodulateur pour un échantillon.

- union LokiModemSample::[union].__unnamed0__

  La composante X de l'échantillon.

- union LokiModemSample::[union].__unnamed1__

  La composante Y de l'échantillon.

Macros

— #define AD8370_VERNIER 0.055744

Le paramètre Vernier pour le réglage du gain de l'amplificateur AD8370.

— #define AD8370_PREGAIN 7.079458

Le paramètre PreGain pour le réglage du gain de l'amplificateur AD8370.

— #define AD8370_HIGH_GAIN_FACTOR (AD8370_VERNIER ∗ AD8370_PREGAIN)

Le facteur appliqué au code de gain dans le mode High Gain de l'amplificateur AD8370.

— #define AD8370_GAIN_CODE_SIZE 7

Le nombre de bits du code de gain de l'amplificateur AD8370.

— #define AD8370_GAIN_CODE_MAX ((1 << AD8370_GAIN_CODE_SIZE) - 1)

La valeur maximale du code de gain de l'amplificateur AD8370.

— #define AD8370_LOW_GAIN_MAX (AD8370_GAIN_CODE_MAX ∗ AD8370_VERNIER)

Le gain maximal de l'amplificateur AD8370 en mode Low Gain.

— #define AD8370_HIGH_GAIN_MAX (AD8370_GAIN_CODE_MAX ∗ AD8370_HIGH_GAIN_FACTOR)

Le gain maximal de l'amplificateur AD8370 en mode High Gain.

— #define SIMULATED_DISP_WAVE_FREQUENCY 0.2

La fréquence du déplacement en mode simulation de capteurs, en Hz.

— #define SIMULATED_DISP_AMPLITUDE (M_PI / 10)

L'amplitude du déplacement en mode simulation de capteurs, en radians.

— #define SIMULATED_ELLIPSE_A 0.1

Le demi-grand axe de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_B 0.15

Le demi-petit axe de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_ANGLE (M_PI / 6)

L'angle de rotation de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_XC 0.01

La coordonnée X du centre de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_YC 0.02

La coordonnée Y du centre de l'ellipse en mode simulation de capteurs.

— #define USE_MMAP

Utiliser la fonction `mmap` pour accéder aux données DMA.

Définitions de type

— typedef uint64_t LokiDMASample

Le type brut d'un échantillon démodulé transféré par DMA.

— typedef struct LokiModem LokiModem

Représentation des périphériques modulateur et démodulateur.

Fonctions

— static uint32_t LokiModem_get_reg (const LokiModem *self, LokiModemRegister reg)

Lire un registre du circuit de modulation/démodulation.

— static void LokiModem_set_reg (const LokiModem *self, LokiModemRegister reg, uint32_t value)

Affecter un registre du circuit de modulation/démodulation.

— static void LokiModem_dma_start (const LokiModem *self)

Demander au circuit de modulation/démodulation de démarrer les transferts DMA.

— static void LokiModem_dma_stop (const LokiModem *self)

Demander au circuit de modulation/démodulation d'arrêter les transferts DMA.

— static bool LokiModem_dma_is_streaming (const LokiModem *self)

Lire l'état des transferts DMA.

— static void LokiModem_dma_get_start_time (const LokiModem *self, struct timespec *res)

Lire la date de démarrage des transferts DMA.

— static void LokiModem_dma_get_packet_time (const LokiModem *self, struct timespec *res)

Lire la date du dernier transfert DMA.

— static unsigned LokiModem_dma_get_dropped_packets (const LokiModem *self)

Lire le nombre de paquets DMA perdus.

— static float LokiModem_scale_xy (const LokiModem *self, float xy)

Mettre à l'échelle une composante X (I) ou Y (Q) d'un échantillon démodulé.

— static float LokiModem_scale_power (const LokiModem *self, float p)

Mettre à l'échelle une valeur d'entrée du démodulateur.

— LokiModem * LokiModem_new (void)

Construire un nouveau modem.

— void LokiModem_del (LokiModem ∗self)

Détruire un modem.

— uint32_t LokiModem_get_clk_frequency (const LokiModem ∗self)

Lire la fréquence d'horloge du circuit d'acquisition.

— float LokiModem_get_mod_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en sortie du modulateur.

— void LokiModem_set_mod_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en sortie du modulateur.

— float LokiModem_get_mod_osc1_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

— void LokiModem_set_mod_osc1_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain_res (const LokiModem ∗self)

Lire le pas de réglage de l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain_max (const LokiModem ∗self)

Lire l'amplitude maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain (const LokiModem ∗self)

Lire l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

— void LokiModem_set_mod_osc1_gain (const LokiModem ∗self, float gain)

Affecter une amplitude au signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc2_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

— void LokiModem_set_mod_osc2_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain_res (const LokiModem ∗self)

Lire le pas de réglage de l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain_max (const LokiModem ∗self)

Lire l'amplitude maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain (const LokiModem ∗self)

Lire l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

— void LokiModem_set_mod_osc2_gain (const LokiModem ∗self, float gain)

Affecter une amplitude au signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

— void LokiModem_set_demod_osc_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_mod_offset_res (const LokiModem ∗self)

Lire le pas de réglage de l'offset en sortie du modulateur.

— float LokiModem_get_mod_offset_max (const LokiModem ∗self)

Lire l'offset maximal supporté en sortie du modulateur.

— float LokiModem_get_mod_offset (const LokiModem ∗self)

Lire l'offset en sortie du modulateur.

— void LokiModem_set_mod_offset (const LokiModem ∗self, float offset)

Affecter un offset en sortie du modulateur.

— float LokiModem_get_mod_dc_value_res (const LokiModem ∗self)

Lire le pas de réglage du courant moyen de commande de la diode laser.

— float LokiModem_get_mod_dc_value_max (const LokiModem ∗self)

Lire le courant moyen maximal supporté pour la commande de la diode laser.

— float LokiModem_get_mod_dc_value (const LokiModem ∗self)

Lire le courant moyen de commande de la diode laser.

— void LokiModem_set_mod_dc_value (const LokiModem ∗self, float dc_value)

Affecter un courant moyen de commande de la diode laser.

— uint32_t LokiModem_get_mod_pga_gain_max (const LokiModem ∗self)

Lire le gain maximal de l'amplificateur de sortie.

— uint32_t LokiModem_get_mod_pga_gain (const LokiModem ∗self)

Lire le gain de l'amplificateur de sortie.

— void LokiModem_set_mod_pga_gain (const LokiModem ∗self, uint32_t gain)

Affecter un gain à l'amplificateur de sortie.

— uint32_t LokiModem_get_demod_channels (const LokiModem ∗self)

Lire le nombre de voies du démodulateur.

— uint32_t LokiModem_get_demod_decimation (const LokiModem ∗self)

Lire le facteur de décimation des filtres du démodulateur.

— float LokiModem_get_demod_bandwidth (const LokiModem ∗self)

Lire la bande passante des filtres du démodulateur.

— float LokiModem_get_demod_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en entrée du démodulateur.

— void LokiModem_set_demod_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en entrée du démodulateur.

— float LokiModem_get_demod_xy_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en sortie du démodulateur.

— void LokiModem_set_demod_xy_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en sortie du démodulateur.

— uint32_t LokiModem_get_demod_delay_x_max (const LokiModem ∗self, uint32_t channel)

Lire le retard maximal applicable à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x_max (const LokiModem ∗self, uint32_t channel)

Lire le déphasage maximal applicable à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x_res (const LokiModem ∗self, uint32_t channel)

Lire le pas de réglage du déphasage applicable à la composante X (I) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_x (const LokiModem ∗self, uint32_t channel)

Lire le retard appliqué à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x (const LokiModem ∗self, uint32_t channel)

Lire le déphasage appliqué à la composante X (I) du signal démodulé pour une voie.

— void LokiModem_set_demod_delay_x (const LokiModem *self, uint32_t channel, uint32_t delay)

Affecter un retard à la composante X (I) du signal démodulé pour une voie.

— void LokiModem_set_demod_phase_x (const LokiModem *self, uint32_t channel, float phase)

Affecter un déphasage à la composante X (I) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_y_max (const LokiModem *self, uint32_t channel)

Lire le retard maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y_max (const LokiModem *self, uint32_t channel)

Lire le déphasage maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y_res (const LokiModem *self, uint32_t channel)

Lire le pas de réglage du déphasage applicable à la composante Y (Q) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_y (const LokiModem *self, uint32_t channel)

Lire le retard appliqué à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y (const LokiModem *self, uint32_t channel)

Lire le déphasage appliqué à la composante Y (Q) du signal démodulé pour une voie.

— void LokiModem_set_demod_delay_y (const LokiModem *self, uint32_t channel, uint32_t delay)

Affecter un retard à la composante Y (Q) du signal démodulé pour une voie.

— void LokiModem_set_demod_phase_y (const LokiModem *self, uint32_t channel, float phase)

Affecter un déphasage à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_pga_gain_max (const LokiModem *self, uint32_t channel)

Lire le gain maximal de l'amplificateur en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_pga_gain (const LokiModem *self, uint32_t channel)

Lire le gain de l'amplificateur en entrée du démodulateur pour une voie.

— void LokiModem_set_demod_pga_gain (const LokiModem *self, uint32_t channel, float gain)

Affecter un gain à l'amplificateur en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_adc_min_ratio (const LokiModem *self, uint32_t channel)

Lire la valeur minimale du signal en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_adc_max_ratio (const LokiModem *self, uint32_t channel)

Lire la valeur maximale du signal en entrée du démodulateur pour une voie.

— uint32_t LokiModem_get_demod_packet_length_max (const LokiModem *self)

Lire la longueur maximale d'un paquet de données en sortie du démodulateur.

— uint32_t LokiModem_get_demod_packet_length (const LokiModem ∗self)

Lire la longueur d'un paquet de données en sortie du démodulateur.

— bool LokiModem_get_demod_mixerless (const LokiModem ∗self)

Identifier le type de démodulateur implanté dans le FPGA.

— bool LokiModem_get_demod_bypass (const LokiModem ∗self)

Lire l'état d'activation du démodulateur.

— void LokiModem_set_demod_bypass (const LokiModem ∗self, bool bypass)

Activer ou désactiver le démodulateur.

— bool LokiModem_get_test_mode (const LokiModem ∗self)

Lire l'état d'activation du mode de test.

— void LokiModem_set_test_mode (const LokiModem ∗self, bool test_mode)

Activer ou désactiver le mode de test.

— void LokiModem_restart (const LokiModem ∗self)

Redémarrer le modem.

— uint32_t LokiModem_get_adc_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons reçus sur une voie ADC en entrée du démodulateur.

— uint32_t LokiModem_get_dac_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons envoyés sur la sortie DAC du modulateur.

— uint32_t LokiModem_get_demod_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons produits sur une voie en sortie du démodulateur.

— static uint32_t LokiModem_get_packet_index (const LokiModem ∗self, LokiDMASample raw_item)

Lire un numéro de paquet dans un échantillon DMA brut.

— static void LokiModem_set_packet_item (const LokiModem ∗self, LokiModemSample ∗item, LokiDMASample raw_item)

Affecter les champs d'un échantillon de sortie du démodulateur.

— static void LokiModem_set_simulated_item (LokiModem ∗self, LokiModemSample ∗item)

Affecter les champs d'un échantillon de sortie du démodulateur en mode simulation de capteurs.

— static void ∗ LokiModem_work (void ∗ptr)

Traiter un tableau d'échantillons issu d'un transfert DMA.

— int LokiModem_start_streaming (LokiModem ∗self, int32_t channels, bool simulate, bool raw, LokiFIFO ∗out)

Démarrer l'acquisition et le traitement des échantillons démodulés.

— void LokiModem_stop_streaming (LokiModem ∗self)

Arrêter l'acquisition.

— void LokiModem_force_stop (const LokiModem ∗self)

Arrêter l'acquisition immédiatement.

— bool LokiModem_is_streaming (const LokiModem ∗self)

Lire l'état de l'acquisition.

— unsigned LokiModem_get_dropped_packets (const LokiModem ∗self)

Lire le nombre de paquets perdus depuis le début de l'acquisition.

— void LokiModem_print_config (const LokiModem ∗self)

Afficher la configuration d'un modem sur la sortie standard.

4.3.3.1 Description détaillée

Ce module fournit une API en espace utilisateur pour accéder aux composants modulateur et démodulateur du FPGA.

Il s'appuie sur un module noyau Linux à travers les interfaces IOCTL et MMAP. Ce module noyau implémente les mécanismes de plus bas niveau pour l'accès au matériel (registres, DMA).

4.3.3.2 Documentation des macros

4.3.3.2.1 AD8370_GAIN_CODE_MAX

```
#define AD8370_GAIN_CODE_MAX ((1 << AD8370_GAIN_CODE_SIZE) - 1)
```

La valeur maximale du code de gain de l'amplificateur AD8370.

Définition à la ligne 693 du fichier libloki-modem.c.

4.3.3.2.2 AD8370_GAIN_CODE_SIZE

```
#define AD8370_GAIN_CODE_SIZE 7
```

Le nombre de bits du code de gain de l'amplificateur AD8370.

Définition à la ligne 691 du fichier libloki-modem.c.

4.3.3.2.3 AD8370_HIGH_GAIN_FACTOR

```
#define AD8370_HIGH_GAIN_FACTOR (AD8370_VERNIER * AD8370_PREGAIN)
```

Le facteur appliqué au code de gain dans le mode *High Gain* de l'amplificateur AD8370.

Définition à la ligne 689 du fichier libloki-modem.c.

4.3.3.2.4 AD8370_HIGH_GAIN_MAX

```
#define AD8370_HIGH_GAIN_MAX (AD8370_GAIN_CODE_MAX * AD8370_HIGH_GAIN_FACTOR)
```

Le gain maximal de l'amplificateur AD8370 en mode *High Gain*.

Définition à la ligne 697 du fichier libloki-modem.c.

4.3.3.2.5 AD8370_LOW_GAIN_MAX

```
#define AD8370_LOW_GAIN_MAX (AD8370_GAIN_CODE_MAX * AD8370_VERNIER)
```

Le gain maximal de l'amplificateur AD8370 en mode *Low Gain*.

Définition à la ligne 695 du fichier libloki-modem.c.

4.3.3.2.6 AD8370_PREGAIN

```
#define AD8370_PREGAIN 7.079458
```

Le paramètre *PreGain* pour le réglage du gain de l'amplificateur AD8370.

Définition à la ligne 687 du fichier libloki-modem.c.

4.3.3.2.7 AD8370_VERNIER

```
#define AD8370_VERNIER 0.055744
```

Le paramètre *Vernier* pour le réglage du gain de l'amplificateur AD8370.

Définition à la ligne 685 du fichier libloki-modem.c.

4.3.3.2.8 SIMULATED_DISP_AMPLITUDE

```
#define SIMULATED_DISP_AMPLITUDE (M_PI / 10)
```

L'amplitude du déplacement en mode simulation de capteurs, en radians.

Définition à la ligne 878 du fichier libloki-modem.c.

4.3.3.2.9 SIMULATED_DISP_WAVE_FREQUENCY

```
#define SIMULATED_DISP_WAVE_FREQUENCY 0.2
```

La fréquence du déplacement en mode simulation de capteurs, en Hz.

Définition à la ligne 876 du fichier libloki-modem.c.

4.3.3.2.10 SIMULATED_ELLIPSE_A

```
#define SIMULATED_ELLIPSE_A 0.1
```

Le demi-grand axe de l'ellipse en mode simulation de capteurs.

Définition à la ligne 880 du fichier libloki-modem.c.

4.3.3.2.11 SIMULATED_ELLIPSE_ANGLE

```
#define SIMULATED_ELLIPSE_ANGLE (M_PI / 6)
```

L'angle de rotation de l'ellipse en mode simulation de capteurs.

Définition à la ligne 884 du fichier libloki-modem.c.

4.3.3.2.12 SIMULATED_ELLIPSE_B

```
#define SIMULATED_ELLIPSE_B 0.15
```

Le demi-petit axe de l'ellipse en mode simulation de capteurs.

Définition à la ligne 882 du fichier libloki-modem.c.

4.3.3.2.13 SIMULATED_ELLIPSE_XC

```
#define SIMULATED_ELLIPSE_XC 0.01
```

La coordonnée X du centre de l'ellipse en mode simulation de capteurs.

Définition à la ligne 886 du fichier libloki-modem.c.

4.3.3.2.14 SIMULATED_ELLIPSE_YC

```
#define SIMULATED_ELLIPSE_YC 0.02
```

La coordonnée Y du centre de l'ellipse en mode simulation de capteurs.

Définition à la ligne 888 du fichier libloki-modem.c.

4.3.3.2.15 USE_MMAP

```
#define USE_MMAP
```

Utiliser la fonction `mmap` pour accéder aux données DMA.

Commenter cette ligne pour utiliser `fread`.

Définition à la ligne 926 du fichier libloki-modem.c.

4.3.3.3 Documentation des définitions de type

4.3.3.3.1 LokiDMASample

```
typedef uint64_t LokiDMASample
```

Le type brut d'un échantillon démodulé transféré par DMA.

Définition à la ligne 129 du fichier libloki-modem.c.

4.3.3.3.2 LokiModem

```
typedef struct LokiModem LokiModem
```

Représentation des périphériques modulateur et démodulateur.

Définition à la ligne 28 du fichier libloki-modem.h.

4.3.3.4 Documentation des fonctions

4.3.3.4.1 LokiModem_del()

```
void LokiModem_del (
            LokiModem * self)
```

Détruire un modem.

Paramètres

self	Le modem à détruire.

Définition à la ligne 372 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.2 LokiModem_dma_get_dropped_packets()

```
unsigned LokiModem_dma_get_dropped_packets (
            const LokiModem * self)  [inline], [static]
```

Lire le nombre de paquets DMA perdus.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-modem`.

Paramètres

self	Le modem courant.

Renvoie

Le nombre de transferts DMA dont les données ont été écrasées avant traitement.

Définition à la ligne 246 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.3 LokiModem_dma_get_packet_time()

```
void LokiModem_dma_get_packet_time (
            const LokiModem * self,
            struct timespec * res)  [inline], [static]
```

Lire la date du dernier transfert DMA.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-modem`.

Paramètres

self	Le modem courant.
res	Un pointeur vers le résultat.

Définition à la ligne 232 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.4 LokiModem_dma_get_start_time()

```
void LokiModem_dma_get_start_time (
            const LokiModem * self,
            struct timespec * res)  [inline], [static]
```

Lire la date de démarrage des transferts DMA.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-modem`.

Paramètres

self	Le modem courant.
res	Un pointeur vers le résultat.

Définition à la ligne 218 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.5 LokiModem_dma_is_streaming()

```
bool LokiModem_dma_is_streaming (
            const LokiModem * self)  [inline], [static]
```

Lire l'état des transferts DMA.

Cette fonction utilise la fonction `ioctl` pour interroger le module noyau `loki-modem`.

Paramètres

self	Le modem courant.

Définition à la ligne 201 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.6 LokiModem_dma_start()

```
void LokiModem_dma_start (
            const LokiModem * self)  [inline], [static]
```

Demander au circuit de modulation/démodulation de démarrer les transferts DMA.

Cette fonction utilise la fonction `ioctl` pour agir sur le circuit modem à travers le module noyau `loki-modem`.

Paramètres

self	Le modem courant.

Définition à la ligne 174 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.7 LokiModem_dma_stop()

```
void LokiModem_dma_stop (
            const LokiModem * self)  [inline], [static]
```

Demander au circuit de modulation/démodulation d'arrêter les transferts DMA.

Cette fonction utilise la fonction `ioctl` pour agir sur le circuit modem à travers le module noyau `loki-modem`.

Paramètres

self	Le modem courant.

Définition à la ligne 188 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.8 LokiModem_force_stop()

```
void LokiModem_force_stop (
            const LokiModem * self)
```

Arrêter l'acquisition immédiatement.

Interrompt l'exécution du thread lancé par la fonction LokiModem_start_streaming. Cette fonction force l'arrêt des transferts DMA sans attendre la fin d'un paquet.

Paramètres

self	Le modem à arrêter.

Définition à la ligne 1105 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.9 LokiModem_get_adc_sample_count()

```
uint32_t LokiModem_get_adc_sample_count (
            const LokiModem * self)
```

Lire le nombre d'échantillons reçus sur une voie ADC en entrée du démodulateur.

Ce compteur est remis à zéro à chaque redémarrage du modem.

Paramètres

self	Le modem à interroger.

Renvoie

Le nombre d'échantillons par voie.

Définition à la ligne 808 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.10 LokiModem_get_clk_frequency()

```
uint32_t LokiModem_get_clk_frequency (
            const LokiModem * self)
```

Lire la fréquence d'horloge du circuit d'acquisition.

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence d'horloge, en Hz.

Définition à la ligne 384 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.11 LokiModem_get_dac_sample_count()

```
uint32_t LokiModem_get_dac_sample_count (
            const LokiModem * self)
```

Lire le nombre d'échantillons envoyés sur la sortie DAC du modulateur.

Ce compteur est remis à zéro à chaque redémarrage du modem.

Paramètres

self	Le modem à interroger.

Renvoie

Le nombre d'échantillons.

Définition à la ligne 812 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.12 LokiModem_get_demod_adc_max_ratio()

```
float LokiModem_get_demod_adc_max_ratio (
            const LokiModem * self,
            uint32_t channel)
```

Lire la valeur maximale du signal en entrée du démodulateur pour une voie.

Cette valeur est mesurée sur une fenêtre glissante gérée dans le FPGA.

La valeur retournée est comprise entre -1.0 et 1.0. Les valeurs extrêmes correspondent à la saturation du convertisseur analogique-numérique d'entrée.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

La valeur maximale du signal sur cette voie.

Définition à la ligne 752 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.13 LokiModem_get_demod_adc_min_ratio()

```
float LokiModem_get_demod_adc_min_ratio (
            const LokiModem * self,
            uint32_t channel)
```

Lire la valeur minimale du signal en entrée du démodulateur pour une voie.

Cette valeur est mesurée sur une fenêtre glissante gérée dans le FPGA.

La valeur retournée est comprise entre -1.0 et 1.0. Les valeurs extrêmes correspondent à la saturation du convertisseur analogique-numérique d'entrée.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

La valeur minimale du signal sur cette voie.

Définition à la ligne 746 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.14 LokiModem_get_demod_bandwidth()

```
float LokiModem_get_demod_bandwidth (
            const LokiModem * self)
```

Lire la bande passante des filtres du démodulateur.

Il s'agit de la bande passante du dernier étage de filtrage passe-bas dans le démodulateur.

Paramètres

self	Le modem à interroger.

Renvoie

La bande passange en sortie du démodulateur.

Définition à la ligne 548 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.15 LokiModem_get_demod_bypass()

```
bool LokiModem_get_demod_bypass (
            const LokiModem * self)
```

Lire l'état d'activation du démodulateur.

En mode "bypass", les données brutes du convertisseur analogique/numérique sont transmises par DMA sans passer par le démodulateur.

Paramètres

self	Le modem à interroger.

Renvoie

true si le mode bypass est activé.

Définition à la ligne 770 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.16 LokiModem_get_demod_channels()

```
uint32_t LokiModem_get_demod_channels (
            const LokiModem * self)
```

Lire le nombre de voies du démodulateur.

Paramètres

self	Le modem à interroger.

Renvoie

Le nombre de voies.

Définition à la ligne 540 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.17 LokiModem_get_demod_decimation()

```
uint32_t LokiModem_get_demod_decimation (
            const LokiModem * self)
```

Lire le facteur de décimation des filtres du démodulateur.

Il s'agit du produit des facteurs de décimation des filtres en cascade dans le démodulateur.

Paramètres

self	Le modem à interroger.

Renvoie

Le facteur de décimation entre l'entrée et la sortie du démodulateur.

Définition à la ligne 544 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.18 LokiModem_get_demod_delay_x()

```
uint32_t LokiModem_get_demod_delay_x (
            const LokiModem * self,
            uint32_t channel)
```

Lire le retard appliqué à la composante X (I) du signal démodulé pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le retard, en échantillons.

Définition à la ligne 597 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.19 LokiModem_get_demod_delay_x_max()

```
uint32_t LokiModem_get_demod_delay_x_max (
             const LokiModem * self,
             uint32_t channel)
```

Lire le retard maximal applicable à la composante X (I) du signal démodulé pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le retard maximal, en échantillons.

Définition à la ligne 578 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.20 LokiModem_get_demod_delay_y()

```
uint32_t LokiModem_get_demod_delay_y (
             const LokiModem * self,
             uint32_t channel)
```

Lire le retard appliqué à la composante Y (Q) du signal démodulé pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le retard, en échantillons.

Définition à la ligne 650 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.21 LokiModem_get_demod_delay_y_max()

```
uint32_t LokiModem_get_demod_delay_y_max (
            const LokiModem * self,
            uint32_t channel)
```

Lire le retard maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le retard maximal, en échantillons.

Définition à la ligne 631 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.22 LokiModem_get_demod_mixerless()

```
bool LokiModem_get_demod_mixerless (
            const LokiModem * self)
```

Identifier le type de démodulateur implanté dans le FPGA.

Il existe deux implémentations du démodulateur :

- une version qui utilise des oscillateurs pour effectuer un décalage en fréquence
- une version "mixerless" qui procède par décimation

Paramètres

self	Le modem à interroger.

Renvoie

true si le démodulateur fonctionne par décimation.

Définition à la ligne 766 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.23 LokiModem_get_demod_osc_wave_frequency()

```
float LokiModem_get_demod_osc_wave_frequency (
            const LokiModem * self)
```

Lire la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence du signal, en Hz.

Définition à la ligne 481 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.24 LokiModem_get_demod_osc_wave_frequency_max()

```
float LokiModem_get_demod_osc_wave_frequency_max (
            const LokiModem * self)
```

Lire la fréquence maximale supportée pour le signal en sortie de l'oscillateur du démodulateur (Fm2).

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence maximale du signal, en Hz.

Définition à la ligne 474 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.25 LokiModem_get_demod_osc_wave_frequency_res()

```
float LokiModem_get_demod_osc_wave_frequency_res (
            const LokiModem * self)
```

Lire le pas de réglage de la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

La fréquence de l'oscillateur est toujours un multiple de cette valeur.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution du réglage de la fréquence du signal, en Hz.

Définition à la ligne 467 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.26 LokiModem_get_demod_packet_length()

```
uint32_t LokiModem_get_demod_packet_length (
            const LokiModem * self)
```

Lire la longueur d'un paquet de données en sortie du démodulateur.

Il s'agit du nombre d'échantillons qui sont échangés lors d'un transfert DMA entre le démodulateur et le processeur.

Paramètres

self	Le modem à interroger.

Renvoie

La longueur des transferts DMA, en échantillons.

Définition à la ligne 762 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.27 LokiModem_get_demod_packet_length_max()

```
uint32_t LokiModem_get_demod_packet_length_max (
            const LokiModem * self)
```

Lire la longueur maximale d'un paquet de données en sortie du démodulateur.

Il s'agit du nombre maximal d'échantillons qui sont échangés lors d'un transfert DMA entre le démodulateur et le processeur.

Paramètres

self	Le modem à interroger.

Renvoie

La longueur maximale des transferts DMA, en échantillons.

Définition à la ligne 758 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.28 LokiModem_get_demod_pga_gain()

```
float LokiModem_get_demod_pga_gain (
            const LokiModem * self,
            uint32_t channel)
```

Lire le gain de l'amplificateur en entrée du démodulateur pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le gain actuel.

Définition à la ligne 716 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.29 LokiModem_get_demod_pga_gain_max()

```
float LokiModem_get_demod_pga_gain_max (
            const LokiModem * self,
            uint32_t channel)
```

Lire le gain maximal de l'amplificateur en entrée du démodulateur pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le gain maximal.

Définition à la ligne 712 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.30 LokiModem_get_demod_phase_x()

```
float LokiModem_get_demod_phase_x (
            const LokiModem * self,
            uint32_t channel)
```

Lire le déphasage appliqué à la composante X (I) du signal démodulé pour une voie.

Voir également

LokiModem_get_demod_delay_x

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le déphasage, en degrés.

Définition à la ligne 607 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.31 LokiModem_get_demod_phase_x_max()

```
float LokiModem_get_demod_phase_x_max (
            const LokiModem * self,
            uint32_t channel)
```

Lire le déphasage maximal applicable à la composante X (I) du signal démodulé pour une voie.

Voir également

LokiModem_get_demod_delay_x_max

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le déphasage maximal, en degrés.

Définition à la ligne 587 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.32 LokiModem_get_demod_phase_x_res()

```
float LokiModem_get_demod_phase_x_res (
            const LokiModem * self,
            uint32_t channel)
```

Lire le pas de réglage du déphasage applicable à la composante X (I) du signal démodulé pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le pas de réglage du déphasage, en degrés.

Définition à la ligne 592 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.33 LokiModem_get_demod_phase_y()

```
float LokiModem_get_demod_phase_y (
            const LokiModem * self,
            uint32_t channel)
```

Lire le déphasage appliqué à la composante Y (Q) du signal démodulé pour une voie.

Voir également

LokiModem_get_demod_delay_y

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le déphasage, en degrés.

Définition à la ligne 660 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.34 LokiModem_get_demod_phase_y_max()

```
float LokiModem_get_demod_phase_y_max (
            const LokiModem * self,
            uint32_t channel)
```

Lire le déphasage maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

Voir également

LokiModem_get_demod_delay_y_max

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le déphasage maximal, en degrés.

Définition à la ligne 640 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.35 LokiModem_get_demod_phase_y_res()

```
float LokiModem_get_demod_phase_y_res (
            const LokiModem * self,
            uint32_t channel)
```

Lire le pas de réglage du déphasage applicable à la composante Y (Q) du signal démodulé pour une voie.

Paramètres

self	Le modem à interroger.
channel	Un numéro de voie.

Renvoie

Le pas de réglage du déphasage, en degrés.

Définition à la ligne 645 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.36 LokiModem_get_demod_sample_count()

```
uint32_t LokiModem_get_demod_sample_count (
            const LokiModem * self)
```

Lire le nombre d'échantillons produits sur une voie en sortie du démodulateur.

Ce compteur est remis à zéro à chaque redémarrage du modem.

Paramètres

self	Le modem à interroger.

Renvoie

Le nombre d'échantillons démodulés par voie.

Définition à la ligne 816 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.37 LokiModem_get_demod_sampling_frequency()

```
float LokiModem_get_demod_sampling_frequency (
            const LokiModem * self)
```

Lire la fréquence d'échantillonnage en entrée du démodulateur.

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence d'échantillonnage, en Hz.

Définition à la ligne 552 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.38 LokiModem_get_demod_xy_sampling_frequency()

```
float LokiModem_get_demod_xy_sampling_frequency (
            const LokiModem * self)
```

Lire la fréquence d'échantillonnage en sortie du démodulateur.

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence d'échantillonnage, en Hz.

Définition à la ligne 570 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.39 LokiModem_get_dropped_packets()

```
unsigned LokiModem_get_dropped_packets (
            const LokiModem * self)
```

Lire le nombre de paquets perdus depuis le début de l'acquisition.

Le pilote du modem compte le nombre de paquets qui ont été reçus par transfert DMA, mais qui ont été écrasés avant d'être traités.

Paramètres

self	Le modem à interroger.

Renvoie

Le nombre de paquets d'échantillons perdus.

Définition à la ligne 1113 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.40 LokiModem_get_mod_dc_value()

```
float LokiModem_get_mod_dc_value (
            const LokiModem * self)
```

Lire le courant moyen de commande de la diode laser.

Paramètres

self	Le modem à interroger.

Renvoie

Le courant moyen, en mA.

Définition à la ligne 520 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.41 LokiModem_get_mod_dc_value_max()

```
float LokiModem_get_mod_dc_value_max (
            const LokiModem * self)
```

Lire le courant moyen maximal supporté pour la commande de la diode laser.

Paramètres

self	Le modem à interroger.

Renvoie

Le courant moyen maximal, en mA.

Définition à la ligne 516 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.42 LokiModem_get_mod_dc_value_res()

```
float LokiModem_get_mod_dc_value_res (
            const LokiModem * self)
```

Lire le pas de réglage du courant moyen de commande de la diode laser.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution du courant moyen, en mA.

Définition à la ligne 512 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.43 LokiModem_get_mod_offset()

```
float LokiModem_get_mod_offset (
            const LokiModem * self)
```

Lire l'offset en sortie du modulateur.

Paramètres

self	Le modem à interroger.

Renvoie

L'offset courant, en mA.

Définition à la ligne 504 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.44 LokiModem_get_mod_offset_max()

```
float LokiModem_get_mod_offset_max (
            const LokiModem * self)
```

Lire l'offset maximal supporté en sortie du modulateur.

Paramètres

self	Le modem à interroger.

Renvoie

L'offset maximal, en mA.

Définition à la ligne 500 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.45 LokiModem_get_mod_offset_res()

```
float LokiModem_get_mod_offset_res (
            const LokiModem * self)
```

Lire le pas de réglage de l'offset en sortie du modulateur.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution de l'offset, en mA.

Définition à la ligne 496 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.46 LokiModem_get_mod_osc1_gain()

```
float LokiModem_get_mod_osc1_gain (
            const LokiModem * self)
```

Lire l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

Paramètres

self	Le modem à interroger.

Renvoie

L'amplitude du signal, en mA.

Définition à la ligne 426 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.47 LokiModem_get_mod_osc1_gain_max()

```
float LokiModem_get_mod_osc1_gain_max (
            const LokiModem * self)
```

Lire l'amplitude maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

Paramètres

self	Le modem à interroger.

Renvoie

L'amplitude maximale du signal, en mA.

Définition à la ligne 422 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.48 LokiModem_get_mod_osc1_gain_res()

```
float LokiModem_get_mod_osc1_gain_res (
            const LokiModem * self)
```

Lire le pas de réglage de l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

L'amplitude du signal est toujours un multiple de cette valeur.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution du réglage de l'amplitude du signal, en mA.

Définition à la ligne 418 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.49 LokiModem_get_mod_osc1_wave_frequency()

```
float LokiModem_get_mod_osc1_wave_frequency (
            const LokiModem * self)
```

Lire la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence du signal, en Hz.

Définition à la ligne 409 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.50 LokiModem_get_mod_osc1_wave_frequency_max()

```
float LokiModem_get_mod_osc1_wave_frequency_max (
            const LokiModem * self)
```

Lire la fréquence maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence maximale du signal, en Hz.

Définition à la ligne 405 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.51 LokiModem_get_mod_osc1_wave_frequency_res()

```
float LokiModem_get_mod_osc1_wave_frequency_res (
            const LokiModem * self)
```

Lire le pas de réglage de la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

La fréquence de l'oscillateur est toujours un multiple de cette valeur.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution du réglage de la fréquence du signal, en Hz.

Définition à la ligne 401 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.52 LokiModem_get_mod_osc2_gain()

```
float LokiModem_get_mod_osc2_gain (
            const LokiModem * self)
```

Lire l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

Paramètres

self	Le modem à interroger.

Renvoie

L'amplitude du signal, en mA.

Définition à la ligne 459 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.53 LokiModem_get_mod_osc2_gain_max()

```
float LokiModem_get_mod_osc2_gain_max (
            const LokiModem * self)
```

Lire l'amplitude maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

Paramètres

self	Le modem à interroger.

Renvoie

L'amplitude maximale du signal, en mA.

Définition à la ligne 455 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.54 LokiModem_get_mod_osc2_gain_res()

```
float LokiModem_get_mod_osc2_gain_res (
            const LokiModem * self)
```

Lire le pas de réglage de l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

L'amplitude du signal est toujours un multiple de cette valeur.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution du réglage de l'amplitude du signal, en mA.

Définition à la ligne 451 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.55 LokiModem_get_mod_osc2_wave_frequency()

```
float LokiModem_get_mod_osc2_wave_frequency (
            const LokiModem * self)
```

Lire la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence du signal, en Hz.

Définition à la ligne 442 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.56 LokiModem_get_mod_osc2_wave_frequency_max()

```
float LokiModem_get_mod_osc2_wave_frequency_max (
            const LokiModem * self)
```

Lire la fréquence maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence maximale du signal, en Hz.

Définition à la ligne 438 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.57 LokiModem_get_mod_osc2_wave_frequency_res()

```
float LokiModem_get_mod_osc2_wave_frequency_res (
            const LokiModem * self)
```

Lire le pas de réglage de la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

La fréquence de l'oscillateur est toujours un multiple de cette valeur.

Paramètres

self	Le modem à interroger.

Renvoie

La résolution du réglage de la fréquence du signal, en Hz.

Définition à la ligne 434 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.58 LokiModem_get_mod_pga_gain()

```
uint32_t LokiModem_get_mod_pga_gain (
            const LokiModem * self)
```

Lire le gain de l'amplificateur de sortie.

Paramètres

self	Le modem à interroger.

Renvoie

La valeur courante du gain.

Définition à la ligne 532 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.59 LokiModem_get_mod_pga_gain_max()

```
uint32_t LokiModem_get_mod_pga_gain_max (
            const LokiModem * self)
```

Lire le gain maximal de l'amplificateur de sortie.

Paramètres

self	Le modem à interroger.

Renvoie

Le gain maximal.

Définition à la ligne 528 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.60 LokiModem_get_mod_sampling_frequency()

```
float LokiModem_get_mod_sampling_frequency (
            const LokiModem * self)
```

Lire la fréquence d'échantillonnage en sortie du modulateur.

Paramètres

self	Le modem à interroger.

Renvoie

La fréquence d'échantillonnage, en Hz.

Définition à la ligne 388 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.61 LokiModem_get_packet_index()

```
uint32_t LokiModem_get_packet_index (
            const LokiModem * self,
            LokiDMASample raw_item)  [inline], [static]
```

Lire un numéro de paquet dans un échantillon DMA brut.

Paramètres

self	Le modem courant.
raw_item	L'échantillon brut à inspecter.

Renvoie

Le numéro de paquet de cet échantillon.

Définition à la ligne 826 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.62 LokiModem_get_reg()

```
uint32_t LokiModem_get_reg (
            const LokiModem * self,
            LokiModemRegister reg)  [inline], [static]
```

Lire un registre du circuit de modulation/démodulation.

Cette fonction utilise la fonction `ioctl` pour interroger le circuit modem à travers le module noyau `loki-modem`.

Paramètres

self	Le modem courant.
reg	Le numéro du registre à lire.

Renvoie

La valeur du registre.

Définition à la ligne 140 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.63 LokiModem_get_test_mode()

```
bool LokiModem_get_test_mode (
            const LokiModem * self)
```

Lire l'état d'activation du mode de test.

En mode "test", la sortie du modulateur est remplacée par un motif qui permet de vérifier les valeurs transmises au convertisseur numérique/analogique.

Paramètres

self	Le modem à interroger.

Renvoie

true si le mode de test est activé.

Définition à la ligne 786 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.64 LokiModem_is_streaming()

```
bool LokiModem_is_streaming (
            const LokiModem * self)
```

Lire l'état de l'acquisition.

Paramètres

self	Le modem à interroger.

Renvoie

true si l'acquisition est en cours.

Définition à la ligne 1109 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.65 LokiModem_new()

```
LokiModem * LokiModem_new (
            void )
```

Construire un nouveau modem.

Renvoie

Une instance de LokiModem.

Définition à la ligne 275 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.66 LokiModem_print_config()

```
void LokiModem_print_config (
            const LokiModem * self)
```

Afficher la configuration d'un modem sur la sortie standard.

Paramètres

self	Le modem à examiner.

Définition à la ligne 1117 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.67 LokiModem_restart()

```
void LokiModem_restart (
            const LokiModem * self)
```

Redémarrer le modem.

Cette fonction déclenche une réinitialisation des oscillateurs internes et des compteurs d'échantillons du modem dans le FPGA. Elle doit être appelée à chaque modification des paramètres de modulation, de démodulation ou de communication avec le modem.

Paramètres

self	Le modem à redémarrer.

Définition à la ligne 802 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.68 LokiModem_scale_power()

```
float LokiModem_scale_power (
            const LokiModem * self,
            float p)  [inline], [static]
```

Mettre à l'échelle une valeur d'entrée du démodulateur.

Paramètres

self	Le modem courant.
p	La valeur à mettre à l'échelle.

Renvoie

La valeur mise à l'échelle.

Définition à la ligne 271 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.69 LokiModem_scale_xy()

```
float LokiModem_scale_xy (
            const LokiModem * self,
            float xy)  [inline], [static]
```

Mettre à l'échelle une composante X (I) ou Y (Q) d'un échantillon démodulé.

Paramètres

self	Le modem courant.
xy	La valeur à mettre à l'échelle.

Renvoie

La valeur mise à l'échelle.

Définition à la ligne 261 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.70 LokiModem_set_demod_bypass()

```
void LokiModem_set_demod_bypass (
            const LokiModem * self,
            bool bypass)
```

Activer ou désactiver le démodulateur.

En mode "bypass", les données brutes du convertisseur analogique/numérique sont transmises par DMA sans passer par le démodulateur.

Paramètres

self	Le modem à interroger.
bypass	true pour utiliser le mode "bypass".

Définition à la ligne 775 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.71 LokiModem_set_demod_delay_x()

```
void LokiModem_set_demod_delay_x (
            const LokiModem * self,
            uint32_t channel,
            uint32_t delay)
```

Affecter un retard à la composante X (I) du signal démodulé pour une voie.

Paramètres

self	Le modem à modifier.
channel	Un numéro de voie.
delay	Le nouveau retard, en échantillons.

Définition à la ligne 613 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.72 LokiModem_set_demod_delay_y()

```
void LokiModem_set_demod_delay_y (
            const LokiModem * self,
            uint32_t channel,
            uint32_t delay)
```

Affecter un retard à la composante Y (Q) du signal démodulé pour une voie.

Paramètres

self	Le modem à modifier.
channel	Un numéro de voie.
delay	Le nouveau retard, en échantillons.

Définition à la ligne 666 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.73 LokiModem_set_demod_osc_wave_frequency()

```
void LokiModem_set_demod_osc_wave_frequency (
            const LokiModem * self,
            float freq)
```

Affecter une fréquence au signal en sortie de l'oscillateur du démodulateur (Fm2).

Paramètres

self	Le modem à modifier.
freq	La fréquence du signal, en Hz.

Définition à la ligne 488 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.74 LokiModem_set_demod_pga_gain()

```
void LokiModem_set_demod_pga_gain (
            const LokiModem * self,
            uint32_t channel,
            float gain)
```

Affecter un gain à l'amplificateur en entrée du démodulateur pour une voie.

Paramètres

self	Le modem à modifier.

channel	Un numéro de voie.
gain	Le nouveau gain.

Définition à la ligne 728 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.75 LokiModem_set_demod_phase_x()

```
void LokiModem_set_demod_phase_x (
            const LokiModem * self,
            uint32_t channel,
            float phase)
```

Affecter un déphasage à la composante X (I) du signal démodulé pour une voie.

Voir également

LokiModem_set_demod_delay_x

Paramètres

self	Le modem à modifier.
channel	Un numéro de voie.
phase	Le nouveau déphasage, en degrés.

Définition à la ligne 624 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.76 LokiModem_set_demod_phase_y()

```
void LokiModem_set_demod_phase_y (
            const LokiModem * self,
            uint32_t channel,
            float phase)
```

Affecter un déphasage à la composante Y (Q) du signal démodulé pour une voie.

Voir également

LokiModem_set_demod_delay_y

Paramètres

self	Le modem à modifier.
channel	Un numéro de voie.

phase	Le nouveau déphasage, en degrés.

Définition à la ligne 677 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.77 LokiModem_set_demod_sampling_frequency()

```
void LokiModem_set_demod_sampling_frequency (
            const LokiModem * self,
            float freq)
```

Affecter une fréquence d'échantillonnage en entrée du démodulateur.

Cette fonction préserve la fréquence réglée précédemment pour l'oscillateur Fm2 du démodulateur.

Paramètres

self	Le modem à modifier.
freq	La fréquence d'échantillonnage, en Hz.

Définition à la ligne 556 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.78 LokiModem_set_demod_xy_sampling_frequency()

```
void LokiModem_set_demod_xy_sampling_frequency (
            const LokiModem * self,
            float freq)
```

Affecter une fréquence d'échantillonnage en sortie du démodulateur.

Connaissant le facteur de décimation des filtres du démodulateur, cette fonction modifie la fréquence d'échantillonnage en entrée pour atteindre la fréquence souhaitée en sortie.

Paramètres

self	Le modem à modifier.
freq	La fréquence d'échantillonnage, en Hz.

Définition à la ligne 574 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.79 LokiModem_set_mod_dc_value()

```
void LokiModem_set_mod_dc_value (
            const LokiModem * self,
            float dc_value)
```

Affecter un courant moyen de commande de la diode laser.

Paramètres

self	Le modem à modifier.
dc_value	Le nouveau courant moyen, en mA.

Définition à la ligne 524 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.80 LokiModem_set_mod_offset()

```
void LokiModem_set_mod_offset (
            const LokiModem * self,
            float offset)
```

Affecter un offset en sortie du modulateur.

Paramètres

self	Le modem à modifier.
offset	Le nouvel offset, en mA.

Définition à la ligne 508 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.81 LokiModem_set_mod_osc1_gain()

```
void LokiModem_set_mod_osc1_gain (
            const LokiModem * self,
            float gain)
```

Affecter une amplitude au signal en sortie du premier oscillateur du modulateur (Fm1).

Paramètres

self	Le modem à modifier.
gain	L'amplitude du signal, en mA.

Définition à la ligne 430 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.82 LokiModem_set_mod_osc1_wave_frequency()

```
void LokiModem_set_mod_osc1_wave_frequency (
            const LokiModem * self,
            float freq)
```

Affecter une fréquence au signal en sortie du premier oscillateur du modulateur (Fm1).

Paramètres

self	Le modem à modifier.
freq	La fréquence du signal, en Hz.

Définition à la ligne 413 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.83 LokiModem_set_mod_osc2_gain()

```
void LokiModem_set_mod_osc2_gain (
            const LokiModem * self,
            float gain)
```

Affecter une amplitude au signal en sortie du second oscillateur du modulateur (Fm2).

Paramètres

self	Le modem à modifier.
gain	L'amplitude du signal, en mA.

Définition à la ligne 463 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.84 LokiModem_set_mod_osc2_wave_frequency()

```
void LokiModem_set_mod_osc2_wave_frequency (
            const LokiModem * self,
            float freq)
```

Affecter une fréquence au signal en sortie du second oscillateur du modulateur (Fm2).

Paramètres

self	Le modem à modifier.
freq	La fréquence du signal, en Hz.

Définition à la ligne 446 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.85 LokiModem_set_mod_pga_gain()

```
void LokiModem_set_mod_pga_gain (
            const LokiModem * self,
            uint32_t gain)
```

Affecter un gain à l'amplificateur de sortie.

Paramètres

self	Le modem à modifier.
gain	Le nouveau gain.

Définition à la ligne 536 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.86 LokiModem_set_mod_sampling_frequency()

```
void LokiModem_set_mod_sampling_frequency (
            const LokiModem * self,
            float freq)
```

Affecter une fréquence d'échantillonnage en sortie du modulateur.

Cette fonction préserve les fréquences réglées précédemment pour les oscillateurs Fm1 et Fm2 du modulateur.

Paramètres

self	Le modem à modifier.
freq	La fréquence d'échantillonnage, en Hz.

Définition à la ligne 392 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.87 LokiModem_set_packet_item()

```
void LokiModem_set_packet_item (
            const LokiModem * self,
            LokiModemSample * item,
            LokiDMASample raw_item)  [static]
```

Affecter les champs d'un échantillon de sortie du démodulateur.

Paramètres

self	Le modem courant.
item	Un pointeur vers l'échantillon de sortie à compléter.

raw_item	L'échantillon DMA brut.

Définition à la ligne 836 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.88 LokiModem_set_reg()

```
void LokiModem_set_reg (
            const LokiModem * self,
            LokiModemRegister reg,
            uint32_t value)  [inline], [static]
```

Affecter un registre du circuit de modulation/démodulation.

Cette fonction utilise la fonction `ioctl` pour agir sur le circuit modem d'acquisition à travers le module noyau `loki-modem`.

Paramètres

self	Le modem courant.
reg	Le numéro du registre à modifier.
value	La nouvelle valeur du registre.

Définition à la ligne 159 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.89 LokiModem_set_simulated_item()

```
void LokiModem_set_simulated_item (
            LokiModem * self,
            LokiModemSample * item)  [static]
```

Affecter les champs d'un échantillon de sortie du démodulateur en mode simulation de capteurs.

Paramètres

self	Le modem courant.
item	Un pointeur vers l'échantillon de sortie à compléter.

Définition à la ligne 895 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.90 LokiModem_set_test_mode()

```
void LokiModem_set_test_mode (
            const LokiModem * self,
            bool test_mode)
```

Activer ou désactiver le mode de test.

En mode "test", la sortie du modulateur est remplacée par un motif qui permet de vérifier les valeurs transmises au convertisseur numérique/analogique.

Paramètres

self	Le modem à interroger.
test_mode	true pour activer le mode de test.

Définition à la ligne 791 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.91 LokiModem_start_streaming()

```
int LokiModem_start_streaming (
            LokiModem * self,
            int32_t channels,
            bool simulate,
            bool raw,
            LokiFIFO * out)
```

Démarrer l'acquisition et le traitement des échantillons démodulés.

Cette fonction est non bloquante. Elle crée un nouveau thread qui démarre les transferts par DMA depuis le démodulateur et qui alimente la FIFO de sortie à chaque transfert.

Paramètres

self	Le modem à démarrer.
channels	Un masque indiquant les voies à traiter, -1 pour toutes les voies.
simulate	Faut-il simuler un capteur logiciellement ?
raw	Faut-il conserver les valeurs d'échantillons en virgule fixe ?
out	La FIFO de destination.

Renvoie

Le code d'erreur retourné par `pthread_create`.

Définition à la ligne 1081 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.92 LokiModem_stop_streaming()

```
void LokiModem_stop_streaming (
            LokiModem * self)
```

Arrêter l'acquisition.

Interrompt l'exécution du thread démarré par la fonction LokiModem_start_streaming.

Paramètres

self	Le modem à arrêter.

Définition à la ligne 1096 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.3.3.4.93 LokiModem_work()

```
void * LokiModem_work (
            void * ptr)  [static]
```

Traiter un tableau d'échantillons issu d'un transfert DMA.

Cette fonction est exécutée dans un thread démarré par LokiModem_start_streaming. Elle se termine lorsqu'une demande d'arrêt est émise par LokiModem_stop_streaming.

Si la constante `USE_MMAP` est définie, elle s'appuie sur la fonction `mmap` pour se synchroniser avec le pilote du circuit modem et accéder au tampon de données de chaque transfert en mode *zero-copy*. Si la constante `USE_MMAP` n'est pas définie, la fonction `fread` est utilisée.

Cette fonction remplit la FIFO de sortie avec des paquets d'échantillons convertis par la fonction LokiModem_set_packet_item.

Paramètres

ptr	Le modem courant.

Définition à la ligne 943 du fichier libloki-modem.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4 Enregistrement et transmission de données

Graphe de collaboration de Enregistrement et transmission de données:

Sujets

— Interface pour la mise en forme de séries temporelles

Définition des méthodes communes pour enregistrer des séries temporelles.

— Enregistrement de séries temporelles au format miniSEED

Ce module implémente l'interface LokiFormatter.

— Enregistrement de séries temporelles brutes

Ce module implémente l'interface LokiFormatter.

— Enregistrement de séries temporelles

Ce module délègue les opérations de mise en forme et d'écriture à une implémentation de l'interface LokiFormatter.

— Serveur de données sur socket Unix

Ce module est utilisé dans la communication des données de déplacement par WebSockets.

4.4.1 Description détaillée

4.4.2 Interface pour la mise en forme de séries temporelles

Définition des méthodes communes pour enregistrer des séries temporelles.

Graphe de collaboration de Interface pour la mise en forme de séries temporelles:

Fichiers

— fichier libloki-formatter.c

— fichier libloki-formatter.h

Structures de données

— struct LokiFormatter

Représentation d'un formateur de séries temporelles.

Définitions de type

— typedef struct LokiFormatter LokiFormatter

Représentation d'un formateur de séries temporelles.

Fonctions

— LokiFormatter ∗ LokiFormatter_new (void ∗config)

Construire un nouveau formateur de séries temporelles.

— LokiFormatter ∗ LokiFormatter_new_with_implementation (const LokiFormatterInterface ∗impl, void ∗config)

Construire un nouveau formateur de séries temporelles avec une implémentation donnée.

— void LokiFormatter_work (LokiFormatter ∗self, const char ∗filename, LokiTimestamp timestamp, bool file_change, void ∗data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency)

Enregistrer un paquet de données.

— void LokiFormatter_close (LokiFormatter ∗self)

Fermer un formateur de séries temporelles.

— void LokiFormatter_del (LokiFormatter ∗self)

Détruire de mise en forme de séries temporelles.

— **LokiClass_interface** (LokiFormatter, LokiClass_method(LokiFormatter, new) LokiClass_method(LokiFormatter, work) LokiClass_method(LokiFormatter, close) LokiClass_method(LokiFormatter, del))

4.4.2.1 Description détaillée

Définition des méthodes communes pour enregistrer des séries temporelles.

4.4.2.2 Documentation des définitions de type

4.4.2.2.1 LokiFormatter

```
typedef struct LokiFormatter LokiFormatter
```

Représentation d'un formateur de séries temporelles.

Définition à la ligne 21 du fichier libloki-formatter.h.

4.4.2.3 Documentation des fonctions

4.4.2.3.1 LokiFormatter_close()

```
void LokiFormatter_close (
            LokiFormatter * self)
```

Fermer un formateur de séries temporelles.

Cette fonction ferme le dernier fichier ouvert. Elle doit être appelée à la fin de l'enregistrement.

Paramètres

self	L'instance à fermer.

Définition à la ligne 47 du fichier libloki-formatter.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.2.3.2 LokiFormatter_del()

```
void LokiFormatter_del (
            LokiFormatter * self)
```

Détruire de mise en forme de séries temporelles.

Paramètres

self	L'instance à détruire.

Définition à la ligne 51 du fichier libloki-formatter.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.2.3.3 LokiFormatter_new()

```
LokiFormatter * LokiFormatter_new (
            void * config)
```

Construire un nouveau formateur de séries temporelles.

Paramètres

config	Une configuration pour le nouveau formateur.

Renvoie

Une instance de LokiFormatter.

Définition à la ligne 22 du fichier libloki-formatter.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.2.3.4 LokiFormatter_new_with_implementation()

```
LokiFormatter * LokiFormatter_new_with_implementation (
            const LokiFormatterInterface * impl,
            void * config)
```

Construire un nouveau formateur de séries temporelles avec une implémentation donnée.

Paramètres

impl	L'implémentation de LokiFormatter à utiliser.
config	Une configuration pour le nouveau formateur.

Renvoie

Une instance de LokiFormatter.

Définition à la ligne 26 du fichier libloki-formatter.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.2.3.5 LokiFormatter_work()

```
void LokiFormatter_work (
            LokiFormatter * self,
            const char * filename,
            LokiTimestamp timestamp,
            bool file_change,
            void * data,
            size_t length,
            size_t max_channels,
            int32_t channels,
            size_t sample_size,
            float sampling_frequency)
```

Enregistrer un paquet de données.

Cette fonction délègue à une implémentation de l'interface LokiFormatter.

Paramètres

self	L'instance à utiliser.
filename	Le nom du fichier de destination.
timestamp	La date du paquet de données.
file_change	Faut-il ouvrir un nouveau fichier de destination ou poursuivre le fichier précédent ?
data	Les données à écrire.
length	Le nombre d'échantillons à écrire.
max_channels	Le nombre maximal de voies à traiter.
channels	Un masque indiquant les voies à traiter.
sample_size	La taille d'un échantillon, en octets.

sampling_frequency	La fréquence d'échantillonage.

Définition à la ligne 43 du fichier libloki-formatter.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.3 Enregistrement de séries temporelles au format miniSEED

Ce module implémente l'interface LokiFormatter.

Graphe de collaboration de Enregistrement de séries temporelles au format miniSEED:

Fichiers

— fichier libloki-mseed.c

— fichier libloki-mseed.h

Structures de données

— struct LokiMSeedConfig

Configuration d'un formateur de séries temporelles miniSEED.

— struct LokiMSeedInputSample

Type des échantillons attendus par LokiMSeed_work.

— union LokiMSeedInputSample::[union].__unnamed0__

La valeur de l'échantillon.

— struct LokiMSeed

Représentation d'un formateur de séries temporelles miniSEED.

— union LokiMSeedOutputSample

Type des échantillons passés à la bibliothèque `libmseed`.

Macros

— #define CHANNEL_NAME_LEN 4

Le nombre maximal de caractères représentant le nom de la voie.

— #define MSEED_FILENAME_SUFFIX_LEN (CHANNEL_NAME_LEN + 7)

Le format du suffixe du nom de fichier: "_{n°de voie}.mseed".

— #define MSEED_RECLEN 512

La longueur d'un record miniSEED.

Définitions de type

— typedef struct LokiMSeedConfig LokiMSeedConfig

Configuration d'un formateur de séries temporelles miniSEED.

— typedef struct LokiMSeed LokiMSeed

Représentation d'un formateur de séries temporelles miniSEED.

Fonctions

— LokiMSeedConfig ∗ LokiMSeedConfig_new (bool separate_channels, const char ∗station, const char ∗location, const char ∗network, bool raw, bool steim2, const char ∗channel_names)

Construire une configuration d'enregistrement de données au format miniSEED.

— void LokiMSeedConfig_del (LokiMSeedConfig ∗self)

Détruire une configuration d'enregistrement de données au format miniSEED.

— LokiMSeed ∗ LokiMSeed_new (LokiMSeedConfig ∗config)

Construire un formateur de séries temporelles miniSEED.

— void LokiMSeed_work (LokiMSeed ∗self, const char ∗filename, LokiTimestamp timestamp, bool file_change, void ∗data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_↩ frequency)

Enregistrer un paquet de données au format miniSEED.

— void LokiMSeed_close (LokiMSeed ∗self)

Fermer un formateur de séries temporelles miniSEED.

— void LokiMSeed_del (LokiMSeed ∗self)

Détruire un formateur de séries temporelles miniSEED.

— **LokiClass_implementation** (LokiFormatter, LokiMSeed, LokiClass_define_method(LokiMSeed, new) LokiClass_define_method(LokiMSeed, work) LokiClass_define_method(LokiMSeed, close) LokiClass_define_method(LokiMSe del))

— **LokiClass_declare_implementation** (LokiFormatter, LokiMSeed)

4.4.3.1 Description détaillée

Ce module implémente l'interface LokiFormatter.

Il s'appuie sur la bibliothèque `libmseed` de l'IRIS.

4.4.3.2 Documentation des macros

4.4.3.2.1 CHANNEL_NAME_LEN

```
#define CHANNEL_NAME_LEN 4
```

Le nombre maximal de caractères représentant le nom de la voie.

Définition à la ligne 133 du fichier libloki-mseed.c.

4.4.3.2.2 MSEED_FILENAME_SUFFIX_LEN

```
#define MSEED_FILENAME_SUFFIX_LEN (CHANNEL_NAME_LEN + 7)
```

Le format du suffixe du nom de fichier: "_{n°de voie}.mseed".

Définition à la ligne 136 du fichier libloki-mseed.c.

4.4.3.2.3 MSEED_RECLEN

```
#define MSEED_RECLEN 512
```

La longueur d'un record miniSEED.

Définition à la ligne 139 du fichier libloki-mseed.c.

4.4.3.3 Documentation des définitions de type

4.4.3.3.1 LokiMSeed

```
typedef struct LokiMSeed LokiMSeed
```

Représentation d'un formateur de séries temporelles miniSEED.

Définition à la ligne 38 du fichier libloki-mseed.h.

4.4.3.3.2 LokiMSeedConfig

```
typedef struct LokiMSeedConfig LokiMSeedConfig
```

Configuration d'un formateur de séries temporelles miniSEED.

Définition à la ligne 16 du fichier libloki-mseed.h.

4.4.3.4 Documentation des fonctions

4.4.3.4.1 LokiMSeed_close()

```
void LokiMSeed_close (
            LokiMSeed * self)
```

Fermer un formateur de séries temporelles miniSEED.

Cette fonction ferme le dernier fichier ouvert. Elle doit être appelée à la fin de l'enregistrement.

Paramètres

self	Le formateur à fermer.

Définition à la ligne 216 du fichier libloki-mseed.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.3.4.2 LokiMSeed_del()

```
void LokiMSeed_del (
            LokiMSeed * self)
```

Détruire un formateur de séries temporelles miniSEED.

Paramètres

self	Le formateur à détruire.

Définition à la ligne 223 du fichier libloki-mseed.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.3.4.3 LokiMSeed_new()

```
LokiMSeed * LokiMSeed_new (
            LokiMSeedConfig * config)
```

Construire un formateur de séries temporelles miniSEED.

Paramètres

config	La configuration à appliquer.

Renvoie

Un nouveau formateur de séries temporelles miniSEED.

Définition à la ligne 112 du fichier libloki-mseed.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.3.4.4 LokiMSeed_work()

```
void LokiMSeed_work (
            LokiMSeed * self,
            const char * filename,
            LokiTimestamp timestamp,
            bool file_change,
            void * data,
            size_t length,
            size_t max_channels,
            int32_t channels,
            size_t sample_size,
            float sampling_frequency)
```

Enregistrer un paquet de données au format miniSEED.

Cette fonction attend un tableau d'échantillons dont les éléments ont un type de la forme :

```
struct Sample {
    uint32_t channel;
    float value;
    ... // Autres données spécifiques
}
```

On doit avoir `sample_size == sizeof(struct Sample)`

Paramètres

self	Le formateur de séries temporelles miniSEED.
filename	Le nom du fichier de destination.
timestamp	La date du paquet de données (inutilisée pour ce format).
file_change	Faut-il ouvrir un nouveau fichier de destination ou poursuivre le fichier précédent ?
data	Les données à écrire.
length	Le nombre d'échantillons à écrire.
max_channels	Le nombre maximal de voies que ce formateur peut traiter.
channels	Un masque indiquant les voies à traiter.
sample_size	La taille d'un échantillon, en octets.
sampling_frequency	La fréquence d'échantillonage (inutilisée pour ce format).

Définition à la ligne 141 du fichier libloki-mseed.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.3.4.5 LokiMSeedConfig_del()

```
void LokiMSeedConfig_del (
            LokiMSeedConfig * self)
```

Détruire une configuration d'enregistrement de données au format miniSEED.

Paramètres

self	La configuration à détruire.

Définition à la ligne 58 du fichier libloki-mseed.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.3.4.6 LokiMSeedConfig_new()

```
LokiMSeedConfig * LokiMSeedConfig_new (
            bool separate_channels,
            const char * station,
            const char * location,
            const char * network,
            bool raw,
            bool steim2,
            const char * channel_names)
```

Construire une configuration d'enregistrement de données au format miniSEED.

Paramètres

separate_channels	Faut-il enregistrer chaque voie dans un fichier séparé ?
station	L'identifiant de station de l'interrogateur.
location	L'identifiant de localisation de l'interrogateur.
network	L'identifiant de réseau de l'interrogateur.
raw	Faut-il utiliser les valeurs brutes des échantillons ?
steim2	Faut-il utiliser l'algorithme de compression STEIM2 ?
channel_names	Un tableau de nom de voies.

Renvoie

Une nouvelle configuration d'enregistrement au format miniSEED.

Définition à la ligne 29 du fichier libloki-mseed.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.4 Enregistrement de séries temporelles brutes

Ce module implémente l'interface LokiFormatter.

Graphe de collaboration de Enregistrement de séries temporelles brutes:

Fichiers

- fichier libloki-raw.c
- fichier libloki-raw.h

Structures de données

— struct LokiRaw

Représentation d'un formateur de données brutes.

Définitions de type

— typedef struct LokiRaw LokiRaw

Représentation d'un formateur de données brutes.

Fonctions

— LokiRaw ∗ LokiRaw_new (void ∗config)

Construire un formateur de données brutes.

— void LokiRaw_work (LokiRaw ∗self, const char ∗filename, LokiTimestamp timestamp, bool file_change, void ∗data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency)

Enregistrer un paquet de données.

— void LokiRaw_close (LokiRaw ∗self)

Fermer un formateur de données brutes.

— void LokiRaw_del (LokiRaw ∗self)

Détruire un formateur de données brutes.

— **LokiClass_implementation** (LokiFormatter, LokiRaw, LokiClass_define_method(LokiRaw, new) LokiClass_define_method(Lo
work) LokiClass_define_method(LokiRaw, close) LokiClass_define_method(LokiRaw, del))

— **LokiClass_declare_implementation** (LokiFormatter, LokiRaw)

4.4.4.1 Description détaillée

Ce module implémente l'interface LokiFormatter.

4.4.4.2 Documentation des définitions de type

4.4.4.2.1 LokiRaw

```
typedef struct LokiRaw LokiRaw
```

Représentation d'un formateur de données brutes.

Définition à la ligne 16 du fichier libloki-raw.h.

4.4.4.3 Documentation des fonctions

4.4.4.3.1 LokiRaw_close()

```
void LokiRaw_close (
            LokiRaw * self)
```

Fermer un formateur de données brutes.

Cette fonction ferme le dernier fichier ouvert. Elle doit être appelée à la fin de l'enregistrement.

Paramètres

self	Le formateur à fermer.

Définition à la ligne 48 du fichier libloki-raw.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.4.3.2 LokiRaw_del()

```
void LokiRaw_del (
            LokiRaw * self)
```

Détruire un formateur de données brutes.

Paramètres

self	Le formateur à détruire.

Définition à la ligne 55 du fichier libloki-raw.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.4.3.3 LokiRaw_new()

```
LokiRaw * LokiRaw_new (
            void * config)
```

Construire un formateur de données brutes.

Paramètres

config	La configuration à appliquer (inutilisée pour ce format).

Renvoie

Un nouveau formateur de données brutes.

Définition à la ligne 20 du fichier libloki-raw.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.4.3.4 LokiRaw_work()

```
void LokiRaw_work (
            LokiRaw * self,
            const char * filename,
            LokiTimestamp timestamp,
            bool file_change,
            void * data,
            size_t length,
            size_t max_channels,
            int32_t channels,
            size_t sample_size,
            float sampling_frequency)
```

Enregistrer un paquet de données.

Paramètres

self	Le formateur de données brutes à utiliser.
filename	Le nom du fichier de destination.
timestamp	La date du paquet de données (inutilisée pour ce format).
file_change	Faut-il ouvrir un nouveau fichier de destination ou poursuivre le fichier précédent ?
data	Les données à écrire.
length	Le nombre d'échantillons à écrire.
max_channels	Le nombre maximal de voies à traiter.
channels	Un masque indiquant les voies à traiter.
sample_size	La taille d'un échantillon, en octets.
sampling_frequency	La fréquence d'échantillonage (inutilisée pour ce format).

Définition à la ligne 27 du fichier libloki-raw.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5 Enregistrement de séries temporelles

Ce module délègue les opérations de mise en forme et d'écriture à une implémentation de l'interface LokiFormatter.

Graphe de collaboration de Enregistrement de séries temporelles:

Fichiers

- fichier libloki-recorder.c
- fichier libloki-recorder.h

Structures de données

- struct LokiRecorder

  Représentation d'un enregistreur de séries temporelles.

Macros

— #define FILENAME_SUFFIX_DATE_LEN 16

Longueur de chaîne à allouer pour mettre en forme la date et l'heure d'un fichier ("_YYYYmmddTHHMMSS").

— #define FILENAME_SUFFIX_MSEC_LEN 6

Longueur de chaîne à allouer pour mettre en forme le nombre de millisecondes de la date d'un fichier (".mmmZ.").

— #define FILENAME_SUFFIX_LEN (FILENAME_SUFFIX_DATE_LEN + FILENAME_SUFFIX_MSEC_LEN)

Longueur de chaîne à allouer pour mettre en forme la date complète d'un fichier.

Définitions de type

— typedef struct LokiRecorder LokiRecorder

Représentation d'un enregistreur de séries temporelles.

Fonctions

— static void LokiRecorder_set_formatter (LokiRecorder *self, void *config)

Affecter un formateur à un enregistreur.

— LokiRecorder * LokiRecorder_new (const char *filename, const char *filetype, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency, float duration, float duration_per_file, void *config)

Construire un enregistreur de séries temporelles.

— char * LokiRecorder_get_filename (const LokiRecorder *self)

Lire le nom des fichiers gérés par un enregistreur.

— size_t LokiRecorder_get_channel_count (const LokiRecorder *self)

Lire le nombre de voies gérées par un enregistreur.

— static void * LokiRecorder_work (void *ptr)

Enregistrer les données.

— int LokiRecorder_start (LokiRecorder *self, LokiFIFO *in, size_t cid)

Démarrer l'enregistrement.

— bool LokiRecorder_is_recording (LokiRecorder *self)

Lire l'état d'un enregistreur.

— static void LokiRecorder_signal_handler (int signum)

Gestionnaire d'événements.

— void LokiRecorder_wait (LokiRecorder *self)

Attendre la fin de l'enregistrement.

— void LokiRecorder_stop (LokiRecorder ∗self)

Arrêter l'enregistrement.

— void LokiRecorder_del (LokiRecorder ∗self)

Détruire un enregistreur.

Variables

— static const char ∗const LokiRecorder_formatter_names [ ]

La liste des noms de formateurs supportés.

— static const LokiFormatterInterface ∗const LokiRecorder_formatter_impls [ ]

La liste des implémentations de formateurs supportées.

— static LokiRecorder ∗ LokiRecorder_waiting_instance

L'instance de LokiRecorder qui fait l'objet d'une attente par la fonction LokiRecorder_wait.

4.4.5.1 Description détaillée

Ce module délègue les opérations de mise en forme et d'écriture à une implémentation de l'interface LokiFormatter.

4.4.5.2 Documentation des macros

4.4.5.2.1 FILENAME_SUFFIX_DATE_LEN

```
#define FILENAME_SUFFIX_DATE_LEN 16
```

Longueur de chaîne à allouer pour mettre en forme la date et l'heure d'un fichier ("_YYYYmmddTHHMMSS").

Définition à la ligne 70 du fichier libloki-recorder.c.

4.4.5.2.2 FILENAME_SUFFIX_LEN

```
#define FILENAME_SUFFIX_LEN (FILENAME_SUFFIX_DATE_LEN + FILENAME_SUFFIX_MSEC_LEN)
```

Longueur de chaîne à allouer pour mettre en forme la date complète d'un fichier.

Définition à la ligne 76 du fichier libloki-recorder.c.

4.4.5.2.3 FILENAME_SUFFIX_MSEC_LEN

```
#define FILENAME_SUFFIX_MSEC_LEN 6
```

Longueur de chaîne à allouer pour mettre en forme le nombre de millisecondes de la date d'un fichier (".mmmZ.").

Définition à la ligne 73 du fichier libloki-recorder.c.

4.4.5.3 Documentation des définitions de type

4.4.5.3.1 LokiRecorder

```
typedef struct LokiRecorder LokiRecorder
```

Représentation d'un enregistreur de séries temporelles.

Définition à la ligne 19 du fichier libloki-recorder.h.

4.4.5.4 Documentation des fonctions

4.4.5.4.1 LokiRecorder_del()

```
void LokiRecorder_del (
            LokiRecorder * self)
```

Détruire un enregistreur.

Paramètres

self	L'enregistreur à détruire.

Définition à la ligne 284 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.2 LokiRecorder_get_channel_count()

```
size_t LokiRecorder_get_channel_count (
            const LokiRecorder * self)
```

Lire le nombre de voies gérées par un enregistreur.

Paramètres

self	L'enregistreur à interroger.

Renvoie

Le nombre de voies à enregistrer.

Définition à la ligne 144 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.3 LokiRecorder_get_filename()

```
char * LokiRecorder_get_filename (
            const LokiRecorder * self)
```

Lire le nom des fichiers gérés par un enregistreur.

Paramètres

self	L'enregistreur à interroger.

Renvoie

Le nom de base des fichiers de destination.

Définition à la ligne 140 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.4 LokiRecorder_is_recording()

```
bool LokiRecorder_is_recording (
            LokiRecorder * self)
```

Lire l'état d'un enregistreur.

Paramètres

self	L'enregistreur à interroger.

Renvoie

true si l'enregistrement est en cours.

Définition à la ligne 239 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.5 LokiRecorder_new()

```
LokiRecorder * LokiRecorder_new (
            const char * filename,
            const char * filetype,
            size_t max_channels,
            int32_t channels,
            size_t sample_size,
            float sampling_frequency,
            float duration,
            float duration_per_file,
            void * config)
```

Construire un enregistreur de séries temporelles.

Paramètres

filename	Le nom de base des fichiers de destination (sans extension).
filetype	Le format des fichiers de destination ("raw" ou "mseed").
max_channels	Le nombre maximal de voies que cet enregistreur peut traiter.
channels	Un masque indiquant les voies à traiter.
sample_size	La taille d'un échantillon, en octets.
sampling_frequency	La fréquence d'échantillonnage, en Hz.
duration	La durée totale d'enregistrement, 0 pour enregistrer indéfiniment.
duration_per_file	La durée approximative d'enregistrement par fichier. La durée effective dépend de la taille des paquets de données.
config	Une configuration à passer au formateur correspondant au format de fichier choisi.

Renvoie

Un nouvel enregistreur.

Définition à la ligne 98 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.6 LokiRecorder_set_formatter()

```
void LokiRecorder_set_formatter (
            LokiRecorder * self,
            void * config)  [static]
```

Affecter un formateur à un enregistreur.

Cette fonction recherche une implémentation de formateur correspondant au type mentionné dans l'enregistreur. Il instancie un formateur en lui passant en paramètre une configuration.

Paramètres

self	L'enregistreur à configurer.
config	La configuration de formateur à appliquer.

Définition à la ligne 87 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.7 LokiRecorder_signal_handler()

```
void LokiRecorder_signal_handler (
            int signum)  [static]
```

Gestionnaire d'événements.

Cette fonction est installée comme gestionnaire d'événement par la fonction LokiRecorder_wait. Elle interrompt l'enregistreur LokiRecorder_waiting_instance sur réception d'un signal KILL, TERM ou INT.

Paramètres

signum	Le numéro du signal reçu.

Définition à la ligne 255 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.8 LokiRecorder_start()

```
int LokiRecorder_start (
            LokiRecorder * self,
            LokiFIFO * in,
            size_t cid)
```

Démarrer l'enregistrement.

Cette fonction est non bloquante. Elle démarre un thread qui lit les données d'une FIFO et les enregistre.

Paramètres

self	L'enregistreur à démarrer.
in	La FIFO de données à enregistrer.
cid	Le numéro de consommateur de cette instance dans la FIFO.

Renvoie

Le code d'erreur de pthread_create.

Définition à la ligne 231 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.9 LokiRecorder_stop()

```
void LokiRecorder_stop (
            LokiRecorder * self)
```

Arrêter l'enregistrement.

Interrompt l'exécution du thread démarré par la fonction LokiRecorder_start.

Paramètres

self	L'enregistreur à arrêter.

Définition à la ligne 275 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.10 LokiRecorder_wait()

```
void LokiRecorder_wait (
            LokiRecorder * self)
```

Attendre la fin de l'enregistrement.

Cette fonction bloque le thread courant jusqu'à ce que l'enregistrement en cours se termine.

Paramètres

self	L'enregistreur à attendre.

Définition à la ligne 262 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.4.11 LokiRecorder_work()

```
void * LokiRecorder_work (
            void * ptr)  [static]
```

Enregistrer les données.

Cette fonction est exécutée dans un thread démarré par la fonction LokiRecorder_start. Elle reçoit les données de la FIFO d'entrée de l'enregistreur courant et les passe à un formateur qui les enregistre. L'enregistrement se poursuit jusqu'à ce qu'une demande d'arrêt soit effectuée par la fonction LokiRecorder_stop, ou jusqu'à ce que la durée totale d'enregistrement soit atteinte.

Paramètres

ptr	L'enregistreur courant.

Renvoie

Rien.

Définition à la ligne 161 du fichier libloki-recorder.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.5.5 Documentation des variables

4.4.5.5.1 LokiRecorder_formatter_impls

```
const LokiFormatterInterface* const LokiRecorder_formatter_impls[]  [static]
```

Valeur initiale :

```
                                                                    = {
    &LokiRawClass, &LokiMSeedClass
}
```

La liste des implémentations de formateurs supportées.

Définition à la ligne 65 du fichier libloki-recorder.c.

4.4.5.5.2 LokiRecorder_formatter_names

```
const char* const LokiRecorder_formatter_names[]  [static]
```

Valeur initiale :

```
                                                 = {
    "raw", "mseed"
}
```

La liste des noms de formateurs supportés.

Définition à la ligne 60 du fichier libloki-recorder.c.

4.4.5.5.3 LokiRecorder_waiting_instance

```
LokiRecorder* LokiRecorder_waiting_instance  [static]
```

L'instance de LokiRecorder qui fait l'objet d'une attente par la fonction LokiRecorder_wait.

Définition à la ligne 244 du fichier libloki-recorder.c.

4.4.6 Serveur de données sur socket Unix

Ce module est utilisé dans la communication des données de déplacement par WebSockets.

Graphe de collaboration de Serveur de données sur socket Unix:

Fichiers

- fichier libloki-socket-server.c
- fichier libloki-socket-server.h

Structures de données

— struct LokiSocketServer

Représentation d'un serveur de données.

Macros

— #define LIBLOKI_SOCKET_SERVER_CONNECTIONS 3

Le nombre maximum de connexions simultanées acceptées par un serveur.

Définitions de type

— typedef struct LokiSocketServer LokiSocketServer

Représentation d'un serveur de données.

Fonctions

— LokiSocketServer ∗ LokiSocketServer_new (const char ∗filename)

Construire un serveur de données.

— static void ∗ LokiSocketServer_receive (void ∗ptr)

Recevoir les données d'entrée.

— static void ∗ LokiSocketServer_serve (void ∗ptr)

Gérer les connexions et transmettre les données aux clients.

— int LokiSocketServer_start (LokiSocketServer ∗self, LokiFIFO ∗in, size_t cid)

Démarrer un serveur.

— bool LokiSocketServer_is_streaming (LokiSocketServer ∗self)

Lire l'état du serveur.

— void LokiSocketServer_stop (LokiSocketServer ∗self)

Arrêter un serveur.

— void LokiSocketServer_del (LokiSocketServer ∗self)

Détruire un serveur.

4.4.6.1 Description détaillée

Ce module est utilisé dans la communication des données de déplacement par WebSockets.

4.4.6.2 Documentation des macros

4.4.6.2.1 LIBLOKI_SOCKET_SERVER_CONNECTIONS

```
#define LIBLOKI_SOCKET_SERVER_CONNECTIONS 3
```

Le nombre maximum de connexions simultanées acceptées par un serveur.

Définition à la ligne 16 du fichier libloki-socket-server.h.

4.4.6.3 Documentation des définitions de type

4.4.6.3.1 LokiSocketServer

```
typedef struct LokiSocketServer LokiSocketServer
```

Représentation d'un serveur de données.

Définition à la ligne 19 du fichier libloki-socket-server.h.

4.4.6.4 Documentation des fonctions

4.4.6.4.1 LokiSocketServer_del()

```
void LokiSocketServer_del (
            LokiSocketServer * self)
```

Détruire un serveur.

Paramètres

self	Le serveur à détruire.

Définition à la ligne 246 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.6.4.2 LokiSocketServer_is_streaming()

```
bool LokiSocketServer_is_streaming (
            LokiSocketServer * self)
```

Lire l'état du serveur.

Paramètres

self	Le serveur à interroger.

Renvoie

true si le serveur est démarré.

Définition à la ligne 223 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.6.4.3 LokiSocketServer_new()

```
LokiSocketServer * LokiSocketServer_new (
            const char * filename)
```

Construire un serveur de données.

Paramètres

filename	Le nom de fichier à utiliser pour représenter la socket Unix dans le système de fichiers.

Renvoie

Un nouveau serveur.

Définition à la ligne 50 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.6.4.4 LokiSocketServer_receive()

```
void * LokiSocketServer_receive (
            void * ptr)  [static]
```

Recevoir les données d'entrée.

Cette fonction est exécutée dans un thread démarré par LokiSocketServer_start. Elle consomme les données de la FIFO d'entrée jusqu'à la demande d'arrêt par LokiSocketServer_stop.

Paramètres

ptr	Le serveur courant.

Renvoie

Rien.

Définition à la ligne 77 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.6.4.5 LokiSocketServer_serve()

```
void * LokiSocketServer_serve (
            void * ptr)  [static]
```

Gérer les connexions et transmettre les données aux clients.

Cette fonction est exécutée dans un thread démarré par LokiSocketServer_start. Elle crée une socket Unix et transmet les données reçues jusqu'à la demande d'arrêt par LokiSocketServer_stop.

Paramètres

ptr	Le serveur courant.

Renvoie

Rien.

Définition à la ligne 114 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.6.4.6 LokiSocketServer_start()

```
int LokiSocketServer_start (
            LokiSocketServer * self,
            LokiFIFO * in,
            size_t cid)
```

Démarrer un serveur.

Cette fonction est non bloquante. Elle démarre deux threads: un pour traiter les paquets de données d'entrée, l'autre pour gérer les connexions des clients et leur transmettre les données aux clients.

Paramètres

self	Le serveur à démarrer.
in	La FIFO contenant les données à transmettre.
cid	Le numéro de consommateur de ce serveur dans sa FIFO d'entrée.

Renvoie

Le code d'erreur retournée par `pthread_create`.

Définition à la ligne 211 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.4.6.4.7 LokiSocketServer_stop()

```
void LokiSocketServer_stop (
            LokiSocketServer * self)
```

Arrêter un serveur.

Interrompt l'exécution des threads démarrés par la fonction LokiSocketServer_start.

Paramètres

self	Le serveur à arrêter.

Définition à la ligne 227 du fichier libloki-socket-server.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5 Calcul de déplacement

Graphe de collaboration de Calcul de déplacement:

Sujets

— Calcul de déplacement

Un calculateur de déplacement est composé d'un filtre adaptatif d'estimation des paramètres d'une ellipse et d'un filtre passe-bas pour éliminer le déplacement fictif.

— Interface d'un filtre adaptatif pour estimer les paramètres d'une ellipse

Définition des méthodes communes pour estimer les paramètres d'une ellipse au cours de la réception d'une série de points.

— Représentation et manipulation d'ellipses

Type de donnée et fonctions permettant de manipuler des ellipses sous différentes représentations.

— Filtre FIR

Implémentation polyphase d'un filtre à réponse impulsionnelle finie.

— Filtre de Kalman

Estimation des paramètres d'une ellipse par filtrage de Kalman.

— Filtre NLMS

Estimation des paramètres d'une ellipse par filtrage Normalized Least Mean Squares.

— Filtre RLS

Estimation des paramètres d'une ellipse par filtrage Recursive Least Squares.

— Valeurs par défaut des paramètres de calcul de déplacement

Constantes utilisées pour la configuration d'un LokiSystem.

— Gestion des acquisitions et du calcul de déplacement

Un système rassemble un modem et un calculateur de déplacement.

Structures de données

— struct LokiSystemSample

Représentation d'un échantillon en sortie d'une instance de LokiSystem.

4.5.1 Description détaillée

4.5.2 Calcul de déplacement

Un calculateur de déplacement est composé d'un filtre adaptatif d'estimation des paramètres d'une ellipse et d'un filtre passe-bas pour éliminer le déplacement fictif.

Graphe de collaboration de Calcul de déplacement:

Fichiers

— fichier libloki-displacement.c

— fichier libloki-displacement.h

Structures de données

— struct LokiDisplacement

Représentation d'un calculateur de déplacement.

Définitions de type

— typedef struct LokiDisplacement LokiDisplacement

Représentation d'un calculateur de déplacement.

Fonctions

— LokiDisplacement ∗ LokiDisplacement_new (LokiEllipseEstimator ∗estimator, LokiFIRFilter ∗filter, unsigned throttle, float scale, float tau[static 3], float sampling_frequency)

Construire un calculateur de déplacement.

— LokiDisplacement ∗ LokiDisplacement_clone (const LokiDisplacement ∗other)

Cloner une instance de LokiDisplacement.

— bool LokiDisplacement_work (LokiDisplacement ∗self, float x, float y, float ∗x_norm, float ∗y_norm, float ∗out)

Calculer le déplacement courant.

— void LokiDisplacement_reset (LokiDisplacement ∗self)

Remettre à zéro le déplacement.

— LokiEllipse ∗ LokiDisplacement_get_ellipse (LokiDisplacement ∗self)

Retourner la dernière ellipse calculée par cette instance.

— void LokiDisplacement_del (LokiDisplacement ∗self)

Détruire une instance de LokiDisplacement.

4.5.2.1 Description détaillée

Un calculateur de déplacement est composé d'un filtre adaptatif d'estimation des paramètres d'une ellipse et d'un filtre passe-bas pour éliminer le déplacement fictif.

4.5.2.2 Documentation des définitions de type

4.5.2.2.1 LokiDisplacement

```
typedef struct LokiDisplacement LokiDisplacement
```

Représentation d'un calculateur de déplacement.

Définition à la ligne 19 du fichier libloki-displacement.h.

4.5.2.3 Documentation des fonctions

4.5.2.3.1 LokiDisplacement_clone()

```
LokiDisplacement * LokiDisplacement_clone (
            const LokiDisplacement * other)
```

Cloner une instance de LokiDisplacement.

Paramètres

other	Une instance de LokiDisplacement à cloner.

Renvoie

Une copie de other.

Définition à la ligne 64 du fichier libloki-displacement.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.2.3.2 LokiDisplacement_del()

```
void LokiDisplacement_del (
            LokiDisplacement * self)
```

Détruire une instance de LokiDisplacement.

Paramètres

self	L'instance de LokiDisplacement à détruire.

Définition à la ligne 151 du fichier libloki-displacement.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.2.3.3 LokiDisplacement_get_ellipse()

```
LokiEllipse * LokiDisplacement_get_ellipse (
            LokiDisplacement * self)
```

Retourner la dernière ellipse calculée par cette instance.

Paramètres

self	Une instance de LokiDisplacement.

Renvoie

un pointeur vers une ellipse.

Définition à la ligne 147 du fichier libloki-displacement.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.2.3.4 LokiDisplacement_new()

```
LokiDisplacement * LokiDisplacement_new (
            LokiEllipseEstimator * estimator,
            LokiFIRFilter * filter,
            unsigned throttle,
            float scale,
            float tau[static 3],
            float sampling_frequency)
```

Construire un calculateur de déplacement.

Le facteur d'échelle détermine l'unité de distance utilisée en sortie de LokiDisplacement_work. On choisira typiquement scale = $\lambda 0$ / 2πN où $\lambda 0$ est la longueur d'onde du laser, dans l'unité de mesure de distance souhaitée, et N est le nombre de réflexions.

Paramètres

estimator	Un filtre adaptatif d'estimation d'ellipse.
filter	Un filtre passe-bas pour éliminer le déplacement fictif.
throttle	La périodicité de mise à jour des paramètres de l'ellipse.
scale	Un facteur d'échelle pour la valeur des déplacements calculés.
tau	Les constantes de temps du filtrage des paramètres de l'ellipse, en secondes (0 : axes; 1 : centre; 2 : angle).
sampling_frequency	La fréquence d'échantillonnage, en Hz

Renvoie

Une instance de LokiDisplacement.

Définition à la ligne 43 du fichier libloki-displacement.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.2.3.5 LokiDisplacement_reset()

```
void LokiDisplacement_reset (
            LokiDisplacement * self)
```

Remettre à zéro le déplacement.

Paramètres

self	Une instance de LokiDisplacement.

Définition à la ligne 141 du fichier libloki-displacement.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.2.3.6 LokiDisplacement_work()

```
bool LokiDisplacement_work (
            LokiDisplacement * self,
            float x,
            float y,
            float * x_norm,
            float * y_norm,
            float * out)
```

Calculer le déplacement courant.

Pour un déplacement d et une longueur d'onde $\lambda 0$, en mètres, cette fonction retourne : scale $*$ 4π / $\lambda 0$ $*$ d

Paramètres

self	Une instance de LokiDisplacement.
x	La coordonnée X du point courant de l'ellipse.
y	La coordonnée Y du point courant de l'ellipse.
x_norm	La coordonnée X du point courant de l'ellipse, ramené sur le cercle unité.
y_norm	La coordonnée Y du point courant de l'ellipse, ramené sur le cercle unité.
out	Le déplacement calculé.

Renvoie

true à chaque période de décimation.

Définition à la ligne 82 du fichier libloki-displacement.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3 Interface d'un filtre adaptatif pour estimer les paramètres d'une ellipse

Définition des méthodes communes pour estimer les paramètres d'une ellipse au cours de la réception d'une série de points.

Graphe de collaboration de Interface d'un filtre adaptatif pour estimer les paramètres d'une ellipse:

Fichiers

- fichier libloki-ellipse-estimator.c
- fichier libloki-ellipse-estimator.h
- fichier libloki-ellipse.c

Structures de données

- struct LokiEllipseEstimator

  Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Définitions de type

— typedef struct LokiEllipseEstimator LokiEllipseEstimator

Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Fonctions

— LokiEllipseEstimator ∗ LokiEllipseEstimator_new (float initial_radius, const float params[ ])

Construire un estimateur.

— LokiEllipseEstimator ∗ LokiEllipseEstimator_new_with_implementation (const LokiEllipseEstimatorInterface ∗impl, float initial_radius, const float params[ ])

Construire un estimateur avec une implémentation donnée.

— LokiEllipseEstimator ∗ LokiEllipseEstimator_clone (const LokiEllipseEstimator ∗other)

Construire un clone d'un estimateur.

— void LokiEllipseEstimator_work (LokiEllipseEstimator ∗self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— float ∗ LokiEllipseEstimator_get_state (LokiEllipseEstimator ∗self)

Retourner l'état courant de l'estimateur.

— void LokiEllipseEstimator_del (LokiEllipseEstimator ∗self)

Détruire un estimateur.

— **LokiClass_interface** (LokiEllipseEstimator, LokiClass_attribute(size_t params_length) LokiClass_method(LokiEllipseEstimator new) LokiClass_method(LokiEllipseEstimator, clone) LokiClass_method(LokiEllipseEstimator, work) LokiClass_method(LokiEllipseEstimator, get_state) LokiClass_method(LokiEllipseEstimator, del))

— void LokiEllipse_set_from_coeffs (LokiEllipse ∗self, const LokiEllipseEquation coeffs)

Affecter les paramètres d'une ellipse en fonction des coefficients de son équation.

— void LokiEllipse_get_coeffs (const LokiEllipse ∗self, LokiEllipseEquation coeffs)

Calculer les coefficients de l'équation d'une ellipse.

— void LokiEllipse_get_point (const LokiEllipse ∗self, float theta, float ∗x, float ∗y)

Calculer les coordonnées d'un point d'une ellipse.

— float LokiEllipse_get_angle (const LokiEllipse ∗self, float x, float y, float ∗x_norm, float ∗y_norm)

Calculer l'angle correspondant à un point d'une ellipse.

4.5.3.1 Description détaillée

Définition des méthodes communes pour estimer les paramètres d'une ellipse au cours de la réception d'une série de points.

4.5.3.2 Documentation des définitions de type

4.5.3.2.1 LokiEllipseEstimator

```
typedef struct LokiEllipseEstimator LokiEllipseEstimator
```

Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Définition à la ligne 19 du fichier libloki-ellipse-estimator.h.

4.5.3.3 Documentation des fonctions

4.5.3.3.1 LokiEllipse_get_angle()

```
float LokiEllipse_get_angle (
            const LokiEllipse * self,
            float x,
            float y,
            float * x_norm,
            float * y_norm)
```

Calculer l'angle correspondant à un point d'une ellipse.

Cette fonction calcule également les coordonées d'un point du cercle unité correspondant au point de l'ellipse passé en argument.

Paramètres

self	L'ellipse de référence.
x	La coordonnée X d'un point de l'ellipse.
y	La coordonnée Y d'un point de l'ellipse.
x_norm	Un pointeur vers l'emplacement où écrire la coordonnée X du point ramené sur le cercle unité.
y_norm	Un pointeur vers l'emplacement où écrire la coordonnée Y du point ramené sur le cercle unité.

Renvoie

L'angle.

Définition à la ligne 68 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.2 LokiEllipse_get_coeffs()

```
void LokiEllipse_get_coeffs (
            const LokiEllipse * self,
            LokiEllipseEquation coeffs)
```

Calculer les coefficients de l'équation d'une ellipse.

Les coefficients sont normalisés de telle sorte que $a + c = 1$. Le résultat ne contient que les coefficients (a, b, d, e, f).

Paramètres

self	L'ellipse de référence.
coeffs	Le tableau de destination.

Définition à la ligne 42 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.3 LokiEllipse_get_point()

```
void LokiEllipse_get_point (
            const LokiEllipse * self,
            float theta,
            float * x,
            float * y)
```

Calculer les coordonnées d'un point d'une ellipse.

Paramètres

self	L'ellipse de référence.
theta	La coordonnée du point sur l'ellipse, en radians.
x	Un pointeur vers la coordonnée X calculée pour ce point.
y	Un pointeur vers la coordonnée Y calculée pour ce point.

Définition à la ligne 60 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.4 LokiEllipse_set_from_coeffs()

```
void LokiEllipse_set_from_coeffs (
            LokiEllipse * self,
            const LokiEllipseEquation coeffs)
```

Affecter les paramètres d'une ellipse en fonction des coefficients de son équation.

L'équation de l'ellipse est considérée comme normalisée avec $a + c = 1$.

Paramètres

self	L'ellipse à calculer.
coeffs	Les coefficients (a, b, d, e, f) de l'équation.

Définition à la ligne 10 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.5 LokiEllipseEstimator_clone()

```
LokiEllipseEstimator * LokiEllipseEstimator_clone (
            const LokiEllipseEstimator * other)
```

Construire un clone d'un estimateur.

Le nouvel estimateur aura la même implémentation et les mêmes paramètres que l'original.

Paramètres

other	L'estimateur à cloner.

Renvoie

Un nouvel estimateur.

Définition à la ligne 43 du fichier libloki-ellipse-estimator.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.6 LokiEllipseEstimator_del()

```
void LokiEllipseEstimator_del (
            LokiEllipseEstimator * self)
```

Détruire un estimateur.

Paramètres

self	L'estimateur à détruire.

Définition à la ligne 68 du fichier libloki-ellipse-estimator.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.7 LokiEllipseEstimator_get_state()

```
float * LokiEllipseEstimator_get_state (
            LokiEllipseEstimator * self)
```

Retourner l'état courant de l'estimateur.

Paramètres

self	Un estimateur.

Renvoie

Le vecteur d'état de cet estimateur.

Définition à la ligne 64 du fichier libloki-ellipse-estimator.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.8 LokiEllipseEstimator_new()

```
LokiEllipseEstimator * LokiEllipseEstimator_new (
            float initial_radius,
            const float params[])
```

Construire un estimateur.

Paramètres

initial_radius	Le rayon initial de l'ellipse estimée.
params	Les paramètres de cet estimateur (selon l'implémentation).

Renvoie

Un nouvel estimateur.

Définition à la ligne 22 du fichier libloki-ellipse-estimator.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.9 LokiEllipseEstimator_new_with_implementation()

```
LokiEllipseEstimator * LokiEllipseEstimator_new_with_implementation (
            const LokiEllipseEstimatorInterface * impl,
            float initial_radius,
            const float params[])
```

Construire un estimateur avec une implémentation donnée.

Paramètres

impl	L'implémentation de LokiEllipseEstimator à utiliser.
initial_radius	Le rayon initial de l'ellipse estimée.
params	Les paramètres de cet estimateur (selon l'implémentation).

Renvoie

Un nouvel estimateur.

Définition à la ligne 26 du fichier libloki-ellipse-estimator.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.3.3.10 LokiEllipseEstimator_work()

```
void LokiEllipseEstimator_work (
            LokiEllipseEstimator * self,
            float x,
            float y)
```

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

Cette fonction met à jour l'état interne du filtre et les paramètres de l'ellipse de destination.

Paramètres

self	L'estimateur à exécuter.
x	La coordonnée X du point courant.
y	La coordonnée Y du point courant.

Définition à la ligne 60 du fichier libloki-ellipse-estimator.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.4 Représentation et manipulation d'ellipses

Type de donnée et fonctions permettant de manipuler des ellipses sous différentes représentations.

Graphe de collaboration de Représentation et manipulation d'ellipses:

Fichiers

— fichier libloki-ellipse.h

Structures de données

— struct LokiEllipse

Structure de données représentant une ellipse.

Définitions de type

— typedef float LokiEllipseEquation[5]

Tableau des coefficients de l'équation d'une ellipse en coordonnées cartésiennes.

Fonctions

— void LokiEllipse_set_from_coeffs (LokiEllipse ∗self, const LokiEllipseEquation coeffs)

Affecter les paramètres d'une ellipse en fonction des coefficients de son équation.

— void LokiEllipse_get_coeffs (const LokiEllipse ∗self, LokiEllipseEquation coeffs)

Calculer les coefficients de l'équation d'une ellipse.

— void LokiEllipse_get_point (const LokiEllipse ∗self, float theta, float ∗x, float ∗y)

Calculer les coordonnées d'un point d'une ellipse.

— float LokiEllipse_get_angle (const LokiEllipse ∗self, float x, float y, float ∗x_norm, float ∗y_norm)

Calculer l'angle correspondant à un point d'une ellipse.

4.5.4.1 Description détaillée

Type de donnée et fonctions permettant de manipuler des ellipses sous différentes représentations.

4.5.4.2 Documentation des définitions de type

4.5.4.2.1 LokiEllipseEquation

```
typedef float LokiEllipseEquation[5]
```

Tableau des coefficients de l'équation d'une ellipse en coordonnées cartésiennes.

Définition à la ligne 29 du fichier libloki-ellipse.h.

4.5.4.3 Documentation des fonctions

4.5.4.3.1 LokiEllipse_get_angle()

```
float LokiEllipse_get_angle (
            const LokiEllipse * self,
            float x,
            float y,
            float * x_norm,
            float * y_norm)
```

Calculer l'angle correspondant à un point d'une ellipse.

Cette fonction calcule également les coordonées d'un point du cercle unité correspondant au point de l'ellipse passé en argument.

Paramètres

self	L'ellipse de référence.
x	La coordonnée X d'un point de l'ellipse.
y	La coordonnée Y d'un point de l'ellipse.
x_norm	Un pointeur vers l'emplacement où écrire la coordonnée X du point ramené sur le cercle unité.
y_norm	Un pointeur vers l'emplacement où écrire la coordonnée Y du point ramené sur le cercle unité.

Renvoie

L'angle.

Définition à la ligne 68 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.4.3.2 LokiEllipse_get_coeffs()

```
void LokiEllipse_get_coeffs (
            const LokiEllipse * self,
            LokiEllipseEquation coeffs)
```

Calculer les coefficients de l'équation d'une ellipse.

Les coefficients sont normalisés de telle sorte que $a + c = 1$. Le résultat ne contient que les coefficients (a, b, d, e, f).

Paramètres

self	L'ellipse de référence.
coeffs	Le tableau de destination.

Définition à la ligne 42 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.4.3.3 LokiEllipse_get_point()

```
void LokiEllipse_get_point (
            const LokiEllipse * self,
            float theta,
            float * x,
            float * y)
```

Calculer les coordonnées d'un point d'une ellipse.

Paramètres

self	L'ellipse de référence.
theta	La coordonnée du point sur l'ellipse, en radians.
x	Un pointeur vers la coordonnée X calculée pour ce point.
y	Un pointeur vers la coordonnée Y calculée pour ce point.

Définition à la ligne 60 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.4.3.4 LokiEllipse_set_from_coeffs()

```
void LokiEllipse_set_from_coeffs (
            LokiEllipse * self,
            const LokiEllipseEquation coeffs)
```

Affecter les paramètres d'une ellipse en fonction des coefficients de son équation.

L'équation de l'ellipse est considérée comme normalisée avec $a + c = 1$.

Paramètres

self	L'ellipse à calculer.
coeffs	Les coefficients (a, b, d, e, f) de l'équation.

Définition à la ligne 10 du fichier libloki-ellipse.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5 Filtre FIR

Implémentation polyphase d'un filtre à réponse impulsionnelle finie.

Graphe de collaboration de Filtre FIR:

Fichiers

— fichier libloki-fir.c

— fichier libloki-fir.h

Structures de données

— struct LokiFIRFilter

Représentation d'un filtre FIR.

Définitions de type

— typedef struct LokiFIRFilter LokiFIRFilter

Représentation d'un filtre FIR.

Fonctions

— static void LokiFIRFilter_allocate_samples (LokiFIRFilter ∗self)

Allouer le tableau des échantillons mémorisés par un filtre FIR.

— LokiFIRFilter ∗ LokiFIRFilter_new (const float ∗coeffs, size_t length, size_t decimation)

Construire un filtre FIR.

— LokiFIRFilter ∗ LokiFIRFilter_clone (const LokiFIRFilter ∗other)

Construire un clone d'un filtre FIR.

— static float bessel0 (float x)

Function de Bessel de première espèce, d'ordre 0.

— LokiFIRFilter ∗ LokiFIRFilter_new_kaiser_bessel (float fs, float fa, float fb, float att, size_t length, size_t decimation)

Construire un filtre FIR en utilisant une fenêtre de Kaiser-Bessel.

— size_t LokiFIRFilter_get_actual_length (LokiFIRFilter ∗self)

Lire la longueur effective d'un filtre FIR.

— bool LokiFIRFilter_work (LokiFIRFilter ∗self, float in, float ∗out)

Exécuter un pas de calcul d'un fitre FIR.

— void LokiFIRFilter_reset (LokiFIRFilter ∗self)

Réinitialiser un filtre FIR.

— void LokiFIRFilter_del (LokiFIRFilter ∗self)

Détruire un filtre FIR.

4.5.5.1 Description détaillée

Implémentation polyphase d'un filtre à réponse impulsionnelle finie.

4.5.5.2 Documentation des définitions de type

4.5.5.2.1 LokiFIRFilter

```
typedef struct LokiFIRFilter LokiFIRFilter
```

Représentation d'un filtre FIR.

Définition à la ligne 17 du fichier libloki-fir.h.

4.5.5.3 Documentation des fonctions

4.5.5.3.1 bessel0()

```
float bessel0 (
            float x)  [static]
```

Function de Bessel de première espèce, d'ordre 0.

Définition à la ligne 125 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.2 LokiFIRFilter_allocate_samples()

```
void LokiFIRFilter_allocate_samples (
            LokiFIRFilter * self)  [inline], [static]
```

Allouer le tableau des échantillons mémorisés par un filtre FIR.

Cette fonction alloue le tableau `samples` du filtre passé en paramètre.

Paramètres

self	Le filtre à allouer.

Définition à la ligne 50 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.3 LokiFIRFilter_clone()

```
LokiFIRFilter * LokiFIRFilter_clone (
            const LokiFIRFilter * other)
```

Construire un clone d'un filtre FIR.

Le nouveau filtre possédera les mêmes coefficients et le même facteur de décimation que celui passé en paramètre.

Paramètres

other	Le filtre à cloner.

Renvoie

Un nouveau filtre FIR.

Définition à la ligne 106 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.4 LokiFIRFilter_del()

```
void LokiFIRFilter_del (
            LokiFIRFilter * self)
```

Détruire un filtre FIR.

Paramètres

self	Le filtre à détruire.

Définition à la ligne 297 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.5 LokiFIRFilter_get_actual_length()

```
size_t LokiFIRFilter_get_actual_length (
            LokiFIRFilter * self)
```

Lire la longueur effective d'un filtre FIR.

La longueur effective est toujours multiple du facteur de décimation. Elle peut également avoir été ajustée pour satisfaire les contraintes d'une méthode de synthèse de filtre. Pour ces raisons, elle peut différer de la valeur du paramètre `length` fourni à la construction.

Paramètres

self	Le filtre à interroger.

Renvoie

La longueur du filtre.

Définition à la ligne 171 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.6 LokiFIRFilter_new()

```
LokiFIRFilter * LokiFIRFilter_new (
            const float * coeffs,
            size_t length,
            size_t decimation)
```

Construire un filtre FIR.

Le paramètre `length` indique la longueur souhaitée. Elle peut être augmentée pour que la longueur effective soit multiple du facteur de décimation et de la taille de vecteur utilisée par les instructions SIMD du processeur. Dans ce cas, le tableau de coefficients est complété avec des zéros.

Paramètres

coeffs	Les coefficients du filtre.
length	Le nombre de coefficients.
decimation	Le facteur de décimation.

Renvoie

Un nouveau filtre FIR.

Définition à la ligne 60 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.7 LokiFIRFilter_new_kaiser_bessel()

```
LokiFIRFilter * LokiFIRFilter_new_kaiser_bessel (
            float fs,
            float fa,
            float fb,
            float att,
            size_t length,
            size_t decimation)
```

Construire un filtre FIR en utilisant une fenêtre de Kaiser-Bessel.

La méthode de synthèse du filtre est détaillée dans l'article `Window method of FIR filter design`.

Le paramètre `length` indique la longueur souhaitée. Elle peut être augmentée pour satisfaire les contraintes de la méthode de synthèse utilisée, et pour que la longueur effective soit multiple du facteur de décimation et de la taille de vecteur utilisée par les instructions SIMD du processeur. Dans ce cas, le tableau de coefficients est complété avec des zéros.

Paramètres

fs	La fréquence d'échantillonnage.
fa	La fréquence de coupure basse (0 pour un filtre passe-bas).
fb	La fréquence de coupure haute.

att	L'atténuation, en dB.
length	Le nombre de coefficients.
decimation	Le facteur de décimation.

Renvoie

Un nouveau filtre FIR.

Définition à la ligne 136 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.8 LokiFIRFilter_reset()

```
void LokiFIRFilter_reset (
            LokiFIRFilter * self)
```

Réinitialiser un filtre FIR.

Cette fonction remet à zéro l'état du filtre.

Paramètres

self	Le filtre à réinitialiser.

Définition à la ligne 293 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.5.3.9 LokiFIRFilter_work()

```
bool LokiFIRFilter_work (
            LokiFIRFilter * self,
            float in,
            float * out)
```

Exécuter un pas de calcul d'un fitre FIR.

Le filtre consomme un échantillon d'entrée, indique si un échantillon de sortie est disponible et le range à l'emplacement indiqué. Si le facteur de décimation de ce filtre est N, il produira un échantillon de sortie tous les N appels.

Paramètres

self	Le filtre à exécuter.
in	L'échantillon d'entrée.
out	Un pointeur vers l'emplacement où ranger l'échantillon de sortie.

Renvoie

true si l'échantillon de sortie est valide.

Définition à la ligne 175 du fichier libloki-fir.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.6 Filtre de Kalman

Estimation des paramètres d'une ellipse par filtrage de Kalman.

Graphe de collaboration de Filtre de Kalman:

Fichiers

— fichier libloki-kalman.c

— fichier libloki-kalman.h

Structures de données

— struct LokiKalman

Représentation d'un filtre de Kalman.

Définitions de type

— typedef struct LokiKalman LokiKalman

Représentation d'un filtre de Kalman.

Fonctions

— LokiKalman * LokiKalman_new (float initial_radius, const float params[static 3])

Construire un nouveau filtre de Kalman.

— LokiKalman * LokiKalman_clone (const LokiKalman *other)

Construire un clone d'un filtre de Kalman.

— float * LokiKalman_get_state (LokiKalman *self)

Retourner l'état courant du filtre.

— void LokiKalman_work (LokiKalman *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— void LokiKalman_del (LokiKalman *self)

Détruire un filtre de Kalman.

— **LokiClass_implementation** (LokiEllipseEstimator, LokiKalman, LokiClass_set_attribute(params_↩ length, 3) LokiClass_define_method(LokiKalman, new) LokiClass_define_method(LokiKalman, clone) LokiClass_define_method(LokiKalman, work) LokiClass_define_method(LokiKalman, get_state) LokiClass_define_method(Lok del))

— **LokiClass_declare_implementation** (LokiEllipseEstimator, LokiKalman)

4.5.6.1 Description détaillée

Estimation des paramètres d'une ellipse par filtrage de Kalman.

Ce module implémente l'interface LokiEllipseEstimator.

4.5.6.2 Documentation des définitions de type

4.5.6.2.1 LokiKalman

```
typedef struct LokiKalman LokiKalman
```

Représentation d'un filtre de Kalman.

Définition à la ligne 17 du fichier libloki-kalman.h.

4.5.6.3 Documentation des fonctions

4.5.6.3.1 LokiKalman_clone()

```
LokiKalman * LokiKalman_clone (
            const LokiKalman * other)
```

Construire un clone d'un filtre de Kalman.

Paramètres

other	Le filtre de Kalman à cloner.

Renvoie

Un nouveau filtre de Kalman.

Définition à la ligne 45 du fichier libloki-kalman.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.6.3.2 LokiKalman_del()

```
void LokiKalman_del (
            LokiKalman * self)
```

Détruire un filtre de Kalman.

Paramètres

self	Le filtre à détruire.

Définition à la ligne 132 du fichier libloki-kalman.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.6.3.3 LokiKalman_get_state()

```
float * LokiKalman_get_state (
            LokiKalman * self)
```

Retourner l'état courant du filtre.

Paramètres

self	Un filtre de Kalman.

Renvoie

Le vecteur d'état du filtre.

Définition à la ligne 56 du fichier libloki-kalman.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.6.3.4 LokiKalman_new()

```
LokiKalman * LokiKalman_new (
            float initial_radius,
            const float params[static 3])
```

Construire un nouveau filtre de Kalman.

Paramètres

initial_radius	Le rayon initial de l'ellipse estimée.
params	Les puissances de bruit Rv et Rw, et la valeur initiale à affecter dans la matrice P.

Renvoie

Un nouveau filtre de Kalman.

Définition à la ligne 24 du fichier libloki-kalman.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.6.3.5 LokiKalman_work()

```
void LokiKalman_work (
            LokiKalman * self,
            float x,
            float y)
```

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

Cette fonction met à jour l'état interne du filtre et les paramètres de l'ellipse de destination.

Paramètres

self	Le filtre de Kalman à exécuter.
x	La coordonnée X du point courant.
y	La coordonnée Y du point courant.

Définition à la ligne 60 du fichier libloki-kalman.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.7 Filtre NLMS

Estimation des paramètres d'une ellipse par filtrage *Normalized Least Mean Squares*.

Graphe de collaboration de Filtre NLMS:

Fichiers

- fichier libloki-nlms.c
- fichier libloki-nlms.h

Structures de données

- struct LokiNLMS

  Représentattion d'un filtre NLMS.

Macros

- #define NLMS_NU 1.0e-30f

  Correction des poids du filtre pour éviter une division par zéro.

Définitions de type

- typedef struct LokiNLMS LokiNLMS

  Représentattion d'un filtre NLMS.

Fonctions

— LokiNLMS * LokiNLMS_new (float initial_radius, const float params[static 2])

Construire un nouveau filtre NLMS.

— LokiNLMS * LokiNLMS_clone (const LokiNLMS *other)

Construire un clone d'un filtre NLMS.

— float * LokiNLMS_get_state (LokiNLMS *self)

Retourner l'état courant du filtre.

— void LokiNLMS_work (LokiNLMS *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— void LokiNLMS_del (LokiNLMS *self)

Détruire un filtre NLMS.

— **LokiClass_implementation** (LokiEllipseEstimator, LokiNLMS, LokiClass_set_attribute(params_↩ length, 2) LokiClass_define_method(LokiNLMS, new) LokiClass_define_method(LokiNLMS, clone) LokiClass_define_method(LokiNLMS, work) LokiClass_define_method(LokiNLMS, get_state) LokiClass_define_method(LokiN del))

— **LokiClass_declare_implementation** (LokiEllipseEstimator, LokiNLMS)

4.5.7.1 Description détaillée

Estimation des paramètres d'une ellipse par filtrage *Normalized Least Mean Squares*.

Ce module implémente l'interface LokiEllipseEstimator.

4.5.7.2 Documentation des macros

4.5.7.2.1 NLMS_NU

```
#define NLMS_NU 1.0e-30f
```

Correction des poids du filtre pour éviter une division par zéro.

Définition à la ligne 12 du fichier libloki-nlms.c.

4.5.7.3 Documentation des définitions de type

4.5.7.3.1 LokiNLMS

```
typedef struct LokiNLMS LokiNLMS
```

Représentattion d'un filtre NLMS.

Définition à la ligne 17 du fichier libloki-nlms.h.

4.5.7.4 Documentation des fonctions

4.5.7.4.1 LokiNLMS_clone()

```
LokiNLMS * LokiNLMS_clone (
            const LokiNLMS * other)
```

Construire un clone d'un filtre NLMS.

Paramètres

other	Le filtre NLMS à cloner.

Renvoie

Un nouveau filtre NLMS.

Définition à la ligne 41 du fichier libloki-nlms.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.7.4.2 LokiNLMS_del()

```
void LokiNLMS_del (
            LokiNLMS * self)
```

Détruire un filtre NLMS.

Paramètres

self	Le filtre à détruire.

Définition à la ligne 79 du fichier libloki-nlms.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.7.4.3 LokiNLMS_get_state()

```
float * LokiNLMS_get_state (
            LokiNLMS * self)
```

Retourner l'état courant du filtre.

Paramètres

self	Un filtre NLMS.

Renvoie

Le vecteur d'état du filtre.

Définition à la ligne 52 du fichier libloki-nlms.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.7.4.4 LokiNLMS_new()

```
LokiNLMS * LokiNLMS_new (
              float initial_radius,
              const float params[static 2])
```

Construire un nouveau filtre NLMS.

Paramètres

initial_radius	Le rayon initial de l'ellipse estimée.
params	Le facteur d'apprentissage du filtre LMS et le coefficients du filtre de mise à jour des facteur de normalisation.

Renvoie

Un nouveau filtre NLMS.

Définition à la ligne 26 du fichier libloki-nlms.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.7.4.5 LokiNLMS_work()

```
void LokiNLMS_work (
              LokiNLMS * self,
              float x,
              float y)
```

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

Cette fonction met à jour l'état interne du filtre et les paramètre de l'ellipse de destination.

Paramètres

self	Le filtre NLMS à exécuter.
x	La coordonnée X du point courant.
y	La coordonnée Y du point courant.

Définition à la ligne 56 du fichier libloki-nlms.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.8 Filtre RLS

Estimation des paramètres d'une ellipse par filtrage *Recursive Least Squares*.

Graphe de collaboration de Filtre RLS:

Fichiers

— fichier libloki-rls.c

— fichier libloki-rls.h

Structures de données

— struct LokiRLS

Représentation d'un filtre RLS.

Définitions de type

— typedef struct LokiRLS LokiRLS

Représentation d'un filtre RLS.

Fonctions

— LokiRLS ∗ LokiRLS_new (float initial_radius, const float params[static 2])

Construire un nouveau filtre RLS.

— LokiRLS ∗ LokiRLS_clone (const LokiRLS ∗other)

Construire un clone d'un filtre RLS.

— float ∗ LokiRLS_get_state (LokiRLS ∗self)

Retourner l'état courant du filtre.

— void LokiRLS_work (LokiRLS ∗self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— void LokiRLS_del (LokiRLS ∗self)

Détruire un filtre RLS.

— **LokiClass_implementation** (LokiEllipseEstimator, LokiRLS, LokiClass_set_attribute(params_length, 2) LokiClass_define_method(LokiRLS, new) LokiClass_define_method(LokiRLS, clone) LokiClass_define_method(LokiRLS, work) LokiClass_define_method(LokiRLS, get_state) LokiClass_define_method(LokiRLS, del))

— **LokiClass_declare_implementation** (LokiEllipseEstimator, LokiRLS)

4.5.8.1 Description détaillée

Estimation des paramètres d'une ellipse par filtrage *Recursive Least Squares*.

Ce module implémente l'interface LokiEllipseEstimator.

4.5.8.2 Documentation des définitions de type

4.5.8.2.1 LokiRLS

```
typedef struct LokiRLS LokiRLS
```

Représentation d'un filtre RLS.

Définition à la ligne 17 du fichier libloki-rls.h.

4.5.8.3 Documentation des fonctions

4.5.8.3.1 LokiRLS_clone()

```
LokiRLS * LokiRLS_clone (
            const LokiRLS * other)
```

Construire un clone d'un filtre RLS.

Paramètres

other	Le filtre RLS à cloner.

Renvoie

Un nouveau filtre RLS.

Définition à la ligne 42 du fichier libloki-rls.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.8.3.2 LokiRLS_del()

```
void LokiRLS_del (
            LokiRLS * self)
```

Détruire un filtre RLS.

Paramètres

self	Le filtre à détruire.

Définition à la ligne 129 du fichier libloki-rls.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.8.3.3 LokiRLS_get_state()

```
float * LokiRLS_get_state (
            LokiRLS * self)
```

Retourner l'état courant du filtre.

Paramètres

self	Un filtre RLS.

Renvoie

Le vecteur d'état du filtre.

Définition à la ligne 53 du fichier libloki-rls.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.8.3.4 LokiRLS_new()

```
LokiRLS * LokiRLS_new (
            float initial_radius,
            const float params[static 2])
```

Construire un nouveau filtre RLS.

Paramètres

initial_radius	Le rayon initial de l'ellipse estimée.
params	Le facteur d'oubli du filtre et la valeur initiale à affecter dans la matrice P.

Renvoie

Un nouveau filtre RLS.

Définition à la ligne 23 du fichier libloki-rls.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.8.3.5 LokiRLS_work()

```
void LokiRLS_work (
            LokiRLS * self,
            float x,
            float y)
```

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

Cette fonction met à jour l'état interne du filtre et les paramètre de l'ellipse de destination.

Paramètres

self	Le filtre RLS à exécuter.
x	La coordonnée X du point courant.
y	La coordonnée Y du point courant.

Définition à la ligne 57 du fichier libloki-rls.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.9 Valeurs par défaut des paramètres de calcul de déplacement

Constantes utilisées pour la configuration d'un LokiSystem.

Graphe de collaboration de Valeurs par défaut des paramètres de calcul de déplacement:

Fichiers

— fichier libloki-system-defaults.h

Macros

— #define LOKI_SYSTEM_DEFAULT_SIMULATE false

Ne pas activer le mode de simulation logicielle des capteurs.

— #define LOKI_SYSTEM_DEFAULT_SKIP 0

Remplir la FIFO des déplacements immédiatement après la réception des premiers échantillons démodulés.

— #define LOKI_SYSTEM_DEFAULT_WAVELENGTH 1310

La longueur d'onde par défaut, 1310 nm.

— #define LOKI_SYSTEM_DEFAULT_REFLECTIONS 1

Supposer qu'il n'y a qu'une seule réflexion.

— #define LOKI_SYSTEM_DEFAULT_CAVITY_INDEX 1.0f

Supposer un indice de cavité de Fabry-Pérot égal à 1.

— #define LOKI_SYSTEM_DEFAULT_DISTANCE_UNIT 'p';

Calculer les déplacements en picomètres.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_RLS LokiRLSClass

L'implémentation du filtre RLS.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_RLS {0.999f, 0.1f}

Les paramètres par défaut du filtre RLS.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN LokiKalmanClass

L'implémentation du filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN {0.999f, 0.0f, 0.1f}

Les paramètres par défaut du filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN

Utiliser un filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN

Utiliser les paramètres par défaut du filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_THROTTLE 1

La périodicité par défaut du calcul des paramètres de l'ellipse.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_INITIAL_RADIUS 1e-3

Le rayon initial par défaut des ellipses estimées.

— #define LOKI_SYSTEM_DEFAULT_FILTER_BANDWIDTH 250.0f

La bande passante par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_FILTER_ATTENUATION 60.0f

L'atténuation par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_FILTER_LENGTH 1000

Le nombre de coefficients par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_FILTER_DECIMATION 50

La facteur de décimation par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_IIR_TAU 1.0f

La constante de temps par défaut des filtres de lissage des paramètres de l'ellipse.

4.5.9.1 Description détaillée

Constantes utilisées pour la configuration d'un LokiSystem.

4.5.9.2 Documentation des macros

4.5.9.2.1 LOKI_SYSTEM_DEFAULT_CAVITY_INDEX

```
#define LOKI_SYSTEM_DEFAULT_CAVITY_INDEX 1.0f
```

Supposer un indice de cavité de Fabry-Pérot égal à 1.

Définition à la ligne 29 du fichier libloki-system-defaults.h.

4.5.9.2.2 LOKI_SYSTEM_DEFAULT_DISTANCE_UNIT

```
#define LOKI_SYSTEM_DEFAULT_DISTANCE_UNIT 'p';
```

Calculer les déplacements en picomètres.

Définition à la ligne 32 du fichier libloki-system-defaults.h.

4.5.9.2.3 LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN
```

Utiliser un filtre de Kalman.

Définition à la ligne 47 du fichier libloki-system-defaults.h.

4.5.9.2.4 LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN LokiKalmanClass
```

L'implémentation du filtre de Kalman.

Définition à la ligne 41 du fichier libloki-system-defaults.h.

4.5.9.2.5 LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_RLS

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_RLS LokiRLSClass
```

L'implémentation du filtre RLS.

Définition à la ligne 35 du fichier libloki-system-defaults.h.

4.5.9.2.6 LOKI_SYSTEM_DEFAULT_ESTIMATOR_INITIAL_RADIUS

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_INITIAL_RADIUS 1e-3
```

Le rayon initial par défaut des ellipses estimées.

Définition à la ligne 56 du fichier libloki-system-defaults.h.

4.5.9.2.7 LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN
```

Utiliser les paramètres par défaut du filtre de Kalman.

Définition à la ligne 50 du fichier libloki-system-defaults.h.

4.5.9.2.8 LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN {0.999f, 0.0f, 0.1f}
```

Les paramètres par défaut du filtre de Kalman.

Définition à la ligne 44 du fichier libloki-system-defaults.h.

4.5.9.2.9 LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_RLS

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_RLS {0.999f, 0.1f}
```

Les paramètres par défaut du filtre RLS.

Définition à la ligne 38 du fichier libloki-system-defaults.h.

4.5.9.2.10 LOKI_SYSTEM_DEFAULT_ESTIMATOR_THROTTLE

```
#define LOKI_SYSTEM_DEFAULT_ESTIMATOR_THROTTLE 1
```

La périodicité par défaut du calcul des paramètres de l'ellipse.

Définition à la ligne 53 du fichier libloki-system-defaults.h.

4.5.9.2.11 LOKI_SYSTEM_DEFAULT_FILTER_ATTENUATION

```
#define LOKI_SYSTEM_DEFAULT_FILTER_ATTENUATION 60.0f
```

L'atténuation par défaut du filtre de suppression du déplacement fictif.

Définition à la ligne 62 du fichier libloki-system-defaults.h.

4.5.9.2.12 LOKI_SYSTEM_DEFAULT_FILTER_BANDWIDTH

```
#define LOKI_SYSTEM_DEFAULT_FILTER_BANDWIDTH 250.0f
```

La bande passante par défaut du filtre de suppression du déplacement fictif.

Définition à la ligne 59 du fichier libloki-system-defaults.h.

4.5.9.2.13 LOKI_SYSTEM_DEFAULT_FILTER_DECIMATION

```
#define LOKI_SYSTEM_DEFAULT_FILTER_DECIMATION 50
```

La facteur de décimation par défaut du filtre de suppression du déplacement fictif.

Définition à la ligne 68 du fichier libloki-system-defaults.h.

4.5.9.2.14 LOKI_SYSTEM_DEFAULT_FILTER_LENGTH

```
#define LOKI_SYSTEM_DEFAULT_FILTER_LENGTH 1000
```

Le nombre de coefficients par défaut du filtre de suppression du déplacement fictif.

Définition à la ligne 65 du fichier libloki-system-defaults.h.

4.5.9.2.15 LOKI_SYSTEM_DEFAULT_IIR_TAU

```
#define LOKI_SYSTEM_DEFAULT_IIR_TAU 1.0f
```

La constante de temps par défaut des filtres de lissage des paramètres de l'ellipse.

Définition à la ligne 71 du fichier libloki-system-defaults.h.

4.5.9.2.16 LOKI_SYSTEM_DEFAULT_REFLECTIONS

```
#define LOKI_SYSTEM_DEFAULT_REFLECTIONS 1
```

Supposer qu'il n'y a qu'une seule réflexion.

Définition à la ligne 26 du fichier libloki-system-defaults.h.

4.5.9.2.17 LOKI_SYSTEM_DEFAULT_SIMULATE

```
#define LOKI_SYSTEM_DEFAULT_SIMULATE false
```

Ne pas activer le mode de simulation logicielle des capteurs.

Définition à la ligne 17 du fichier libloki-system-defaults.h.

4.5.9.2.18 LOKI_SYSTEM_DEFAULT_SKIP

```
#define LOKI_SYSTEM_DEFAULT_SKIP 0
```

Remplir la FIFO des déplacements immédiatement après la réception des premiers échantillons démodulés.

Définition à la ligne 20 du fichier libloki-system-defaults.h.

4.5.9.2.19 LOKI_SYSTEM_DEFAULT_WAVELENGTH

```
#define LOKI_SYSTEM_DEFAULT_WAVELENGTH 1310
```

La longueur d'onde par défaut, 1310 nm.

Définition à la ligne 23 du fichier libloki-system-defaults.h.

4.5.10 Gestion des acquisitions et du calcul de déplacement

Un système rassemble un modem et un calculateur de déplacement.

Graphe de collaboration de Gestion des acquisitions et du calcul de déplacement:

Fichiers

- fichier libloki-system.c
- fichier libloki-system.h

Structures de données

- struct LokiSystemConfig

  Les paramètres de configuration d'un système d'acquisition et de traitement.

- struct LokiSystem

  Représentation d'un système d'acquisition et de traitement.

- struct LokiSystemSample

  Représentation d'un échantillon en sortie d'une instance de LokiSystem.

Définitions de type

- typedef struct LokiSystemConfig LokiSystemConfig

  Paramètres de configuration d'une instance de LokiSystem.

- typedef struct LokiSystem LokiSystem

  Représentation d'un système d'acquisition et de calcul de déplacement.

Fonctions

— LokiSystemConfig ∗ LokiSystemConfig_new (void)

Construire une configuration pour une instance de LokiSystem.

— void LokiSystemConfig_del (LokiSystemConfig ∗self)

Détruire une configuration pour une instance de LokiSystem.

— float LokiSystemConfig_get_skip (LokiSystemConfig ∗self)

Lire le temps d'attente au démarrage.

— void LokiSystemConfig_set_skip (LokiSystemConfig ∗self, float value)

Affecter un temps d'attente au démarrage.

— bool LokiSystemConfig_get_simulate (LokiSystemConfig ∗self)

Lire l'état d'activation de la simulation des capteurs de déplacement.

— void LokiSystemConfig_set_simulate (LokiSystemConfig ∗self, bool value)

Activer ou désactiver la simulation des capteurs de déplacement.

— float LokiSystemConfig_get_scale (LokiSystemConfig ∗self)

Lire le facteur d'échelle de déplacement.

— float LokiSystemConfig_get_wavelength (LokiSystemConfig ∗self)

Lire la longueur d'onde de la diode laser.

— void LokiSystemConfig_set_wavelength (LokiSystemConfig ∗self, float value)

Écrire la longueur d'onde de la diode laser.

— unsigned LokiSystemConfig_get_reflections (LokiSystemConfig ∗self)

Lire le nombre de réflexions supposé.

— void LokiSystemConfig_set_reflections (LokiSystemConfig ∗self, unsigned value)

Écrire le nombre de réflexions supposé.

— float LokiSystemConfig_get_cavity_index (LokiSystemConfig ∗self)

Lire l'indice de la cavité de Fabry-Pérot.

— void LokiSystemConfig_set_cavity_index (LokiSystemConfig ∗self, float value)

Écrire l'indice de la cavité de Fabry-Pérot.

— char LokiSystemConfig_get_distance_unit (LokiSystemConfig ∗self)

Lire l'unité du déplacement calculé.

— void LokiSystemConfig_set_distance_unit (LokiSystemConfig ∗self, char value)

Écrire l'unité du déplacement calculé.

— float * LokiSystemConfig_get_time_constants (LokiSystemConfig *self)

Lire le tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— size_t LokiSystemConfig_get_time_constants_length (LokiSystemConfig *self)

Lire la longueur du tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— void LokiSystemConfig_set_time_constants (LokiSystemConfig *self, const float *value)

Affecter un tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— const char * LokiSystemConfig_get_estimator_name (LokiSystemConfig *self)

Lire le nom de l'estimateur de paramètres d'ellipse.

— void LokiSystemConfig_set_estimator_name (LokiSystemConfig *self, const char *name)

Affecter un nom de l'estimateur de paramètres d'ellipse.

— unsigned LokiSystemConfig_get_estimator_throttle (LokiSystemConfig *self)

— void LokiSystemConfig_set_estimator_throttle (LokiSystemConfig *self, unsigned value)

— float LokiSystemConfig_get_estimator_initial_radius (LokiSystemConfig *self)

Lire le rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

— void LokiSystemConfig_set_estimator_initial_radius (LokiSystemConfig *self, float value)

Affecter un rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

— float * LokiSystemConfig_get_estimator_params (LokiSystemConfig *self)

Lire les paramètres du filtre d'estimation des paramètres de l'ellipse.

— size_t LokiSystemConfig_get_estimator_params_length (LokiSystemConfig *self)

Lire le nombre de paramètres du filtre d'estimation des paramètres de l'ellipse.

— void LokiSystemConfig_set_estimator_params (LokiSystemConfig *self, const float *value)

Affecter des paramètres pour le filtre d'estimation des paramètres de l'ellipse.

— float LokiSystemConfig_get_filter_bandwidth (LokiSystemConfig *self)

Lire la bande passante du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_bandwidth (LokiSystemConfig *self, float value)

Affecter un bande passante pour le filtre d'élimination du déplacement fictif.

— float LokiSystemConfig_get_filter_attenuation (LokiSystemConfig *self)

Lire l'atténuation du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_attenuation (LokiSystemConfig *self, float value)

Affecter une atténuation pour le filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_filter_length (LokiSystemConfig ∗self)

Lire la longueur du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_length (LokiSystemConfig ∗self, size_t value)

Affecter une longueur au filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_filter_decimation (LokiSystemConfig ∗self)

Lire le facteur de décimation du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_decimation (LokiSystemConfig ∗self, size_t value)

Affecter un facteur de décimation au filtre d'élimination du déplacement fictif.

— LokiSystem ∗ LokiSystem_new (LokiSystemConfig ∗config, LokiModem ∗modem)

Construire un nouveau système.

— LokiSystemConfig ∗ LokiSystem_get_config (const LokiSystem ∗self)

Lire la configuration d'un système.

— LokiModem ∗ LokiSystem_get_modem (const LokiSystem ∗self)

Lire le modem utilisé par ce système.

— float LokiSystem_get_sampling_frequency (const LokiSystem ∗self)

Lire la fréquence d'échantillonnage en sortie de ce système.

— void LokiSystem_displacement_reset (LokiSystem ∗self)

Mettre à zéro le déplacement calculé.

— static void ∗ LokiSystem_work (void ∗ptr)

Calculer le déplacement à partir des données démodulées.

— int LokiSystem_start_streaming (LokiSystem ∗self, int32_t channels, LokiFIFO ∗modem_out, LokiFIFO ∗disp_out, size_t cid)

Démarrer l'acquisition et le traitement.

— void LokiSystem_stop_streaming (LokiSystem ∗self)

Arrêter l'acquisition et le traitement.

— bool LokiSystem_is_streaming (LokiSystem ∗self)

Lire l'état de l'acquisition.

— LokiEllipse ∗ LokiSystem_get_ellipse (LokiSystem ∗self, size_t channel)

Lire la dernière ellipse estimée.

— void LokiSystem_del (LokiSystem ∗self)

Détruire un système.

Variables

— static const char ∗const LokiSystemConfig_estimator_names [ ]

Les noms des filtres adaptatifs supportés pour l'estimation des paramètres de l'ellipse.

— static const LokiEllipseEstimatorInterface ∗const LokiSystemConfig_estimator_impls [ ]

Les implémentations des filtres adaptatifs supportés pour l'estimation des paramètres de l'ellipse.

4.5.10.1 Description détaillée

Un système rassemble un modem et un calculateur de déplacement.

4.5.10.2 Documentation des définitions de type

4.5.10.2.1 LokiSystem

```
typedef struct LokiSystem LokiSystem
```

Représentation d'un système d'acquisition et de calcul de déplacement.

Définition à la ligne 293 du fichier libloki-system.h.

4.5.10.2.2 LokiSystemConfig

```
typedef struct LokiSystemConfig LokiSystemConfig
```

Paramètres de configuration d'une instance de LokiSystem.

Définition à la ligne 33 du fichier libloki-system.h.

4.5.10.3 Documentation des fonctions

4.5.10.3.1 LokiSystem_del()

```
void LokiSystem_del (
            LokiSystem * self)
```

Détruire un système.

Paramètres

self	Le système à détruire.

Définition à la ligne 573 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.2 LokiSystem_displacement_reset()

```
void LokiSystem_displacement_reset (
            LokiSystem * self)
```

Mettre à zéro le déplacement calculé.

Paramètres

self	Le système à modifier.

Voir également

LokiDisplacement_reset

Définition à la ligne 363 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.3 LokiSystem_get_config()

```
LokiSystemConfig * LokiSystem_get_config (
            const LokiSystem * self)
```

Lire la configuration d'un système.

Paramètres

self	Le système à interroger.

Renvoie

La configuration attachée à ce système.

Définition à la ligne 351 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.4 LokiSystem_get_ellipse()

```
LokiEllipse * LokiSystem_get_ellipse (
            LokiSystem * self,
            size_t channel)
```

Lire la dernière ellipse estimée.

Paramètres

self	Le système à interroger.

channel	La voie à observer.

Renvoie

La dernière ellipse estimée pour cette voie.

Définition à la ligne 569 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.5 LokiSystem_get_modem()

```
LokiModem * LokiSystem_get_modem (
              const LokiSystem * self)
```

Lire le modem utilisé par ce système.

Paramètres

self	Le système à interroger.

Renvoie

Le modem attaché à ce système.

Définition à la ligne 355 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.6 LokiSystem_get_sampling_frequency()

```
float LokiSystem_get_sampling_frequency (
              const LokiSystem * self)
```

Lire la fréquence d'échantillonnage en sortie de ce système.

Paramètres

self	Le système à interroger.

Renvoie

La fréquence d'échantillonnage du signal de déplacement.

Définition à la ligne 359 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.7 LokiSystem_is_streaming()

```
bool LokiSystem_is_streaming (
            LokiSystem * self)
```

Lire l'état de l'acquisition.

Paramètres

self	Le système à interroger.

Renvoie

true si l'acquisition est en cours.

Définition à la ligne 565 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.8 LokiSystem_new()

```
LokiSystem * LokiSystem_new (
            LokiSystemConfig * config,
            LokiModem * modem)
```

Construire un nouveau système.

Paramètres

config	La configuration à utiliser.
modem	Le modem à utliser, `NULL` pour en instancier un nouveau.

Renvoie

Une nouvelle instance de LokiSystem.

Définition à la ligne 295 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.9 LokiSystem_start_streaming()

```
int LokiSystem_start_streaming (
            LokiSystem * self,
            int32_t channels,
            LokiFIFO * modem_out,
            LokiFIFO * disp_out,
            size_t cid)
```

Démarrer l'acquisition et le traitement.

Cette fonction est non bloquante. Elle crée un nouveau thread pour le traitement des données en provenance du modem et démarre celui-ci.

Paramètres

self	Le système à démarrer.
channels	Un masque indiquant les voies à traiter, -1 pour toutes les voies.
modem_out	La FIFO qui recevra les données de sortie du démodulateur.
disp_out	La FIFO qui recevra les valeurs de déplacement.
cid	Le numéro de consommateur de ce système pour la FIFO `modem_out`.

Renvoie

Le code d'erreur retourné par LokiModem_start_streaming

Définition à la ligne 527 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.10 LokiSystem_stop_streaming()

```
void LokiSystem_stop_streaming (
            LokiSystem * self)
```

Arrêter l'acquisition et le traitement.

Interrompt l'exécution du thread lancé par la fonction LokiSystem_start_streaming.

Paramètres

self	Le système à arrêter.

Définition à la ligne 550 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.11 LokiSystem_work()

```
void * LokiSystem_work (
            void * ptr)  [static]
```

Calculer le déplacement à partir des données démodulées.

Cette fonction est exécutée dans un thread démarré par LokiSystem_start_streaming. Elle consomme les données de la FIFO d'entrée et alimente la FIFO de sortie, jusqu'à la demande d'arrêt par LokiConsumer_stop.

Paramètres

ptr	Le système courant.

Renvoie

Rien.

Définition à la ligne 380 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.12 LokiSystemConfig_del()

```
void LokiSystemConfig_del (
            LokiSystemConfig * self)
```

Détruire une configuration pour une instance de LokiSystem.

Paramètres

self	L'instance de LokiSystemConfig à détruire.

Définition à la ligne 102 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.13 LokiSystemConfig_get_cavity_index()

```
float LokiSystemConfig_get_cavity_index (
            LokiSystemConfig * self)
```

Lire l'indice de la cavité de Fabry-Pérot.

Paramètres

self	La configuration à interroger.

Renvoie

Le nombre de réflexions.

Définition à la ligne 152 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.14 LokiSystemConfig_get_distance_unit()

```
char LokiSystemConfig_get_distance_unit (
            LokiSystemConfig * self)
```

Lire l'unité du déplacement calculé.

Paramètres

self	La configuration à interroger.

Renvoie

Le préfixe de l'unité de déplacement ('p', 'n' 'u', 'm').

Définition à la ligne 160 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.15 LokiSystemConfig_get_estimator_initial_radius()

```
float LokiSystemConfig_get_estimator_initial_radius (
            LokiSystemConfig * self)
```

Lire le rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

Paramètres

self	La configuration à interroger.

Renvoie

Le rayon initial.

Définition à la ligne 209 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.16 LokiSystemConfig_get_estimator_name()

```
const char * LokiSystemConfig_get_estimator_name (
            LokiSystemConfig * self)
```

Lire le nom de l'estimateur de paramètres d'ellipse.

Paramètres

self	La configuration à interroger.

Renvoie

Le nom du filtre d'estimation.

Définition à la ligne 182 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.17 LokiSystemConfig_get_estimator_params()

```
float * LokiSystemConfig_get_estimator_params (
            LokiSystemConfig * self)
```

Lire les paramètres du filtre d'estimation des paramètres de l'ellipse.

Les paramètres se présentent sous la forme d'un tableau dont la longueur est accessible à l'aide de la fonction LokiSystemConfig_get_estimator_params_length.

Paramètres

self	La configuration à interroger.

Renvoie

Un tableau de paramètres.

Définition à la ligne 217 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.18 LokiSystemConfig_get_estimator_params_length()

```
size_t LokiSystemConfig_get_estimator_params_length (
            LokiSystemConfig * self)
```

Lire le nombre de paramètres du filtre d'estimation des paramètres de l'ellipse.

Paramètres

self	La configuration à interroger.

Renvoie

Le nombre de paramètres.

Définition à la ligne 221 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.19 LokiSystemConfig_get_estimator_throttle()

```
unsigned LokiSystemConfig_get_estimator_throttle (
            LokiSystemConfig * self)
```

Définition à la ligne 201 du fichier libloki-system.c.

4.5.10.3.20 LokiSystemConfig_get_filter_attenuation()

```
float LokiSystemConfig_get_filter_attenuation (
            LokiSystemConfig * self)
```

Lire l'atténuation du filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à interroger.

Renvoie

L'atténuation, en dB.

Définition à la ligne 243 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.21 LokiSystemConfig_get_filter_bandwidth()

```
float LokiSystemConfig_get_filter_bandwidth (
            LokiSystemConfig * self)
```

Lire la bande passante du filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à interroger.

Renvoie

La bande passante du filtre, en Hz.

Définition à la ligne 235 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.22 LokiSystemConfig_get_filter_decimation()

```
size_t LokiSystemConfig_get_filter_decimation (
            LokiSystemConfig * self)
```

Lire le facteur de décimation du filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à interroger.

Renvoie

Le facteur de décimation du filtre.

Définition à la ligne 259 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.23 LokiSystemConfig_get_filter_length()

```
size_t LokiSystemConfig_get_filter_length (
            LokiSystemConfig * self)
```

Lire la longueur du filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à interroger.

Renvoie

Le nombre de coefficients du filtre.

Définition à la ligne 251 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.24 LokiSystemConfig_get_reflections()

```
unsigned LokiSystemConfig_get_reflections (
            LokiSystemConfig * self)
```

Lire le nombre de réflexions supposé.

Paramètres

self	La configuration à interroger.

Renvoie

Le nombre de réflexions.

Définition à la ligne 144 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.25 LokiSystemConfig_get_scale()

```
float LokiSystemConfig_get_scale (
            LokiSystemConfig * self)
```

Lire le facteur d'échelle de déplacement.

Le résultat est calculé en fonction des paramètres de longueur d'onde, nombre de réflexions, indice de la cavité, unité de distance.

Paramètres

self	La configuration à interroger.

Renvoie

La valeur du facteur d'échelle de déplacement.

Définition à la ligne 125 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.26 LokiSystemConfig_get_simulate()

```
bool LokiSystemConfig_get_simulate (
            LokiSystemConfig * self)
```

Lire l'état d'activation de la simulation des capteurs de déplacement.

Le paramètre `simulate` active un mode de simulation des capteurs de déplacement dans le système à configurer.

Paramètres

self	Une configuration à interroger.

Renvoie

true si la configuration prévoit une simulation des capteurs de déplacement.

Définition à la ligne 117 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.27 LokiSystemConfig_get_skip()

```
float LokiSystemConfig_get_skip (
            LokiSystemConfig * self)
```

Lire le temps d'attente au démarrage.

Le paramètre `skip` définit le nombre de secondes d'attente avant de commencer à remplir la FIFO des données de déplacement.

Paramètres

self	Une configuration à interroger.

Renvoie

Le temps d'attente, en secondes.

Définition à la ligne 109 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.28 LokiSystemConfig_get_time_constants()

```
float * LokiSystemConfig_get_time_constants (
            LokiSystemConfig * self)
```

Lire le tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

La longueur de ce tableau peut être connue en appelant la fonction LokiSystemConfig_get_time_constants_length.

Paramètres

self	La configuration à interroger.

Renvoie

Le tableau de constantes de temps.

Définition à la ligne 168 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.29 LokiSystemConfig_get_time_constants_length()

```
size_t LokiSystemConfig_get_time_constants_length (
            LokiSystemConfig * self)
```

Lire la longueur du tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

Paramètres

self	La configuration à interroger.

Renvoie

Le nombre de constantes de temps.

Définition à la ligne 172 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.30 LokiSystemConfig_get_wavelength()

```
float LokiSystemConfig_get_wavelength (
            LokiSystemConfig * self)
```

Lire la longueur d'onde de la diode laser.

Paramètres

self	La configuration à interroger.

Renvoie

La longueur d'onde de la diode laser, en nm.

Définition à la ligne 136 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.31 LokiSystemConfig_new()

```
LokiSystemConfig * LokiSystemConfig_new (
            void )
```

Construire une configuration pour une instance de LokiSystem.

Renvoie

Une instance de LokiSystemConfig.

Définition à la ligne 65 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.32 LokiSystemConfig_set_cavity_index()

```
void LokiSystemConfig_set_cavity_index (
            LokiSystemConfig * self,
            float value)
```

Écrire l'indice de la cavité de Fabry-Pérot.

Paramètres

self	La configuration à modifier.
value	Le nouvel indice de la cavité de Fabry-Pérot.

Définition à la ligne 156 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.33 LokiSystemConfig_set_distance_unit()

```
void LokiSystemConfig_set_distance_unit (
            LokiSystemConfig * self,
            char value)
```

Écrire l'unité du déplacement calculé.

Paramètres

self	La configuration à modifier.
value	Le nouveau préfixe de l'unité de déplacement ('p', 'n' 'u', 'm').

Définition à la ligne 164 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.34 LokiSystemConfig_set_estimator_initial_radius()

```
void LokiSystemConfig_set_estimator_initial_radius (
            LokiSystemConfig * self,
            float value)
```

Affecter un rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

Paramètres

self	La configuration à modifier.
value	Le rayon initial.

Définition à la ligne 213 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.35 LokiSystemConfig_set_estimator_name()

```
void LokiSystemConfig_set_estimator_name (
            LokiSystemConfig * self,
            const char * name)
```

Affecter un nom de l'estimateur de paramètres d'ellipse.

Paramètres

self	La configuration à modifier.
name	Le nom du filtre d'estimation à utiliser.

Définition à la ligne 191 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.36 LokiSystemConfig_set_estimator_params()

```
void LokiSystemConfig_set_estimator_params (
            LokiSystemConfig * self,
            const float * value)
```

Affecter des paramètres pour le filtre d'estimation des paramètres de l'ellipse.

Les paramètres se présentent sous la forme d'un tableau dont la longueur doit être égale à la valeur retournée par la fonction LokiSystemConfig_get_estimator_params_length.

Paramètres

self	La configuration à modifier.
value	Un tableau de paramètres.

Définition à la ligne 225 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.37 LokiSystemConfig_set_estimator_throttle()

```
void LokiSystemConfig_set_estimator_throttle (
            LokiSystemConfig * self,
            unsigned value)
```

Définition à la ligne 205 du fichier libloki-system.c.

4.5.10.3.38 LokiSystemConfig_set_filter_attenuation()

```
void LokiSystemConfig_set_filter_attenuation (
            LokiSystemConfig * self,
            float value)
```

Affecter une atténuation pour le filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à modifier.
value	L'atténuation, en dB.

Définition à la ligne 247 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.39 LokiSystemConfig_set_filter_bandwidth()

```
void LokiSystemConfig_set_filter_bandwidth (
            LokiSystemConfig * self,
            float value)
```

Affecter un bande passante pour le filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à modifier.
value	La bande passante du filtre, en Hz.

Définition à la ligne 239 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.40 LokiSystemConfig_set_filter_decimation()

```
void LokiSystemConfig_set_filter_decimation (
            LokiSystemConfig * self,
            size_t value)
```

Affecter un facteur de décimation au filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à modifier.
value	Le facteur de décimation du filtre.

Définition à la ligne 263 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.41 LokiSystemConfig_set_filter_length()

```
void LokiSystemConfig_set_filter_length (
            LokiSystemConfig * self,
            size_t value)
```

Affecter une longueur au filtre d'élimination du déplacement fictif.

Paramètres

self	La configuration à modifier.
value	Le nombre de coefficients du filtre.

Définition à la ligne 255 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.42 LokiSystemConfig_set_reflections()

```
void LokiSystemConfig_set_reflections (
            LokiSystemConfig * self,
            unsigned value)
```

Écrire le nombre de réflexions supposé.

Paramètres

self	La configuration à modifier.
value	Le nouveau nombre de réflexions.

Définition à la ligne 148 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.43 LokiSystemConfig_set_simulate()

```
void LokiSystemConfig_set_simulate (
            LokiSystemConfig * self,
            bool value)
```

Activer ou désactiver la simulation des capteurs de déplacement.

Le paramètre `simulate` active un mode de simulation des capteurs de déplacement dans le système à configurer.

Paramètres

self	Une configuration à modifier.
value	true pour indiquer qu'il faut simuler les capteurs de déplacement.

Définition à la ligne 121 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.44 LokiSystemConfig_set_skip()

```
void LokiSystemConfig_set_skip (
            LokiSystemConfig * self,
            float value)
```

Affecter un temps d'attente au démarrage.

Le paramètre `skip` définit le nombre de secondes d'attente avant de commencer à remplir la FIFO des données de déplacement.

Paramètres

self	Une configuration à modifier.

value	Le nouveau temps d'attente, en secondes.

Définition à la ligne 113 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.45 LokiSystemConfig_set_time_constants()

```
void LokiSystemConfig_set_time_constants (
            LokiSystemConfig * self,
            const float * value)
```

Affecter un tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

La longueur de ce tableau doit être égale à la valeur retournée par la fonction LokiSystemConfig_get_time_constants_length.

Paramètres

self	La configuration à interroger.
value	Le nombre de constantes de temps.

Définition à la ligne 176 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.3.46 LokiSystemConfig_set_wavelength()

```
void LokiSystemConfig_set_wavelength (
            LokiSystemConfig * self,
            float value)
```

Écrire la longueur d'onde de la diode laser.

Paramètres

self	La configuration à modifier.
value	La nouvelle longueur d'onde de la diode laser, en nm.

Définition à la ligne 140 du fichier libloki-system.c.

Voici le graphe d'appel pour cette fonction : Voici le graphe des appelants de cette fonction :

4.5.10.4 Documentation des variables

4.5.10.4.1 LokiSystemConfig_estimator_impls

```
const LokiEllipseEstimatorInterface* const LokiSystemConfig_estimator_impls[]  [static]
```

Valeur initiale :

```
                                                                    = {
    &LokiRLSClass, &LokiNLMSClass, &LokiKalmanClass
}
```

Les implémentations des filtres adaptatifs supportés pour l'estimation des paramètres de l'ellipse.

Définition à la ligne 61 du fichier libloki-system.c.

4.5.10.4.2 LokiSystemConfig_estimator_names

```
const char* const LokiSystemConfig_estimator_names[]  [static]
```

Valeur initiale :

```
                                                    = {
    "rls", "nlms", "kalman"
}
```

Les noms des filtres adaptatifs supportés pour l'estimation des paramètres de l'ellipse.

Définition à la ligne 56 du fichier libloki-system.c.

Chapitre 5

Documentation des structures de données

5.1 Référence de la structure AD8370ControlWord::[struct].__unnamed0__

Les champs du mot de contrôle.

Champs de données

- unsigned **gain_code**: AD8370_GAIN_CODE_SIZE

  Le code de gain.

- unsigned **high_gain**: 1

  Le bit d'activation du mode High Gain.

5.1.1 Description détaillée

Les champs du mot de contrôle.

Définition à la ligne 704 du fichier libloki-modem.c.

La documentation de cette structure a été générée à partir des fichiers suivants :

5.2 Référence de la structure float32x4_t::[struct].__unnamed0__

Décomposition d'un vecteur en deux moitiés.

Champs de données

— float32x2_t **low**

Un vecteur contenant les deux premiers éléments de ce vecteur.

— float32x2_t **high**

Un vecteur contenant les deux derniers éléments de ce vecteur.

5.2.1 Description détaillée

Décomposition d'un vecteur en deux moitiés.

Définition à la ligne 36 du fichier libloki-neon.h.

La documentation de cette structure a été générée à partir des fichiers suivants :

5.3 Référence de l'union LokiEnvSensorsSample::[union].__unnamed0__

La valeur de l'échantillon.

Champs de données

— float **data**

La valeur de l'échantillon, en virgule flottante après mise à l'échelle.

— int32_t **data_raw**

La valeur de l'échantillon, en virgule fixe.

5.3.1 Description détaillée

La valeur de l'échantillon.

Définition à la ligne 30 du fichier libloki-env-sensors.h.

La documentation de cette union a été générée à partir des fichiers suivants :

5.4 Référence de l'union LokiModemSample::[union].__unnamed0__

La composante X de l'échantillon.

Champs de données

— float **x**

La composante X de l'échantillon, en virgule flottante.

— int32_t **x_raw**

La composante X de l'échantillon, en virgule fixe.

5.4.1 Description détaillée

La composante X de l'échantillon.

Définition à la ligne 35 du fichier libloki-modem.h.

La documentation de cette union a été générée à partir des fichiers suivants :

5.5 Référence de l'union LokiMSeedInputSample::[union].__unnamed0__

La valeur de l'échantillon.

Champs de données

— int32_t **value_raw**

— float **value**

5.5.1 Description détaillée

La valeur de l'échantillon.

Définition à la ligne 87 du fichier libloki-mseed.c.

La documentation de cette union a été générée à partir des fichiers suivants :

5.6 Référence de l'union LokiModemSample::[union].__unnamed1__

La composante Y de l'échantillon.

Champs de données

— float **y**

La composante Y de l'échantillon, en virgule flottante après mise à l'échelle.

— int32_t **y_raw**

La composante Y de l'échantillon, en virgule fixe.

5.6.1 Description détaillée

La composante Y de l'échantillon.

Définition à la ligne 42 du fichier libloki-modem.h.

La documentation de cette union a été générée à partir des fichiers suivants :

5.7 Référence de l'union AD8370ControlWord

Représentation du mot de contrôle du gain pour l'amplificateur AD8370.

Structures de données

— struct [struct].__unnamed0__

Les champs du mot de contrôle.

Champs de données

— uint8_t raw

La valeur brute du mot de contrôle.

— struct

5.7.1 Description détaillée

Représentation du mot de contrôle du gain pour l'amplificateur AD8370.

Définition à la ligne 700 du fichier libloki-modem.c.

5.7.2 Documentation des champs

5.7.2.1 raw

```
uint8_t AD8370ControlWord::raw
```

La valeur brute du mot de contrôle.

Définition à la ligne 702 du fichier libloki-modem.c.

La documentation de cette union a été générée à partir du fichier suivant :

— libloki-modem.c

5.8 Référence de la structure float32x2_t

Vecteur de deux nombres flottants 32 bits.

```
#include <libloki-neon.h>
```

Champs de données

— float val [2]

Les éléments de ce vecteur.

5.8.1 Description détaillée

Vecteur de deux nombres flottants 32 bits.

Définition à la ligne 26 du fichier libloki-neon.h.

5.8.2 Documentation des champs

5.8.2.1 val

```
float float32x2_t::val[2]
```

Les éléments de ce vecteur.

Définition à la ligne 28 du fichier libloki-neon.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-neon.h

5.9 Référence de l'union float32x4_t

Vecteur de quatre nombres flottants 32 bits.

`#include <libloki-neon.h>`

Structures de données

— struct [struct].__unnamed0__

Décomposition d'un vecteur en deux moitiés.

Champs de données

— float val [4]

Les éléments de ce vecteur.

— struct

5.9.1 Description détaillée

Vecteur de quatre nombres flottants 32 bits.

Définition à la ligne 32 du fichier libloki-neon.h.

5.9.2 Documentation des champs

5.9.2.1 val

`float float32x4_t::val[4]`

Les éléments de ce vecteur.

Définition à la ligne 34 du fichier libloki-neon.h.

La documentation de cette union a été générée à partir du fichier suivant :

— libloki-neon.h

5.10 Référence de la structure LokiConsumer

Définition d'un consommateur.

Graphe de collaboration de LokiConsumer:

Champs de données

- LokiFIFO * in

  La FIFO lue par ce consommateur.

- size_t cid

  Le numéro de consommateur pour la FIFO.

- volatile bool stop_request

  Indicateur de demande d'arrêt.

- pthread_t thread

  Le thread démarré par LokiConsumer_start.

5.10.1 Description détaillée

Définition d'un consommateur.

Définition à la ligne 20 du fichier libloki-consumer.c.

5.10.2 Documentation des champs

5.10.2.1 cid

```
size_t LokiConsumer::cid
```

Le numéro de consommateur pour la FIFO.

Définition à la ligne 24 du fichier libloki-consumer.c.

5.10.2.2 in

```
LokiFIFO* LokiConsumer::in
```

La FIFO lue par ce consommateur.

Définition à la ligne 22 du fichier libloki-consumer.c.

5.10.2.3 stop_request

```
volatile bool LokiConsumer::stop_request
```

Indicateur de demande d'arrêt.

Définition à la ligne 26 du fichier libloki-consumer.c.

5.10.2.4 thread

```
pthread_t LokiConsumer::thread
```

Le thread démarré par LokiConsumer_start.

Définition à la ligne 28 du fichier libloki-consumer.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-consumer.c

5.11 Référence de la structure LokiDisplacement

Représentation d'un calculateur de déplacement.

Graphe de collaboration de LokiDisplacement:

Champs de données

— LokiEllipse ellipse

La dernière ellipse calculée, après lissage.

— LokiEllipseEstimator * estimator

Le filtre adaptatif d'estimation des paramètres de l'ellipse.

— LokiFIRFilter * filter

Le filtre passe-bas de suppression du déplacement fictif.

— unsigned estimator_throttle

Périodicité du calcul des paramètres de l'ellipse.

— unsigned estimator_cnt

Compteur pour l'activation du calcul des paramètres de l'ellipse.

— float phi_prev

La phase précédente, pour le déroulage de phase.

— float phi_filt

La phase après filtrage du déplacement fictif.

— float phi_ref

La phase de référence correspondant à un déplacement nul.

— int32_t phi_turns

Le nombre de tours effectués lors du déroulage de phase.

— float alpha_prev

L'angle précédent de l'ellipse, pour détecter les sauts de pi/2.

— float alpha_ref

L'angle de référence pour corriger l'angle de l'ellipse en cas de saut.

— float scale

Le facteur de conversion d'angle en distance.

— float iir_a [3]

Les coefficients a des filtres de lissage des paramètres de l'ellipse.

— float iir_b [3]

Les coefficients b des filtres de lissage des paramètres de l'ellipse.

5.11.1 Description détaillée

Représentation d'un calculateur de déplacement.

Définition à la ligne 12 du fichier libloki-displacement.c.

5.11.2 Documentation des champs

5.11.2.1 alpha_prev

```
float LokiDisplacement::alpha_prev
```

L'angle précédent de l'ellipse, pour détecter les sauts de pi/2.

Définition à la ligne 32 du fichier libloki-displacement.c.

5.11.2.2 alpha_ref

```
float LokiDisplacement::alpha_ref
```

L'angle de référence pour corriger l'angle de l'ellipse en cas de saut.

Définition à la ligne 34 du fichier libloki-displacement.c.

5.11.2.3 ellipse

```
LokiEllipse LokiDisplacement::ellipse
```

La dernière ellipse calculée, après lissage.

Définition à la ligne 14 du fichier libloki-displacement.c.

5.11.2.4 estimator

```
LokiEllipseEstimator* LokiDisplacement::estimator
```

Le filtre adaptatif d'estimation des paramètres de l'ellipse.

Définition à la ligne 16 du fichier libloki-displacement.c.

5.11.2.5 estimator_cnt

```
unsigned LokiDisplacement::estimator_cnt
```

Compteur pour l'activation du calcul des paramètres de l'ellipse.

Définition à la ligne 22 du fichier libloki-displacement.c.

5.11.2.6 estimator_throttle

```
unsigned LokiDisplacement::estimator_throttle
```

Périodicité du calcul des paramètres de l'ellipse.

Définition à la ligne 20 du fichier libloki-displacement.c.

5.11.2.7 filter

```
LokiFIRFilter* LokiDisplacement::filter
```

Le filtre passe-bas de suppression du déplacement fictif.

Définition à la ligne 18 du fichier libloki-displacement.c.

5.11.2.8 iir_a

```
float LokiDisplacement::iir_a[3]
```

Les coefficients a des filtres de lissage des paramètres de l'ellipse.

Définition à la ligne 38 du fichier libloki-displacement.c.

5.11.2.9 iir_b

```
float LokiDisplacement::iir_b[3]
```

Les coefficients b des filtres de lissage des paramètres de l'ellipse.

Définition à la ligne 40 du fichier libloki-displacement.c.

5.11.2.10 phi_filt

```
float LokiDisplacement::phi_filt
```

La phase après filtrage du déplacement fictif.

Définition à la ligne 26 du fichier libloki-displacement.c.

5.11.2.11 phi_prev

```
float LokiDisplacement::phi_prev
```

La phase précédente, pour le déroulage de phase.

Définition à la ligne 24 du fichier libloki-displacement.c.

5.11.2.12 phi_ref

```
float LokiDisplacement::phi_ref
```

La phase de référence correspondant à un déplacement nul.

Définition à la ligne 28 du fichier libloki-displacement.c.

5.11.2.13 phi_turns

```
int32_t LokiDisplacement::phi_turns
```

Le nombre de tours effectués lors du déroulage de phase.

Définition à la ligne 30 du fichier libloki-displacement.c.

5.11.2.14 scale

```
float LokiDisplacement::scale
```

Le facteur de conversion d'angle en distance.

Définition à la ligne 36 du fichier libloki-displacement.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-displacement.c

5.12 Référence de la structure LokiEllipse

Structure de données représentant une ellipse.

```
#include <libloki-ellipse.h>
```

Champs de données

- float A

  Le demi-grand axe de l'ellipse.

- float B

  Le demi-petit axe de l'ellipse.

- float xc

  La coordonnée X du centre de l'ellipse.

- float yc

  La coordonnée Y du centre de l'ellipse.

- float alpha

  L'angle entre le grand axe et l'axe horizontal.

5.12.1 Description détaillée

Structure de données représentant une ellipse.

Définition à la ligne 15 du fichier libloki-ellipse.h.

5.12.2 Documentation des champs

5.12.2.1 A

```
float LokiEllipse::A
```

Le demi-grand axe de l'ellipse.

Définition à la ligne 17 du fichier libloki-ellipse.h.

5.12.2.2 alpha

```
float LokiEllipse::alpha
```

L'angle entre le grand axe et l'axe horizontal.

Définition à la ligne 25 du fichier libloki-ellipse.h.

5.12.2.3 B

```
float LokiEllipse::B
```

Le demi-petit axe de l'ellipse.

Définition à la ligne 19 du fichier libloki-ellipse.h.

5.12.2.4 xc

```
float LokiEllipse::xc
```

La coordonnée X du centre de l'ellipse.

Définition à la ligne 21 du fichier libloki-ellipse.h.

5.12.2.5 yc

```
float LokiEllipse::yc
```

La coordonnée Y du centre de l'ellipse.

Définition à la ligne 23 du fichier libloki-ellipse.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-ellipse.h

5.13 Référence de la structure LokiEllipseEstimator

Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Champs de données

— void * instance

Les données spécifiques à l'implémentation de l'estimateur.

— LokiEllipseEstimatorInterface impl

L'implémentation des méthodes de cet estimateur.

5.13.1 Description détaillée

Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Définition à la ligne 11 du fichier libloki-ellipse-estimator.c.

5.13.2 Documentation des champs

5.13.2.1 impl

```
LokiEllipseEstimatorInterface LokiEllipseEstimator::impl
```

L'implémentation des méthodes de cet estimateur.

Les pointeurs vers les méthodes seront copiés ici pour éviter un niveau d'indirection supplémentaire et réduire la durée d'un appel de méthode.

Définition à la ligne 19 du fichier libloki-ellipse-estimator.c.

5.13.2.2 instance

```
void* LokiEllipseEstimator::instance
```

Les données spécifiques à l'implémentation de l'estimateur.

Définition à la ligne 13 du fichier libloki-ellipse-estimator.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-ellipse-estimator.c

5.14 Référence de la structure LokiEnvSensors

Représentation de l'interface d'acquisition des capteurs auxiliaires.

Graphe de collaboration de LokiEnvSensors:

Champs de données

— FILE * file

Le fichier représentant le périphérique qui gère les capteurs.

— int fileno

Le numéro du fichier représentant le périphérique qui gère les capteurs.

— volatile bool stop_request

Indicateur de demande d'arrêt d'acquisition.

— LokiFIFO * out

La FIFO de sortie de ce module d'acquisition.

— bool raw

Conserver les données brutes (entiers sans mise à l'échelle).

— int32_t stream_channels

Un masque indiquant les voies à traiter, -1 pour toutes les voies.

— pthread_t thread

Le thread démarré par LokiEnvSensors_start_streaming.

— uint32_t clk_frequency

La fréquence d'horloge du circuit d'acquisition.

— uint32_t channels

Le nombre de voies supportées par le circuit d'acquisition.

— uint32_t cycles_per_sample_max

Le nombre maximal de cycles d'horloge par échantillon.

— uint32_t channel_output_width

Le nombre de bits représentant le numéro de voie dans un échantillon brut.

— uint32_t data_width

Le nombre de bits de données dans un échantillon brut.

— uint32_t channel_mask

Le masque binaire permettant d'isoler le numéro de voie dans un échantillon brut.

— uint32_t data_mask

Le masque binaire permettant d'isoler la valeur d'un échantillon brut.

— uint32_t data_sign_mask

Le masque binaire permettant d'isoler le bit de signe de la valeur d'un échantillon brut.

— int64_t bmp390_t_fine

La dernière valeur de température lue sur le capteur BMP390 après compensation.

— float * scales

Le tableau des facteurs d'échelle à appliquer à chaque voie.

5.14.1 Description détaillée

Représentation de l'interface d'acquisition des capteurs auxiliaires.

Définition à la ligne 25 du fichier libloki-env-sensors.c.

5.14.2 Documentation des champs

5.14.2.1 bmp390_t_fine

```
int64_t LokiEnvSensors::bmp390_t_fine
```

La dernière valeur de température lue sur le capteur BMP390 après compensation.

Définition à la ligne 57 du fichier libloki-env-sensors.c.

5.14.2.2 channel_mask

```
uint32_t LokiEnvSensors::channel_mask
```

Le masque binaire permettant d'isoler le numéro de voie dans un échantillon brut.

Définition à la ligne 51 du fichier libloki-env-sensors.c.

5.14.2.3 channel_output_width

```
uint32_t LokiEnvSensors::channel_output_width
```

Le nombre de bits représentant le numéro de voie dans un échantillon brut.

Définition à la ligne 47 du fichier libloki-env-sensors.c.

5.14.2.4 channels

```
uint32_t LokiEnvSensors::channels
```

Le nombre de voies supportées par le circuit d'acquisition.

Définition à la ligne 43 du fichier libloki-env-sensors.c.

5.14.2.5 clk_frequency

```
uint32_t LokiEnvSensors::clk_frequency
```

La fréquence d'horloge du circuit d'acquisition.

Définition à la ligne 41 du fichier libloki-env-sensors.c.

5.14.2.6 cycles_per_sample_max

```
uint32_t LokiEnvSensors::cycles_per_sample_max
```

Le nombre maximal de cycles d'horloge par échantillon.

Définition à la ligne 45 du fichier libloki-env-sensors.c.

5.14.2.7 data_mask

```
uint32_t LokiEnvSensors::data_mask
```

Le masque binaire permettant d'isoler la valeur d'un échantillon brut.

Définition à la ligne 53 du fichier libloki-env-sensors.c.

5.14.2.8 data_sign_mask

```
uint32_t LokiEnvSensors::data_sign_mask
```

Le masque binaire permettant d'isoler le bit de signe de la valeur d'un échantillon brut.

Définition à la ligne 55 du fichier libloki-env-sensors.c.

5.14.2.9 data_width

```
uint32_t LokiEnvSensors::data_width
```

Le nombre de bits de données dans un échantillon brut.

Définition à la ligne 49 du fichier libloki-env-sensors.c.

5.14.2.10 file

```
FILE* LokiEnvSensors::file
```

Le fichier représentant le périphérique qui gère les capteurs.

Définition à la ligne 27 du fichier libloki-env-sensors.c.

5.14.2.11 fileno

```
int LokiEnvSensors::fileno
```

Le numéro du fichier représentant le périphérique qui gère les capteurs.

Définition à la ligne 29 du fichier libloki-env-sensors.c.

5.14.2.12 out

```
LokiFIFO* LokiEnvSensors::out
```

La FIFO de sortie de ce module d'acquisition.

Définition à la ligne 33 du fichier libloki-env-sensors.c.

5.14.2.13 raw

```
bool LokiEnvSensors::raw
```

Conserver les données brutes (entiers sans mise à l'échelle).

Définition à la ligne 35 du fichier libloki-env-sensors.c.

5.14.2.14 scales

```
float* LokiEnvSensors::scales
```

Le tableau des facteurs d'échelle à appliquer à chaque voie.

Définition à la ligne 59 du fichier libloki-env-sensors.c.

5.14.2.15 stop_request

```
volatile bool LokiEnvSensors::stop_request
```

Indicateur de demande d'arrêt d'acquisition.

Définition à la ligne 31 du fichier libloki-env-sensors.c.

5.14.2.16 stream_channels

```
int32_t LokiEnvSensors::stream_channels
```

Un masque indiquant les voies à traiter, -1 pour toutes les voies.

Définition à la ligne 37 du fichier libloki-env-sensors.c.

5.14.2.17 thread

```
pthread_t LokiEnvSensors::thread
```

Le thread démarré par LokiEnvSensors_start_streaming.

Définition à la ligne 39 du fichier libloki-env-sensors.c.

La documentation de cette structure a été générée à partir du fichier suivant :
- libloki-env-sensors.c

5.15 Référence de la structure LokiEnvSensorsSample

Représentation d'un échantillon en provenance des capteurs auxiliaires.

```
#include <libloki-env-sensors.h>
```

Structures de données

- union [union].__unnamed0__

  La valeur de l'échantillon.

Champs de données

- uint32_t channel

  Le numéro de la voie sur lequel l'échantillon a été reçu.

- union

5.15.1 Description détaillée

Représentation d'un échantillon en provenance des capteurs auxiliaires.

Définition à la ligne 26 du fichier libloki-env-sensors.h.

5.15.2 Documentation des champs

5.15.2.1 channel

```
uint32_t LokiEnvSensorsSample::channel
```

Le numéro de la voie sur lequel l'échantillon a été reçu.

Définition à la ligne 28 du fichier libloki-env-sensors.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-env-sensors.h

5.16 Référence de la structure LokiFIFO

Représentation d'une FIFO.

Graphe de collaboration de LokiFIFO:

Champs de données

— size_t item_size

La taille d'un élément, en octets.

— int all_read

Le masque binaire indiquant que tous les consommateurs ont lu le dernier élément.

— volatile bool abort

Faut-il interrompre l'opération en cours.

— pthread_mutex_t mutex

Un mutex pour protéger les accès concurrents à la FIFO.

— pthread_cond_t pushed

Variable conditionnelle indiquant qu'un élément vient d'être ajouté à la FIFO.

— pthread_cond_t pulled

Variable conditionnelle indiquant qu'un élément vient d'être retiré de la FIFO.

— LokiFIFORecord * producer_record

Un pointeur vers le record courant en écriture.

— LokiFIFORecord * consumer_records [0]

Pointeurs vers les records courants en lecture pour chaque consommateur.

5.16.1 Description détaillée

Représentation d'une FIFO.

Le contenu de la FIFO est construit comme une liste cirulaire dont les maillons sont de type LokiFIFORecord. Cette liste circulaire n'est pas représentée explicitement dans la structure LokiFIFO. Elle est allouée à la suite du tableau `consumer_records`.

Définition à la ligne 20 du fichier libloki-fifo.c.

5.16.2 Documentation des champs

5.16.2.1 abort

```
volatile bool LokiFIFO::abort
```

Faut-il interrompre l'opération en cours.

Définition à la ligne 26 du fichier libloki-fifo.c.

5.16.2.2 all_read

```
int LokiFIFO::all_read
```

Le masque binaire indiquant que tous les consommateurs ont lu le dernier élément.

Définition à la ligne 24 du fichier libloki-fifo.c.

5.16.2.3 consumer_records

```
LokiFIFORecord* LokiFIFO::consumer_records[0]
```

Pointeurs vers les records courants en lecture pour chaque consommateur.

Ce tableau est alloué dans LokiFIFO_new selon le nombre de consommateurs.

Définition à la ligne 39 du fichier libloki-fifo.c.

5.16.2.4 item_size

```
size_t LokiFIFO::item_size
```

La taille d'un élément, en octets.

Définition à la ligne 22 du fichier libloki-fifo.c.

5.16.2.5 mutex

```
pthread_mutex_t LokiFIFO::mutex
```

Un mutex pour protéger les accès concurrents à la FIFO.

Définition à la ligne 28 du fichier libloki-fifo.c.

5.16.2.6 producer_record

```
LokiFIFORecord* LokiFIFO::producer_record
```

Un pointeur vers le record courant en écriture.

Définition à la ligne 34 du fichier libloki-fifo.c.

5.16.2.7 pulled

```
pthread_cond_t LokiFIFO::pulled
```

Variable conditionnelle indiquant qu'un élément vient d'être retiré de la FIFO.

Définition à la ligne 32 du fichier libloki-fifo.c.

5.16.2.8 pushed

```
pthread_cond_t LokiFIFO::pushed
```

Variable conditionnelle indiquant qu'un élément vient d'être ajouté à la FIFO.

Définition à la ligne 30 du fichier libloki-fifo.c.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-fifo.c

5.17 Référence de la structure LokiFIFORecord

Représentation d'un enregistrement dans une FIFO.

Graphe de collaboration de LokiFIFORecord:

Champs de données

— volatile int status

Le masque indiquant l'état de lecture de cet enregistrement par les consommateurs de la FIFO.

— LokiFIFORecord * next

Un pointeur vers le prochain enregistrement dans la FIFO.

— char item [0]

Les données logées dans cet enregistrement.

5.17.1 Description détaillée

Représentation d'un enregistrement dans une FIFO.

Un enregistrement constitue un maillon dans une liste chaînée circulaire.

Définition à la ligne 47 du fichier libloki-fifo.c.

5.17.2 Documentation des champs

5.17.2.1 item

```
char LokiFIFORecord::item[0]
```

Les données logées dans cet enregistrement.

La taille effective de ce tableau dépend du champ `item_size` de la FIFO.

Définition à la ligne 56 du fichier libloki-fifo.c.

5.17.2.2 next

```
LokiFIFORecord* LokiFIFORecord::next
```

Un pointeur vers le prochain enregistrement dans la FIFO.

Définition à la ligne 51 du fichier libloki-fifo.c.

5.17.2.3 status

```
volatile int LokiFIFORecord::status
```

Le masque indiquant l'état de lecture de cet enregistrement par les consommateurs de la FIFO.

Définition à la ligne 49 du fichier libloki-fifo.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-fifo.c

5.18 Référence de la structure LokiFIRFilter

Représentation d'un filtre FIR.

Champs de données

— bool is_clone

Ce filtre est-il un clone d'un autre filtre ?

— float * coeffs

Le tableau des coefficients de ce filtre.

— float * coeffs_end

Un pointeur vers la région située après le dernier coefficient de ce filtre.

— float * coeff_ptr

Un pointeur vers le premier coefficient du filtre pour la phase courante.

— float * samples

Le tableau des échantillons d'entrée mémorisés.

— float * samples_end

Un pointeur vers la région située après le dernier échantillon mémorisé.

— size_t sample_index

L'indice de l'échantillon courant dans la phase courante.

— float * phase_ptr

Un pointeur vers les échantillons de la phase courante.

— float acc

Le résultat partiel du filtrage.

— size_t length

Le nombre de coefficients de ce filtre.

— size_t decimation

Le facteur de décimation de ce filtre.

— size_t phase_length

Le nombre de coefficients par phase.

5.18.1 Description détaillée

Représentation d'un filtre FIR.

Définition à la ligne 17 du fichier libloki-fir.c.

5.18.2 Documentation des champs

5.18.2.1 acc

```
float LokiFIRFilter::acc
```

Le résultat partiel du filtrage.

Définition à la ligne 35 du fichier libloki-fir.c.

5.18.2.2 coeff_ptr

```
float* LokiFIRFilter::coeff_ptr
```

Un pointeur vers le premier coefficient du filtre pour la phase courante.

Définition à la ligne 25 du fichier libloki-fir.c.

5.18.2.3 coeffs

```
float* LokiFIRFilter::coeffs
```

Le tableau des coefficients de ce filtre.

Définition à la ligne 21 du fichier libloki-fir.c.

5.18.2.4 coeffs_end

```
float* LokiFIRFilter::coeffs_end
```

Un pointeur vers la région située après le dernier coefficient de ce filtre.

Définition à la ligne 23 du fichier libloki-fir.c.

5.18.2.5 decimation

```
size_t LokiFIRFilter::decimation
```

Le facteur de décimation de ce filtre.

Définition à la ligne 39 du fichier libloki-fir.c.

5.18.2.6 is_clone

```
bool LokiFIRFilter::is_clone
```

Ce filtre est-il un clone d'un autre filtre ?

Définition à la ligne 19 du fichier libloki-fir.c.

5.18.2.7 length

```
size_t LokiFIRFilter::length
```

Le nombre de coefficients de ce filtre.

Définition à la ligne 37 du fichier libloki-fir.c.

5.18.2.8 phase_length

```
size_t LokiFIRFilter::phase_length
```

Le nombre de coefficients par phase.

Définition à la ligne 41 du fichier libloki-fir.c.

5.18.2.9 phase_ptr

```
float* LokiFIRFilter::phase_ptr
```

Un pointeur vers les échantillons de la phase courante.

Définition à la ligne 33 du fichier libloki-fir.c.

5.18.2.10 sample_index

```
size_t LokiFIRFilter::sample_index
```

L'indice de l'échantillon courant dans la phase courante.

Définition à la ligne 31 du fichier libloki-fir.c.

5.18.2.11 samples

```
float* LokiFIRFilter::samples
```

Le tableau des échantillons d'entrée mémorisés.

Définition à la ligne 27 du fichier libloki-fir.c.

5.18.2.12 samples_end

```
float* LokiFIRFilter::samples_end
```

Un pointeur vers la région située après le dernier échantillon mémorisé.

Définition à la ligne 29 du fichier libloki-fir.c.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-fir.c

5.19 Référence de la structure LokiFormatter

Représentation d'un formateur de séries temporelles.

Champs de données

— void * instance

Les données spécifiques à l'implémentation du formateur.

— LokiFormatterInterface impl

L'implémentation des méthodes de ce formateur.

5.19.1 Description détaillée

Représentation d'un formateur de séries temporelles.

Définition à la ligne 11 du fichier libloki-formatter.c.

5.19.2 Documentation des champs

5.19.2.1 impl

```
LokiFormatterInterface LokiFormatter::impl
```

L'implémentation des méthodes de ce formateur.

Les pointeurs vers les méthodes seront copiés ici pour éviter un niveau d'indirection supplémentaire et réduire la durée d'un appel de méthode.

Définition à la ligne 19 du fichier libloki-formatter.c.

5.19.2.2 instance

```
void* LokiFormatter::instance
```

Les données spécifiques à l'implémentation du formateur.

Définition à la ligne 13 du fichier libloki-formatter.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-formatter.c

5.20 Référence de la structure LokiKalman

Représentation d'un filtre de Kalman.

Champs de données

— float rv

La puissance du bruit de sortie du filtre.

— float rw

La puissance du bruit d'état du filtre.

— float P [5][5]

La matrice de covariance de l'erreur d'estimation.

— LokiEllipseEquation state

Les coefficients de l'équation de l'ellipse.

5.20.1 Description détaillée

Représentation d'un filtre de Kalman.

Définition à la ligne 13 du fichier libloki-kalman.c.

5.20.2 Documentation des champs

5.20.2.1 P

```
float LokiKalman::P[5][5]
```

La matrice de covariance de l'erreur d'estimation.

Définition à la ligne 19 du fichier libloki-kalman.c.

5.20.2.2 rv

```
float LokiKalman::rv
```

La puissance du bruit de sortie du filtre.

Définition à la ligne 15 du fichier libloki-kalman.c.

5.20.2.3 rw

```
float LokiKalman::rw
```

La puissance du bruit d'état du filtre.

Définition à la ligne 17 du fichier libloki-kalman.c.

5.20.2.4 state

```
LokiEllipseEquation LokiKalman::state
```

Les coefficients de l'équation de l'ellipse.

Définition à la ligne 21 du fichier libloki-kalman.c.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-kalman.c

5.21 Référence de la structure LokiModem

Représentation des périphériques modulateur et démodulateur.

Graphe de collaboration de LokiModem:

Champs de données

— FILE * file

Le fichier représentant le périphérique qui gère le modem.

— int fileno

Le numéro du fichier représentant le périphérique qui gère le modem.

— volatile bool stop_request

Indicateur de demande d'arrêt d'acquisition.

— LokiFIFO * out

La FIFO de sortie des données démodulées.

— bool raw

Conserver les données démodulées brutes (entiers sans mise à l'échelle).

— int32_t stream_channels

Un masque indiquant les voies à traiter, -1 pour toutes les voies.

— pthread_t thread

Le thread démarré par LokiModem_start_streaming.

— bool simulate

Faut-il simuler les capteurs logiciellement ?

— uint32_t sim_channel

Le numéro de la voie courante pendant une simulation logicielle des capteurs.

— float sim_osc1_phase

La phase courante du déplacement fictif pendant une simulation logicielle des capteurs.

— float sim_disp_phase

La phase courante correspondant au déplacement réel pendant une simulation logicielle des capteurs.

— uint32_t clk_frequency

La fréquence d'horloge du circuit modem.

— uint32_t mod_sample_frequency_ref

La fréquence d'échantillonnage de référence en sortie du modulateur.

— uint32_t mod_cycles_per_sample_max

Le nombre maximal de périodes d'horloge par échantillon en sortie du modulateur.

— float mod_cycles_per_sample_ref

Le nombre de périodes d'horloge de référence par échantillon en sortie du modulateur.

— uint32_t mod_osc1_wave_frequency_min

La fréquence minimale du premier oscillateur du modulateur pour la fréquence d'échantillonnage de référence, en Hz.

— uint32_t mod_osc2_wave_frequency_min

La fréquence minimale du second oscillateur du modulateur pour la fréquence d'échantillonnage de référence, en Hz.

— uint32_t mod_gain_integer_width

Le nombre de bits de la partie entière de l'amplitude de sortie du modulateur.

— uint32_t mod_gain_fractional_width

Le nombre de bits de la partie fractionnaire de l'amplitude de sortie du modulateur.

— float mod_gain_res

La résolution du réglage d'amplitude en sortie du modulateur.

— float mod_phase_amplitude_max

L'amplitude maximale de sortie du modulateur, en mA.

— uint32_t mod_phase_integer_width

Le nombre de bits de la partie entière de la valeur de sortie du modulateur.

— uint32_t mod_phase_fractional_width

Le nombre de bits de la partie fractionnaire de la valeur de sortie du modulateur.

— float mod_offset_res

La résolution du réglage d'offset en sortie du modulateur.

— float mod_dc_value_max

Le courant moyen maximal applicable en sortie du modulateur.

— uint32_t mod_dc_value_width

Le nombre de bits pour représenter le courant moyen en sortie du modulateur.

— float mod_dc_value_res

La résolution du réglage du courant moyen en sortie du modulateur.

— uint32_t mod_pga_gain_width

Le nombre de bits pour représenter le gain de l'amplificateur de sortie du modulateur.

— uint32_t mod_pga_gain_max

Le gain maximal de l'amplificateur en sortie du modulateur.

— bool demod_mixerless

Le démodulateur fonctionne-t-il par décimation ?

— uint32_t demod_channels

Le nombre de voies d'entrée du démodulateur.

— uint32_t demod_channel_output_width

Le nombre de bits pour représenter le numéro de voie dans un échantillon brut produit par le démodulateur.

— uint32_t demod_channel_mask

Le masque binaire pour lire le numéro de voie dans un échantillon démodulé.

— uint32_t demod_power_integer_width

Le nombre de bits de la partie entière des échantillons en entrée du démodulateur.

— uint32_t demod_power_fractional_width

Le nombre de bits de la partie fractionnaire des échantillons en entrée du démodulateur.

— float demod_power_max

La valeur maximale des échantillons en entrée du démodulateur.

— uint32_t demod_sample_frequency_ref

La fréquence d'échantillonnage de référence en entrée du démodulateur.

— uint32_t demod_cycles_per_sample_max

Le nombre maximal de périodes d'horloge par échantillon en entrée du démodulateur.

— float demod_cycles_per_sample_ref

Le nombre de périodes d'horloge de référence par échantillon en entrée du démodulateur.

— uint32_t demod_osc_wave_frequency_min

La fréquence minimale de l'oscillateur du démodulateur pour la fréquence d'échantillonnage de référence, en Hz.

— uint32_t demod_fir1_decimation

Le facteur de décimation du premier étage de filtrage dans le démodulateur.

— uint32_t demod_fir1_length

Le nombre de coefficients du premier étage de filtrage dans le démodulateur.

— uint32_t demod_fir2_decimation

Le facteur de décimation du second étage de filtrage dans le démodulateur.

— uint32_t demod_fir2_length

Le nombre de coefficients du second étage de filtrage dans le démodulateur.

— uint32_t demod_fir2_bandwidth

La bande passante de sortie du second étage de filtrage dans le démodulateur.

— uint32_t demod_xy_integer_width

Le nombre de bits de la partie entière des échantillons de chaque composante X (I) ou Y (Q) du signal démodulé.

— uint32_t demod_xy_fractional_width

Le nombre de bits de la partie fractionnaire des échantillons de chaque composante X (I) ou Y (Q) du signal démodulé.

— uint32_t demod_xy_width

Le nombre total de bits d'un échantillon de chaque composante X (I) ou Y (Q) du signal démodulé.

— uint32_t demod_xy_mask

Le masque binaire pour lire une composante X (I) ou Y (Q) d'un échantillon du signal démodulé.

— uint32_t demod_xy_sign_mask

Le masque binaire pour lire le bit de signe d'une composante X (I) ou Y (Q) d'un échantillon du signal démodulé.

— float demod_xy_max

La valeur maximale d'un échantillon pour une composante X (I) ou Y (Q) du signal démodulé.

5.21.1 Description détaillée

Représentation des périphériques modulateur et démodulateur.

Définition à la ligne 23 du fichier libloki-modem.c.

5.21.2 Documentation des champs

5.21.2.1 clk_frequency

```
uint32_t LokiModem::clk_frequency
```

La fréquence d'horloge du circuit modem.

Définition à la ligne 47 du fichier libloki-modem.c.

5.21.2.2 demod_channel_mask

```
uint32_t LokiModem::demod_channel_mask
```

Le masque binaire pour lire le numéro de voie dans un échantillon démodulé.

Définition à la ligne 89 du fichier libloki-modem.c.

5.21.2.3 demod_channel_output_width

```
uint32_t LokiModem::demod_channel_output_width
```

Le nombre de bits pour représenter le numéro de voie dans un échantillon brut produit par le démodulateur.

Définition à la ligne 87 du fichier libloki-modem.c.

5.21.2.4 demod_channels

```
uint32_t LokiModem::demod_channels
```

Le nombre de voies d'entrée du démodulateur.

Définition à la ligne 85 du fichier libloki-modem.c.

5.21.2.5 demod_cycles_per_sample_max

```
uint32_t LokiModem::demod_cycles_per_sample_max
```

Le nombre maximal de périodes d'horloge par échantillon en entrée du démodulateur.

Définition à la ligne 99 du fichier libloki-modem.c.

5.21.2.6 demod_cycles_per_sample_ref

```
float LokiModem::demod_cycles_per_sample_ref
```

Le nombre de périodes d'horloge de référence par échantillon en entrée du démodulateur.

Définition à la ligne 101 du fichier libloki-modem.c.

5.21.2.7 demod_fir1_decimation

```
uint32_t LokiModem::demod_fir1_decimation
```

Le facteur de décimation du premier étage de filtrage dans le démodulateur.

Définition à la ligne 105 du fichier libloki-modem.c.

5.21.2.8 demod_fir1_length

```
uint32_t LokiModem::demod_fir1_length
```

Le nombre de coefficients du premier étage de filtrage dans le démodulateur.

Définition à la ligne 107 du fichier libloki-modem.c.

5.21.2.9 demod_fir2_bandwidth

```
uint32_t LokiModem::demod_fir2_bandwidth
```

La bande passante de sortie du second étage de filtrage dans le démodulateur.

Définition à la ligne 113 du fichier libloki-modem.c.

5.21.2.10 demod_fir2_decimation

```
uint32_t LokiModem::demod_fir2_decimation
```

Le facteur de décimation du second étage de filtrage dans le démodulateur.

Définition à la ligne 109 du fichier libloki-modem.c.

5.21.2.11 demod_fir2_length

```
uint32_t LokiModem::demod_fir2_length
```

Le nombre de coefficients du second étage de filtrage dans le démodulateur.

Définition à la ligne 111 du fichier libloki-modem.c.

5.21.2.12 demod_mixerless

```
bool LokiModem::demod_mixerless
```

Le démodulateur fonctionne-t-il par décimation ?

Définition à la ligne 83 du fichier libloki-modem.c.

5.21.2.13 demod_osc_wave_frequency_min

```
uint32_t LokiModem::demod_osc_wave_frequency_min
```

La fréquence minimale de l'oscillateur du démodulateur pour la fréquence d'échantillonnage de référence, en Hz.

Définition à la ligne 103 du fichier libloki-modem.c.

5.21.2.14 demod_power_fractional_width

```
uint32_t LokiModem::demod_power_fractional_width
```

Le nombre de bits de la partie fractionnaire des échantillons en entrée du démodulateur.

Définition à la ligne 93 du fichier libloki-modem.c.

5.21.2.15 demod_power_integer_width

```
uint32_t LokiModem::demod_power_integer_width
```

Le nombre de bits de la partie entière des échantillons en entrée du démodulateur.

Définition à la ligne 91 du fichier libloki-modem.c.

5.21.2.16 demod_power_max

```
float LokiModem::demod_power_max
```

La valeur maximale des échantillons en entrée du démodulateur.

Définition à la ligne 95 du fichier libloki-modem.c.

5.21.2.17 demod_sample_frequency_ref

```
uint32_t LokiModem::demod_sample_frequency_ref
```

La fréquence d'échantillonnage de référence en entrée du démodulateur.

Définition à la ligne 97 du fichier libloki-modem.c.

5.21.2.18 demod_xy_fractional_width

```
uint32_t LokiModem::demod_xy_fractional_width
```

Le nombre de bits de la partie fractionnaire des échantillons de chaque composante X (I) ou Y (Q) du signal démodulé.

Définition à la ligne 117 du fichier libloki-modem.c.

5.21.2.19 demod_xy_integer_width

```
uint32_t LokiModem::demod_xy_integer_width
```

Le nombre de bits de la partie entière des échantillons de chaque composante X (I) ou Y (Q) du signal démodulé.

Définition à la ligne 115 du fichier libloki-modem.c.

5.21.2.20 demod_xy_mask

```
uint32_t LokiModem::demod_xy_mask
```

Le masque binaire pour lire une composante X (I) ou Y (Q) d'un échantillon du signal démodulé.

Définition à la ligne 121 du fichier libloki-modem.c.

5.21.2.21 demod_xy_max

```
float LokiModem::demod_xy_max
```

La valeur maximale d'un échantillon pour une composante X (I) ou Y (Q) du signal démodulé.

Définition à la ligne 125 du fichier libloki-modem.c.

5.21.2.22 demod_xy_sign_mask

```
uint32_t LokiModem::demod_xy_sign_mask
```

Le masque binaire pour lire le bit de signe d'une composante X (I) ou Y (Q) d'un échantillon du signal démodulé.

Définition à la ligne 123 du fichier libloki-modem.c.

5.21.2.23 demod_xy_width

```
uint32_t LokiModem::demod_xy_width
```

Le nombre total de bits d'un échantillon de chaque composante X (I) ou Y (Q) du signal démodulé.

Définition à la ligne 119 du fichier libloki-modem.c.

5.21.2.24 file

```
FILE* LokiModem::file
```

Le fichier représentant le périphérique qui gère le modem.

Définition à la ligne 25 du fichier libloki-modem.c.

5.21.2.25 fileno

```
int LokiModem::fileno
```

Le numéro du fichier représentant le périphérique qui gère le modem.

Définition à la ligne 27 du fichier libloki-modem.c.

5.21.2.26 mod_cycles_per_sample_max

```
uint32_t LokiModem::mod_cycles_per_sample_max
```

Le nombre maximal de périodes d'horloge par échantillon en sortie du modulateur.

Définition à la ligne 51 du fichier libloki-modem.c.

5.21.2.27 mod_cycles_per_sample_ref

```
float LokiModem::mod_cycles_per_sample_ref
```

Le nombre de périodes d'horloge de référence par échantillon en sortie du modulateur.

Définition à la ligne 53 du fichier libloki-modem.c.

5.21.2.28 mod_dc_value_max

```
float LokiModem::mod_dc_value_max
```

Le courant moyen maximal applicable en sortie du modulateur.

Définition à la ligne 73 du fichier libloki-modem.c.

5.21.2.29 mod_dc_value_res

```
float LokiModem::mod_dc_value_res
```

La résolution du réglage du courant moyen en sortie du modulateur.

Définition à la ligne 77 du fichier libloki-modem.c.

5.21.2.30 mod_dc_value_width

```
uint32_t LokiModem::mod_dc_value_width
```

Le nombre de bits pour représenter le courant moyen en sortie du modulateur.

Définition à la ligne 75 du fichier libloki-modem.c.

5.21.2.31 mod_gain_fractional_width

```
uint32_t LokiModem::mod_gain_fractional_width
```

Le nombre de bits de la partie fractionnaire de l'amplitude de sortie du modulateur.

Définition à la ligne 61 du fichier libloki-modem.c.

5.21.2.32 mod_gain_integer_width

```
uint32_t LokiModem::mod_gain_integer_width
```

Le nombre de bits de la partie entière de l'amplitude de sortie du modulateur.

Définition à la ligne 59 du fichier libloki-modem.c.

5.21.2.33 mod_gain_res

```
float LokiModem::mod_gain_res
```

La résolution du réglage d'amplitude en sortie du modulateur.

Définition à la ligne 63 du fichier libloki-modem.c.

5.21.2.34 mod_offset_res

```
float LokiModem::mod_offset_res
```

La résolution du réglage d'offset en sortie du modulateur.

Définition à la ligne 71 du fichier libloki-modem.c.

5.21.2.35 mod_osc1_wave_frequency_min

```
uint32_t LokiModem::mod_osc1_wave_frequency_min
```

La fréquence minimale du premier oscillateur du modulateur pour la fréquence d'échantillonnage de référence, en Hz.

Définition à la ligne 55 du fichier libloki-modem.c.

5.21.2.36 mod_osc2_wave_frequency_min

```
uint32_t LokiModem::mod_osc2_wave_frequency_min
```

La fréquence minimale du second oscillateur du modulateur pour la fréquence d'échantillonnage de référence, en Hz.

Définition à la ligne 57 du fichier libloki-modem.c.

5.21.2.37 mod_pga_gain_max

```
uint32_t LokiModem::mod_pga_gain_max
```

Le gain maximal de l'amplificateur en sortie du modulateur.

Définition à la ligne 81 du fichier libloki-modem.c.

5.21.2.38 mod_pga_gain_width

```
uint32_t LokiModem::mod_pga_gain_width
```

Le nombre de bits pour représenter le gain de l'amplificateur de sortie du modulateur.

Définition à la ligne 79 du fichier libloki-modem.c.

5.21.2.39 mod_phase_amplitude_max

```
float LokiModem::mod_phase_amplitude_max
```

L'amplitude maximale de sortie du modulateur, en mA.

Définition à la ligne 65 du fichier libloki-modem.c.

5.21.2.40 mod_phase_fractional_width

```
uint32_t LokiModem::mod_phase_fractional_width
```

Le nombre de bits de la partie fractionnaire de la valeur de sortie du modulateur.

Définition à la ligne 69 du fichier libloki-modem.c.

5.21.2.41 mod_phase_integer_width

```
uint32_t LokiModem::mod_phase_integer_width
```

Le nombre de bits de la partie entière de la valeur de sortie du modulateur.

Définition à la ligne 67 du fichier libloki-modem.c.

5.21.2.42 mod_sample_frequency_ref

```
uint32_t LokiModem::mod_sample_frequency_ref
```

La fréquence d'échantillonnage de référence en sortie du modulateur.

Définition à la ligne 49 du fichier libloki-modem.c.

5.21.2.43 out

```
LokiFIFO* LokiModem::out
```

La FIFO de sortie des données démodulées.

Définition à la ligne 31 du fichier libloki-modem.c.

5.21.2.44 raw

```
bool LokiModem::raw
```

Conserver les données démodulées brutes (entiers sans mise à l'échelle).

Définition à la ligne 33 du fichier libloki-modem.c.

5.21.2.45 sim_channel

```
uint32_t LokiModem::sim_channel
```

Le numéro de la voie courante pendant une simulation logicielle des capteurs.

Définition à la ligne 41 du fichier libloki-modem.c.

5.21.2.46 sim_disp_phase

```
float LokiModem::sim_disp_phase
```

La phase courante correspondant au déplacement réel pendant une simulation logicielle des capteurs.

Définition à la ligne 45 du fichier libloki-modem.c.

5.21.2.47 sim_osc1_phase

```
float LokiModem::sim_osc1_phase
```

La phase courante du déplacement fictif pendant une simulation logicielle des capteurs.

Définition à la ligne 43 du fichier libloki-modem.c.

5.21.2.48 simulate

```
bool LokiModem::simulate
```

Faut-il simuler les capteurs logiciellement ?

Définition à la ligne 39 du fichier libloki-modem.c.

5.21.2.49 stop_request

```
volatile bool LokiModem::stop_request
```

Indicateur de demande d'arrêt d'acquisition.

Définition à la ligne 29 du fichier libloki-modem.c.

5.21.2.50 stream_channels

```
int32_t LokiModem::stream_channels
```

Un masque indiquant les voies à traiter, -1 pour toutes les voies.

Définition à la ligne 35 du fichier libloki-modem.c.

5.21.2.51 thread

```
pthread_t LokiModem::thread
```

Le thread démarré par LokiModem_start_streaming.

Définition à la ligne 37 du fichier libloki-modem.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-modem.c

5.22 Référence de la structure LokiModemSample

Données retournées par le démodulateur pour un échantillon.

```
#include <libloki-modem.h>
```

Structures de données

— union [union].__unnamed0__

La composante X de l'échantillon.

— union [union].__unnamed1__

La composante Y de l'échantillon.

Champs de données

— uint32_t channel

La voie d'entrée sur lequel l'échantillon a été reçu.

— union

— union

5.22.1 Description détaillée

Données retournées par le démodulateur pour un échantillon.

Définition à la ligne 31 du fichier libloki-modem.h.

5.22.2 Documentation des champs

5.22.2.1 channel

```
uint32_t LokiModemSample::channel
```

La voie d'entrée sur lequel l'échantillon a été reçu.

Définition à la ligne 33 du fichier libloki-modem.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-modem.h

5.23 Référence de la structure LokiMSeed

Représentation d'un formateur de séries temporelles miniSEED.

Graphe de collaboration de LokiMSeed:

Champs de données

— LokiMSeedConfig * config

La configuration d'enregistrement à utiliser.

— LokiMSeedOutputSample * buffer

Buffer d'échantillons préalloué, sans numéro de voie ni numéro de séquence.

— MSRecord * msr

Record MiniSEED préalloué pour toutes les écritures.

5.23.1 Description détaillée

Représentation d'un formateur de séries temporelles miniSEED.

Définition à la ligne 103 du fichier libloki-mseed.c.

5.23.2 Documentation des champs

5.23.2.1 buffer

```
LokiMSeedOutputSample* LokiMSeed::buffer
```

Buffer d'échantillons préalloué, sans numéro de voie ni numéro de séquence.

Définition à la ligne 107 du fichier libloki-mseed.c.

5.23.2.2 config

```
LokiMSeedConfig* LokiMSeed::config
```

La configuration d'enregistrement à utiliser.

Définition à la ligne 105 du fichier libloki-mseed.c.

5.23.2.3 msr

```
MSRecord* LokiMSeed::msr
```

Record MiniSEED préalloué pour toutes les écritures.

Définition à la ligne 109 du fichier libloki-mseed.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-mseed.c

5.24 Référence de la structure LokiMSeedConfig

Configuration d'un formateur de séries temporelles miniSEED.

Champs de données

— bool separate_channels

Faut-il enregistrer chaque voie dans un fichier séparé ?

— char * station

L'identifiant de station de l'interrogateur.

— char * location

L'identifiant de localisation de l'interrogateur.

— char * network

L'identifiant de réseau de l'interrogateur.

— bool raw

Faut-il considérer les valeurs brutes des échantillons ?

— bool steim2

Faut-il utiliser l'algorithme de compression STEIM2 ?

— char * channel_names

Un tableau de nom de voies.

5.24.1 Description détaillée

Configuration d'un formateur de séries temporelles miniSEED.

Définition à la ligne 12 du fichier libloki-mseed.c.

5.24.2 Documentation des champs

5.24.2.1 channel_names

```
char* LokiMSeedConfig::channel_names
```

Un tableau de nom de voies.

Définition à la ligne 26 du fichier libloki-mseed.c.

5.24.2.2 location

```
char* LokiMSeedConfig::location
```

L'identifiant de localisation de l'interrogateur.

Définition à la ligne 18 du fichier libloki-mseed.c.

5.24.2.3 network

```
char* LokiMSeedConfig::network
```

L'identifiant de réseau de l'interrogateur.

Définition à la ligne 20 du fichier libloki-mseed.c.

5.24.2.4 raw

```
bool LokiMSeedConfig::raw
```

Faut-il considérer les valeurs brutes des échantillons ?

Définition à la ligne 22 du fichier libloki-mseed.c.

5.24.2.5 separate_channels

```
bool LokiMSeedConfig::separate_channels
```

Faut-il enregistrer chaque voie dans un fichier séparé ?

Définition à la ligne 14 du fichier libloki-mseed.c.

5.24.2.6 station

```
char* LokiMSeedConfig::station
```

L'identifiant de station de l'interrogateur.

Définition à la ligne 16 du fichier libloki-mseed.c.

5.24.2.7 steim2

```
bool LokiMSeedConfig::steim2
```

Faut-il utiliser l'algorithme de compression STEIM2 ?

Définition à la ligne 24 du fichier libloki-mseed.c.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-mseed.c

5.25 Référence de la structure LokiMSeedInputSample

Type des échantillons attendus par LokiMSeed_work.

Structures de données

— union [union].__unnamed0__

La valeur de l'échantillon.

Champs de données

— uint32_t channel

Le numéro de voie.

— union

5.25.1 Description détaillée

Type des échantillons attendus par LokiMSeed_work.

Définition à la ligne 83 du fichier libloki-mseed.c.

5.25.2 Documentation des champs

5.25.2.1 channel

```
uint32_t LokiMSeedInputSample::channel
```

Le numéro de voie.

Définition à la ligne 85 du fichier libloki-mseed.c.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-mseed.c

5.26 Référence de l'union LokiMSeedOutputSample

Type des échantillons passés à la bibliothèque `libmseed`.

Champs de données

— int32_t i

Valeur d'un échantillon entier.

— float f

Valeur d'un échantillon flottant.

5.26.1 Description détaillée

Type des échantillons passés à la bibliothèque `libmseed`.

Définition à la ligne 95 du fichier libloki-mseed.c.

5.26.2 Documentation des champs

5.26.2.1 f

```
float LokiMSeedOutputSample::f
```

Valeur d'un échantillon flottant.

Définition à la ligne 99 du fichier libloki-mseed.c.

5.26.2.2 i

```
int32_t LokiMSeedOutputSample::i
```

Valeur d'un échantillon entier.

Définition à la ligne 97 du fichier libloki-mseed.c.

La documentation de cette union a été générée à partir du fichier suivant :

— libloki-mseed.c

5.27 Référence de la structure LokiNLMS

Représentattion d'un filtre NLMS.

Champs de données

— float mu

Le facteur d'apprentissage du filtre LMS.

— float alpha

Le coefficient du filtre passe-bas de mise à jour des poids.

— LokiEllipseEquation state

Les coefficients de l'équation de l'ellipse.

— float weights [5]

Les poids de normalisation du filtre LMS.

5.27.1 Description détaillée

Représentattion d'un filtre NLMS.

Définition à la ligne 15 du fichier libloki-nlms.c.

5.27.2 Documentation des champs

5.27.2.1 alpha

```
float LokiNLMS::alpha
```

Le coefficient du filtre passe-bas de mise à jour des poids.

Définition à la ligne 19 du fichier libloki-nlms.c.

5.27.2.2 mu

```
float LokiNLMS::mu
```

Le facteur d'apprentissage du filtre LMS.

Définition à la ligne 17 du fichier libloki-nlms.c.

5.27.2.3 state

```
LokiEllipseEquation LokiNLMS::state
```

Les coefficients de l'équation de l'ellipse.

Définition à la ligne 21 du fichier libloki-nlms.c.

5.27.2.4 weights

```
float LokiNLMS::weights[5]
```

Les poids de normalisation du filtre LMS.

Définition à la ligne 23 du fichier libloki-nlms.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-nlms.c

5.28 Référence de la structure LokiPacket

Structure d'un paquet d'échantillons échangés entre tâches concurrentes.

```
#include <libloki-packet.h>
```

Champs de données

— LokiTimestamp timestamp

La date de réception du paquet, en microsecondes depuis l'epoch Unix.

— char buffer [0]

Les valeurs d'échantillons. La taille et le type de ce tableau son spécifiés ultérieurement.

5.28.1 Description détaillée

Structure d'un paquet d'échantillons échangés entre tâches concurrentes.

Définition à la ligne 16 du fichier libloki-packet.h.

5.28.2 Documentation des champs

5.28.2.1 buffer

```
char LokiPacket::buffer[0]
```

Les valeurs d'échantillons. La taille et le type de ce tableau son spécifiés ultérieurement.

Définition à la ligne 20 du fichier libloki-packet.h.

5.28.2.2 timestamp

```
LokiTimestamp LokiPacket::timestamp
```

La date de réception du paquet, en microsecondes depuis l'epoch Unix.

Définition à la ligne 18 du fichier libloki-packet.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-packet.h

5.29 Référence de la structure LokiRaw

Représentation d'un formateur de données brutes.

Champs de données

— int out

Le numéro du fichier de destination.

5.29.1 Description détaillée

Représentation d'un formateur de données brutes.

Définition à la ligne 15 du fichier libloki-raw.c.

5.29.2 Documentation des champs

5.29.2.1 out

```
int LokiRaw::out
```

Le numéro du fichier de destination.

Définition à la ligne 17 du fichier libloki-raw.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-raw.c

5.30 Référence de la structure LokiRecorder

Représentation d'un enregistreur de séries temporelles.

Graphe de collaboration de LokiRecorder:

Champs de données

— char * filename

Le nom de base des fichiers de destination (sans extension).

— char * filetype

Le format des fichiers de destination ("raw" ou "mseed").

— LokiFormatter * formatter

Le formateur à utiliser pour écrire dans les fichiers.

— size_t max_channels

Le nombre maximal de voies que cet enregistreur peut traiter.

— int32_t channels

Un masque indiquant quelles voies enregistrer.

— size_t channel_count

Le nombre de voies à enregistrer.

— float sampling_frequency

La fréquence d'échantillonnage, en Hz.

— size_t sample_size

La taille d'un échantillon, en octets.

— size_t sample_count

Le nombre total d'échantillons à enregistrer toutes voies confondues, 0 pour enregistrer indéfiniment.

— size_t sample_count_per_file

Le nombre total d'échantillons à enregistrer par fichier toutes voies confondues, 0 pour tout enregistrer dans un seul fichier.

— bool multifile

Faut-il produire plusieurs fichiers (true si sample_count_per_file > 0).

— LokiFIFO * in

La FIFO contenant les données à enregistrer.

— size_t cid

Le numéro de consommateur de cette instance dans la FIFO.

— volatile bool stop_request

Indicateur de demande d'arrêt de l'enregistrement.

— bool is_recording

Indicateur d'enregistrement en cours.

— pthread_mutex_t mutex

Un mutex pour gérer les accès concurrents à l'état de l'enregistrement.

— pthread_cond_t recording_done

Variable conditionnelle activée en fin d'enregistrement.

— pthread_t thread

Le thread qui lit les données de la FIFO et les enregistre.

5.30.1 Description détaillée

Représentation d'un enregistreur de séries temporelles.

Définition à la ligne 20 du fichier libloki-recorder.c.

5.30.2 Documentation des champs

5.30.2.1 channel_count

```
size_t LokiRecorder::channel_count
```

Le nombre de voies à enregistrer.

Définition à la ligne 32 du fichier libloki-recorder.c.

5.30.2.2 channels

```
int32_t LokiRecorder::channels
```

Un masque indiquant quelles voies enregistrer.

Définition à la ligne 30 du fichier libloki-recorder.c.

5.30.2.3 cid

```
size_t LokiRecorder::cid
```

Le numéro de consommateur de cette instance dans la FIFO.

Définition à la ligne 46 du fichier libloki-recorder.c.

5.30.2.4 filename

```
char* LokiRecorder::filename
```

Le nom de base des fichiers de destination (sans extension).

Définition à la ligne 22 du fichier libloki-recorder.c.

5.30.2.5 filetype

```
char* LokiRecorder::filetype
```

Le format des fichiers de destination ("raw" ou "mseed").

Définition à la ligne 24 du fichier libloki-recorder.c.

5.30.2.6 formatter

```
LokiFormatter* LokiRecorder::formatter
```

Le formateur à utiliser pour écrire dans les fichiers.

Définition à la ligne 26 du fichier libloki-recorder.c.

5.30.2.7 in

```
LokiFIFO* LokiRecorder::in
```

La FIFO contenant les données à enregistrer.

Définition à la ligne 44 du fichier libloki-recorder.c.

5.30.2.8 is_recording

```
bool LokiRecorder::is_recording
```

Indicateur d'enregistrement en cours.

Définition à la ligne 50 du fichier libloki-recorder.c.

5.30.2.9 max_channels

```
size_t LokiRecorder::max_channels
```

Le nombre maximal de voies que cet enregistreur peut traiter.

Définition à la ligne 28 du fichier libloki-recorder.c.

5.30.2.10 multifile

```
bool LokiRecorder::multifile
```

Faut-il produire plusieurs fichiers (true si sample_count_per_file $>$ 0).

Définition à la ligne 42 du fichier libloki-recorder.c.

5.30.2.11 mutex

```
pthread_mutex_t LokiRecorder::mutex
```

Un mutex pour gérer les accès concurrents à l'état de l'enregistrement.

Définition à la ligne 52 du fichier libloki-recorder.c.

5.30.2.12 recording_done

```
pthread_cond_t LokiRecorder::recording_done
```

Variable conditionnelle activée en fin d'enregistrement.

Définition à la ligne 54 du fichier libloki-recorder.c.

5.30.2.13 sample_count

```
size_t LokiRecorder::sample_count
```

Le nombre total d'échantillons à enregistrer toutes voies confondues, 0 pour enregistrer indéfiniment.

Définition à la ligne 38 du fichier libloki-recorder.c.

5.30.2.14 sample_count_per_file

```
size_t LokiRecorder::sample_count_per_file
```

Le nombre total d'échantillons à enregistrer par fichier toutes voies confondues, 0 pour tout enregistrer dans un seul fichier.

Définition à la ligne 40 du fichier libloki-recorder.c.

5.30.2.15 sample_size

```
size_t LokiRecorder::sample_size
```

La taille d'un échantillon, en octets.

Définition à la ligne 36 du fichier libloki-recorder.c.

5.30.2.16 sampling_frequency

```
float LokiRecorder::sampling_frequency
```

La fréquence d'échantillonnage, en Hz.

Définition à la ligne 34 du fichier libloki-recorder.c.

5.30.2.17 stop_request

```
volatile bool LokiRecorder::stop_request
```

Indicateur de demande d'arrêt de l'enregistrement.

Définition à la ligne 48 du fichier libloki-recorder.c.

5.30.2.18 thread

```
pthread_t LokiRecorder::thread
```

Le thread qui lit les données de la FIFO et les enregistre.

Définition à la ligne 56 du fichier libloki-recorder.c.

La documentation de cette structure a été générée à partir du fichier suivant :
- libloki-recorder.c

5.31 Référence de la structure LokiRLS

Représentation d'un filtre RLS.

Champs de données

- float lambda

  Le facteur d'oubli du filtre.

- float lambda_inv

  L'inverse du facteur d'oubli du filtre.

- float P [5][5]

  L'inverse de la matrice de covariance du signal d'entrée.

- LokiEllipseEquation state

  Les coefficients de l'équation de l'ellipse.

5.31.1 Description détaillée

Représentation d'un filtre RLS.

Définition à la ligne 12 du fichier libloki-rls.c.

5.31.2 Documentation des champs

5.31.2.1 lambda

```
float LokiRLS::lambda
```

Le facteur d'oubli du filtre.

Définition à la ligne 14 du fichier libloki-rls.c.

5.31.2.2 lambda_inv

```
float LokiRLS::lambda_inv
```

L'inverse du facteur d'oubli du filtre.

Définition à la ligne 16 du fichier libloki-rls.c.

5.31.2.3 P

```
float LokiRLS::P[5][5]
```

L'inverse de la matrice de covariance du signal d'entrée.

Définition à la ligne 18 du fichier libloki-rls.c.

5.31.2.4 state

```
LokiEllipseEquation LokiRLS::state
```

Les coefficients de l'équation de l'ellipse.

Définition à la ligne 20 du fichier libloki-rls.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-rls.c

5.32 Référence de la structure LokiSocketServer

Représentation d'un serveur de données.

Graphe de collaboration de LokiSocketServer:

Champs de données

— char * filename

Le nom de fichier à utiliser pour représenter la socket Unix dans le système de fichiers.

— LokiFIFO * in

La FIFO contenant les données à transmettre.

— size_t cid

Le numéro de consommateur de ce serveur dans sa FIFO d'entrée.

— volatile bool stop_request

Indicateur de demande d'arrêt du serveur.

— bool is_streaming

Le serveur est-il en service ?

— pthread_t receiver_thread

Le thread de réception des données.

— pthread_t server_thread

Le thread qui gère les connexions des clients et transmet les données.

— pthread_mutex_t mutex

Un mutex pour gérer les accès concurrents au paquet de données courant.

— void * data

Le dernier paquet de données reçu.

— size_t data_size

La taille totale d'un paquet de données, en octets.

— bool has_data

Indicateur de présence d'un nouveau paquet à transmettre.

— pthread_cond_t has_data_cond

Variable conditionnelle pour débloquer le thread serveur lorsqu'un nouveau paquet est disponible.

5.32.1 Description détaillée

Représentation d'un serveur de données.

Définition à la ligne 23 du fichier libloki-socket-server.c.

5.32.2 Documentation des champs

5.32.2.1 cid

```
size_t LokiSocketServer::cid
```

Le numéro de consommateur de ce serveur dans sa FIFO d'entrée.

Définition à la ligne 29 du fichier libloki-socket-server.c.

5.32.2.2 data

```
void* LokiSocketServer::data
```

Le dernier paquet de données reçu.

Définition à la ligne 41 du fichier libloki-socket-server.c.

5.32.2.3 data_size

```
size_t LokiSocketServer::data_size
```

La taille totale d'un paquet de données, en octets.

Définition à la ligne 43 du fichier libloki-socket-server.c.

5.32.2.4 filename

```
char* LokiSocketServer::filename
```

Le nom de fichier à utiliser pour représenter la socket Unix dans le système de fichiers.

Définition à la ligne 25 du fichier libloki-socket-server.c.

5.32.2.5 has_data

```
bool LokiSocketServer::has_data
```

Indicateur de présence d'un nouveau paquet à transmettre.

Définition à la ligne 45 du fichier libloki-socket-server.c.

5.32.2.6 has_data_cond

```
pthread_cond_t LokiSocketServer::has_data_cond
```

Variable conditionnelle pour débloquer le thread serveur lorsqu'un nouveau paquet est disponible.

Définition à la ligne 47 du fichier libloki-socket-server.c.

5.32.2.7 in

```
LokiFIFO* LokiSocketServer::in
```

La FIFO contenant les données à transmettre.

Définition à la ligne 27 du fichier libloki-socket-server.c.

5.32.2.8 is_streaming

```
bool LokiSocketServer::is_streaming
```

Le serveur est-il en service ?

Définition à la ligne 33 du fichier libloki-socket-server.c.

5.32.2.9 mutex

```
pthread_mutex_t LokiSocketServer::mutex
```

Un mutex pour gérer les accès concurrents au paquet de données courant.

Définition à la ligne 39 du fichier libloki-socket-server.c.

5.32.2.10 receiver_thread

```
pthread_t LokiSocketServer::receiver_thread
```

Le thread de réception des données.

Définition à la ligne 35 du fichier libloki-socket-server.c.

5.32.2.11 server_thread

```
pthread_t LokiSocketServer::server_thread
```

Le thread qui gère les connexions des clients et transmet les données.

Définition à la ligne 37 du fichier libloki-socket-server.c.

5.32.2.12 stop_request

```
volatile bool LokiSocketServer::stop_request
```

Indicateur de demande d'arrêt du serveur.

Définition à la ligne 31 du fichier libloki-socket-server.c.

La documentation de cette structure a été générée à partir du fichier suivant :
— libloki-socket-server.c

5.33 Référence de la structure LokiSystem

Représentation d'un système d'acquisition et de traitement.

Graphe de collaboration de LokiSystem:

Champs de données

— LokiSystemConfig * config

La configuration de ce système.

— size_t channels

Le nombre de voies prises en charge par ce système.

— bool has_own_modem

Ce système a-t-il construit sa propre instance de LokiModem ?

— LokiModem * modem

Le modem utilisé par ce système.

— LokiDisplacement ** disp

Un tableau de calculateurs de déplacement.

— volatile bool stop_request

Indicateur de demande d'arrêt de l'acquisition.

— bool is_streaming

Une acquisition est-elle en cours ?

— pthread_t thread

Le thread de traitement.

— LokiFIFO * in

La FIFO des données démodulées (alimentée par le modem).

— LokiFIFO * out

La FIFO des déplacements (alimentée par ce système).

— size_t cid

Le numéro de consommateur de ce système pour la FIFO des données démodulées.

— pthread_mutex_t mutex

Un mutex pour gérer les accès concurrents au calculateur du déplacement.

5.33.1 Description détaillée

Représentation d'un système d'acquisition et de traitement.

Définition à la ligne 268 du fichier libloki-system.c.

5.33.2 Documentation des champs

5.33.2.1 channels

```
size_t LokiSystem::channels
```

Le nombre de voies prises en charge par ce système.

Définition à la ligne 272 du fichier libloki-system.c.

5.33.2.2 cid

```
size_t LokiSystem::cid
```

Le numéro de consommateur de ce système pour la FIFO des données démodulées.

Définition à la ligne 290 du fichier libloki-system.c.

5.33.2.3 config

```
LokiSystemConfig* LokiSystem::config
```

La configuration de ce système.

Définition à la ligne 270 du fichier libloki-system.c.

5.33.2.4 disp

```
LokiDisplacement** LokiSystem::disp
```

Un tableau de calculateurs de déplacement.

Définition à la ligne 278 du fichier libloki-system.c.

5.33.2.5 has_own_modem

```
bool LokiSystem::has_own_modem
```

Ce système a-t-il construit sa propre instance de LokiModem ?

Définition à la ligne 274 du fichier libloki-system.c.

5.33.2.6 in

```
LokiFIFO* LokiSystem::in
```

La FIFO des données démodulées (alimentée par le modem).

Définition à la ligne 286 du fichier libloki-system.c.

5.33.2.7 is_streaming

```
bool LokiSystem::is_streaming
```

Une acquisition est-elle en cours ?

Définition à la ligne 282 du fichier libloki-system.c.

5.33.2.8 modem

```
LokiModem* LokiSystem::modem
```

Le modem utilisé par ce système.

Définition à la ligne 276 du fichier libloki-system.c.

5.33.2.9 mutex

```
pthread_mutex_t LokiSystem::mutex
```

Un mutex pour gérer les accès concurrents au calculateur du déplacement.

Définition à la ligne 292 du fichier libloki-system.c.

5.33.2.10 out

```
LokiFIFO* LokiSystem::out
```

La FIFO des déplacements (alimentée par ce système).

Définition à la ligne 288 du fichier libloki-system.c.

5.33.2.11 stop_request

```
volatile bool LokiSystem::stop_request
```

Indicateur de demande d'arrêt de l'acquisition.

Définition à la ligne 280 du fichier libloki-system.c.

5.33.2.12 thread

```
pthread_t LokiSystem::thread
```

Le thread de traitement.

Définition à la ligne 284 du fichier libloki-system.c.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-system.c

5.34 Référence de la structure LokiSystemConfig

Les paramètres de configuration d'un système d'acquisition et de traitement.

Champs de données

— bool simulate

Faut-il simuler les capteurs de déplacement ?

— float skip

Le délai d'attente avant d'alimenter la FIFO de sortie, en secondes.

— float wavelength

La longueur d'onde de la diode laser, en nm.

— unsigned reflections

Le nombre de réflexions supposé.

— float cavity_index

L'indice optique de la cavité.

— char distance_unit

Unité de distance.

— LokiEllipseEstimatorInterface ∗ estimator_impl

Le filtre adaptatif pour l'estimation des paramètres de l'ellipse.

— unsigned estimator_throttle

La périodicité de mise à jour des paramètres de l'ellipse, en échantillons.

— float estimator_initial_radius

Le rayon initial à utiliser dans le filtre d'estimation.

— float ∗ estimator_params

Le tableau des paramètres de l'estimateur.

— float filter_bandwidth

La bande passante du filtre d'élimination du déplacement fictif, en Hz.

— float filter_attenuation

L'atténuation du filtre d'élimination du déplacement fictif, en dB.

— size_t filter_length

Le nombre de coefficients du filtre d'élimination du déplacement fictif.

— size_t filter_decimation

Le facteur de décimation du filtre d'élimination du déplacement fictif.

— float time_constants [3]

Les constantes de temps des filtres de lissage des paramètres de l'ellipse.

5.34.1 Description détaillée

Les paramètres de configuration d'un système d'acquisition et de traitement.

Définition à la ligne 22 du fichier libloki-system.c.

5.34.2 Documentation des champs

5.34.2.1 cavity_index

```
float LokiSystemConfig::cavity_index
```

L'indice optique de la cavité.

Définition à la ligne 32 du fichier libloki-system.c.

5.34.2.2 distance_unit

```
char LokiSystemConfig::distance_unit
```

Unité de distance.

Définition à la ligne 34 du fichier libloki-system.c.

5.34.2.3 estimator_impl

```
LokiEllipseEstimatorInterface* LokiSystemConfig::estimator_impl
```

Le filtre adaptatif pour l'estimation des paramètres de l'ellipse.

Définition à la ligne 36 du fichier libloki-system.c.

5.34.2.4 estimator_initial_radius

```
float LokiSystemConfig::estimator_initial_radius
```

Le rayon initial à utiliser dans le filtre d'estimation.

Définition à la ligne 40 du fichier libloki-system.c.

5.34.2.5 estimator_params

```
float* LokiSystemConfig::estimator_params
```

Le tableau des paramètres de l'estimateur.

Définition à la ligne 42 du fichier libloki-system.c.

5.34.2.6 estimator_throttle

```
unsigned LokiSystemConfig::estimator_throttle
```

La périodicité de mise à jour des paramètres de l'ellipse, en échantillons.

Définition à la ligne 38 du fichier libloki-system.c.

5.34.2.7 filter_attenuation

```
float LokiSystemConfig::filter_attenuation
```

L'atténuation du filtre d'élimination du déplacement fictif, en dB.

Définition à la ligne 46 du fichier libloki-system.c.

5.34.2.8 filter_bandwidth

```
float LokiSystemConfig::filter_bandwidth
```

La bande passante du filtre d'élimination du déplacement fictif, en Hz.

Définition à la ligne 44 du fichier libloki-system.c.

5.34.2.9 filter_decimation

```
size_t LokiSystemConfig::filter_decimation
```

Le facteur de décimation du filtre d'élimination du déplacement fictif.

Définition à la ligne 50 du fichier libloki-system.c.

5.34.2.10 filter_length

```
size_t LokiSystemConfig::filter_length
```

Le nombre de coefficients du filtre d'élimination du déplacement fictif.

Définition à la ligne 48 du fichier libloki-system.c.

5.34.2.11 reflections

```
unsigned LokiSystemConfig::reflections
```

Le nombre de réflexions supposé.

Définition à la ligne 30 du fichier libloki-system.c.

5.34.2.12 simulate

```
bool LokiSystemConfig::simulate
```

Faut-il simuler les capteurs de déplacement ?

Définition à la ligne 24 du fichier libloki-system.c.

5.34.2.13 skip

```
float LokiSystemConfig::skip
```

Le délai d'attente avant d'alimenter la FIFO de sortie, en secondes.

Définition à la ligne 26 du fichier libloki-system.c.

5.34.2.14 time_constants

```
float LokiSystemConfig::time_constants[3]
```

Les constantes de temps des filtres de lissage des paramètres de l'ellipse.

Définition à la ligne 52 du fichier libloki-system.c.

5.34.2.15 wavelength

```
float LokiSystemConfig::wavelength
```

La longueur d'onde de la diode laser, en nm.

Définition à la ligne 28 du fichier libloki-system.c.

La documentation de cette structure a été générée à partir du fichier suivant :
- libloki-system.c

5.35 Référence de la structure LokiSystemSample

Représentation d'un échantillon en sortie d'une instance de LokiSystem.

`#include <libloki-system.h>`

Champs de données

- uint32_t channel

  La voie d'entrée sur lequel l'échantillon a été reçu.

- float value

  La valeur de l'échantillon.

- float x_norm

  La coordonnée X du point courant de l'ellipse, ramené sur le cercle unité.

- float y_norm

  La coordonnée Y du point courant de l'ellipse, ramené sur le cercle unité.

5.35.1 Description détaillée

Représentation d'un échantillon en sortie d'une instance de LokiSystem.

Définition à la ligne 21 du fichier libloki-system.h.

5.35.2 Documentation des champs

5.35.2.1 channel

`uint32_t LokiSystemSample::channel`

La voie d'entrée sur lequel l'échantillon a été reçu.

Définition à la ligne 23 du fichier libloki-system.h.

5.35.2.2 value

`float LokiSystemSample::value`

La valeur de l'échantillon.

Définition à la ligne 25 du fichier libloki-system.h.

5.35.2.3 x_norm

```
float LokiSystemSample::x_norm
```

La coordonnée X du point courant de l'ellipse, ramené sur le cercle unité.

Définition à la ligne 27 du fichier libloki-system.h.

5.35.2.4 y_norm

```
float LokiSystemSample::y_norm
```

La coordonnée Y du point courant de l'ellipse, ramené sur le cercle unité.

Définition à la ligne 29 du fichier libloki-system.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— libloki-system.h

Chapitre 6

Documentation des fichiers

6.1 Référence du fichier libloki-class.h

```
#include "libloki-common.h"
```

Graphe des dépendances par inclusion de libloki-class.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Macros

- #define LokiClass_interface_name(prefix)

  Construit un nom d'interface pour le préfixe donné.

- #define LokiClass_class_name(prefix)

  Construit un nom de classe pour le préfixe donné.

- #define LokiClass_function_name(prefix, name)

  Construit un nom de fonction pour une méthode d'une classe.

- #define LokiClass_interface(prefix, methods)

  Déclare une interface.

- #define LokiClass_attribute(decl)

  Dans une interface, déclare un attribut.

- #define LokiClass_method(prefix, name)

  Dans une interface, déclare une méthode.

- #define LokiClass_declare_implementation(superclass, prefix)

  Déclare une classe qui implémente une interface donnée.

- #define LokiClass_implementation(superclass, prefix, impls)

  Définit le contenu d'une classe qui implémente une interface donnée.

— #define LokiClass_set_attribute(name, value)

Dans une implémentation, affecte une valeur à un attribut.

— #define LokiClass_define_method(prefix, name)

Dans une implémentation, associe une fonction à une méthode.

6.1.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-class.h.

6.2 libloki-class.h

Aller à la documentation de ce fichier.

```
00001
00065
00067
00068 #ifndef LIBLOKI_CLASS_H_
00069 #define LIBLOKI_CLASS_H_
00070
00071 #include "libloki-common.h"
00072
00082 #define LokiClass_interface_name(prefix) CAT(prefix, Interface)
00083
00093 #define LokiClass_class_name(prefix) CAT(prefix, Class)
00094
00105 #define LokiClass_function_name(prefix, name) CAT3(prefix, _, name)
00106
00112 #define LokiClass_interface(prefix, methods) \
00113     typedef struct {                         \
00114         methods                              \
00115     } LokiClass_interface_name(prefix)
00116
00124 #define LokiClass_attribute(decl) \
00125     decl;
00126
00132 #define LokiClass_method(prefix, name) \
00133     typeof(LokiClass_function_name(prefix, name)) *name;
00134
00140 #define LokiClass_declare_implementation(superclass, prefix) \
00141     extern LokiClass_interface_name(superclass) LokiClass_class_name(prefix)
00142
00149 #define LokiClass_implementation(superclass, prefix, impls)               \
00150     LokiClass_interface_name(superclass) LokiClass_class_name(prefix) = { \
00151         impls                                                             \
00152     }
00153
00159 #define LokiClass_set_attribute(name, value) \
00160     .name = value,
00161
00167 #define LokiClass_define_method(prefix, name) \
00168     .name = (typeof(LokiClass_class_name(prefix).name))LokiClass_function_name(prefix, name),
00169
00170 #endif
00171
```

6.3 Référence du fichier libloki-common.h

Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Macros

— #define LEN(a)

Retourner le nombre d'éléments d'un tableau.

— #define CAT(a, b)

Concaténer deux identifiants.

— #define CAT3(a, b, c)

Concaténer deux identifiants en insérant un séparateur.

6.3.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-common.h.

6.4 libloki-common.h

Aller à la documentation de ce fichier.

```
00001
00003
00006
00007 #ifndef LIBLOKI_COMMON_H_
00008 #define LIBLOKI_COMMON_H_
00009
00015 #define LEN(a) (sizeof(a) / sizeof(a[0]))
00016
00023 #define CAT(a, b) a##b
00024
00032 #define CAT3(a, b, c) a##b##c
00033
00034 #endif
00035
00038
```

6.5 Référence du fichier libloki-consumer.c

```
#include "libloki.h"
#include <math.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <pthread.h>
#include <signal.h>
#include <inttypes.h>
```

Graphe des dépendances par inclusion de libloki-consumer.c:

Structures de données

— struct LokiConsumer

Définition d'un consommateur.

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Fonctions

— LokiConsumer * LokiConsumer_new (void)

Construire un consommateur.

— static void * LokiConsumer_work (void *ptr)

Lire les données de la FIFO.

— int LokiConsumer_start (LokiConsumer *self, LokiFIFO *in, size_t cid)

Démarrer un consommateur.

— void LokiConsumer_stop (LokiConsumer *self)

Arrêter un consommateur.

— void LokiConsumer_del (LokiConsumer *self)

Détruire un consommateur.

6.5.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-consumer.c.

6.5.2 Documentation des macros

6.5.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Définition à la ligne 3 du fichier libloki-consumer.c.

6.6 libloki-consumer.c

Aller à la documentation de ce fichier.

```
00001
00003 #define _GNU_SOURCE
00004
00005 #include "libloki.h"
00006
00007 #include <math.h>
00008 #include <stdlib.h>
00009 #include <stdbool.h>
00010 #include <stdio.h>
00011 #include <string.h>
00012 #include <pthread.h>
00013 #include <signal.h>
00014 #include <inttypes.h>
00015
00018
00020 struct LokiConsumer {
00022     LokiFIFO *in;
00024     size_t cid;
00026     volatile bool stop_request;
00028     pthread_t thread;
00029 };
00030
00031 LokiConsumer *LokiConsumer_new(void) {
00032     LokiConsumer *self = calloc(1, sizeof(LokiConsumer));
00033     if (!self) {
00034         return NULL;
00035     }
00036
00037     return self;
00038 }
00039
00047 static void *LokiConsumer_work(void *ptr) {
00048     LokiConsumer *self = ptr;
00049
00050     pthread_setname_np(self->thread, "Consumer");
00051
00052     while (!self->stop_request) {
00053         if (!LokiFIFO_get_consumer_item(self->in, self->cid)) {
00054             break;
00055         }
00056         LokiFIFO_pull(self->in, self->cid);
00057     }
00058 }
00059
00060 int LokiConsumer_start(LokiConsumer *self, LokiFIFO *in, size_t cid) {
```

```
00061     self->in = in;
00062     self->cid = cid;
00063     self->stop_request = false;
00064     return pthread_create(&self->thread, NULL, LokiConsumer_work, self);
00065 }
00066
00067 void LokiConsumer_stop(LokiConsumer *self) {
00068     if (self->stop_request) {
00069         return;
00070     }
00071     self->stop_request = true;
00072     LokiFIFO_abort(self->in);
00073     pthread_join(self->thread, NULL);
00074 }
00075
00076 void LokiConsumer_del(LokiConsumer *self) {
00077     if (!self) {
00078         return;
00079     }
00080
00081     free(self);
00082 }
00083
```

6.7 Référence du fichier libloki-consumer.h

`#include "libloki-fifo.h"`
Graphe des dépendances par inclusion de libloki-consumer.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiConsumer LokiConsumer

Le type consommateur.

Fonctions

— LokiConsumer * LokiConsumer_new (void)

Construire un consommateur.

— int LokiConsumer_start (LokiConsumer *self, LokiFIFO *in, size_t cid)

Démarrer un consommateur.

— void LokiConsumer_stop (LokiConsumer *self)

Arrêter un consommateur.

— void LokiConsumer_del (LokiConsumer *self)

Détruire un consommateur.

6.7.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-consumer.h.

6.8 libloki-consumer.h

Aller à la documentation de ce fichier.

```
00001
00013
00015
00016 #ifndef LIBLOKI_CONSUMER_H_
00017 #define LIBLOKI_CONSUMER_H_
00018
00019 #include "libloki-fifo.h"
00020
00022 typedef struct LokiConsumer LokiConsumer;
00023
00028 LokiConsumer *LokiConsumer_new(void);
00029
00040 int LokiConsumer_start(LokiConsumer *self, LokiFIFO *in, size_t cid);
00041
00049 void LokiConsumer_stop(LokiConsumer *self);
00050
00057 void LokiConsumer_del(LokiConsumer *self);
00058
00059 #endif
00060
```

6.9 Référence du fichier libloki-displacement.c

```
#include "libloki.h"
#include <stdlib.h>
#include <string.h>
#include <math.h>
```

Graphe des dépendances par inclusion de libloki-displacement.c:

Structures de données

— struct LokiDisplacement

Représentation d'un calculateur de déplacement.

Fonctions

— LokiDisplacement * LokiDisplacement_new (LokiEllipseEstimator *estimator, LokiFIRFilter *filter, unsigned throttle, float scale, float tau[static 3], float sampling_frequency)

Construire un calculateur de déplacement.

— LokiDisplacement * LokiDisplacement_clone (const LokiDisplacement *other)

Cloner une instance de LokiDisplacement.

— bool LokiDisplacement_work (LokiDisplacement *self, float x, float y, float *x_norm, float *y_norm, float *out)

Calculer le déplacement courant.

— void LokiDisplacement_reset (LokiDisplacement ∗self)

Remettre à zéro le déplacement.

— LokiEllipse ∗ LokiDisplacement_get_ellipse (LokiDisplacement ∗self)

Retourner la dernière ellipse calculée par cette instance.

— void LokiDisplacement_del (LokiDisplacement ∗self)

Détruire une instance de LokiDisplacement.

6.9.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-displacement.c.

6.10 libloki-displacement.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki.h"
00003
00004 #include <stdlib.h>
00005 #include <string.h>
00006 #include <math.h>
00007
00010
00012 struct LokiDisplacement {
00014     LokiEllipse ellipse;
00016     LokiEllipseEstimator *estimator;
00018     LokiFIRFilter *filter;
00020     unsigned estimator_throttle;
00022     unsigned estimator_cnt;
00024     float phi_prev;
00026     float phi_filt;
00028     float phi_ref;
00030     int32_t phi_turns;
00032     float alpha_prev;
00034     float alpha_ref;
00036     float scale;
00038     float iir_a[3];
00040     float iir_b[3];
00041 };
00042
00043 LokiDisplacement *LokiDisplacement_new(LokiEllipseEstimator *estimator, LokiFIRFilter *filter,
      unsigned throttle, float scale, float tau[static 3], float sampling_frequency) {
00044     LokiDisplacement *self = calloc(1, sizeof(LokiDisplacement));
00045     if (!self) {
00046         return NULL;
00047     }
00048
00049     self->estimator_throttle = throttle;
00050     self->estimator_cnt      = 0;
00051     self->scale              = scale;
00052     self->estimator          = estimator;
00053     self->filter             = filter;
00054
```

```
00055     for (size_t i = 0; i < 3; i ++) {
00056         const float ts = tau[i] * sampling_frequency;
00057         self->iir_a[i] = ts   / (ts + 1);
00058         self->iir_b[i] = 1.0f / (ts + 1);
00059     }
00060
00061     return self;
00062 }
00063
00064 LokiDisplacement *LokiDisplacement_clone(const LokiDisplacement *other) {
00065     LokiDisplacement *self = calloc(1, sizeof(LokiDisplacement));
00066     if (!self) {
00067         return NULL;
00068     }
00069
00070     memcpy(self, other, sizeof(LokiDisplacement));
00071
00072     self->estimator = LokiEllipseEstimator_clone(other->estimator);
00073     self->filter    = LokiFIRFilter_clone(other->filter);
00074     if (!self->estimator || !self->filter) {
00075         LokiDisplacement_del(self);
00076         return NULL;
00077     }
00078
00079     return self;
00080 }
00081
00082 bool LokiDisplacement_work(LokiDisplacement *self, float x, float y, float *x_norm, float *y_norm,
      float *out) {
00083     // Calcul des paramètres de l'ellipse et de l'angle courant.
00084     self->estimator_cnt ++;
00085     if (self->estimator_cnt >= self->estimator_throttle) {
00086         LokiEllipseEstimator_work(self->estimator, x, y);
00087         self->estimator_cnt = 0;
00088     }
00089
00090     LokiEllipse ellipse;
00091     LokiEllipse_set_from_coeffs(&ellipse, LokiEllipseEstimator_get_state(self->estimator));
00092
00093     // Éviter les sauts entre -pi/2 et +pi/2 lorsque l'ellipse est presque verticale.
00094     float dalpha = ellipse.alpha - self->alpha_prev;
00095     self->alpha_prev = ellipse.alpha;
00096
00097     if (dalpha > M_PI_2 || dalpha < -M_PI_2) {
00098         self->alpha_ref = M_PI - self->alpha_ref;
00099     }
00100
00101     ellipse.alpha += self->alpha_ref;
00102
00103     // Lisser les paramètres de l'ellipse.
00104     const float *const a = self->iir_a;
00105     const float *const b = self->iir_b;
00106     self->ellipse.A     = a[0] * self->ellipse.A     + b[0] * ellipse.A;
00107     self->ellipse.B     = a[0] * self->ellipse.B     + b[0] * ellipse.B;
00108     self->ellipse.xc    = a[1] * self->ellipse.xc    + b[1] * ellipse.xc;
00109     self->ellipse.yc    = a[1] * self->ellipse.yc    + b[1] * ellipse.yc;
00110     self->ellipse.alpha = a[2] * self->ellipse.alpha + b[2] * ellipse.alpha;
00111
00112     // Calculer la phase courante.
00113     float phi = LokiEllipse_get_angle(&self->ellipse, x, y, x_norm, y_norm);
00114
00115     // Dérouler la phase pour éviter les discontinuités au passage de -pi à +pi.
00116     float dphi = phi - self->phi_prev;
00117     self->phi_prev = phi;
00118
00119     if (dphi > M_PI) {
00120         self->phi_turns --;
00121     }
00122     else if (dphi < -M_PI) {
00123         self->phi_turns ++;
00124     }
00125
00126     phi += M_PI * 2 * self->phi_turns;
00127
00128     // Filtrage passe-bas pour éliminer le déplacement fictif.
00129     if (LokiFIRFilter_work(self->filter, phi, &phi)) {
00130         // Mémoriser la dernière valeur filtrée, pour utilisation comme
00131         // référence dans LokiDisplacement_reset.
00132         self->phi_filt = phi;
00133         // Calculer le déplacement et affecter l'échantillon de destination.
00134         *out = (phi - self->phi_ref) * self->scale;
00135         return true;
00136     }
00137
00138     return false;
00139 }
00140
```

```
00141 void LokiDisplacement_reset(LokiDisplacement *self) {
00142     self->phi_turns = 0;
00143     self->phi_ref   = self->phi_filt;
00144     LokiFIRFilter_reset(self->filter);
00145 }
00146
00147 LokiEllipse *LokiDisplacement_get_ellipse(LokiDisplacement *self) {
00148     return &self->ellipse;
00149 }
00150
00151 void LokiDisplacement_del(LokiDisplacement *self) {
00152     if (!self) {
00153         return;
00154     }
00155
00156     if (self->estimator) {
00157         LokiEllipseEstimator_del(self->estimator);
00158     }
00159
00160     if (self->filter) {
00161         LokiFIRFilter_del(self->filter);
00162     }
00163
00164     free(self);
00165 }
00166
```

6.11 Référence du fichier libloki-displacement.h

```
#include "libloki-fir.h"
#include "libloki-ellipse-estimator.h"
```

Graphe des dépendances par inclusion de libloki-displacement.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiDisplacement LokiDisplacement

Représentation d'un calculateur de déplacement.

Fonctions

— LokiDisplacement * LokiDisplacement_new (LokiEllipseEstimator *estimator, LokiFIRFilter *filter, unsigned throttle, float scale, float tau[static 3], float sampling_frequency)

Construire un calculateur de déplacement.

— LokiDisplacement * LokiDisplacement_clone (const LokiDisplacement *other)

Cloner une instance de LokiDisplacement.

— bool LokiDisplacement_work (LokiDisplacement *self, float x, float y, float *x_norm, float *y_norm, float *out)

Calculer le déplacement courant.

— void LokiDisplacement_reset (LokiDisplacement *self)

Remettre à zéro le déplacement.

— LokiEllipse * LokiDisplacement_get_ellipse (LokiDisplacement *self)

Retourner la dernière ellipse calculée par cette instance.

— void LokiDisplacement_del (LokiDisplacement *self)

Détruire une instance de LokiDisplacement.

6.11.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-displacement.h.

6.12 libloki-displacement.h

Aller à la documentation de ce fichier.

```
00001
00009
00011
00012 #ifndef LIBLOKI_DISPLACEMENT_H_
00013 #define LIBLOKI_DISPLACEMENT_H_
00014
00015 #include "libloki-fir.h"
00016 #include "libloki-ellipse-estimator.h"
00017
00019 typedef struct LokiDisplacement LokiDisplacement;
00020
00037 LokiDisplacement *LokiDisplacement_new(LokiEllipseEstimator *estimator, LokiFIRFilter *filter,
      unsigned throttle, float scale, float tau[static 3], float sampling_frequency);
00038
00044 LokiDisplacement *LokiDisplacement_clone(const LokiDisplacement *other);
00045
00059 bool LokiDisplacement_work(LokiDisplacement *self, float x, float y, float *x_norm, float *y_norm,
      float *out);
00060
00065 void LokiDisplacement_reset(LokiDisplacement *self);
00066
00072 LokiEllipse *LokiDisplacement_get_ellipse(LokiDisplacement *self);
00073
00078 void LokiDisplacement_del(LokiDisplacement *self);
00079
00080 #endif
00081
```

6.13 Référence du fichier libloki-ellipse-estimator.c

```
#include <stdlib.h>
#include <string.h>
#include "libloki-ellipse-estimator.h"
```

Graphe des dépendances par inclusion de libloki-ellipse-estimator.c:

Structures de données

— struct LokiEllipseEstimator

Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Fonctions

— LokiEllipseEstimator * LokiEllipseEstimator_new (float initial_radius, const float params[ ])

Construire un estimateur.

— LokiEllipseEstimator * LokiEllipseEstimator_new_with_implementation (const LokiEllipseEstimatorInterface *impl, float initial_radius, const float params[ ])

Construire un estimateur avec une implémentation donnée.

— LokiEllipseEstimator * LokiEllipseEstimator_clone (const LokiEllipseEstimator *other)

Construire un clone d'un estimateur.

— void LokiEllipseEstimator_work (LokiEllipseEstimator *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— float * LokiEllipseEstimator_get_state (LokiEllipseEstimator *self)

Retourner l'état courant de l'estimateur.

— void LokiEllipseEstimator_del (LokiEllipseEstimator *self)

Détruire un estimateur.

6.13.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-ellipse-estimator.c.

6.14 libloki-ellipse-estimator.c

Aller à la documentation de ce fichier.

```
00001
00002 #include <stdlib.h>
00003 #include <string.h>
00004
00005 #include "libloki-ellipse-estimator.h"
00006
00009
00011 struct LokiEllipseEstimator {
00013     void *instance;
00019     LokiEllipseEstimatorInterface impl;
00020 };
00021
00022 LokiEllipseEstimator *LokiEllipseEstimator_new(float initial_radius, const float params[]) {
00023     return NULL;
00024 }
00025
00026 LokiEllipseEstimator *LokiEllipseEstimator_new_with_implementation(const LokiEllipseEstimatorInterface
      *impl, float initial_radius, const float params[]) {
```

```
00027     LokiEllipseEstimator *self = calloc(1, sizeof(LokiEllipseEstimator));
00028     if (!self) {
00029         return NULL;
00030     }
00031
00032     self->instance = impl->new(initial_radius, params);
00033     if (!self->instance) {
00034         LokiEllipseEstimator_del(self);
00035         return NULL;
00036     }
00037
00038     memcpy(&self->impl, impl, sizeof(LokiEllipseEstimatorInterface));
00039
00040     return self;
00041 }
00042
00043 LokiEllipseEstimator *LokiEllipseEstimator_clone(const LokiEllipseEstimator *other) {
00044     LokiEllipseEstimator *self = calloc(1, sizeof(LokiEllipseEstimator));
00045     if (!self) {
00046         return NULL;
00047     }
00048
00049     memcpy(self, other, sizeof(LokiEllipseEstimator));
00050
00051     self->instance = self->impl.clone(other->instance);
00052     if (!self->instance) {
00053         LokiEllipseEstimator_del(self);
00054         return NULL;
00055     }
00056
00057     return self;
00058 }
00059
00060 void LokiEllipseEstimator_work(LokiEllipseEstimator *self, float x, float y) {
00061     self->impl.work(self->instance, x, y);
00062 }
00063
00064 float *LokiEllipseEstimator_get_state(LokiEllipseEstimator *self) {
00065     return self->impl.get_state(self->instance);
00066 }
00067
00068 void LokiEllipseEstimator_del(LokiEllipseEstimator *self) {
00069     if (!self) {
00070         return;
00071     }
00072
00073     self->impl.del(self->instance);
00074     free(self);
00075 }
00076
```

6.15 Référence du fichier libloki-ellipse-estimator.h

```
#include "libloki-class.h"
#include <stddef.h>
```

Graphe des dépendances par inclusion de libloki-ellipse-estimator.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiEllipseEstimator LokiEllipseEstimator

Représentation d'un filtre d'estimation des paramètres d'une ellipse.

Fonctions

— LokiEllipseEstimator * LokiEllipseEstimator_new (float initial_radius, const float params[ ])

Construire un estimateur.

— LokiEllipseEstimator ∗ LokiEllipseEstimator_clone (const LokiEllipseEstimator ∗other)

Construire un clone d'un estimateur.

— void LokiEllipseEstimator_work (LokiEllipseEstimator ∗self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— float ∗ LokiEllipseEstimator_get_state (LokiEllipseEstimator ∗self)

Retourner l'état courant de l'estimateur.

— void LokiEllipseEstimator_del (LokiEllipseEstimator ∗self)

Détruire un estimateur.

— **LokiClass_interface** (LokiEllipseEstimator, LokiClass_attribute(size_t params_length) LokiClass_method(LokiEllipseEstimator new) LokiClass_method(LokiEllipseEstimator, clone) LokiClass_method(LokiEllipseEstimator, work) LokiClass_method(LokiEllipseEstimator, get_state) LokiClass_method(LokiEllipseEstimator, del))

— LokiEllipseEstimator ∗ LokiEllipseEstimator_new_with_implementation (const LokiEllipseEstimatorInterface ∗impl, float initial_radius, const float params[ ])

Construire un estimateur avec une implémentation donnée.

6.15.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-ellipse-estimator.h.

6.16 libloki-ellipse-estimator.h

Aller à la documentation de ce fichier.

```
00001
00008
00010
00011 #ifndef LIBLOKI_ELLIPSE_ESTIMATOR_H_
00012 #define LIBLOKI_ELLIPSE_ESTIMATOR_H_
00013
00014 #include "libloki-class.h"
00015
00016 #include <stddef.h>
00017
00019 typedef struct LokiEllipseEstimator LokiEllipseEstimator;
00020
00027 LokiEllipseEstimator *LokiEllipseEstimator_new(float initial_radius, const float params[]);
00028
00037 LokiEllipseEstimator *LokiEllipseEstimator_clone(const LokiEllipseEstimator *other);
00038
00048 void LokiEllipseEstimator_work(LokiEllipseEstimator *self, float x, float y);
00049
00055 float *LokiEllipseEstimator_get_state(LokiEllipseEstimator *self);
00056
00061 void LokiEllipseEstimator_del(LokiEllipseEstimator *self);
00062
```

```
00063 LokiClass_interface(LokiEllipseEstimator,
00064     LokiClass_attribute(size_t params_length)
00065     LokiClass_method(LokiEllipseEstimator, new)
00066     LokiClass_method(LokiEllipseEstimator, clone)
00067     LokiClass_method(LokiEllipseEstimator, work)
00068     LokiClass_method(LokiEllipseEstimator, get_state)
00069     LokiClass_method(LokiEllipseEstimator, del)
00070 );
00071
00079 LokiEllipseEstimator *LokiEllipseEstimator_new_with_implementation(const LokiEllipseEstimatorInterface
      *impl, float initial_radius, const float params[]);
00080
00081 #endif
00082
```

6.17 Référence du fichier libloki-ellipse.c

```
#include "libloki-ellipse.h"
#include <math.h>
#include <complex.h>
```

Graphe des dépendances par inclusion de libloki-ellipse.c:

Fonctions

— void LokiEllipse_set_from_coeffs (LokiEllipse *self, const LokiEllipseEquation coeffs)

Affecter les paramètres d'une ellipse en fonction des coefficients de son équation.

— void LokiEllipse_get_coeffs (const LokiEllipse *self, LokiEllipseEquation coeffs)

Calculer les coefficients de l'équation d'une ellipse.

— void LokiEllipse_get_point (const LokiEllipse *self, float theta, float *x, float *y)

Calculer les coordonnées d'un point d'une ellipse.

— float LokiEllipse_get_angle (const LokiEllipse *self, float x, float y, float *x_norm, float *y_norm)

Calculer l'angle correspondant à un point d'une ellipse.

6.17.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-ellipse.c.

6.18 libloki-ellipse.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki-ellipse.h"
00003
00004 #include <math.h>
00005 #include <complex.h>
00006
00009
00010 void LokiEllipse_set_from_coeffs(LokiEllipse *self, const LokiEllipseEquation coeffs) {
00011     float a   = coeffs[0];
00012     float b   = coeffs[1];
00013     float c   = 1.0f - a;
00014     float d   = coeffs[2];
00015     float e   = coeffs[3];
00016     float f   = coeffs[4];
00017
00018     float ac  = a * c;
00019     float ae  = a * e;
00020     float cma = c - a;
00021     float b2  = b * b;
00022     float bd  = b * d;
00023     float be  = b * e;
00024     float cd  = c * d;
00025
00026     float b2m4ac  = b2 - ac * 4;
00027     float samcpb2 = sqrtf(cma * cma + b2) / 2;
00028     float A0      = 0.5f + samcpb2; // 0.5 == (a + c) / 2
00029     float B0      = 0.5f - samcpb2; // 0.5 == (a + c) / 2
00030     float xc      = (cd * 2 - be) / b2m4ac;
00031     float yc      = (ae * 2 - bd) / b2m4ac;
00032     float df      = a * xc * xc + b * xc * yc + c * yc * yc - f;
00033
00034     // FIXME Est-il utile de calculer alpha sachant que par la suite, on recalcule son cos et son sin?
00035     self->alpha = atan2f(-b, cma) / 2;
00036     self->xc    = xc;
00037     self->yc    = yc;
00038     self->A     = sqrtf(fabsf(df / B0));
00039     self->B     = sqrtf(fabsf(df / A0));
00040 }
00041
00042 void LokiEllipse_get_coeffs(const LokiEllipse *self, LokiEllipseEquation coeffs) {
00043     float A2         = self->A * self->A;
00044     float B2         = self->B * self->B;
00045     float cos_alpha  = cosf(self->alpha);
00046     float sin_alpha  = sinf(self->alpha);
00047     float cos_alpha2 = cos_alpha * cos_alpha;
00048     float sin_alpha2 = sin_alpha * sin_alpha;
00049     float xc2        = self->xc * self->xc;
00050     float yc2        = self->yc * self->yc;
00051     float K          = A2 + B2;
00052
00053     coeffs[0] = (A2 * sin_alpha2 + B2 * cos_alpha2) / K;
00054     coeffs[1] = (B2 - A2) * sin_alpha * cos_alpha * 2 / K;
00055     coeffs[2] = -coeffs[0] * self->xc * 2 - coeffs[1] * self->yc;
00056     coeffs[3] = (coeffs[0] - 1) * self->yc * 2 - coeffs[1] * self->xc;
00057     coeffs[4] = coeffs[0] * xc2 + coeffs[1] * self->xc * self->yc + (1.0f - coeffs[0]) * yc2 - A2 * B2
    / K;
00058 }
00059
00060 void LokiEllipse_get_point(const LokiEllipse *self, float theta, float *x, float *y) {
00061     complex float z = (self->A * cosf(theta)) + I * (self->B * sinf(theta));
00062     z *= cosf(self->alpha) + I * sinf(self->alpha);
00063
00064     *x = self->xc + crealf(z);
00065     *y = self->yc + cimagf(z);
00066 }
00067
00068 float LokiEllipse_get_angle(const LokiEllipse *self, float x, float y, float *x_norm, float *y_norm) {
00069     complex float z  = x + I * y;
00070     // Translation pour recentrer l'ellipse autour de (0, 0).
00071     z -= self->xc + I * self->yc;
00072     // Rotation pour aligner le grand axe sur l'axe horizontal.
00073     z *= cosf(self->alpha) - I * sinf(self->alpha);
00074     // Normalisation pour transformer l'ellipse en un cercle unité.
00075     *x_norm = x = crealf(z) / self->A;
00076     *y_norm = y = cimagf(z) / self->B;
00077     // Calcul de l'angle.
00078     return atan2f(y, x);
00079 }
00080
```

6.19 Référence du fichier libloki-ellipse.h

Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Structures de données

— struct LokiEllipse

Structure de données représentant une ellipse.

Définitions de type

— typedef float LokiEllipseEquation[5]

Tableau des coefficients de l'équation d'une ellipse en coordonnées cartésiennes.

Fonctions

— void LokiEllipse_set_from_coeffs (LokiEllipse ∗self, const LokiEllipseEquation coeffs)

Affecter les paramètres d'une ellipse en fonction des coefficients de son équation.

— void LokiEllipse_get_coeffs (const LokiEllipse ∗self, LokiEllipseEquation coeffs)

Calculer les coefficients de l'équation d'une ellipse.

— void LokiEllipse_get_point (const LokiEllipse ∗self, float theta, float ∗x, float ∗y)

Calculer les coordonnées d'un point d'une ellipse.

— float LokiEllipse_get_angle (const LokiEllipse ∗self, float x, float y, float ∗x_norm, float ∗y_norm)

Calculer l'angle correspondant à un point d'une ellipse.

6.19.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-ellipse.h.

6.20 libloki-ellipse.h

Aller à la documentation de ce fichier.

```
00001
00008
00010
00011 #ifndef LIBLOKI_ELLIPSE_H_
00012 #define LIBLOKI_ELLIPSE_H_
00013
00015 typedef struct {
00017     float A;
00019     float B;
00021     float xc;
00023     float yc;
00025     float alpha;
00026 } LokiEllipse;
00027
00029 typedef float LokiEllipseEquation[5];
00030
00038 void LokiEllipse_set_from_coeffs(LokiEllipse *self, const LokiEllipseEquation coeffs);
00039
00048 void LokiEllipse_get_coeffs(const LokiEllipse *self, LokiEllipseEquation coeffs);
00049
00057 void LokiEllipse_get_point(const LokiEllipse *self, float theta, float *x, float *y);
00058
00071 float LokiEllipse_get_angle(const LokiEllipse *self, float x, float y, float *x_norm, float *y_norm);
00072
00073 #endif
00074
```

6.21 Référence du fichier libloki-env-sensors.c

```
#include <math.h>
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <loki-env-sensors-regs.h>
#include <loki-device-ioctl.h>
#include "libloki.h"
```

Graphe des dépendances par inclusion de libloki-env-sensors.c:

Structures de données

— struct LokiEnvSensors

Représentation de l'interface d'acquisition des capteurs auxiliaires.

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

— #define USE_MMAP

Utiliser la fonction `mmap` pour accéder aux données DMA.

Définitions de type

— typedef uint32_t LokiDMASample

Le type de donnée brut d'un échantillon passé par transfert DMA.

Fonctions

— static uint32_t LokiEnvSensors_get_reg (const LokiEnvSensors *self, LokiEnvSensorsRegister reg)

Lire un registre du circuit d'acquisition.

— static void LokiEnvSensors_set_reg (const LokiEnvSensors *self, LokiEnvSensorsRegister reg, uint32_t value)

Affecter un registre du circuit d'acquisition.

— static void LokiEnvSensors_dma_start (const LokiEnvSensors *self)

Demander au circuit d'acquisition de démarrer les transferts DMA.

— static void LokiEnvSensors_dma_stop (const LokiEnvSensors *self)

Demander au circuit d'acquisition d'arrêter les transferts DMA.

— static bool LokiEnvSensors_dma_is_streaming (const LokiEnvSensors *self)

Lire l'état des transferts DMA.

— static void LokiEnvSensors_dma_get_start_time (const LokiEnvSensors *self, struct timespec *res)

Lire la date de démarrage des transferts DMA.

— static void LokiEnvSensors_dma_get_packet_time (const LokiEnvSensors *self, struct timespec *res)

Lire la date du dernier transfert DMA.

— static int32_t LokiEnvSensors_bmp390_compensate_temperature (LokiEnvSensors *self, uint32_t data)

— static int32_t LokiEnvSensors_bmp390_compensate_pressure (const LokiEnvSensors *self, uint32_t data)

— LokiEnvSensors * LokiEnvSensors_new (void)

Construire un module d'acquisition pour les capteurs auxiliaires.

— void LokiEnvSensors_del (LokiEnvSensors *self)

Détruire un module d'acquisition pour les capteurs auxiliaires.

— void LokiEnvSensors_restart (const LokiEnvSensors *self)

Redémarrer le module d'acquisition pour les capteurs auxiliaires.

— uint32_t LokiEnvSensors_get_clk_frequency (const LokiEnvSensors *self)

Lire la fréquence d'horloge du circuit d'acquisition.

— size_t LokiEnvSensors_get_channels (const LokiEnvSensors *self)

Lire le nombre de voies du circuit d'acquisition.

— float LokiEnvSensors_get_sampling_frequency (const LokiEnvSensors ∗self)

Lire la fréquence d'échantillonnage du circuit d'acquisition.

— void LokiEnvSensors_set_sampling_frequency (const LokiEnvSensors ∗self, float freq)

Affecter une fréquence d'échantillonnage au circuit d'acquisition.

— uint32_t LokiEnvSensors_get_packet_length_max (const LokiEnvSensors ∗self)

Lire la longueur maximale des paquets DMA.

— uint32_t LokiEnvSensors_get_packet_length (const LokiEnvSensors ∗self)

Lire la longueur courante d'un paquet DMA.

— float LokiEnvSensors_get_scaling_factor (const LokiEnvSensors ∗self, size_t channel)

Lire le facteur de mise à l'échelle des échantillons d'une voie.

— void LokiEnvSensors_set_scaling_factor (LokiEnvSensors ∗self, size_t channel, float scale)

Affecter un facteur de mise à l'échelle des échantillons d'une voie.

— float LokiEnvSensors_get_last_data (const LokiEnvSensors ∗self, size_t channel)

Lire la valeur du dernier échantillon acquis sur une voie.

— static uint32_t LokiEnvSensors_get_packet_index (const LokiEnvSensors ∗self, LokiDMASample raw_item)

Lire l'indice d'un paquet reçu.

— static void LokiEnvSensors_set_packet_item (const LokiEnvSensors ∗self, LokiEnvSensorsSample ∗item, LokiDMASample raw_item)

Convertir un échantillon issu du DMA.

— static void ∗ LokiEnvSensors_work (void ∗ptr)

Traiter un tableau d'échantillons issu d'un transfert DMA.

— int LokiEnvSensors_start_streaming (LokiEnvSensors ∗self, int32_t channels, bool raw, LokiFIFO ∗out)

Démarrer l'acquisition.

— void LokiEnvSensors_stop_streaming (LokiEnvSensors ∗self)

Arrêter l'acquisition.

— void LokiEnvSensors_force_stop (const LokiEnvSensors ∗self)

Arrêter l'acquisition immédiatement.

— bool LokiEnvSensors_is_streaming (const LokiEnvSensors ∗self)

Lire l'état de l'acquisition.

6.21.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-env-sensors.c.

6.21.2 Documentation des macros

6.21.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Définition à la ligne 5 du fichier libloki-env-sensors.c.

6.22 libloki-env-sensors.c

Aller à la documentation de ce fichier.

```
00001
00002 // TODO Ajouter les sources XADC et capteur de température I2C
00003
00005 #define _GNU_SOURCE
00006
00007 #include <math.h>
00008 #include <errno.h>
00009 #include <stdio.h>
00010 #include <string.h>
00011 #include <stdlib.h>
00012 #include <pthread.h>
00013 #include <unistd.h>
00014 #include <sys/ioctl.h>
00015 #include <sys/mman.h>
00016
00017 #include <loki-env-sensors-regs.h>
00018 #include <loki-device-ioctl.h>
00019 #include "libloki.h"
00020
00023
00025 struct LokiEnvSensors {
00027     FILE *file;
00029     int fileno;
00031     volatile bool stop_request;
00033     LokiFIFO *out;
00035     bool raw;
00037     int32_t stream_channels;
00039     pthread_t thread;
00041     uint32_t clk_frequency;
00043     uint32_t channels;
00045     uint32_t cycles_per_sample_max;
00047     uint32_t channel_output_width;
00049     uint32_t data_width;
00051     uint32_t channel_mask;
00053     uint32_t data_mask;
00055     uint32_t data_sign_mask;
00057     int64_t bmp390_t_fine;
00059     float *scales;
00060 };
00061
00063 typedef uint32_t LokiDMASample;
00064
```

```
00074 static inline uint32_t LokiEnvSensors_get_reg(const LokiEnvSensors* self, LokiEnvSensorsRegister reg)
      {
00075     LokiRegisterQuery query = {.reg = reg};
00076     int err = ioctl(self->fileno, LOKI_IOCTL_REG_READ, &query);
00077     if (err) {
00078         fprintf(stderr, "LokiEnvSensors_get_reg: ioctl returned %d\n", err);
00079         return 0;
00080     }
00081     return query.value;
00082 }
00083
00093 static inline void LokiEnvSensors_set_reg(const LokiEnvSensors* self, LokiEnvSensorsRegister reg,
      uint32_t value) {
00094     LokiRegisterQuery query = {.reg = reg, .value = value};
00095     int err = ioctl(self->fileno, LOKI_IOCTL_REG_WRITE, &query);
00096     if (err) {
00097         fprintf(stderr, "LokiEnvSensors_set_reg: ioctl returned %d\n", err);
00098     }
00099 }
00100
00108 static inline void LokiEnvSensors_dma_start(const LokiEnvSensors *self) {
00109     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_START);
00110     if (err) {
00111         fprintf(stderr, "LokiEnvSensors_dma_start: ioctl returned %d\n", err);
00112     }
00113 }
00114
00122 static inline void LokiEnvSensors_dma_stop(const LokiEnvSensors *self) {
00123     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_STOP);
00124     if (err) {
00125         fprintf(stderr, "LokiEnvSensors_dma_stop: ioctl returned %d\n", err);
00126     }
00127 }
00128
00135 static inline bool LokiEnvSensors_dma_is_streaming(const LokiEnvSensors *self) {
00136     bool res;
00137     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_IS_STREAMING, &res);
00138     if (err) {
00139         fprintf(stderr, "LokiEnvSensors_dma_is_streaming: ioctl returned %d\n", err);
00140         return false;
00141     }
00142     return res;
00143 }
00144
00152 static inline void LokiEnvSensors_dma_get_start_time(const LokiEnvSensors *self, struct timespec *res)
      {
00153     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_START_TIME, res);
00154     if (err) {
00155         fprintf(stderr, "LokiEnvSensors_dma_get_start_time: ioctl returned %d\n", err);
00156     }
00157 }
00158
00166 static inline void LokiEnvSensors_dma_get_packet_time(const LokiEnvSensors *self, struct timespec
      *res) {
00167     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_PACKET_TIME, res);
00168     if (err) {
00169         fprintf(stderr, "LokiEnvSensors_dma_get_packet_time: ioctl returned %d\n", err);
00170     }
00171 }
00172
00173 // Adapté de : https://github.com/hepingood/bmp390
00174 static int32_t LokiEnvSensors_bmp390_compensate_temperature(LokiEnvSensors *self, uint32_t data) {
00175     uint32_t nvm_reg0 = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_BMP390_NVM0);
00176     uint32_t nvm_reg1 = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_BMP390_NVM1);
00177
00178     uint16_t t1 = nvm_reg0 » 16;
00179     uint16_t t2 = nvm_reg0;
00180     int8_t   t3 = nvm_reg1;
00181
00182     uint64_t partial_data1 = (uint64_t)data - (uint64_t)t1 * 256;
00183     int64_t partial_data2 = t2 * partial_data1;
00184     int64_t partial_data3 = partial_data1 * partial_data1;
00185     int64_t partial_data4 = partial_data3 * t3;
00186     int64_t partial_data5 = partial_data2 * 262144 + partial_data4;
00187     int64_t partial_data6 = partial_data5 / 4294967296U;
00188     self->bmp390_t_fine = partial_data6;
00189     return partial_data6 * 25 / 16384;
00190 }
00191
00192 // Adapté de : https://github.com/hepingood/bmp390
00193 static int32_t LokiEnvSensors_bmp390_compensate_pressure(const LokiEnvSensors *self, uint32_t data) {
00194     uint32_t nvm_reg2 = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_BMP390_NVM2);
00195     uint32_t nvm_reg3 = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_BMP390_NVM3);
00196     uint32_t nvm_reg4 = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_BMP390_NVM4);
00197     uint32_t nvm_reg5 = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_BMP390_NVM5);
00198
00199     int16_t  p1  = nvm_reg2 » 16;
```

```
00200     int16_t  p2  = nvm_reg2;
00201     int8_t   p3  = nvm_reg3 » 24;
00202     int8_t   p4  = nvm_reg3 » 16;
00203     uint16_t p5  = nvm_reg3;
00204     uint16_t p6  = nvm_reg4 » 16;
00205     int8_t   p7  = nvm_reg4 » 8;
00206     int8_t   p8  = nvm_reg4;
00207     int16_t  p9  = nvm_reg5 » 16;
00208     int8_t   p10 = nvm_reg5 » 8;
00209     int8_t   p11 = nvm_reg5;
00210
00211     int64_t t_fine = self->bmp390_t_fine;
00212     int64_t partial_data1 = t_fine * t_fine;
00213     int64_t partial_data2 = partial_data1 / 64;
00214     int64_t partial_data3 = partial_data2 * t_fine / 256;
00215     int64_t partial_data4 = p8 * partial_data3 / 32;
00216     int64_t partial_data5 = p7 * partial_data1 * 16;
00217     int64_t partial_data6 = p6 * t_fine * 4194304;
00218     int64_t offset = (int64_t)p5 * (int64_t)140737488355328U + partial_data4 + partial_data5 +
    partial_data6;
00219     partial_data2 = (int64_t)p4 * partial_data3 / 32;
00220     partial_data4 = p3 * partial_data1 * 4;
00221     partial_data5 = ((int64_t)p2 - 16384) * t_fine * 2097152;
00222     int64_t sensitivity = ((int64_t)p1 - 16384) * (int64_t)70368744177664U + partial_data2 +
    partial_data4 + partial_data5;
00223     partial_data1 = sensitivity / 16777216 * data;
00224     partial_data2 = (int64_t)p10 * t_fine;
00225     partial_data3 = partial_data2 + 65536 * (int64_t)p9;
00226     partial_data4 = partial_data3 * data / 8192;
00227     partial_data5 = partial_data4 * data / 512;
00228     partial_data6 = (uint64_t)data * (uint64_t)data;
00229     partial_data2 = (int64_t)p11 * partial_data6 / 65536;
00230     partial_data3 = partial_data2 * data / 128;
00231     partial_data4 = offset / 4 + partial_data1 + partial_data5 + partial_data3;
00232     return (uint64_t)partial_data4 * 25 / 1099511627776U;
00233 }
00234
00235 LokiEnvSensors *LokiEnvSensors_new(void) {
00236     LokiEnvSensors *self = calloc(1, sizeof(LokiEnvSensors));
00237     if (!self) {
00238         return NULL;
00239     }
00240
00241     self->file = fopen("/dev/loki-env-sensors", "r");
00242     if (!self->file) {
00243         LokiEnvSensors_del(self);
00244         return NULL;
00245     }
00246
00247     self->fileno = fileno(self->file);
00248
00249     self->clk_frequency         = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_CLK_FREQUENCY);
00250     self->channels              = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_CHANNELS);
00251     self->channel_output_width  = LokiEnvSensors_get_reg(self,
    LOKI_ENV_SENSORS_REG_CHANNEL_OUTPUT_WIDTH);
00252     self->data_width            = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_DATA_WIDTH);
00253     self->cycles_per_sample_max = LokiEnvSensors_get_reg(self,
    LOKI_ENV_SENSORS_REG_CYCLES_PER_SAMPLE_MAX);
00254     self->channel_mask          = (1 « self->channel_output_width) - 1;
00255     self->data_mask             = (1 « self->data_width) - 1;
00256     self->data_sign_mask        = 1 « (self->data_width - 1);
00257
00258     // Allouer un tableau pour les facteurs de mise à l'échelle,
00259     // et les initialiser à 1.
00260     self->scales = calloc(self->channels, sizeof(float));
00261     if (!self->scales) {
00262         LokiEnvSensors_del(self);
00263         return NULL;
00264     }
00265
00266     for (size_t i = 0; i < self->channels; i ++) {
00267         self->scales[i] = 1.0f;
00268     }
00269
00270     // Le nombre d'octets d'un transfert DMA doit être multiple de
00271     // * la taille d'une page mémoire
00272     // * la taille d'un échantillon
00273     uint32_t page_size = getpagesize();
00274     uint32_t a         = page_size;
00275     uint32_t b         = sizeof(LokiDMASample);
00276     while (a != b) {
00277         if (a > b) {
00278             a -= b;
00279         }
00280         if (b > a) {
00281             b -= a;
00282         }
```

```
00283     }
00284
00285     LokiEnvSensors_set_reg(self, LOKI_ENV_SENSORS_REG_PACKET_LENGTH, page_size / a);
00286
00287     return self;
00288 }
00289
00290 void LokiEnvSensors_del(LokiEnvSensors *self) {
00291     if (!self) {
00292         return;
00293     }
00294
00295     if (self->file) {
00296         fclose(self->file);
00297     }
00298
00299     if (self->scales) {
00300         free(self->scales);
00301     }
00302
00303     free(self);
00304 }
00305
00306 void LokiEnvSensors_restart(const LokiEnvSensors *self) {
00307     uint32_t value = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_CONTROL);
00308     value |= LOKI_ENV_SENSORS_REG_CONTROL_RESTART_SHIFTED_MASK;
00309     LokiEnvSensors_set_reg(self, LOKI_ENV_SENSORS_REG_CONTROL, value);
00310 }
00311
00312 uint32_t LokiEnvSensors_get_clk_frequency(const LokiEnvSensors *self) {
00313     return self->clk_frequency;
00314 }
00315
00316 size_t LokiEnvSensors_get_channels(const LokiEnvSensors *self) {
00317     return self->channels;
00318 }
00319
00320 float LokiEnvSensors_get_sampling_frequency(const LokiEnvSensors *self) {
00321     return (float)self->clk_frequency / LokiEnvSensors_get_reg(self,
      LOKI_ENV_SENSORS_REG_CYCLES_PER_SAMPLE);
00322 }
00323
00324 void LokiEnvSensors_set_sampling_frequency(const LokiEnvSensors *self, float freq) {
00325     LokiEnvSensors_set_reg(self, LOKI_ENV_SENSORS_REG_CYCLES_PER_SAMPLE,
      roundf((float)self->clk_frequency / freq));
00326 }
00327
00328 uint32_t LokiEnvSensors_get_packet_length_max(const LokiEnvSensors *self) {
00329     return LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_PACKET_LENGTH_MAX);
00330 }
00331
00332 uint32_t LokiEnvSensors_get_packet_length(const LokiEnvSensors *self) {
00333     return LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_PACKET_LENGTH);
00334 }
00335
00336 float LokiEnvSensors_get_scaling_factor(const LokiEnvSensors *self, size_t channel) {
00337     return self->scales[channel];
00338 }
00339
00340 void LokiEnvSensors_set_scaling_factor(LokiEnvSensors *self, size_t channel, float scale) {
00341     self->scales[channel] = scale;
00342 }
00343
00344 float LokiEnvSensors_get_last_data(const LokiEnvSensors *self, size_t channel) {
00345     int32_t value;
00346     if (channel < self->channels - 2) {
00347         value = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_LAST_DATA + channel);
00348     }
00349     else {
00350         uint32_t t_raw = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_LAST_DATA + self->channels
      - 2);
00351         value = LokiEnvSensors_bmp390_compensate_temperature(self, t_raw);
00352         if (channel == self->channels - 1) {
00353             uint32_t p_raw = LokiEnvSensors_get_reg(self, LOKI_ENV_SENSORS_REG_LAST_DATA + channel);
00354             value = LokiEnvSensors_bmp390_compensate_pressure(self, p_raw);
00355         }
00356     }
00357     return self->scales[channel] * value;
00358 }
00359
00368 static inline uint32_t LokiEnvSensors_get_packet_index(const LokiEnvSensors *self, LokiDMASample
      raw_item) {
00369     return raw_item » (self->data_width + self->channel_output_width);
00370 }
00371
00382 static void LokiEnvSensors_set_packet_item(const LokiEnvSensors *self, LokiEnvSensorsSample *item,
      LokiDMASample raw_item) {
```

```
00383     // Lire la valeur de l'échantillon, alignée à droite en complément à deux.
00384     int32_t data_raw = raw_item & self->data_mask;
00385     raw_item »= self->data_width;
00386 
00387     // Lire le numéro de la voie.
00388     item->channel = raw_item & self->channel_mask;
00389 
00390     // BMP390 : effectuer la compensation des mesures de température et de pression.
00391     // Autres voies: effectuer l'extension du bit de signe.
00392     if (item->channel == self->channels - 2) {
00393         data_raw = LokiEnvSensors_bmp390_compensate_temperature(self, data_raw);
00394     }
00395     else if (item->channel == self->channels - 1) {
00396         data_raw = LokiEnvSensors_bmp390_compensate_pressure(self, data_raw);
00397     }
00398     else if (data_raw & self->data_sign_mask) {
00399         data_raw |= ~self->data_mask;
00400     }
00401 
00402     // Convertir les coordonnées en flottants si nécessaire.
00403     if (self->raw) {
00404         item->data_raw = data_raw;
00405     }
00406     else {
00407         item->data = self->scales[item->channel] * data_raw;
00408     }
00409 }
00410 
00415 #define USE_MMAP
00416 
00433 static void *LokiEnvSensors_work(void *ptr) {
00434     LokiEnvSensors *self = ptr;
00435 
00436     pthread_setname_np(self->thread, "LokiEnvSensors");
00437 
00438     // Obtenir un pointeur vers le buffer courant des échantillons mise en forme.
00439     const size_t out_length = LokiPacket_get_length(self->out, sizeof(LokiEnvSensorsSample));
00440     LokiPacket *out_packet  = LokiFIFO_get_producer_item(self->out);
00441     if (!out_packet) {
00442         return NULL;
00443     }
00444     LokiEnvSensorsSample *out_ptr = (LokiEnvSensorsSample*)out_packet->buffer;
00445     LokiEnvSensorsSample *out_end = out_ptr + out_length;
00446 
00447     // Avec la fonction fread, allouer un buffer de données.
00448     // Avec l'accès par mmap, le buffer est fourni par le pilote.
00449     const size_t in_length = LokiEnvSensors_get_packet_length(self);
00450     #ifndef USE_MMAP
00451         LokiDMASample *in_buffer = calloc(in_length, sizeof(LokiDMASample));
00452         LokiDMASample *in_end    = in_buffer + in_length;
00453     #else
00454         size_t in_length_bytes = in_length * sizeof(LokiDMASample);
00455     #endif
00456 
00457     // Calculer la durée moyenne d'un paquet d'entrée, en nanosecondes.
00458     double in_period_ns = 1.0e9 * in_length / self->channels /
    LokiEnvSensors_get_sampling_frequency(self);
00459 
00460     // Calculer le modulo du compteur de paquets du FPGA.
00461     uint32_t packet_count_mod = 1 « (sizeof(LokiDMASample) * 8 - self->channel_output_width -
    self->data_width);
00462 
00463     bool first_packet = true;
00464     uint32_t packet_index_prev = 0;
00465     LokiTimestamp in_timestamp;
00466 
00467     // Démarrer l'acquisition.
00468     LokiEnvSensors_dma_start(self);
00469 
00470     while (!self->stop_request) {
00471         // Lire un buffer d'échantillons en provenance du DMA.
00472         #ifndef USE_MMAP
00473             if (fread(in_buffer, sizeof(LokiDMASample), in_length, self->file) != in_length) {
00474                 fprintf(stderr, "LokiEnvSensors_work: read failed (error: %d, %s)\n", errno,
    strerror(errno));
00475                 break;
00476             }
00477         #else
00478             // Accès au buffer DMA en mode zero-copy.
00479             // Contrairement à l'acquisition des voies optiques, l'appel à mmap
00480             // peut bloquer le processus pendant une longue durée.
00481             // On utilise le timeout pour le débloquer jusqu'au succès.
00482             LokiDMASample *in_buffer;
00483             do {
00484                 in_buffer = mmap(NULL, in_length_bytes, PROT_READ, MAP_PRIVATE, self->fileno, 0);
00485             } while (in_buffer == MAP_FAILED && errno == ETIMEDOUT);
00486 
```

```
00487             if (in_buffer == MAP_FAILED) {
00488                 fprintf(stderr, "LokiEnvSensors_work: mmap failed (error: %d, %s)\n", errno,
    strerror(errno));
00489                 break;
00490             }
00491
00492             LokiDMASample *in_end = in_buffer + in_length;
00493         #endif
00494
00495         // Attendre le paquet numéro 0 pour commencer.
00496         uint32_t packet_index = LokiEnvSensors_get_packet_index(self, *in_buffer);
00497
00498         // Prendre pour référence temporelle la date du premier paquet.
00499         // Les dates des autres paquets seront calculées en fonction du nombre d'échantillons.
00500         if (first_packet && !packet_index) {
00501             struct timespec ts;
00502             LokiEnvSensors_dma_get_packet_time(self, &ts);
00503             in_timestamp = LokiTime_to_timestamp(ts) - in_period_ns;
00504         }
00505
00506         if (!first_packet || !packet_index) {
00507             // Calculer le timestamp du paquet d'entrée, en nanosecondes,
00508             // en utilisant le numéro de paquet fourni par le FPGA.
00509             int32_t packet_interval = packet_index - packet_index_prev;
00510             if (packet_interval < 0) {
00511                 fprintf(stderr, "LokiEnvSensors_work: packet index rollover (%u).\n", packet_index);
00512                 packet_interval += packet_count_mod;
00513             }
00514
00515             if (packet_interval > 1) {
00516                 fprintf(stderr, "LokiEnvSensors_work: packet index gap (%u -> %u, %u).\n",
    packet_index_prev, packet_index, packet_interval);
00517                 //goto LokiEnvSensors_work_continue;
00518             }
00519
00520             packet_index_prev = packet_index;
00521             first_packet = false;
00522
00523             in_timestamp += round(in_period_ns * packet_interval);
00524
00525             // En commençant un nouveau paquet de sortie, copier la date du paquet d'entrée.
00526             if (out_ptr == (LokiEnvSensorsSample*)out_packet->buffer) {
00527                 out_packet->timestamp = in_timestamp;
00528             }
00529
00530             LokiDMASample *in_ptr = in_buffer;
00531             while (in_ptr != in_end && !self->stop_request) {
00532                 LokiDMASample in = *in_ptr ++;
00533
00534                 LokiEnvSensorsSample s;
00535                 LokiEnvSensors_set_packet_item(self, &s, in);
00536
00537                 if (self->stream_channels & (1 « s.channel)) {
00538                     *out_ptr ++ = s;
00539
00540                     if (out_ptr == out_end) {
00541                         LokiFIFO_push(self->out);
00542                         out_packet = LokiFIFO_get_producer_item(self->out);
00543                         if (!out_packet) {
00544                             break;
00545                         }
00546                         if (in_ptr != in_end) {
00547                             out_packet->timestamp = in_timestamp + round(in_period_ns * (in_ptr -
    in_buffer) / in_length);
00548                         }
00549                         out_ptr = (LokiEnvSensorsSample*)out_packet->buffer;
00550                         out_end = out_ptr + out_length;
00551                     }
00552                 }
00553             }
00554         }
00555
00556     LokiEnvSensors_work_continue:
00557         #ifdef USE_MMAP
00558             munmap(in_buffer, in_length_bytes);
00559         #endif
00560     }
00561
00562     #ifndef USE_MMAP
00563         free(in_buffer);
00564     #endif
00565
00566     LokiEnvSensors_dma_stop(self);
00567 }
00568
00569 int LokiEnvSensors_start_streaming(LokiEnvSensors *self, int32_t channels, bool raw, LokiFIFO *out) {
00570     if (LokiEnvSensors_dma_is_streaming(self)) {
```

```
00571         return EBUSY;
00572     }
00573     self->stop_request    = false;
00574     self->out             = out;
00575     self->raw             = raw;
00576     self->stream_channels = channels;
00577     return pthread_create(&self->thread, NULL, LokiEnvSensors_work, self);
00578 }
00579
00580 void LokiEnvSensors_stop_streaming(LokiEnvSensors *self) {
00581     if (self->stop_request) {
00582         return;
00583     }
00584     self->stop_request = true;
00585     LokiFIFO_abort(self->out);
00586     pthread_join(self->thread, NULL);
00587 }
00588
00589 void LokiEnvSensors_force_stop(const LokiEnvSensors *self) {
00590     LokiEnvSensors_dma_stop(self);
00591 }
00592
00593 bool LokiEnvSensors_is_streaming(const LokiEnvSensors *self) {
00594     return LokiEnvSensors_dma_is_streaming(self);
00595 }
00596
```

6.23 Référence du fichier libloki-env-sensors.h

```
#include <stdint.h>
#include <stdbool.h>
#include <sys/types.h>
```

Graphe des dépendances par inclusion de libloki-env-sensors.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Structures de données

— struct LokiEnvSensorsSample

Représentation d'un échantillon en provenance des capteurs auxiliaires.

Définitions de type

— typedef struct LokiEnvSensors LokiEnvSensors

Représentation des capteurs auxiliaires.

Fonctions

— LokiEnvSensors * LokiEnvSensors_new (void)

Construire un module d'acquisition pour les capteurs auxiliaires.

— void LokiEnvSensors_del (LokiEnvSensors *self)

Détruire un module d'acquisition pour les capteurs auxiliaires.

— void LokiEnvSensors_restart (const LokiEnvSensors *self)

Redémarrer le module d'acquisition pour les capteurs auxiliaires.

— uint32_t LokiEnvSensors_get_clk_frequency (const LokiEnvSensors ∗self)

Lire la fréquence d'horloge du circuit d'acquisition.

— size_t LokiEnvSensors_get_channels (const LokiEnvSensors ∗self)

Lire le nombre de voies du circuit d'acquisition.

— float LokiEnvSensors_get_sampling_frequency (const LokiEnvSensors ∗self)

Lire la fréquence d'échantillonnage du circuit d'acquisition.

— void LokiEnvSensors_set_sampling_frequency (const LokiEnvSensors ∗self, float freq)

Affecter une fréquence d'échantillonnage au circuit d'acquisition.

— uint32_t LokiEnvSensors_get_packet_length_max (const LokiEnvSensors ∗self)

Lire la longueur maximale des paquets DMA.

— uint32_t LokiEnvSensors_get_packet_length (const LokiEnvSensors ∗self)

Lire la longueur courante d'un paquet DMA.

— float LokiEnvSensors_get_scaling_factor (const LokiEnvSensors ∗self, size_t channel)

Lire le facteur de mise à l'échelle des échantillons d'une voie.

— void LokiEnvSensors_set_scaling_factor (LokiEnvSensors ∗self, size_t channel, float scale)

Affecter un facteur de mise à l'échelle des échantillons d'une voie.

— float LokiEnvSensors_get_last_data (const LokiEnvSensors ∗self, size_t channel)

Lire la valeur du dernier échantillon acquis sur une voie.

— int LokiEnvSensors_start_streaming (LokiEnvSensors ∗self, int32_t channels, bool raw, LokiFIFO ∗out)

Démarrer l'acquisition.

— void LokiEnvSensors_stop_streaming (LokiEnvSensors ∗self)

Arrêter l'acquisition.

— void LokiEnvSensors_force_stop (const LokiEnvSensors ∗self)

Arrêter l'acquisition immédiatement.

— bool LokiEnvSensors_is_streaming (const LokiEnvSensors ∗self)

Lire l'état de l'acquisition.

6.23.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-env-sensors.h.

6.24 libloki-env-sensors.h

Aller à la documentation de ce fichier.

```
00001
00012
00014
00015 #ifndef LIBLOKI_ENV_SENSORS_H_
00016 #define LIBLOKI_ENV_SENSORS_H_
00017
00018 #include <stdint.h>
00019 #include <stdbool.h>
00020 #include <sys/types.h>
00021
00023 typedef struct LokiEnvSensors LokiEnvSensors;
00024
00026 typedef struct {
00028     uint32_t channel;
00030     union {
00032         float data;
00034         int32_t data_raw;
00035     };
00036 } LokiEnvSensorsSample;
00037
00042 LokiEnvSensors *LokiEnvSensors_new(void);
00043
00048 void LokiEnvSensors_del(LokiEnvSensors *self);
00049
00057 void LokiEnvSensors_restart(const LokiEnvSensors *self);
00058
00064 uint32_t LokiEnvSensors_get_clk_frequency(const LokiEnvSensors *self);
00065
00071 size_t LokiEnvSensors_get_channels(const LokiEnvSensors *self);
00072
00078 float LokiEnvSensors_get_sampling_frequency(const LokiEnvSensors *self);
00079
00085 void LokiEnvSensors_set_sampling_frequency(const LokiEnvSensors *self, float freq);
00086
00092 uint32_t LokiEnvSensors_get_packet_length_max(const LokiEnvSensors *self);
00093
00099 uint32_t LokiEnvSensors_get_packet_length(const LokiEnvSensors *self);
00100
00110 float LokiEnvSensors_get_scaling_factor(const LokiEnvSensors *self, size_t channel);
00111
00121 void LokiEnvSensors_set_scaling_factor(LokiEnvSensors *self, size_t channel, float scale);
00122
00129 float LokiEnvSensors_get_last_data(const LokiEnvSensors *self, size_t channel);
00130
00143 int LokiEnvSensors_start_streaming(LokiEnvSensors *self, int32_t channels, bool raw, LokiFIFO *out);
00144
00152 void LokiEnvSensors_stop_streaming(LokiEnvSensors *self);
00153
00161 void LokiEnvSensors_force_stop(const LokiEnvSensors *self);
00162
00168 bool LokiEnvSensors_is_streaming(const LokiEnvSensors *self);
00169
00170 #endif
00171
```

6.25 Référence du fichier libloki-fifo.c

```
#include "libloki-fifo.h"
#include <stdlib.h>
#include <pthread.h>
```

Graphe des dépendances par inclusion de libloki-fifo.c:

Structures de données

— struct LokiFIFO

Représentation d'une FIFO.

— struct LokiFIFORecord

Représentation d'un enregistrement dans une FIFO.

Définitions de type

— typedef struct LokiFIFORecord LokiFIFORecord

Représentation d'un enregistrement dans une FIFO.

Fonctions

— LokiFIFO ∗ LokiFIFO_new (size_t item_size, size_t length, size_t consumers)

Créer une FIFO.

— void LokiFIFO_del (LokiFIFO ∗self)

Détruire une FIFO.

— size_t LokiFIFO_get_item_size (const LokiFIFO ∗self)

Obtenir la taille d'un élément de la FIFO, en octets.

— void ∗ LokiFIFO_get_producer_item (LokiFIFO ∗self)

Obtenir une référence vers l'élément courant en écriture.

— void LokiFIFO_push (LokiFIFO ∗self)

Valider l'élément courant en écriture.

— void ∗ LokiFIFO_get_consumer_item (LokiFIFO ∗self, size_t cid)

Obtenir une référence vers l'élément courant en lecture.

— void LokiFIFO_pull (LokiFIFO ∗self, size_t cid)

Libérer l'élément courant en lecture.

— void LokiFIFO_abort (LokiFIFO ∗self)

Terminer les attentes en cours.

6.25.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-fifo.c.

6.26 libloki-fifo.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki-fifo.h"
00003
00004 #include <stdlib.h>
00005 #include <pthread.h>
00006
00009
00011 typedef struct LokiFIFORecord LokiFIFORecord;
00012
00020 struct LokiFIFO {
00022     size_t item_size;
00024     int all_read;
00026     volatile bool abort;
00028     pthread_mutex_t mutex;
00030     pthread_cond_t pushed;
00032     pthread_cond_t pulled;
00034     LokiFIFORecord *producer_record;
00039     LokiFIFORecord *consumer_records[0];
00040     // Ne rien ajouter ici.
00041 };
00042
00047 struct LokiFIFORecord {
00049     volatile int status;
00051     LokiFIFORecord *next;
00056     char item[0];
00057     // Ne rien ajouter ici.
00058 };
00059
00060 LokiFIFO *LokiFIFO_new(size_t item_size, size_t length, size_t consumers) {
00061     size_t record_size = sizeof(LokiFIFORecord) + item_size;
00062     LokiFIFO *self = calloc(1, sizeof(LokiFIFO) + length * record_size + consumers *
      sizeof(LokiFIFORecord*));
00063     if (!self) {
00064         return NULL;
00065     }
00066
00067     LokiFIFORecord *first_record = (LokiFIFORecord*)(self->consumer_records + consumers);
00068
00069     self->abort           = false;
00070     self->item_size       = item_size;
00071     self->all_read        = (1 << consumers) - 1;
00072     self->producer_record = first_record;
00073     for (size_t c = 0; c < consumers; c ++) {
00074         self->consumer_records[c] = first_record;
00075     }
00076
00077     // Initialiser les records comme une liste chaînée circulaire.
00078     // Marquer tous les éléments comme lus par tous les consommateurs.
00079     LokiFIFORecord *rec = first_record;
00080     for (size_t i = 0; i < length; i ++) {
00081         rec->status = self->all_read;
00082         rec->next   = i == length - 1 ?
00083                       first_record :
00084                       (LokiFIFORecord*)((char*)rec + record_size);
00085         rec = rec->next;
00086     }
00087
00088     pthread_mutex_init(&self->mutex, NULL);
00089     pthread_cond_init(&self->pushed, NULL);
00090     pthread_cond_init(&self->pulled, NULL);
00091
00092     return self;
00093 }
00094
```

```
00095 void LokiFIFO_del(LokiFIFO *self) {
00096     if (!self) {
00097         return;
00098     }
00099     free(self);
00100 }
00101
00102 size_t LokiFIFO_get_item_size(const LokiFIFO *self) {
00103     return self->item_size;
00104 }
00105
00106 void *LokiFIFO_get_producer_item(LokiFIFO *self) {
00107     pthread_mutex_lock(&self->mutex);
00108
00109     // Bloquer tant que l'élément courant n'a pas été lu par tous les consommateurs.
00110     LokiFIFORecord *rec = self->producer_record;
00111     while (!self->abort && rec->status != self->all_read) {
00112         pthread_cond_wait(&self->pulled, &self->mutex);
00113     }
00114
00115     // Retourner NULL en cas d'annulation.
00116     void *item = self->abort ? NULL : rec->item;
00117
00118     pthread_mutex_unlock(&self->mutex);
00119
00120     return item;
00121 }
00122
00123 void LokiFIFO_push(LokiFIFO *self) {
00124     if (self->abort) {
00125         return;
00126     }
00127
00128     pthread_mutex_lock(&self->mutex);
00129
00130     // Marquer l'élément courant comme non lu.
00131     self->producer_record->status = 0;
00132
00133     // Signaler la présence d'un nouvel élément à tous les consommateurs en attente.
00134     pthread_cond_broadcast(&self->pushed);
00135
00136     // Passer à l'élément suivant.
00137     self->producer_record = self->producer_record->next;
00138
00139     pthread_mutex_unlock(&self->mutex);
00140 }
00141
00142 void *LokiFIFO_get_consumer_item(LokiFIFO *self, size_t cid) {
00143     pthread_mutex_lock(&self->mutex);
00144
00145     // Bloquer tant que l'élément courant est marqué comme lu par ce consommateur.
00146     int mask = 1 « cid;
00147     LokiFIFORecord *rec = self->consumer_records[cid];
00148     while (!self->abort && (rec->status & mask)) {
00149         pthread_cond_wait(&self->pushed, &self->mutex);
00150     }
00151
00152     // Retourner NULL en cas d'annulation.
00153     void *item = self->abort ? NULL : rec->item;
00154
00155     pthread_mutex_unlock(&self->mutex);
00156
00157     return item;
00158 }
00159
00160 void LokiFIFO_pull(LokiFIFO *self, size_t cid) {
00161     if (self->abort) {
00162         return;
00163     }
00164
00165     pthread_mutex_lock(&self->mutex);
00166
00167     // Marquer l'élément courant comme lu par ce consommateur.
00168     LokiFIFORecord *rec = self->consumer_records[cid];
00169     rec->status |= 1 « cid;
00170
00171     // Signaler qu'une lecture vient d'avoir lieu.
00172     pthread_cond_signal(&self->pulled);
00173
00174     // Passer à l'élément suivant.
00175     self->consumer_records[cid] = rec->next;
00176
00177     pthread_mutex_unlock(&self->mutex);
00178 }
00179
00180 void LokiFIFO_abort(LokiFIFO *self) {
00181     pthread_mutex_lock(&self->mutex);
```

```
00182     self->abort = true;
00183     pthread_cond_signal(&self->pulled);
00184     pthread_cond_broadcast(&self->pushed);
00185     pthread_mutex_unlock(&self->mutex);
00186 }
00187
```

6.27 Référence du fichier libloki-fifo.h

```
#include <stddef.h>
#include <stdbool.h>
```

Graphe des dépendances par inclusion de libloki-fifo.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiFIFO LokiFIFO

Représentation d'une FIFO.

Fonctions

— LokiFIFO * LokiFIFO_new (size_t item_size, size_t length, size_t consumers)

Créer une FIFO.

— void LokiFIFO_del (LokiFIFO *self)

Détruire une FIFO.

— size_t LokiFIFO_get_item_size (const LokiFIFO *self)

Obtenir la taille d'un élément de la FIFO, en octets.

— void * LokiFIFO_get_producer_item (LokiFIFO *self)

Obtenir une référence vers l'élément courant en écriture.

— void LokiFIFO_push (LokiFIFO *self)

Valider l'élément courant en écriture.

— void * LokiFIFO_get_consumer_item (LokiFIFO *self, size_t cid)

Obtenir une référence vers l'élément courant en lecture.

— void LokiFIFO_pull (LokiFIFO *self, size_t cid)

Libérer l'élément courant en lecture.

— void LokiFIFO_abort (LokiFIFO *self)

Terminer les attentes en cours.

6.27.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-fifo.h.

6.28 libloki-fifo.h

Aller à la documentation de ce fichier.

```
00001
00004
00010
00012
00013 #ifndef LIBLOKI_FIFO_H
00014 #define LIBLOKI_FIFO_H
00015
00016 #include <stddef.h>
00017 #include <stdbool.h>
00018
00020 typedef struct LokiFIFO LokiFIFO;
00021
00032 LokiFIFO *LokiFIFO_new(size_t item_size, size_t length, size_t consumers);
00033
00038 void LokiFIFO_del(LokiFIFO *self);
00039
00045 size_t LokiFIFO_get_item_size(const LokiFIFO *self);
00046
00059 void *LokiFIFO_get_producer_item(LokiFIFO *self);
00060
00069 void LokiFIFO_push(LokiFIFO *self);
00070
00084 void *LokiFIFO_get_consumer_item(LokiFIFO *self, size_t cid);
00085
00098 void LokiFIFO_pull(LokiFIFO *self, size_t cid);
00099
00104 void LokiFIFO_abort(LokiFIFO *self);
00105
00106 #endif
00107
00110
```

6.29 Référence du fichier libloki-fir.c

```
#include "libloki-fir.h"
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <libloki-neon.h>
```

Graphe des dépendances par inclusion de libloki-fir.c:

Structures de données

— struct LokiFIRFilter

Représentation d'un filtre FIR.

Fonctions

— static void LokiFIRFilter_allocate_samples (LokiFIRFilter ∗self)

Allouer le tableau des échantillons mémorisés par un filtre FIR.

— LokiFIRFilter ∗ LokiFIRFilter_new (const float ∗coeffs, size_t length, size_t decimation)

Construire un filtre FIR.

— LokiFIRFilter ∗ LokiFIRFilter_clone (const LokiFIRFilter ∗other)

Construire un clone d'un filtre FIR.

— static float bessel0 (float x)

Function de Bessel de première espèce, d'ordre 0.

— LokiFIRFilter ∗ LokiFIRFilter_new_kaiser_bessel (float fs, float fa, float fb, float att, size_t length, size_t decimation)

Construire un filtre FIR en utilisant une fenêtre de Kaiser-Bessel.

— size_t LokiFIRFilter_get_actual_length (LokiFIRFilter ∗self)

Lire la longueur effective d'un filtre FIR.

— bool LokiFIRFilter_work (LokiFIRFilter ∗self, float in, float ∗out)

Exécuter un pas de calcul d'un fitre FIR.

— void LokiFIRFilter_reset (LokiFIRFilter ∗self)

Réinitialiser un filtre FIR.

— void LokiFIRFilter_del (LokiFIRFilter ∗self)

Détruire un filtre FIR.

6.29.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-fir.c.

6.30 libloki-fir.c

Aller à la documentation de ce fichier.

```
00001
```

```
00002 #include "libloki-fir.h"
00003
00004 #include <stdlib.h>
00005 #include <stdio.h>
00006 #include <string.h>
00007 #include <math.h>
00008
00009 #ifndef NO_SIMD
00010 #include <libloki-neon.h>
00011 #endif
00012
00015
00017 struct LokiFIRFilter {
00019     bool is_clone;
00021     float *coeffs;
00023     float *coeffs_end;
00025     float *coeff_ptr;
00027     float *samples;
00029     float *samples_end;
00031     size_t sample_index;
00033     float *phase_ptr;
00035     float acc;
00037     size_t length;
00039     size_t decimation;
00041     size_t phase_length;
00042 };
00043
00050 static inline void LokiFIRFilter_allocate_samples(LokiFIRFilter *self) {
00051     size_t buffer_length = self->length;
00052     #ifndef NO_SIMD
00053         // Si la vectorisation est activée, prévoir 3 échantillons de plus par phase.
00054         buffer_length += self->decimation * 3;
00055     #endif
00056     self->samples     = self->phase_ptr = calloc(buffer_length, sizeof(float));
00057     self->samples_end = self->samples + buffer_length;
00058 }
00059
00060 LokiFIRFilter *LokiFIRFilter_new(const float *coeffs, size_t length, size_t decimation) {
00061     LokiFIRFilter * self = calloc(1, sizeof(LokiFIRFilter));
00062     if (!self) {
00063         return NULL;
00064     }
00065
00066     // Ajuster la longueur du filtre pour qu'elle soit multiple de decimation.
00067     self->phase_length = length / decimation;
00068     if (length % decimation) {
00069         self->phase_length ++;
00070     }
00071     #ifndef NO_SIMD
00072         // Si la vectorisation est activée, la longueur de chaque sous-filtre sera
00073         // ajustée au multiple de 4 immédiatement supérieur.
00074         if (self->phase_length % 4) {
00075             self->phase_length += 4 - self->phase_length % 4;
00076         }
00077     #endif
00078
00079     self->length     = self->phase_length * decimation;
00080     self->decimation = decimation;
00081
00082     if (length != self->length) {
00083         fprintf(stderr, "LokiFIRFilter_new: Filter length was expanded from %zu to %zu\n", length,
      self->length);
00084     }
00085
00086     // Allouer les tableaux des coefficients et des échantillons précédents.
00087     self->coeffs      = self->coeff_ptr = calloc(self->length, sizeof(float));
00088     self->coeffs_end  = self->coeffs  + self->length;
00089     LokiFIRFilter_allocate_samples(self);
00090     if (!self->coeffs || !self->samples) {
00091         LokiFIRFilter_del(self);
00092         return NULL;
00093     }
00094
00095     // Ranger les coefficients en les répartissant par phase
00096     // et en renversant l'ordre des coefficients.
00097     for (size_t i = 0; i < length; i ++) {
00098         size_t p = i % decimation;       // Numéro de phase (< decimation)
00099         size_t q = (i - p) / decimation; // Indice dans la phase (< n)
00100         self->coeffs[p * self->phase_length + q] = coeffs[length - i - 1];
00101     }
00102
00103     return self;
00104 }
00105
00106 LokiFIRFilter *LokiFIRFilter_clone(const LokiFIRFilter *other) {
00107     LokiFIRFilter * self = calloc(1, sizeof(LokiFIRFilter));
00108     if (!self) {
```

```
00109         return NULL;
00110     }
00111
00112     memcpy(self, other, sizeof(LokiFIRFilter));
00113
00114     self->is_clone = true;
00115     LokiFIRFilter_allocate_samples(self);
00116     if (!self->samples) {
00117         LokiFIRFilter_del(self);
00118         return NULL;
00119     }
00120
00121     return self;
00122 }
00123
00125 static float bessel0(float x) {
00126     float d = 0, ds = 1, s = 1;
00127     do {
00128         d  += 2;
00129         ds *= x * x / d / d;
00130         s  += ds;
00131     }
00132     while (ds > s * 1e-6f);
00133     return s;
00134 }
00135
00136 LokiFIRFilter *LokiFIRFilter_new_kaiser_bessel(float fs, float fa, float fb, float att, size_t length,
      size_t decimation) {
00137     // Ajuster la taille du filtre à un nombre impair de coefficients.
00138     const size_t half_length = (length - 1) / 2;
00139     length = half_length * 2 + 1;
00140
00141     // Allouer le tableau des coefficients.
00142     float *coeffs = calloc(length, sizeof(float));
00143     if (!coeffs) {
00144         return NULL;
00145     }
00146
00147     // Calculer la partie droite de la réponse impulsionnelle du fitre idéal.
00148     coeffs[half_length] = (fb - fa) * 2 / fs;
00149     for (size_t i = 1; i <= half_length; i ++) {
00150         coeffs[half_length + i] = (sinf(M_PI * 2 * i * fb / fs) - sinf(M_PI * 2 * i * fa / fs)) / i /
      M_PI;
00151     }
00152
00153     // Calculer le facteur de forme de la fenêtre de Kaiser-Bessel.
00154     float alpha = att < 21 ? 0.0f :
00155                   att > 50 ? 0.1102f * (att - 8.7f) :
00156                              0.5842f * powf((att - 21.0f), 0.4f) + 0.07886f * (att - 21.0f);
00157
00158     // Appliquer la fenêtre sur la réponse idéale.
00159     float alpha0 = bessel0(alpha);
00160     for (size_t i = 0; i <= half_length; i ++) {
00161         coeffs[half_length + i] *= bessel0(alpha * sqrtf(1.0f - (float)(i * i) / half_length /
      half_length)) / alpha0;
00162         coeffs[half_length - i] = coeffs[half_length + i];
00163     }
00164
00165     LokiFIRFilter *self = LokiFIRFilter_new(coeffs, length, decimation);
00166     free(coeffs);
00167
00168     return self;
00169 }
00170
00171 size_t LokiFIRFilter_get_actual_length(LokiFIRFilter *self) {
00172     return self->length;
00173 }
00174
00175 bool LokiFIRFilter_work(LokiFIRFilter *self, float in, float *out) {
00176     // Copie locale de l'état du filtre.
00177     const size_t phase_length     = self->phase_length;
00178     const size_t decimation       = self->decimation;
00179     float *phase_ptr              = self->phase_ptr;
00180     float *const phase_end        = phase_ptr + phase_length;
00181     size_t sample_index           = self->sample_index;
00182     float *sample_ptr             = phase_ptr + sample_index;
00183     float *const samples          = self->samples;
00184     float *const samples_end      = self->samples_end;
00185     const float *coeff_ptr        = self->coeff_ptr;
00186     const float *const coeffs     = self->coeffs;
00187     const float *const coeffs_end = self->coeffs_end;
00188     float acc                     = self->acc;
00189
00190     #ifdef NO_SIMD
00191         // Version de référence sans vectorisation
00192
00193         // Insérer le nouvel échantillon dans le tampon de réception.
```

```
00194         *sample_ptr++ = in;
00195
00196         for (size_t i = 0; i < phase_length; i ++) {
00197             if (sample_ptr == phase_end) {
00198                 sample_ptr = phase_ptr;
00199             }
00200
00201             acc += (*sample_ptr ++) * (*coeff_ptr ++);
00202
00203             if (coeff_ptr == coeffs_end) {
00204                 coeff_ptr = coeffs;
00205             }
00206         }
00207     #else
00208         // Vectorisation ARM NEON
00209         float32x4_t vsamp, vcoeff;
00210         float32x4_t vacc = vmovq_n_f32(0);
00211
00212         // Insérer le nouvel échantillon dans le tampon de réception.
00213         // Les trois premiers échantillons du buffer sont dupliqués en fin
00214         // de buffer de la phase courante. Cela permet de traiter le buffer
00215         // par vecteurs de 4 échantillons même si sample_ptr n'est pas aligné
00216         // sur un multiple de 4.
00217         if (sample_index < 3) {
00218             sample_ptr[phase_length] = in;
00219         }
00220         *sample_ptr++ = in;
00221
00222         float *saved_sample_ptr = sample_ptr;
00223
00224         // Traiter les échantillons par vecteurs de 4 jusqu'à la fin du buffer
00225         // pour la phase courante.
00226         while (sample_ptr < phase_end) {
00227             vsamp  = vld1q_f32(sample_ptr);
00228             vcoeff = vld1q_f32(coeff_ptr);
00229             vacc   = vmlaq_f32(vacc, vsamp, vcoeff);
00230
00231             sample_ptr += 4;
00232             coeff_ptr  += 4;
00233
00234             if (coeff_ptr == coeffs_end) {
00235                 coeff_ptr = coeffs;
00236             }
00237         }
00238
00239         // Lorsque la fin du buffer est dépassée, reprendre au début.
00240         sample_ptr -= phase_length;
00241
00242         // Traiter les échantillons à partir du début du buffer
00243         // jusqu'à l'emplacement de départ.
00244         while (sample_ptr != saved_sample_ptr) {
00245             vsamp  = vld1q_f32(sample_ptr);
00246             vcoeff = vld1q_f32(coeff_ptr);
00247             vacc   = vmlaq_f32(vacc, vsamp, vcoeff);
00248
00249             sample_ptr += 4;
00250             coeff_ptr  += 4;
00251
00252             if (coeff_ptr == coeffs_end) {
00253                 coeff_ptr = coeffs;
00254             }
00255         }
00256
00257         // Calculer la somme des quatre accumulateurs.
00258         float32x2_t vacc2 = vadd_f32(vget_low_f32(vacc), vget_high_f32(vacc));
00259         acc += vget_lane_f32(vacc2, 0) + vget_lane_f32(vacc2, 1);
00260
00261         phase_ptr += 3;
00262     #endif
00263
00264     // Passer à la phase suivante dans le tableau des échantillons.
00265     phase_ptr += phase_length;
00266     if (phase_ptr == samples_end) {
00267         phase_ptr = samples;
00268         sample_index ++;
00269         if (sample_index == phase_length) {
00270             sample_index = 0;
00271         }
00272     }
00273
00274     // Lorsqu'on revient au premier coefficient,
00275     // le calcul d'un échantillon de sortie est terminé.
00276     bool res = coeff_ptr == coeffs;
00277     if (res) {
00278         *out = acc;
00279         self->acc = 0;
00280     }
```

```
00281     else {
00282         self->acc = acc;
00283     }
00284 
00285     // Mise à jour de l'état du filtre.
00286     self->phase_ptr    = phase_ptr;
00287     self->sample_index = sample_index;
00288     self->coeff_ptr    = (float*)coeff_ptr;
00289 
00290     return res;
00291 }
00292 
00293 void LokiFIRFilter_reset(LokiFIRFilter *self) {
00294     memset(self->samples, 0, (self->samples_end - self->samples) * sizeof(float));
00295 }
00296 
00297 void LokiFIRFilter_del(LokiFIRFilter *self) {
00298     if (!self) {
00299         return;
00300     }
00301 
00302     if (!self->is_clone && self->coeffs) {
00303         free(self->coeffs);
00304     }
00305 
00306     if (self->samples) {
00307         free(self->samples);
00308     }
00309 
00310     free(self);
00311 }
00312 
```

6.31 Référence du fichier libloki-fir.h

```
#include <stdbool.h>
#include <stddef.h>
```

Graphe des dépendances par inclusion de libloki-fir.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiFIRFilter LokiFIRFilter

Représentation d'un filtre FIR.

Fonctions

— LokiFIRFilter * LokiFIRFilter_new (const float *coeffs, size_t length, size_t decimation)

Construire un filtre FIR.

— LokiFIRFilter * LokiFIRFilter_clone (const LokiFIRFilter *other)

Construire un clone d'un filtre FIR.

— LokiFIRFilter * LokiFIRFilter_new_kaiser_bessel (float fs, float fa, float fb, float att, size_t length, size_t decimation)

Construire un filtre FIR en utilisant une fenêtre de Kaiser-Bessel.

— bool LokiFIRFilter_work (LokiFIRFilter *self, float in, float *out)

Exécuter un pas de calcul d'un fitre FIR.

— void LokiFIRFilter_reset (LokiFIRFilter ∗self)

Réinitialiser un filtre FIR.

— size_t LokiFIRFilter_get_actual_length (LokiFIRFilter ∗self)

Lire la longueur effective d'un filtre FIR.

— void LokiFIRFilter_del (LokiFIRFilter ∗self)

Détruire un filtre FIR.

6.31.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-fir.h.

6.32 libloki-fir.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_FIR_H_
00011 #define LIBLOKI_FIR_H_
00012
00013 #include <stdbool.h>
00014 #include <stddef.h>
00015
00017 typedef struct LokiFIRFilter LokiFIRFilter;
00018
00032 LokiFIRFilter *LokiFIRFilter_new(const float *coeffs, size_t length, size_t decimation);
00033
00042 LokiFIRFilter *LokiFIRFilter_clone(const LokiFIRFilter *other);
00043
00064 LokiFIRFilter *LokiFIRFilter_new_kaiser_bessel(float fs, float fa, float fb, float att, size_t length,
      size_t decimation);
00065
00078 bool LokiFIRFilter_work(LokiFIRFilter *self, float in, float *out);
00079
00086 void LokiFIRFilter_reset(LokiFIRFilter *self);
00087
00099 size_t LokiFIRFilter_get_actual_length(LokiFIRFilter *self);
00100
00105 void LokiFIRFilter_del(LokiFIRFilter *self);
00106
00107 #endif
00108
```

6.33 Référence du fichier libloki-formatter.c

```
#include "libloki-formatter.h"
#include <stdlib.h>
#include <string.h>
```

Graphe des dépendances par inclusion de libloki-formatter.c:

Structures de données

— struct LokiFormatter

Représentation d'un formateur de séries temporelles.

Fonctions

— LokiFormatter ∗ LokiFormatter_new (void ∗config)

Construire un nouveau formateur de séries temporelles.

— LokiFormatter ∗ LokiFormatter_new_with_implementation (const LokiFormatterInterface ∗impl, void ∗config)

Construire un nouveau formateur de séries temporelles avec une implémentation donnée.

— void LokiFormatter_work (LokiFormatter ∗self, const char ∗filename, LokiTimestamp timestamp, bool file_change, void ∗data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency)

Enregistrer un paquet de données.

— void LokiFormatter_close (LokiFormatter ∗self)

Fermer un formateur de séries temporelles.

— void LokiFormatter_del (LokiFormatter ∗self)

Détruire de mise en forme de séries temporelles.

6.33.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-formatter.c.

6.34 libloki-formatter.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki-formatter.h"
00003
00004 #include <stdlib.h>
00005 #include <string.h>
00006
00009
00011 struct LokiFormatter {
00013     void *instance;
00019     LokiFormatterInterface impl;
```

```
00020 };
00021
00022 LokiFormatter * LokiFormatter_new(void *config) {
00023     return NULL;
00024 }
00025
00026 LokiFormatter *LokiFormatter_new_with_implementation(const LokiFormatterInterface *impl, void *config)
      {
00027     LokiFormatter *self = calloc(1, sizeof(LokiFormatter));
00028     if (!self) {
00029         return NULL;
00030     }
00031
00032     self->instance = impl->new(config);
00033     if (!self->instance) {
00034         LokiFormatter_del(self);
00035         return NULL;
00036     }
00037
00038     memcpy(&self->impl, impl, sizeof(LokiFormatterInterface));
00039
00040     return self;
00041 }
00042
00043 void LokiFormatter_work(LokiFormatter *self, const char *filename, LokiTimestamp timestamp, bool
      file_change, void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size,
      float sampling_frequency) {
00044     self->impl.work(self->instance, filename, timestamp, file_change, data, length, max_channels,
      channels, sample_size, sampling_frequency);
00045 }
00046
00047 void LokiFormatter_close(LokiFormatter *self) {
00048     self->impl.close(self->instance);
00049 }
00050
00051 void LokiFormatter_del(LokiFormatter *self) {
00052     if (!self) {
00053         return;
00054     }
00055
00056     self->impl.del(self->instance);
00057     free(self);
00058 }
00059
```

6.35 Référence du fichier libloki-formatter.h

```
#include "libloki-class.h"
#include "libloki-time.h"
#include <stdbool.h>
#include <stddef.h>
#include <inttypes.h>
```

Graphe des dépendances par inclusion de libloki-formatter.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiFormatter LokiFormatter

Représentation d'un formateur de séries temporelles.

Fonctions

— LokiFormatter ∗ LokiFormatter_new (void ∗config)

Construire un nouveau formateur de séries temporelles.

— void LokiFormatter_work (LokiFormatter ∗self, const char ∗filename, LokiTimestamp timestamp, bool file_change, void ∗data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency)

Enregistrer un paquet de données.

— void LokiFormatter_close (LokiFormatter ∗self)

Fermer un formateur de séries temporelles.

— void LokiFormatter_del (LokiFormatter ∗self)

Détruire de mise en forme de séries temporelles.

— **LokiClass_interface** (LokiFormatter, LokiClass_method(LokiFormatter, new) LokiClass_method(LokiFormatter, work) LokiClass_method(LokiFormatter, close) LokiClass_method(LokiFormatter, del))

— LokiFormatter ∗ LokiFormatter_new_with_implementation (const LokiFormatterInterface ∗impl, void ∗config)

Construire un nouveau formateur de séries temporelles avec une implémentation donnée.

6.35.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-formatter.h.

6.36 libloki-formatter.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_FORMATTER_H_
00011 #define LIBLOKI_FORMATTER_H_
00012
00013 #include "libloki-class.h"
00014 #include "libloki-time.h"
00015
00016 #include <stdbool.h>
00017 #include <stddef.h>
00018 #include <inttypes.h>
00019
00021 typedef struct LokiFormatter LokiFormatter;
00022
00028 LokiFormatter *LokiFormatter_new(void *config);
00029
00045 void LokiFormatter_work(LokiFormatter *self, const char *filename, LokiTimestamp timestamp, bool
      file_change, void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size,
      float sampling_frequency);
00046
00054 void LokiFormatter_close(LokiFormatter *self);
00055
00060 void LokiFormatter_del(LokiFormatter *self);
00061
00062 LokiClass_interface(LokiFormatter,
00063     LokiClass_method(LokiFormatter, new)
00064     LokiClass_method(LokiFormatter, work)
```

```
00065     LokiClass_method(LokiFormatter, close)
00066     LokiClass_method(LokiFormatter, del)
00067 );
00068
00075 LokiFormatter *LokiFormatter_new_with_implementation(const LokiFormatterInterface *impl, void
      *config);
00076
00077 #endif
00078
```

6.37 Référence du fichier libloki-hooks.c

```
#include <dlfcn.h>
#include <malloc.h>
#include <stdio.h>
```

Graphe des dépendances par inclusion de libloki-hooks.c:

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `dladdr` pour connaître le nom de la fonction référencée par un pointeur.

Fonctions

— static void * LokiHooks_malloc (size_t size, const void *caller)

Une version instrumentée du hook de la fonction `malloc`.

— static void LokiHooks_free (void *ptr, const void *caller)

Une version instrumentée du hook de la fonction `free`.

— static void LokiHooks_setup (void)

Affecter les hooks des fonctions `malloc` et `free`.

— static void LokiHooks_save (void)

Sauvegarder les hooks des fonctions `malloc` et `free`.

— void LokiHooks_restore (void)

Restaurer les hooks d'origine pour l'allocation/désallocation de mémoire.

— void LokiHooks_init (void)

Initialiser l'utilisation des hooks instrumentés pour l'allocation/désallocation de mémoire.

Variables

— static void *(*) LokiHooks_saved_malloc (size_t, const void *)

Pointeur vers le hook qui exécute la fonction `malloc` d'origine.

— static void(∗) LokiHooks_saved_free (void ∗, const void ∗)

Pointeur vers le hook qui exécute la fonction `free` d'origine.

6.37.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-hooks.c.

6.37.2 Documentation des macros

6.37.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `dladdr` pour connaître le nom de la fonction référencée par un pointeur.

Définition à la ligne 3 du fichier libloki-hooks.c.

6.38 libloki-hooks.c

Aller à la documentation de ce fichier.

```
00001
00003 #define _GNU_SOURCE
00004
00005 #include <dlfcn.h>
00006 #include <malloc.h>
00007 #include <stdio.h>
00008
00011
00013 static void* (*LokiHooks_saved_malloc)(size_t, const void*);
00014
00016 static void (*LokiHooks_saved_free)(void*, const void*);
00017
00018 static void *LokiHooks_malloc(size_t, const void*);
00019
00020 static void LokiHooks_free(void*, const void*);
00021
00023 static void LokiHooks_setup(void) {
00024     __malloc_hook = LokiHooks_malloc;
00025     __free_hook   = LokiHooks_free;
00026 }
00027
00029 static void LokiHooks_save(void) {
00030     LokiHooks_saved_malloc = __malloc_hook;
00031     LokiHooks_saved_free   = __free_hook;
00032 }
00033
00034 void LokiHooks_restore(void) {
00035     __malloc_hook = LokiHooks_saved_malloc;
00036     __free_hook   = LokiHooks_saved_free;
00037 }
00038
00039 void LokiHooks_init(void) {
00040     if (__malloc_hook == LokiHooks_malloc) {
```

```
00041         return;
00042     }
00043     LokiHooks_save();
00044     LokiHooks_setup();
00045 }
00046
00056 static void *LokiHooks_malloc(size_t size, const void *caller) {
00057     LokiHooks_restore();
00058     void *result = malloc(size);
00059     LokiHooks_save();
00060
00061     Dl_info info;
00062     dladdr(caller, &info);
00063
00064     // fprintf utilise peut-être malloc.
00065     // Ne pas restaurer les hooks avant cet appel.
00066     fprintf(stderr, "LokiHooks_malloc: %p (allocated %zu bytes), called by %s\n", result, size,
      info.dli_sname);
00067
00068     LokiHooks_setup();
00069     return result;
00070 }
00071
00080 static void LokiHooks_free(void *ptr, const void *caller) {
00081     LokiHooks_restore();
00082     free(ptr);
00083     LokiHooks_save();
00084
00085     Dl_info info;
00086     dladdr(caller, &info);
00087
00088     // fprintf utilise peut-être malloc.
00089     // Ne pas restaurer les hooks avant cet appel.
00090     fprintf (stderr, "LokiHooks_free: %p, called by %s\n", ptr, info.dli_sname);
00091
00092     LokiHooks_setup();
00093 }
00094
```

6.39 Référence du fichier libloki-hooks.h

Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Fonctions

— void LokiHooks_init (void)

Initialiser l'utilisation des hooks instrumentés pour l'allocation/désallocation de mémoire.

— void LokiHooks_restore (void)

Restaurer les hooks d'origine pour l'allocation/désallocation de mémoire.

6.39.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-hooks.h.

6.40 libloki-hooks.h

Aller à la documentation de ce fichier.

```
00001
00008
00010
00011 #ifndef LOKI_HOOKS_H_
00012 #define LOKI_HOOKS_H_
00013
00015 void LokiHooks_init(void);
00016
00018 void LokiHooks_restore(void);
00019
00020 #endif
00021
```

6.41 Référence du fichier libloki-kalman.c

```
#include "libloki-kalman.h"
#include "libloki-ellipse.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
```

Graphe des dépendances par inclusion de libloki-kalman.c:

Structures de données

— struct LokiKalman

Représentation d'un filtre de Kalman.

Fonctions

— LokiKalman * LokiKalman_new (float initial_radius, const float params[static 3])

Construire un nouveau filtre de Kalman.

— LokiKalman * LokiKalman_clone (const LokiKalman *other)

Construire un clone d'un filtre de Kalman.

— float * LokiKalman_get_state (LokiKalman *self)

Retourner l'état courant du filtre.

— void LokiKalman_work (LokiKalman *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— void LokiKalman_del (LokiKalman *self)

Détruire un filtre de Kalman.

— **LokiClass_implementation** (LokiEllipseEstimator, LokiKalman, LokiClass_set_attribute(params_↩ length, 3) LokiClass_define_method(LokiKalman, new) LokiClass_define_method(LokiKalman, clone) LokiClass_define_method(LokiKalman, work) LokiClass_define_method(LokiKalman, get_state) LokiClass_define_method(Lok del))

6.41.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-kalman.c.

6.42 libloki-kalman.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki-kalman.h"
00003 #include "libloki-ellipse.h"
00004
00005 #include <stdlib.h>
00006 #include <string.h>
00007 #include <stdio.h>
00008
00011
00013 struct LokiKalman {
00015     float rv;
00017     float rw;
00019     float P[5][5];
00021     LokiEllipseEquation state;
00022 };
00023
00024 LokiKalman *LokiKalman_new(float initial_radius, const float params[static 3]) {
00025     LokiKalman * self = calloc(1, sizeof(LokiKalman));
00026     if (!self) {
00027         return NULL;
00028     }
00029
00030     fprintf(stderr, "LokiKalman_new: creating Kalman filter with Rv=%f Rw=%f P=%f\n", params[0],
      params[1], params[2]);
00031
00032     self->rv = params[0];
00033     self->rw = params[1];
00034
00035     self->state[0] = 0.5f;
00036     self->state[4] = -initial_radius * initial_radius / 2;
00037
00038     for (size_t i = 0; i < 5; i ++) {
00039         self->P[i][i] = params[2];
00040     }
00041
00042     return self;
00043 }
00044
00045 LokiKalman *LokiKalman_clone(const LokiKalman *other) {
00046     LokiKalman * self = calloc(1, sizeof(LokiKalman));
00047     if (!self) {
00048         return NULL;
00049     }
00050
00051     memcpy(self, other, sizeof(LokiKalman));
00052
00053     return self;
00054 }
00055
00056 float *LokiKalman_get_state(LokiKalman *self) {
00057     return self->state;
00058 }
00059
00060 void LokiKalman_work(LokiKalman *self, float x, float y) {
00061     const float out  = -y * y;
00062     const float in[] = {x * x + out, x * y, x, y, 1.0f};
00063
00064     // Copies locales de l'état du filtre.
00065     float (* const P)[5] = self->P;
00066     float *const state   = self->state;
```

```
00067     float rw             = self->rw;
00068
00069     // Dans les calculs suivants, on tient compte du fait que in[4] = 1
00070
00071     // Calcul de l'erreur courante :
00072     // epsilon = out - in * state
00073     float epsilon = out - state[4];
00074     for (size_t i = 0; i < 4; i ++) {
00075         epsilon -= in[i] * state[i];
00076     }
00077
00078     // Calcul du produits L = P * in'
00079     float L[5];
00080     for (size_t i = 0; i < 5; i ++) {
00081         L[i] = P[i][4];
00082         for (size_t j = 0; j < 4; j ++) {
00083             L[i] += P[i][j] * in[j];
00084         }
00085     }
00086
00087     // Calcul du dénominateur K = Rv + in * P * in'
00088     float K = self->rv + L[4];
00089     for (size_t i = 0; i < 4; i ++) {
00090         K += in[i] * L[i];
00091     }
00092
00093     // Normalisation du vecteur L et mise à jour de l'état.
00094     float K_inv = 1.0f / K;
00095     for (size_t i = 0; i < 5; i ++) {
00096         L[i] *= K_inv;
00097         state[i] += epsilon * L[i];
00098     }
00099
00100     // Calcul de J = I - L * in
00101     float J[5][5];
00102     for (size_t i = 0; i < 5; i ++) {
00103         J[i][i] = 1.0f - L[i] * in[i];
00104         for (size_t j = i + 1; j < 5; j ++) {
00105             J[i][j] = -L[i] * in[j];
00106             J[j][i] = -L[j] * in[i];
00107         }
00108     }
00109
00110     // Mise à jour de la matrice P_next = J * P + Rw
00111     float P_next[5][5];
00112     for (size_t i = 0; i < 5; i ++) {
00113         for (size_t j = 0; j < 5; j ++) {
00114             P_next[i][j] = J[i][0] * P[0][j];
00115             for (size_t k = 1; k < 5; k ++) {
00116                 P_next[i][j] += J[i][k] * P[k][j];
00117             }
00118         }
00119         P_next[i][i] += rw;
00120     }
00121
00122     // Cette boucle est plus rapide que memcpy(P, P_next, sizeof(P_next)).
00123     // Il est possible que le compilateur optimise le graphe flot de données
00124     // sans passer par l'allocation du tableau P_next.
00125     for (size_t i = 0; i < 5; i ++) {
00126         for (size_t j = 0; j < 5; j ++) {
00127             P[i][j] = P_next[i][j];
00128         }
00129     }
00130 }
00131
00132 void LokiKalman_del(LokiKalman *self) {
00133     if (!self) {
00134         return;
00135     }
00136     free(self);
00137 }
00138
00139 LokiClass_implementation(LokiEllipseEstimator, LokiKalman,
00140     LokiClass_set_attribute(params_length, 3)
00141     LokiClass_define_method(LokiKalman, new)
00142     LokiClass_define_method(LokiKalman, clone)
00143     LokiClass_define_method(LokiKalman, work)
00144     LokiClass_define_method(LokiKalman, get_state)
00145     LokiClass_define_method(LokiKalman, del)
00146 );
00147
```

6.43 Référence du fichier libloki-kalman.h

```
#include "libloki-ellipse-estimator.h"
```

Graphe des dépendances par inclusion de libloki-kalman.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiKalman LokiKalman

Représentation d'un filtre de Kalman.

Fonctions

— LokiKalman * LokiKalman_new (float initial_radius, const float params[static 3])

Construire un nouveau filtre de Kalman.

— LokiKalman * LokiKalman_clone (const LokiKalman *other)

Construire un clone d'un filtre de Kalman.

— void LokiKalman_work (LokiKalman *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— float * LokiKalman_get_state (LokiKalman *self)

Retourner l'état courant du filtre.

— void LokiKalman_del (LokiKalman *self)

Détruire un filtre de Kalman.

— **LokiClass_declare_implementation** (LokiEllipseEstimator, LokiKalman)

6.43.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-kalman.h.

6.44 libloki-kalman.h

Aller à la documentation de ce fichier.

```
00001
00008
00010
00011 #ifndef LIBLOKI_KALMAN_H_
00012 #define LIBLOKI_KALMAN_H_
00013
00014 #include "libloki-ellipse-estimator.h"
00015
00017 typedef struct LokiKalman LokiKalman;
00018
00025 LokiKalman *LokiKalman_new(float initial_radius, const float params[static 3]);
00026
00032 LokiKalman *LokiKalman_clone(const LokiKalman *other);
00033
00043 void LokiKalman_work(LokiKalman *self, float x, float y);
00044
00050 float *LokiKalman_get_state(LokiKalman *self);
00051
00056 void LokiKalman_del(LokiKalman *self);
00057
00058 LokiClass_declare_implementation(LokiEllipseEstimator, LokiKalman);
00059
00060 #endif
00061
```

6.45 Référence du fichier libloki-modem.c

```
#include <math.h>
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <loki-modem-regs.h>
#include <loki-device-ioctl.h>
#include "libloki.h"
```

Graphe des dépendances par inclusion de libloki-modem.c:

Structures de données

— struct LokiModem

Représentation des périphériques modulateur et démodulateur.

— union AD8370ControlWord

Représentation du mot de contrôle du gain pour l'amplificateur AD8370.

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

— #define AD8370_VERNIER 0.055744

Le paramètre Vernier pour le réglage du gain de l'amplificateur AD8370.

— #define AD8370_PREGAIN 7.079458

Le paramètre PreGain pour le réglage du gain de l'amplificateur AD8370.

— #define AD8370_HIGH_GAIN_FACTOR (AD8370_VERNIER ∗ AD8370_PREGAIN)

Le facteur appliqué au code de gain dans le mode High Gain de l'amplificateur AD8370.

— #define AD8370_GAIN_CODE_SIZE 7

Le nombre de bits du code de gain de l'amplificateur AD8370.

— #define AD8370_GAIN_CODE_MAX ((1 << AD8370_GAIN_CODE_SIZE) - 1)

La valeur maximale du code de gain de l'amplificateur AD8370.

— #define AD8370_LOW_GAIN_MAX (AD8370_GAIN_CODE_MAX ∗ AD8370_VERNIER)

Le gain maximal de l'amplificateur AD8370 en mode Low Gain.

— #define AD8370_HIGH_GAIN_MAX (AD8370_GAIN_CODE_MAX ∗ AD8370_HIGH_GAIN_FACTOR)

Le gain maximal de l'amplificateur AD8370 en mode High Gain.

— #define SIMULATED_DISP_WAVE_FREQUENCY 0.2

La fréquence du déplacement en mode simulation de capteurs, en Hz.

— #define SIMULATED_DISP_AMPLITUDE (M_PI / 10)

L'amplitude du déplacement en mode simulation de capteurs, en radians.

— #define SIMULATED_ELLIPSE_A 0.1

Le demi-grand axe de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_B 0.15

Le demi-petit axe de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_ANGLE (M_PI / 6)

L'angle de rotation de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_XC 0.01

La coordonnée X du centre de l'ellipse en mode simulation de capteurs.

— #define SIMULATED_ELLIPSE_YC 0.02

La coordonnée Y du centre de l'ellipse en mode simulation de capteurs.

— #define USE_MMAP

Utiliser la fonction `mmap` pour accéder aux données DMA.

Définitions de type

— typedef uint64_t LokiDMASample

Le type brut d'un échantillon démodulé transféré par DMA.

Fonctions

— static uint32_t LokiModem_get_reg (const LokiModem *self, LokiModemRegister reg)

Lire un registre du circuit de modulation/démodulation.

— static void LokiModem_set_reg (const LokiModem *self, LokiModemRegister reg, uint32_t value)

Affecter un registre du circuit de modulation/démodulation.

— static void LokiModem_dma_start (const LokiModem *self)

Demander au circuit de modulation/démodulation de démarrer les transferts DMA.

— static void LokiModem_dma_stop (const LokiModem *self)

Demander au circuit de modulation/démodulation d'arrêter les transferts DMA.

— static bool LokiModem_dma_is_streaming (const LokiModem *self)

Lire l'état des transferts DMA.

— static void LokiModem_dma_get_start_time (const LokiModem *self, struct timespec *res)

Lire la date de démarrage des transferts DMA.

— static void LokiModem_dma_get_packet_time (const LokiModem *self, struct timespec *res)

Lire la date du dernier transfert DMA.

— static unsigned LokiModem_dma_get_dropped_packets (const LokiModem *self)

Lire le nombre de paquets DMA perdus.

— static float LokiModem_scale_xy (const LokiModem *self, float xy)

Mettre à l'échelle une composante X (I) ou Y (Q) d'un échantillon démodulé.

— static float LokiModem_scale_power (const LokiModem *self, float p)

Mettre à l'échelle une valeur d'entrée du démodulateur.

— LokiModem * LokiModem_new (void)

Construire un nouveau modem.

— void LokiModem_del (LokiModem *self)

Détruire un modem.

— uint32_t LokiModem_get_clk_frequency (const LokiModem *self)

Lire la fréquence d'horloge du circuit d'acquisition.

— float LokiModem_get_mod_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en sortie du modulateur.

— void LokiModem_set_mod_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en sortie du modulateur.

— float LokiModem_get_mod_osc1_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

— void LokiModem_set_mod_osc1_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain_res (const LokiModem ∗self)

Lire le pas de réglage de l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain_max (const LokiModem ∗self)

Lire l'amplitude maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain (const LokiModem ∗self)

Lire l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

— void LokiModem_set_mod_osc1_gain (const LokiModem ∗self, float gain)

Affecter une amplitude au signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc2_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

— void LokiModem_set_mod_osc2_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain_res (const LokiModem ∗self)

Lire le pas de réglage de l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain_max (const LokiModem ∗self)

Lire l'amplitude maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain (const LokiModem ∗self)

Lire l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

— void LokiModem_set_mod_osc2_gain (const LokiModem ∗self, float gain)

Affecter une amplitude au signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

— void LokiModem_set_demod_osc_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_mod_offset_res (const LokiModem ∗self)

Lire le pas de réglage de l'offset en sortie du modulateur.

— float LokiModem_get_mod_offset_max (const LokiModem ∗self)

Lire l'offset maximal supporté en sortie du modulateur.

— float LokiModem_get_mod_offset (const LokiModem ∗self)

Lire l'offset en sortie du modulateur.

— void LokiModem_set_mod_offset (const LokiModem ∗self, float offset)

Affecter un offset en sortie du modulateur.

— float LokiModem_get_mod_dc_value_res (const LokiModem ∗self)

Lire le pas de réglage du courant moyen de commande de la diode laser.

— float LokiModem_get_mod_dc_value_max (const LokiModem ∗self)

Lire le courant moyen maximal supporté pour la commande de la diode laser.

— float LokiModem_get_mod_dc_value (const LokiModem ∗self)

Lire le courant moyen de commande de la diode laser.

— void LokiModem_set_mod_dc_value (const LokiModem ∗self, float dc_value)

Affecter un courant moyen de commande de la diode laser.

— uint32_t LokiModem_get_mod_pga_gain_max (const LokiModem ∗self)

Lire le gain maximal de l'amplificateur de sortie.

— uint32_t LokiModem_get_mod_pga_gain (const LokiModem ∗self)

Lire le gain de l'amplificateur de sortie.

— void LokiModem_set_mod_pga_gain (const LokiModem ∗self, uint32_t gain)

Affecter un gain à l'amplificateur de sortie.

— uint32_t LokiModem_get_demod_channels (const LokiModem ∗self)

Lire le nombre de voies du démodulateur.

— uint32_t LokiModem_get_demod_decimation (const LokiModem ∗self)

Lire le facteur de décimation des filtres du démodulateur.

— float LokiModem_get_demod_bandwidth (const LokiModem ∗self)

Lire la bande passante des filtres du démodulateur.

— float LokiModem_get_demod_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en entrée du démodulateur.

— void LokiModem_set_demod_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en entrée du démodulateur.

— float LokiModem_get_demod_xy_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en sortie du démodulateur.

— void LokiModem_set_demod_xy_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en sortie du démodulateur.

— uint32_t LokiModem_get_demod_delay_x_max (const LokiModem ∗self, uint32_t channel)

Lire le retard maximal applicable à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x_max (const LokiModem ∗self, uint32_t channel)

Lire le déphasage maximal applicable à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x_res (const LokiModem ∗self, uint32_t channel)

Lire le pas de réglage du déphasage applicable à la composante X (I) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_x (const LokiModem ∗self, uint32_t channel)

Lire le retard appliqué à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x (const LokiModem ∗self, uint32_t channel)

Lire le déphasage appliqué à la composante X (I) du signal démodulé pour une voie.

— void LokiModem_set_demod_delay_x (const LokiModem ∗self, uint32_t channel, uint32_t delay)

Affecter un retard à la composante X (I) du signal démodulé pour une voie.

— void LokiModem_set_demod_phase_x (const LokiModem ∗self, uint32_t channel, float phase)

Affecter un déphasage à la composante X (I) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_y_max (const LokiModem ∗self, uint32_t channel)

Lire le retard maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y_max (const LokiModem ∗self, uint32_t channel)

Lire le déphasage maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y_res (const LokiModem ∗self, uint32_t channel)

Lire le pas de réglage du déphasage applicable à la composante Y (Q) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_y (const LokiModem ∗self, uint32_t channel)

Lire le retard appliqué à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y (const LokiModem ∗self, uint32_t channel)

Lire le déphasage appliqué à la composante Y (Q) du signal démodulé pour une voie.

— void LokiModem_set_demod_delay_y (const LokiModem ∗self, uint32_t channel, uint32_t delay)

Affecter un retard à la composante Y (Q) du signal démodulé pour une voie.

— void LokiModem_set_demod_phase_y (const LokiModem ∗self, uint32_t channel, float phase)

Affecter un déphasage à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_pga_gain_max (const LokiModem ∗self, uint32_t channel)

Lire le gain maximal de l'amplificateur en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_pga_gain (const LokiModem ∗self, uint32_t channel)

Lire le gain de l'amplificateur en entrée du démodulateur pour une voie.

— void LokiModem_set_demod_pga_gain (const LokiModem ∗self, uint32_t channel, float gain)

Affecter un gain à l'amplificateur en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_adc_min_ratio (const LokiModem ∗self, uint32_t channel)

Lire la valeur minimale du signal en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_adc_max_ratio (const LokiModem ∗self, uint32_t channel)

Lire la valeur maximale du signal en entrée du démodulateur pour une voie.

— uint32_t LokiModem_get_demod_packet_length_max (const LokiModem ∗self)

Lire la longueur maximale d'un paquet de données en sortie du démodulateur.

— uint32_t LokiModem_get_demod_packet_length (const LokiModem ∗self)

Lire la longueur d'un paquet de données en sortie du démodulateur.

— bool LokiModem_get_demod_mixerless (const LokiModem ∗self)

Identifier le type de démodulateur implanté dans le FPGA.

— bool LokiModem_get_demod_bypass (const LokiModem ∗self)

Lire l'état d'activation du démodulateur.

— void LokiModem_set_demod_bypass (const LokiModem ∗self, bool bypass)

Activer ou désactiver le démodulateur.

— bool LokiModem_get_test_mode (const LokiModem ∗self)

Lire l'état d'activation du mode de test.

— void LokiModem_set_test_mode (const LokiModem ∗self, bool test_mode)

Activer ou désactiver le mode de test.

— void LokiModem_restart (const LokiModem ∗self)

Redémarrer le modem.

— uint32_t LokiModem_get_adc_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons reçus sur une voie ADC en entrée du démodulateur.

— uint32_t LokiModem_get_dac_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons envoyés sur la sortie DAC du modulateur.

— uint32_t LokiModem_get_demod_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons produits sur une voie en sortie du démodulateur.

— static uint32_t LokiModem_get_packet_index (const LokiModem ∗self, LokiDMASample raw_item)

Lire un numéro de paquet dans un échantillon DMA brut.

— static void LokiModem_set_packet_item (const LokiModem ∗self, LokiModemSample ∗item, LokiDMASample raw_item)

Affecter les champs d'un échantillon de sortie du démodulateur.

— static void LokiModem_set_simulated_item (LokiModem ∗self, LokiModemSample ∗item)

Affecter les champs d'un échantillon de sortie du démodulateur en mode simulation de capteurs.

— static void ∗ LokiModem_work (void ∗ptr)

Traiter un tableau d'échantillons issu d'un transfert DMA.

— int LokiModem_start_streaming (LokiModem ∗self, int32_t channels, bool simulate, bool raw, LokiFIFO ∗out)

Démarrer l'acquisition et le traitement des échantillons démodulés.

— void LokiModem_stop_streaming (LokiModem ∗self)

Arrêter l'acquisition.

— void LokiModem_force_stop (const LokiModem ∗self)

Arrêter l'acquisition immédiatement.

— bool LokiModem_is_streaming (const LokiModem *self)

Lire l'état de l'acquisition.

— unsigned LokiModem_get_dropped_packets (const LokiModem *self)

Lire le nombre de paquets perdus depuis le début de l'acquisition.

— void LokiModem_print_config (const LokiModem *self)

Afficher la configuration d'un modem sur la sortie standard.

6.45.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-modem.c.

6.45.2 Documentation des macros

6.45.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Définition à la ligne 3 du fichier libloki-modem.c.

6.46 libloki-modem.c

Aller à la documentation de ce fichier.

```
00001
00003 #define _GNU_SOURCE
00004
00005 #include <math.h>
00006 #include <errno.h>
00007 #include <stdio.h>
00008 #include <string.h>
00009 #include <stdlib.h>
00010 #include <pthread.h>
00011 #include <unistd.h>
00012 #include <sys/ioctl.h>
00013 #include <sys/mman.h>
00014
00015 #include <loki-modem-regs.h>
00016 #include <loki-device-ioctl.h>
00017 #include "libloki.h"
00018
00021
00023 struct LokiModem {
00025     FILE *file;
00027     int fileno;
00029     volatile bool stop_request;
```

```
00031     LokiFIFO *out;
00033     bool raw;
00035     int32_t stream_channels;
00037     pthread_t thread;
00039     bool simulate;
00041     uint32_t sim_channel;
00043     float sim_osc1_phase;
00045     float sim_disp_phase;
00047     uint32_t clk_frequency;
00049     uint32_t mod_sample_frequency_ref;
00051     uint32_t mod_cycles_per_sample_max;
00053     float mod_cycles_per_sample_ref;
00055     uint32_t mod_osc1_wave_frequency_min;
00057     uint32_t mod_osc2_wave_frequency_min;
00059     uint32_t mod_gain_integer_width;
00061     uint32_t mod_gain_fractional_width;
00063     float mod_gain_res;
00065     float mod_phase_amplitude_max;
00067     uint32_t mod_phase_integer_width;
00069     uint32_t mod_phase_fractional_width;
00071     float mod_offset_res;
00073     float mod_dc_value_max;
00075     uint32_t mod_dc_value_width;
00077     float mod_dc_value_res;
00079     uint32_t mod_pga_gain_width;
00081     uint32_t mod_pga_gain_max;
00083     bool demod_mixerless;
00085     uint32_t demod_channels;
00087     uint32_t demod_channel_output_width;
00089     uint32_t demod_channel_mask;
00091     uint32_t demod_power_integer_width;
00093     uint32_t demod_power_fractional_width;
00095     float demod_power_max;
00097     uint32_t demod_sample_frequency_ref;
00099     uint32_t demod_cycles_per_sample_max;
00101     float demod_cycles_per_sample_ref;
00103     uint32_t demod_osc_wave_frequency_min;
00105     uint32_t demod_fir1_decimation;
00107     uint32_t demod_fir1_length;
00109     uint32_t demod_fir2_decimation;
00111     uint32_t demod_fir2_length;
00113     uint32_t demod_fir2_bandwidth;
00115     uint32_t demod_xy_integer_width;
00117     uint32_t demod_xy_fractional_width;
00119     uint32_t demod_xy_width;
00121     uint32_t demod_xy_mask;
00123     uint32_t demod_xy_sign_mask;
00125     float demod_xy_max;
00126 };
00127
00129 typedef uint64_t LokiDMASample;
00130
00140 static inline uint32_t LokiModem_get_reg(const LokiModem *self, LokiModemRegister reg) {
00141     LokiRegisterQuery query = {.reg = reg};
00142     int err = ioctl(self->fileno, LOKI_IOCTL_REG_READ, &query);
00143     if (err) {
00144         fprintf(stderr, "LokiModem_get_reg: ioctl returned %d\n", err);
00145         return 0;
00146     }
00147     return query.value;
00148 }
00149
00159 static inline void LokiModem_set_reg(const LokiModem *self, LokiModemRegister reg, uint32_t value) {
00160     LokiRegisterQuery query = {.reg = reg, .value = value};
00161     int err = ioctl(self->fileno, LOKI_IOCTL_REG_WRITE, &query);
00162     if (err) {
00163         fprintf(stderr, "LokiModem_set_reg: ioctl returned %d\n", err);
00164     }
00165 }
00166
00174 static inline void LokiModem_dma_start(const LokiModem *self) {
00175     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_START);
00176     if (err) {
00177         fprintf(stderr, "LokiModem_dma_start: ioctl returned %d\n", err);
00178     }
00179 }
00180
00188 static inline void LokiModem_dma_stop(const LokiModem *self) {
00189     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_STOP);
00190     if (err) {
00191         fprintf(stderr, "LokiModem_dma_stop: ioctl returned %d\n", err);
00192     }
00193 }
00194
00201 static inline bool LokiModem_dma_is_streaming(const LokiModem *self) {
00202     bool res;
00203     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_IS_STREAMING, &res);
```

```
00204     if (err) {
00205         fprintf(stderr, "LokiModem_dma_is_streaming: ioctl returned %d\n", err);
00206         return false;
00207     }
00208     return res;
00209 }
00210
00218 static inline void LokiModem_dma_get_start_time(const LokiModem *self, struct timespec *res) {
00219     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_START_TIME, res);
00220     if (err) {
00221         fprintf(stderr, "LokiModem_dma_get_start_time: ioctl returned %d\n", err);
00222     }
00223 }
00224
00232 static inline void LokiModem_dma_get_packet_time(const LokiModem *self, struct timespec *res) {
00233     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_PACKET_TIME, res);
00234     if (err) {
00235         fprintf(stderr, "LokiModem_dma_get_packet_time: ioctl returned %d\n", err);
00236     }
00237 }
00238
00246 static inline unsigned LokiModem_dma_get_dropped_packets(const LokiModem *self) {
00247     unsigned res = 0;
00248     int err = ioctl(self->fileno, LOKI_IOCTL_DMA_DROPPED_PACKETS, &res);
00249     if (err) {
00250         fprintf(stderr, "LokiModem_dma_get_dropped_packets: ioctl returned %d\n", err);
00251     }
00252     return res;
00253 }
00254
00261 static inline float LokiModem_scale_xy(const LokiModem *self, float xy) {
00262     return ldexpf(xy, -self->demod_xy_fractional_width);
00263 }
00264
00271 static inline float LokiModem_scale_power(const LokiModem *self, float p) {
00272     return ldexpf(p, -self->demod_power_fractional_width);
00273 }
00274
00275 LokiModem *LokiModem_new(void) {
00276     LokiModem *self = calloc(1, sizeof(LokiModem));
00277     if (!self) {
00278         return NULL;
00279     }
00280
00281     self->file = fopen("/dev/loki-modem", "r");
00282     if (!self->file) {
00283         fprintf(stderr, "LokiModem_new: failed to open file /dev/loki-modem (error: %d, %s)\n", errno,
      strerror(errno));
00284         LokiModem_del(self);
00285         return NULL;
00286     }
00287
00288     self->fileno = fileno(self->file);
00289
00290     self->clk_frequency                = LokiModem_get_reg(self, LOKI_MODEM_REG_CLK_FREQUENCY);
00291     self->mod_sample_frequency_ref     = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_SAMPLE_FREQUENCY_REF);
00292     self->mod_cycles_per_sample_max    = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE_MAX);
00293     self->mod_osc1_wave_frequency_min  = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_OSC1_WAVE_FREQUENCY_MIN);
00294     self->mod_osc2_wave_frequency_min  = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_OSC2_WAVE_FREQUENCY_MIN);
00295     self->mod_gain_integer_width       = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_GAIN_INTEGER_WIDTH);
00296     self->mod_gain_fractional_width    = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_GAIN_FRACTIONAL_WIDTH);
00297
00298     union {
00299         uint32_t u;
00300         float f;
00301     } mod_phase_amplitude_max, mod_dc_value_max;
00302
00303     mod_phase_amplitude_max.u          = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_PHASE_AMPLITUDE_MAX);
00304     self->mod_phase_amplitude_max      = mod_phase_amplitude_max.f;
00305     self->mod_phase_integer_width      = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_PHASE_INTEGER_WIDTH);
00306     self->mod_phase_fractional_width   = LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_PHASE_FRACTIONAL_WIDTH);
00307     mod_dc_value_max.u                 = LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_DC_VALUE_MAX);
00308     self->mod_dc_value_max             = mod_dc_value_max.f;
00309     self->mod_dc_value_width           = LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_DC_VALUE_WIDTH);
00310     self->mod_pga_gain_width           = LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_PGA_GAIN_WIDTH);
00311     self->demod_mixerless              = !!(LokiModem_get_reg(self, LOKI_MODEM_REG_CONTROL) &
      LOKI_MODEM_REG_CONTROL_MIXERLESS_SHIFTED_MASK);
00312     self->demod_channels               = LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_CHANNELS);
```

```
00313     self->demod_channel_output_width   = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_CHANNEL_OUTPUT_WIDTH);
00314     self->demod_power_integer_width    = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_POWER_INTEGER_WIDTH);
00315     self->demod_power_fractional_width = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_POWER_FRACTIONAL_WIDTH);
00316     self->demod_sample_frequency_ref   = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_SAMPLE_FREQUENCY_REF);
00317     self->demod_cycles_per_sample_max  = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_CYCLES_PER_SAMPLE_MAX);
00318     self->demod_osc_wave_frequency_min = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_OSC_WAVE_FREQUENCY_MIN);
00319     self->demod_fir1_decimation        = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_FIR1_DECIMATION);
00320     self->demod_fir1_length            = LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_FIR1_LENGTH);
00321     self->demod_fir2_decimation        = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_FIR2_DECIMATION);
00322     self->demod_fir2_length            = LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_FIR2_LENGTH);
00323     self->demod_fir2_bandwidth         = LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_FIR2_BANDWIDTH);
00324     self->demod_xy_integer_width       = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_XY_INTEGER_WIDTH);
00325     self->demod_xy_fractional_width    = LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_XY_FRACTIONAL_WIDTH);
00326
00327     self->demod_channel_mask           = (1 << self->demod_channel_output_width) - 1;
00328
00329     self->mod_cycles_per_sample_ref    = (float)self->clk_frequency / self->mod_sample_frequency_ref;
00330     self->mod_gain_res                 = self->mod_phase_amplitude_max / (1 <<
      (self->mod_gain_integer_width  + self->mod_gain_fractional_width  - 1) - 1);
00331     self->mod_offset_res               = self->mod_phase_amplitude_max / (1 <<
      (self->mod_phase_integer_width + self->mod_phase_fractional_width - 1) - 1);
00332     self->mod_dc_value_res             = self->mod_dc_value_max / ((1 << self->mod_dc_value_width) -
      1);
00333     self->mod_pga_gain_max             = (1 << self->mod_pga_gain_width) - 1;
00334     self->demod_power_max              = LokiModem_scale_power(self, ((1 <<
      (self->demod_power_integer_width + self->demod_power_fractional_width - 1))) - 1);
00335     self->demod_cycles_per_sample_ref  = (float)self->clk_frequency /
      self->demod_sample_frequency_ref;
00336     self->demod_xy_max                 = LokiModem_scale_xy(self, ((1 << (self->demod_xy_integer_width
      + self->demod_xy_fractional_width - 1))) - 1);
00337     self->demod_xy_width               = self->demod_xy_integer_width +
      self->demod_xy_fractional_width;
00338     self->demod_xy_mask                = (1 << self->demod_xy_width) - 1;
00339     self->demod_xy_sign_mask           = 1 << (self->demod_xy_width - 1);
00340
00341     if (!LokiModem_is_streaming(self)) {
00342         // Le nombre d'octets d'un transfert DMA doit être multiple de
00343         // * la taille d'une page mémoire
00344         // * la taille d'un échantillon
00345         uint32_t page_size = getpagesize();
00346         uint32_t a         = page_size;
00347         uint32_t b         = sizeof(LokiDMASample);
00348         while (a != b) {
00349             if (a > b) {
00350                 a -= b;
00351             }
00352             if (b > a) {
00353                 b -= a;
00354             }
00355         }
00356
00357         uint32_t packet_len_min = page_size / a;
00358         uint32_t packet_len_max = LokiModem_get_demod_packet_length_max(self);
00359         packet_len_max -= packet_len_max % packet_len_min;
00360
00361         uint32_t packet_len = self->demod_channels * packet_len_min;
00362         if (packet_len > packet_len_max) {
00363             packet_len = packet_len_max;
00364         }
00365
00366         LokiModem_set_reg(self, LOKI_MODEM_REG_DEMOD_PACKET_LENGTH, packet_len);
00367     }
00368
00369     return self;
00370 }
00371
00372 void LokiModem_del(LokiModem *self) {
00373     if (!self) {
00374         return;
00375     }
00376
00377     if (self->file) {
00378         fclose(self->file);
00379     }
00380
00381     free(self);
00382 }
```

```
00383
00384 uint32_t LokiModem_get_clk_frequency(const LokiModem *self) {
00385     return self->clk_frequency;
00386 }
00387
00388 float LokiModem_get_mod_sampling_frequency(const LokiModem *self) {
00389     return (float)self->clk_frequency / LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE);
00390 }
00391
00392 void LokiModem_set_mod_sampling_frequency(const LokiModem *self, float freq) {
00393     // Cette fonction doit préserver la fréquence des oscillateurs.
00394     float osc1_freq = LokiModem_get_mod_osc1_wave_frequency(self);
00395     float osc2_freq = LokiModem_get_mod_osc2_wave_frequency(self);
00396     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE, roundf((float)self->clk_frequency /
      freq));
00397     LokiModem_set_mod_osc1_wave_frequency(self, osc1_freq);
00398     LokiModem_set_mod_osc2_wave_frequency(self, osc2_freq);
00399 }
00400
00401 float LokiModem_get_mod_osc1_wave_frequency_res(const LokiModem *self) {
00402     return (float)self->mod_osc1_wave_frequency_min * self->mod_cycles_per_sample_ref /
      LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE);
00403 }
00404
00405 float LokiModem_get_mod_osc1_wave_frequency_max(const LokiModem *self) {
00406     return LokiModem_get_mod_sampling_frequency(self) / 4;
00407 }
00408
00409 float LokiModem_get_mod_osc1_wave_frequency(const LokiModem *self) {
00410     return LokiModem_get_mod_osc1_wave_frequency_res(self) * LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_OSC1_STEP);
00411 }
00412
00413 void LokiModem_set_mod_osc1_wave_frequency(const LokiModem *self, float freq) {
00414     float ref_freq = freq * LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE) /
      self->mod_cycles_per_sample_ref;
00415     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_OSC1_STEP, roundf(ref_freq /
      self->mod_osc1_wave_frequency_min));
00416 }
00417
00418 float LokiModem_get_mod_osc1_gain_res(const LokiModem *self) {
00419     return self->mod_gain_res;
00420 }
00421
00422 float LokiModem_get_mod_osc1_gain_max(const LokiModem *self) {
00423     return self->mod_phase_amplitude_max;
00424 }
00425
00426 float LokiModem_get_mod_osc1_gain(const LokiModem *self) {
00427     return (float)LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_OSC1_GAIN) * self->mod_gain_res;
00428 }
00429
00430 void LokiModem_set_mod_osc1_gain(const LokiModem *self, float gain) {
00431     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_OSC1_GAIN, roundf(gain / self->mod_gain_res));
00432 }
00433
00434 float LokiModem_get_mod_osc2_wave_frequency_res(const LokiModem *self) {
00435     return (float)self->mod_osc2_wave_frequency_min * self->mod_cycles_per_sample_ref /
      LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE);
00436 }
00437
00438 float LokiModem_get_mod_osc2_wave_frequency_max(const LokiModem *self) {
00439     return LokiModem_get_mod_sampling_frequency(self) / 4;
00440 }
00441
00442 float LokiModem_get_mod_osc2_wave_frequency(const LokiModem *self) {
00443     return LokiModem_get_mod_osc2_wave_frequency_res(self) * LokiModem_get_reg(self,
      LOKI_MODEM_REG_MOD_OSC2_STEP);
00444 }
00445
00446 void LokiModem_set_mod_osc2_wave_frequency(const LokiModem *self, float freq) {
00447     float ref_freq = freq * LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_CYCLES_PER_SAMPLE) /
      self->mod_cycles_per_sample_ref;
00448     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_OSC2_STEP, roundf(ref_freq /
      self->mod_osc2_wave_frequency_min));
00449 }
00450
00451 float LokiModem_get_mod_osc2_gain_res(const LokiModem *self) {
00452     return self->mod_gain_res;
00453 }
00454
00455 float LokiModem_get_mod_osc2_gain_max(const LokiModem *self) {
00456     return self->mod_phase_amplitude_max;
00457 }
00458
00459 float LokiModem_get_mod_osc2_gain(const LokiModem *self) {
00460     return (float)LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_OSC2_GAIN) * self->mod_gain_res;
```

```
00461 }
00462
00463 void LokiModem_set_mod_osc2_gain(const LokiModem *self, float gain) {
00464     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_OSC2_GAIN, roundf(gain / self->mod_gain_res));
00465 }
00466
00467 float LokiModem_get_demod_osc_wave_frequency_res(const LokiModem *self) {
00468     if (self->demod_mixerless) {
00469         return 0.0f;
00470     }
00471     return (float)self->demod_osc_wave_frequency_min * self->demod_cycles_per_sample_ref /
      LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_CYCLES_PER_SAMPLE);
00472 }
00473
00474 float LokiModem_get_demod_osc_wave_frequency_max(const LokiModem *self) {
00475     if (self->demod_mixerless) {
00476         return 0.0f;
00477     }
00478     return LokiModem_get_demod_sampling_frequency(self) / 4;
00479 }
00480
00481 float LokiModem_get_demod_osc_wave_frequency(const LokiModem *self) {
00482     if (self->demod_mixerless) {
00483         return 0.0f;
00484     }
00485     return LokiModem_get_demod_osc_wave_frequency_res(self) * LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_OSC_STEP);
00486 }
00487
00488 void LokiModem_set_demod_osc_wave_frequency(const LokiModem *self, float freq) {
00489     if (self->demod_mixerless) {
00490         return;
00491     }
00492     float ref_freq = freq * LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_CYCLES_PER_SAMPLE) /
      self->demod_cycles_per_sample_ref;
00493     LokiModem_set_reg(self, LOKI_MODEM_REG_DEMOD_OSC_STEP, roundf(ref_freq /
      self->demod_osc_wave_frequency_min));
00494 }
00495
00496 float LokiModem_get_mod_offset_res(const LokiModem *self) {
00497     return self->mod_offset_res;
00498 }
00499
00500 float LokiModem_get_mod_offset_max(const LokiModem *self) {
00501     return self->mod_phase_amplitude_max;
00502 }
00503
00504 float LokiModem_get_mod_offset(const LokiModem *self) {
00505     return (float)(int32_t)LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_OFFSET) * self->mod_offset_res;
00506 }
00507
00508 void LokiModem_set_mod_offset(const LokiModem *self, float offset) {
00509     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_OFFSET, roundf(offset / self->mod_offset_res));
00510 }
00511
00512 float LokiModem_get_mod_dc_value_res(const LokiModem *self) {
00513     return self->mod_dc_value_res;
00514 }
00515
00516 float LokiModem_get_mod_dc_value_max(const LokiModem *self) {
00517     return self->mod_dc_value_max;
00518 }
00519
00520 float LokiModem_get_mod_dc_value(const LokiModem *self) {
00521     return (float)LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_DC_VALUE) * self->mod_dc_value_res;
00522 }
00523
00524 void LokiModem_set_mod_dc_value(const LokiModem *self, float dc_value) {
00525     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_DC_VALUE, roundf(dc_value / self->mod_dc_value_res));
00526 }
00527
00528 uint32_t LokiModem_get_mod_pga_gain_max(const LokiModem *self) {
00529     return self->mod_pga_gain_max;
00530 }
00531
00532 uint32_t LokiModem_get_mod_pga_gain(const LokiModem *self) {
00533     return LokiModem_get_reg(self, LOKI_MODEM_REG_MOD_PGA_GAIN);
00534 }
00535
00536 void LokiModem_set_mod_pga_gain(const LokiModem *self, uint32_t gain) {
00537     LokiModem_set_reg(self, LOKI_MODEM_REG_MOD_PGA_GAIN, gain);
00538 }
00539
00540 uint32_t LokiModem_get_demod_channels(const LokiModem *self) {
00541     return self->demod_channels;
00542 }
00543
```

```
00544 uint32_t LokiModem_get_demod_decimation(const LokiModem *self) {
00545     return self->demod_fir1_decimation * self->demod_fir2_decimation;
00546 }
00547
00548 float LokiModem_get_demod_bandwidth(const LokiModem *self) {
00549     return (float)self->demod_fir2_bandwidth * self->demod_cycles_per_sample_ref /
      LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_CYCLES_PER_SAMPLE);
00550 }
00551
00552 float LokiModem_get_demod_sampling_frequency(const LokiModem *self) {
00553     return (float)self->clk_frequency / LokiModem_get_reg(self,
      LOKI_MODEM_REG_DEMOD_CYCLES_PER_SAMPLE);
00554 }
00555
00556 void LokiModem_set_demod_sampling_frequency(const LokiModem *self, float freq) {
00557     float osc_freq;
00558     if (!self->demod_mixerless) {
00559         // Cette fonction doit préserver la fréquence de l'oscillateur.
00560         osc_freq = LokiModem_get_demod_osc_wave_frequency(self);
00561     }
00562
00563     LokiModem_set_reg(self, LOKI_MODEM_REG_DEMOD_CYCLES_PER_SAMPLE, roundf((float)self->clk_frequency
      / freq));
00564
00565     if (!self->demod_mixerless) {
00566         LokiModem_set_demod_osc_wave_frequency(self, osc_freq);
00567     }
00568 }
00569
00570 float LokiModem_get_demod_xy_sampling_frequency(const LokiModem *self) {
00571     return LokiModem_get_demod_sampling_frequency(self) / self->demod_fir1_decimation /
      self->demod_fir2_decimation;
00572 }
00573
00574 void LokiModem_set_demod_xy_sampling_frequency(const LokiModem *self, float freq) {
00575     LokiModem_set_demod_sampling_frequency(self, freq * self->demod_fir1_decimation *
      self->demod_fir2_decimation);
00576 }
00577
00578 uint32_t LokiModem_get_demod_delay_x_max(const LokiModem *self, uint32_t channel) {
00579     if (self->demod_mixerless) {
00580         return self->demod_fir2_decimation * self->demod_fir1_decimation - 1;
00581     }
00582     else {
00583         return self->demod_sample_frequency_ref / self->demod_osc_wave_frequency_min - 1;
00584     }
00585 }
00586
00587 float LokiModem_get_demod_phase_x_max(const LokiModem *self, uint32_t channel) {
00588     const uint32_t delay_max = LokiModem_get_demod_delay_x_max(self, channel);
00589     return 360.0f * delay_max / (delay_max + 1);
00590 }
00591
00592 float LokiModem_get_demod_phase_x_res(const LokiModem *self, uint32_t channel) {
00593     const uint32_t delay_max = LokiModem_get_demod_delay_x_max(self, channel);
00594     return 360.0f / (delay_max + 1);
00595 }
00596
00597 uint32_t LokiModem_get_demod_delay_x(const LokiModem *self, uint32_t channel) {
00598     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00599     uint32_t lowpass_delay_x  = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_LOWPASS_X);
00600     if (self->demod_mixerless) {
00601         return lowpass_delay_x;
00602     }
00603     uint32_t bandpass_delay_x = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_BANDPASS_X);
00604     return lowpass_delay_x + bandpass_delay_x * self->demod_fir1_decimation;
00605 }
00606
00607 float LokiModem_get_demod_phase_x(const LokiModem *self, uint32_t channel) {
00608     const uint32_t delay     = LokiModem_get_demod_delay_x(self, channel);
00609     const uint32_t delay_max = LokiModem_get_demod_delay_x_max(self, channel);
00610     return 360.0f * delay / (delay_max + 1);
00611 }
00612
00613 void LokiModem_set_demod_delay_x(const LokiModem *self, uint32_t channel, uint32_t delay) {
00614     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00615     if (self->demod_mixerless) {
00616         LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_LOWPASS_X,  delay %
      self->demod_fir1_decimation);
00617         LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_BANDPASS_X, delay /
      self->demod_fir1_decimation);
00618     }
00619     else {
00620         LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_LOWPASS_X,  delay);
00621     }
```

```
00622 }
00623
00624 void LokiModem_set_demod_phase_x(const LokiModem *self, uint32_t channel, float phase) {
00625     // delay_range needs to be signed for the % operation below.
00626     const int32_t delay_range = LokiModem_get_demod_delay_x_max(self, channel) + 1;
00627     const int32_t delay = (int32_t)roundf(phase * delay_range / 360.0f) % delay_range;
00628     LokiModem_set_demod_delay_x(self, channel, delay >= 0 ? delay : delay + delay_range);
00629 }
00630
00631 uint32_t LokiModem_get_demod_delay_y_max(const LokiModem *self, uint32_t channel) {
00632     if (self->demod_mixerless) {
00633         return self->demod_fir2_decimation * self->demod_fir1_decimation - 1;
00634     }
00635     else {
00636         return self->demod_sample_frequency_ref / self->demod_osc_wave_frequency_min - 1;
00637     }
00638 }
00639
00640 float LokiModem_get_demod_phase_y_max(const LokiModem *self, uint32_t channel) {
00641     const uint32_t delay_max = LokiModem_get_demod_delay_y_max(self, channel);
00642     return 360.0f * delay_max / (delay_max + 1);
00643 }
00644
00645 float LokiModem_get_demod_phase_y_res(const LokiModem *self, uint32_t channel) {
00646     const uint32_t delay_max = LokiModem_get_demod_delay_y_max(self, channel);
00647     return 360.0f / (delay_max + 1);
00648 }
00649
00650 uint32_t LokiModem_get_demod_delay_y(const LokiModem *self, uint32_t channel) {
00651     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00652     uint32_t lowpass_delay_y  = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_LOWPASS_Y);
00653     if (self->demod_mixerless) {
00654         return lowpass_delay_y;
00655     }
00656     uint32_t bandpass_delay_y = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_BANDPASS_Y);
00657     return lowpass_delay_y + bandpass_delay_y * self->demod_fir1_decimation;
00658 }
00659
00660 float LokiModem_get_demod_phase_y(const LokiModem *self, uint32_t channel) {
00661     const uint32_t delay     = LokiModem_get_demod_delay_y(self, channel);
00662     const uint32_t delay_max = LokiModem_get_demod_delay_y_max(self, channel);
00663     return 360.0f * delay / (delay_max + 1);
00664 }
00665
00666 void LokiModem_set_demod_delay_y(const LokiModem *self, uint32_t channel, uint32_t delay) {
00667     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00668     if (self->demod_mixerless) {
00669         LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_LOWPASS_Y,  delay %
      self->demod_fir1_decimation);
00670         LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_BANDPASS_Y, delay /
      self->demod_fir1_decimation);
00671     }
00672     else {
00673         LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_DELAY_LOWPASS_Y,  delay);
00674     }
00675 }
00676
00677 void LokiModem_set_demod_phase_y(const LokiModem *self, uint32_t channel, float phase) {
00678     // delay_range needs to be signed for the % operation below.
00679     const int32_t delay_range = LokiModem_get_demod_delay_y_max(self, channel) + 1;
00680     const int32_t delay = (int32_t)roundf(phase * delay_range / 360.0f) % delay_range;
00681     LokiModem_set_demod_delay_y(self, channel, delay >= 0 ? delay : delay + delay_range);
00682 }
00683
00685 #define AD8370_VERNIER 0.055744
00687 #define AD8370_PREGAIN 7.079458
00689 #define AD8370_HIGH_GAIN_FACTOR (AD8370_VERNIER * AD8370_PREGAIN)
00691 #define AD8370_GAIN_CODE_SIZE 7
00693 #define AD8370_GAIN_CODE_MAX ((1 « AD8370_GAIN_CODE_SIZE) - 1)
00695 #define AD8370_LOW_GAIN_MAX (AD8370_GAIN_CODE_MAX * AD8370_VERNIER)
00697 #define AD8370_HIGH_GAIN_MAX (AD8370_GAIN_CODE_MAX * AD8370_HIGH_GAIN_FACTOR)
00698
00700 typedef union {
00702     uint8_t raw;
00704     struct {
00706         unsigned gain_code : AD8370_GAIN_CODE_SIZE;
00708         unsigned high_gain : 1;
00709     };
00710 } AD8370ControlWord;
00711
00712 float LokiModem_get_demod_pga_gain_max(const LokiModem *self, uint32_t channel) {
00713     return AD8370_HIGH_GAIN_MAX;
00714 }
00715
00716 float LokiModem_get_demod_pga_gain(const LokiModem *self, uint32_t channel) {
```

```
00717     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00718     AD8370ControlWord control_word = {
00719         .raw = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_PGA_GAIN)
00720     };
00721     float gain = AD8370_VERNIER * control_word.gain_code;
00722     if (control_word.high_gain) {
00723         gain *= AD8370_PREGAIN;
00724     }
00725     return gain;
00726 }
00727
00728 void LokiModem_set_demod_pga_gain(const LokiModem *self, uint32_t channel, float gain) {
00729     if (gain < 0) {
00730         gain = 0;
00731     }
00732     else if (gain > AD8370_HIGH_GAIN_MAX) {
00733         gain = AD8370_HIGH_GAIN_MAX;
00734     }
00735
00736     AD8370ControlWord control_word = {
00737         .high_gain = gain > AD8370_LOW_GAIN_MAX
00738     };
00739
00740     control_word.gain_code = gain / (control_word.high_gain ? AD8370_HIGH_GAIN_FACTOR :
      AD8370_VERNIER);
00741
00742     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00743     LokiModem_set_reg(self, base + LOKI_MODEM_REG_DEMOD_PGA_GAIN, control_word.raw);
00744 }
00745
00746 float LokiModem_get_demod_adc_min_ratio(const LokiModem *self, uint32_t channel) {
00747     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00748     int32_t value = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_ADC_MIN);
00749     return LokiModem_scale_power(self, value) / self->demod_power_max;
00750 }
00751
00752 float LokiModem_get_demod_adc_max_ratio(const LokiModem *self, uint32_t channel) {
00753     uint32_t base = LOKI_MODEM_REG_DEMOD_CHANNEL_CFG + channel *
      LOKI_MODEM_REG_DEMOD_CHANNEL_CFG_COUNT;
00754     int32_t value = LokiModem_get_reg(self, base + LOKI_MODEM_REG_DEMOD_ADC_MAX);
00755     return LokiModem_scale_power(self, value) / self->demod_power_max;
00756 }
00757
00758 uint32_t LokiModem_get_demod_packet_length_max(const LokiModem *self) {
00759     return LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_PACKET_LENGTH_MAX);
00760 }
00761
00762 uint32_t LokiModem_get_demod_packet_length(const LokiModem *self) {
00763     return LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_PACKET_LENGTH);
00764 }
00765
00766 bool LokiModem_get_demod_mixerless(const LokiModem *self) {
00767     return self->demod_mixerless;
00768 }
00769
00770 bool LokiModem_get_demod_bypass(const LokiModem *self) {
00771     uint32_t value = LokiModem_get_reg(self, LOKI_MODEM_REG_CONTROL) &
      LOKI_MODEM_REG_CONTROL_BYPASS_SHIFTED_MASK;
00772     return !!value;
00773 }
00774
00775 void LokiModem_set_demod_bypass(const LokiModem *self, bool bypass) {
00776     uint32_t value = LokiModem_get_reg(self, LOKI_MODEM_REG_CONTROL);
00777     if (bypass) {
00778         value |= LOKI_MODEM_REG_CONTROL_BYPASS_SHIFTED_MASK;
00779     }
00780     else {
00781         value &= ~LOKI_MODEM_REG_CONTROL_BYPASS_SHIFTED_MASK;
00782     }
00783     LokiModem_set_reg(self, LOKI_MODEM_REG_CONTROL, value);
00784 }
00785
00786 bool LokiModem_get_test_mode(const LokiModem *self) {
00787     uint32_t value = LokiModem_get_reg(self, LOKI_MODEM_REG_CONTROL) &
      LOKI_MODEM_REG_CONTROL_TEST_MODE_SHIFTED_MASK;
00788     return !!value;
00789 }
00790
00791 void LokiModem_set_test_mode(const LokiModem *self, bool test_mode) {
00792     uint32_t value = LokiModem_get_reg(self, LOKI_MODEM_REG_CONTROL);
00793     if (test_mode) {
00794         value |= LOKI_MODEM_REG_CONTROL_TEST_MODE_SHIFTED_MASK;
00795     }
00796     else {
```

```
00797         value &= ~LOKI_MODEM_REG_CONTROL_TEST_MODE_SHIFTED_MASK;
00798     }
00799     LokiModem_set_reg(self, LOKI_MODEM_REG_CONTROL, value);
00800 }
00801
00802 void LokiModem_restart(const LokiModem *self) {
00803     uint32_t value = LokiModem_get_reg(self, LOKI_MODEM_REG_CONTROL);
00804     value |= LOKI_MODEM_REG_CONTROL_RESTART_SHIFTED_MASK;
00805     LokiModem_set_reg(self, LOKI_MODEM_REG_CONTROL, value);
00806 }
00807
00808 uint32_t LokiModem_get_adc_sample_count(const LokiModem *self) {
00809     return LokiModem_get_reg(self, LOKI_MODEM_REG_ADC_SAMPLE_COUNT);
00810 }
00811
00812 uint32_t LokiModem_get_dac_sample_count(const LokiModem *self) {
00813     return LokiModem_get_reg(self, LOKI_MODEM_REG_DAC_SAMPLE_COUNT);
00814 }
00815
00816 uint32_t LokiModem_get_demod_sample_count(const LokiModem *self) {
00817     return LokiModem_get_reg(self, LOKI_MODEM_REG_DEMOD_SAMPLE_COUNT);
00818 }
00819
00826 static inline uint32_t LokiModem_get_packet_index(const LokiModem *self, LokiDMASample raw_item) {
00827     return raw_item » (self->demod_xy_width * 2 + self->demod_channel_output_width);
00828 }
00829
00836 static void LokiModem_set_packet_item(const LokiModem *self, LokiModemSample *item, LokiDMASample
      raw_item) {
00837     // Lire la valeur de l'échantillon, alignée à droite en complément à deux.
00838     // Effectuer l'extension du bit de signe.
00839     int32_t y_raw = raw_item & self->demod_xy_mask;
00840     raw_item »= self->demod_xy_width;
00841
00842     if (y_raw & self->demod_xy_sign_mask) {
00843         y_raw |= ~self->demod_xy_mask;
00844     }
00845     else {
00846         y_raw &= self->demod_xy_mask;
00847     }
00848
00849     int32_t x_raw = raw_item & self->demod_xy_mask;
00850     raw_item »= self->demod_xy_width;
00851
00852     if (x_raw & self->demod_xy_sign_mask) {
00853         x_raw |= ~self->demod_xy_mask;
00854     }
00855     else {
00856         x_raw &= self->demod_xy_mask;
00857     }
00858
00859     // Convertir les coordonnées en flottants si nécessaire.
00860     if (self->raw) {
00861         item->x_raw = x_raw;
00862         item->y_raw = y_raw;
00863     }
00864     else {
00865         item->x = LokiModem_scale_xy(self, x_raw);
00866         item->y = LokiModem_scale_xy(self, y_raw);
00867     }
00868
00869     // Lire le numéro de la voie.
00870     item->channel = raw_item & self->demod_channel_mask;
00871 }
00872
00873 // TODO Rendre ces valeurs configurables
00874
00876 #define SIMULATED_DISP_WAVE_FREQUENCY 0.2
00878 #define SIMULATED_DISP_AMPLITUDE (M_PI / 10)
00880 #define SIMULATED_ELLIPSE_A 0.1
00882 #define SIMULATED_ELLIPSE_B 0.15
00884 #define SIMULATED_ELLIPSE_ANGLE (M_PI / 6)
00886 #define SIMULATED_ELLIPSE_XC 0.01
00888 #define SIMULATED_ELLIPSE_YC 0.02
00889
00895 static void LokiModem_set_simulated_item(LokiModem *self, LokiModemSample *item) {
00896     item->channel = self->sim_channel;
00897     float phi = LokiModem_get_mod_osc1_gain(self) * cosf(self->sim_osc1_phase) * M_PI /
      self->mod_phase_amplitude_max +
00898                 SIMULATED_DISP_AMPLITUDE          * cosf(self->sim_disp_phase);
00899     float x = SIMULATED_ELLIPSE_A * cosf(phi);
00900     float y = SIMULATED_ELLIPSE_B * sinf(phi);
00901     float c = cosf(SIMULATED_ELLIPSE_ANGLE);
00902     float s = sinf(SIMULATED_ELLIPSE_ANGLE);
00903     item->x = SIMULATED_ELLIPSE_XC + x * c - y * s;
00904     item->y = SIMULATED_ELLIPSE_YC + y * c + x * s;
00905
```

```
00906     // Passer à la voie suivante.
00907     // En revenant à la voie 0, mettre à jour les phases des sinusoïdes simulées.
00908     self->sim_channel ++;
00909     if (self->sim_channel == self->demod_channels) {
00910         self->sim_channel = 0;
00911         self->sim_osc1_phase += M_PI * 2 * LokiModem_get_mod_osc1_wave_frequency(self) /
      LokiModem_get_demod_xy_sampling_frequency(self);
00912         if (self->sim_osc1_phase >= M_PI) {
00913             self->sim_osc1_phase -= M_PI * 2;
00914         }
00915         self->sim_disp_phase += M_PI * 2 * SIMULATED_DISP_WAVE_FREQUENCY /
      LokiModem_get_demod_xy_sampling_frequency(self);
00916         if (self->sim_disp_phase >= M_PI) {
00917             self->sim_disp_phase -= M_PI * 2;
00918         }
00919     }
00920 }
00921
00926 #define USE_MMAP
00927
00943 static void *LokiModem_work(void *ptr) {
00944     LokiModem *self = ptr;
00945
00946     pthread_setname_np(self->thread, "LokiModem");
00947
00948     // Obtenir un pointeur vers le buffer courant des échantillons mise en forme.
00949     const size_t out_length = LokiPacket_get_length(self->out, sizeof(LokiModemSample));
00950     LokiPacket *out_packet  = LokiFIFO_get_producer_item(self->out);
00951     if (!out_packet) {
00952         return NULL;
00953     }
00954     LokiModemSample *out_ptr = (LokiModemSample*)out_packet->buffer;
00955     LokiModemSample *out_end = out_ptr + out_length;
00956
00957     // Avec la fonction fread, allouer un buffer de données.
00958     // Avec l'accès par mmap, le buffer est fourni par le pilote.
00959     const size_t in_length = LokiModem_get_demod_packet_length(self);
00960     #ifndef USE_MMAP
00961         LokiDMASample *in_buffer = calloc(in_length, sizeof(LokiDMASample));
00962         LokiDMASample *in_end    = in_buffer + in_length;
00963     #else
00964         size_t in_length_bytes = in_length * sizeof(LokiDMASample);
00965     #endif
00966
00967     // Calculer la durée d'un paquet d'entrée, en nanosecondes.
00968     double in_period_ns = 1.0e9 * in_length / self->demod_channels /
      LokiModem_get_demod_xy_sampling_frequency(self);
00969
00970     // Calculer le modulo du compteur de paquets du FPGA.
00971     const uint32_t packet_count_mod = 1 « (sizeof(LokiDMASample) * 8 -
      self->demod_channel_output_width - self->demod_xy_width * 2);
00972
00973     bool first_packet = true;
00974     uint32_t packet_index_prev = 0;
00975     LokiTimestamp in_timestamp;
00976
00977     // Démarrer l'acquisition.
00978     LokiModem_dma_start(self);
00979
00980     while (!self->stop_request) {
00981         // Lire un buffer d'échantillons en provenance du DMA.
00982         #ifndef USE_MMAP
00983             if (fread(in_buffer, sizeof(LokiDMASample), in_length, self->file) != in_length) {
00984                 fprintf(stderr, "LokiModem_work: read failed (error: %d, %s)\n", errno,
      strerror(errno));
00985                 break;
00986             }
00987         #else
00988             // Accès au buffer DMA en mode zero-copy.
00989             LokiDMASample *in_buffer = mmap(NULL, in_length_bytes, PROT_READ, MAP_PRIVATE,
      self->fileno, 0);
00990             if (in_buffer == MAP_FAILED) {
00991                 fprintf(stderr, "LokiModem_work: mmap failed (error: %d, %s)\n", errno,
      strerror(errno));
00992                 break;
00993             }
00994             LokiDMASample *in_end = in_buffer + in_length;
00995         #endif
00996
00997         // Attendre le paquet numéro 0 pour commencer.
00998         uint32_t packet_index = LokiModem_get_packet_index(self, *in_buffer);
00999
01000         // Prendre pour référence temporelle la date du premier paquet.
01001         // Les dates des autres paquets seront calculées en fonction du nombre d'échantillons.
01002         if (first_packet && !packet_index) {
01003             struct timespec ts;
01004             LokiModem_dma_get_packet_time(self, &ts);
```

```
01005             in_timestamp = LokiTime_to_timestamp(ts) - in_period_ns;
01006         }
01007
01008         if (!first_packet || !packet_index) {
01009             // Calculer le timestamp du paquet d'entrée, en nanosecondes,
01010             // en utilisant le numéro de paquet fourni par le FPGA.
01011             int32_t packet_interval = packet_index - packet_index_prev;
01012             if (packet_interval < 0) {
01013                 fprintf(stderr, "LokiModem_work: packet index rollover (%u -> %u).\n",
      packet_index_prev, packet_index);
01014                 packet_interval += packet_count_mod;
01015             }
01016
01017             if (packet_interval < 0) {
01018                 fprintf(stderr, "LokiModem_work: packet index overlap (%u -> %u, %u).\n",
      packet_index_prev, packet_index, -packet_interval);
01019             }
01020             else if (packet_interval > 1) {
01021                 fprintf(stderr, "LokiModem_work: packet index gap (%u -> %u, %u).\n",
      packet_index_prev, packet_index, packet_interval);
01022                 //goto LokiModem_work_continue;
01023             }
01024
01025             packet_index_prev = packet_index;
01026             first_packet = false;
01027
01028             in_timestamp += round(in_period_ns * packet_interval);
01029
01030             // En commençant un nouveau paquet de sortie, copier la date du paquet d'entrée.
01031             if (out_ptr == (LokiModemSample*)out_packet->buffer) {
01032                 out_packet->timestamp = in_timestamp;
01033             }
01034
01035             LokiDMASample *in_ptr = in_buffer;
01036             while (in_ptr != in_end && !self->stop_request) {
01037                 LokiDMASample in = *in_ptr ++;
01038
01039                 LokiModemSample s;
01040                 if (self->simulate) {
01041                     LokiModem_set_simulated_item(self, &s);
01042                 }
01043                 else {
01044                     LokiModem_set_packet_item(self, &s, in);
01045                 }
01046
01047                 // Produire l'échantillon en sortie si la voie correspondante
01048                 // est activée dans le masque des voies à traiter.
01049                 if (self->stream_channels & (1 « s.channel)) {
01050                     *out_ptr ++ = s;
01051
01052                     if (out_ptr == out_end) {
01053                         LokiFIFO_push(self->out);
01054                         out_packet = LokiFIFO_get_producer_item(self->out);
01055                         if (!out_packet) {
01056                             break;
01057                         }
01058                         if (in_ptr != in_end) {
01059                             out_packet->timestamp = in_timestamp + round(in_period_ns * (in_ptr -
      in_buffer) / in_length);
01060                         }
01061                         out_ptr = (LokiModemSample*)out_packet->buffer;
01062                         out_end = out_ptr + out_length;
01063                     }
01064                 }
01065             }
01066         }
01067
01068     LokiModem_work_continue:
01069         #ifdef USE_MMAP
01070             munmap(in_buffer, in_length_bytes);
01071         #endif
01072     }
01073
01074     #ifndef USE_MMAP
01075         free(in_buffer);
01076     #endif
01077
01078     LokiModem_dma_stop(self);
01079 }
01080
01081 int LokiModem_start_streaming(LokiModem *self, int32_t channels, bool simulate, bool raw, LokiFIFO
      *out) {
01082     if (LokiModem_is_streaming(self)) {
01083         return EBUSY;
01084     }
01085     self->stop_request    = false;
01086     self->out             = out;
```

```
01087     self->simulate        = simulate;
01088     self->sim_channel     = 0;
01089     self->sim_osc1_phase  = 0;
01090     self->sim_disp_phase  = 0;
01091     self->raw             = raw;
01092     self->stream_channels = channels;
01093     return pthread_create(&self->thread, NULL, LokiModem_work, self);
01094 }
01095
01096 void LokiModem_stop_streaming(LokiModem *self) {
01097     if (self->stop_request) {
01098         return;
01099     }
01100     self->stop_request = true;
01101     LokiFIFO_abort(self->out);
01102     pthread_join(self->thread, NULL);
01103 }
01104
01105 void LokiModem_force_stop(const LokiModem *self) {
01106     LokiModem_dma_stop(self);
01107 }
01108
01109 bool LokiModem_is_streaming(const LokiModem *self) {
01110     return LokiModem_dma_is_streaming(self);
01111 }
01112
01113 unsigned LokiModem_get_dropped_packets(const LokiModem *self) {
01114     return LokiModem_dma_get_dropped_packets(self);
01115 }
01116
01117 void LokiModem_print_config(const LokiModem *self) {
01118     printf(
01119         "Clock frequency:               %u Hz\n"
01120         "Modulator:\n"
01121         "  Reference sample frequency:  %u samples/sec\n"
01122         "  Current sample frequency:    %f samples/sec\n"
01123         "  Oscillator 1:\n"
01124         "    Wave frequency resolution: %f Hz\n"
01125         "    Wave frequency max:        %f Hz\n"
01126         "    Wave frequency:            %f Hz\n"
01127         "    Gain resolution:           %f\n"
01128         "    Gain max:                  %f\n"
01129         "    Gain:                      %f\n"
01130         "  Oscillator 2:\n"
01131         "    Wave frequency resolution: %f Hz\n"
01132         "    Wave frequency max:        %f Hz\n"
01133         "    Wave frequency:            %f Hz\n"
01134         "    Gain resolution:           %f\n"
01135         "    Gain max:                  %f\n"
01136         "    Gain:                      %f\n"
01137         "  Output:\n"
01138         "    Resolution:                %f\n"
01139         "    Max:                       %f\n"
01140         "    Offset:                    %f\n"
01141         "  DC value:\n"
01142         "    Resolution:                %f\n"
01143         "    Max:                       %f\n"
01144         "    Offset:                    %f\n"
01145         "  PGA gain:                    %u\n"
01146         "Demodulator:\n"
01147         "  Bypass:                      %s\n"
01148         "  Channels:                    %u\n"
01149         "  Input sample frequency:      %f samples/sec\n"
01150         "  Input resolution:            %f\n"
01151         "  Input max:                   %f\n"
01152         "  Output sample frequency:     %f samples/sec\n"
01153         "  Output resolution:           %f\n"
01154         "  Output max:                  %f\n"
01155         "  Oscillator:                  %s\n"
01156         "    Wave frequency resolution: %f Hz\n"
01157         "    Wave frequency max:        %f Hz\n"
01158         "    Wave frequency:            %f Hz\n"
01159         "  Filter 1:\n"
01160         "    Length:                    %u\n"
01161         "    Decimation:                %u\n"
01162         "  Filter 2:\n"
01163         "    Length:                    %u\n"
01164         "    Decimation:                %u\n"
01165         "DMA interface:\n"
01166         "  Packet length max:           %u samples/transfer\n"
01167         "  Packet length:               %u samples/transfer\n",
01168         self->clk_frequency,
01169         // Modulator
01170         self->mod_sample_frequency_ref,
01171         LokiModem_get_mod_sampling_frequency(self),
01172         LokiModem_get_mod_osc1_wave_frequency_res(self),
01173         LokiModem_get_mod_osc1_wave_frequency_max(self),
```

```
01174         LokiModem_get_mod_osc1_wave_frequency(self),
01175         LokiModem_get_mod_osc1_gain_res(self),
01176         LokiModem_get_mod_osc1_gain_max(self),
01177         LokiModem_get_mod_osc1_gain(self),
01178         LokiModem_get_mod_osc2_wave_frequency_res(self),
01179         LokiModem_get_mod_osc2_wave_frequency_max(self),
01180         LokiModem_get_mod_osc2_wave_frequency(self),
01181         LokiModem_get_mod_osc2_gain_res(self),
01182         LokiModem_get_mod_osc2_gain_max(self),
01183         LokiModem_get_mod_osc2_gain(self),
01184         LokiModem_get_mod_offset_res(self),
01185         LokiModem_get_mod_offset_max(self),
01186         LokiModem_get_mod_offset(self),
01187         LokiModem_get_mod_dc_value_res(self),
01188         LokiModem_get_mod_dc_value_max(self),
01189         LokiModem_get_mod_dc_value(self),
01190         LokiModem_get_mod_pga_gain(self),
01191         // Demodulator
01192         LokiModem_get_demod_bypass(self) ? "Yes" : "No",
01193         self->demod_channels,
01194         LokiModem_get_demod_sampling_frequency(self),
01195         LokiModem_scale_power(self, 1),
01196         self->demod_power_max,
01197         LokiModem_get_demod_xy_sampling_frequency(self),
01198         LokiModem_scale_xy(self, 1),
01199         self->demod_xy_max,
01200         self->demod_mixerless ? "No" : "Yes",
01201         LokiModem_get_demod_osc_wave_frequency_res(self),
01202         LokiModem_get_demod_osc_wave_frequency_max(self),
01203         LokiModem_get_demod_osc_wave_frequency(self),
01204         self->demod_fir1_length,
01205         self->demod_fir1_decimation,
01206         self->demod_fir2_length,
01207         self->demod_fir2_decimation,
01208         LokiModem_get_demod_packet_length_max(self),
01209         LokiModem_get_demod_packet_length(self)
01210     );
01211 }
01212
```

6.47 Référence du fichier libloki-modem.h

```
#include "libloki-fifo.h"
#include <stdint.h>
#include <stdbool.h>
#include <sys/types.h>
```

Graphe des dépendances par inclusion de libloki-modem.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Structures de données

— struct LokiModemSample

Données retournées par le démodulateur pour un échantillon.

Définitions de type

— typedef struct LokiModem LokiModem

Représentation des périphériques modulateur et démodulateur.

Fonctions

— LokiModem ∗ LokiModem_new (void)

Construire un nouveau modem.

— void LokiModem_del (LokiModem ∗self)

Détruire un modem.

— uint32_t LokiModem_get_clk_frequency (const LokiModem ∗self)

Lire la fréquence d'horloge du circuit d'acquisition.

— float LokiModem_get_mod_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en sortie du modulateur.

— void LokiModem_set_mod_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en sortie du modulateur.

— float LokiModem_get_mod_osc1_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie du premier oscillateur du modulateur (Fm1).

— void LokiModem_set_mod_osc1_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain_res (const LokiModem ∗self)

Lire le pas de réglage de l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain_max (const LokiModem ∗self)

Lire l'amplitude maximale supportée pour le signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc1_gain (const LokiModem ∗self)

Lire l'amplitude du signal en sortie du premier oscillateur du modulateur (Fm1).

— void LokiModem_set_mod_osc1_gain (const LokiModem ∗self, float gain)

Affecter une amplitude au signal en sortie du premier oscillateur du modulateur (Fm1).

— float LokiModem_get_mod_osc2_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie du second oscillateur du modulateur (Fm2).

— void LokiModem_set_mod_osc2_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain_res (const LokiModem ∗self)

Lire le pas de réglage de l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain_max (const LokiModem ∗self)

Lire l'amplitude maximale supportée pour le signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_mod_osc2_gain (const LokiModem ∗self)

Lire l'amplitude du signal en sortie du second oscillateur du modulateur (Fm2).

— void LokiModem_set_mod_osc2_gain (const LokiModem ∗self, float gain)

Affecter une amplitude au signal en sortie du second oscillateur du modulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency_res (const LokiModem ∗self)

Lire le pas de réglage de la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency_max (const LokiModem ∗self)

Lire la fréquence maximale supportée pour le signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_demod_osc_wave_frequency (const LokiModem ∗self)

Lire la fréquence du signal en sortie de l'oscillateur du démodulateur (Fm2).

— void LokiModem_set_demod_osc_wave_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence au signal en sortie de l'oscillateur du démodulateur (Fm2).

— float LokiModem_get_mod_offset_res (const LokiModem ∗self)

Lire le pas de réglage de l'offset en sortie du modulateur.

— float LokiModem_get_mod_offset_max (const LokiModem ∗self)

Lire l'offset maximal supporté en sortie du modulateur.

— float LokiModem_get_mod_offset (const LokiModem ∗self)

Lire l'offset en sortie du modulateur.

— void LokiModem_set_mod_offset (const LokiModem ∗self, float offset)

Affecter un offset en sortie du modulateur.

— float LokiModem_get_mod_dc_value_res (const LokiModem ∗self)

Lire le pas de réglage du courant moyen de commande de la diode laser.

— float LokiModem_get_mod_dc_value_max (const LokiModem ∗self)

Lire le courant moyen maximal supporté pour la commande de la diode laser.

— float LokiModem_get_mod_dc_value (const LokiModem ∗self)

Lire le courant moyen de commande de la diode laser.

— void LokiModem_set_mod_dc_value (const LokiModem ∗self, float dc_value)

Affecter un courant moyen de commande de la diode laser.

— uint32_t LokiModem_get_mod_pga_gain_max (const LokiModem ∗self)

Lire le gain maximal de l'amplificateur de sortie.

— uint32_t LokiModem_get_mod_pga_gain (const LokiModem ∗self)

Lire le gain de l'amplificateur de sortie.

— void LokiModem_set_mod_pga_gain (const LokiModem ∗self, uint32_t gain)

Affecter un gain à l'amplificateur de sortie.

— uint32_t LokiModem_get_demod_channels (const LokiModem ∗self)

Lire le nombre de voies du démodulateur.

— uint32_t LokiModem_get_demod_decimation (const LokiModem ∗self)

Lire le facteur de décimation des filtres du démodulateur.

— float LokiModem_get_demod_bandwidth (const LokiModem ∗self)

Lire la bande passante des filtres du démodulateur.

— float LokiModem_get_demod_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en entrée du démodulateur.

— void LokiModem_set_demod_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en entrée du démodulateur.

— float LokiModem_get_demod_xy_sampling_frequency (const LokiModem ∗self)

Lire la fréquence d'échantillonnage en sortie du démodulateur.

— void LokiModem_set_demod_xy_sampling_frequency (const LokiModem ∗self, float freq)

Affecter une fréquence d'échantillonnage en sortie du démodulateur.

— uint32_t LokiModem_get_demod_delay_x_max (const LokiModem ∗self, uint32_t channel)

Lire le retard maximal applicable à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x_max (const LokiModem ∗self, uint32_t channel)

Lire le déphasage maximal applicable à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x_res (const LokiModem ∗self, uint32_t channel)

Lire le pas de réglage du déphasage applicable à la composante X (I) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_x (const LokiModem ∗self, uint32_t channel)

Lire le retard appliqué à la composante X (I) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_x (const LokiModem ∗self, uint32_t channel)

Lire le déphasage appliqué à la composante X (I) du signal démodulé pour une voie.

— void LokiModem_set_demod_delay_x (const LokiModem ∗self, uint32_t channel, uint32_t delay)

Affecter un retard à la composante X (I) du signal démodulé pour une voie.

— void LokiModem_set_demod_phase_x (const LokiModem ∗self, uint32_t channel, float phase)

Affecter un déphasage à la composante X (I) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_y_max (const LokiModem ∗self, uint32_t channel)

Lire le retard maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y_max (const LokiModem ∗self, uint32_t channel)

Lire le déphasage maximal applicable à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y_res (const LokiModem ∗self, uint32_t channel)

Lire le pas de réglage du déphasage applicable à la composante Y (Q) du signal démodulé pour une voie.

— uint32_t LokiModem_get_demod_delay_y (const LokiModem ∗self, uint32_t channel)

Lire le retard appliqué à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_phase_y (const LokiModem ∗self, uint32_t channel)

Lire le déphasage appliqué à la composante Y (Q) du signal démodulé pour une voie.

— void LokiModem_set_demod_delay_y (const LokiModem ∗self, uint32_t channel, uint32_t delay)

Affecter un retard à la composante Y (Q) du signal démodulé pour une voie.

— void LokiModem_set_demod_phase_y (const LokiModem ∗self, uint32_t channel, float phase)

Affecter un déphasage à la composante Y (Q) du signal démodulé pour une voie.

— float LokiModem_get_demod_pga_gain_max (const LokiModem ∗self, uint32_t channel)

Lire le gain maximal de l'amplificateur en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_pga_gain (const LokiModem ∗self, uint32_t channel)

Lire le gain de l'amplificateur en entrée du démodulateur pour une voie.

— void LokiModem_set_demod_pga_gain (const LokiModem ∗self, uint32_t channel, float gain)

Affecter un gain à l'amplificateur en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_adc_min_ratio (const LokiModem ∗self, uint32_t channel)

Lire la valeur minimale du signal en entrée du démodulateur pour une voie.

— float LokiModem_get_demod_adc_max_ratio (const LokiModem ∗self, uint32_t channel)

Lire la valeur maximale du signal en entrée du démodulateur pour une voie.

— uint32_t LokiModem_get_demod_packet_length_max (const LokiModem ∗self)

Lire la longueur maximale d'un paquet de données en sortie du démodulateur.

— uint32_t LokiModem_get_demod_packet_length (const LokiModem ∗self)

Lire la longueur d'un paquet de données en sortie du démodulateur.

— bool LokiModem_get_demod_mixerless (const LokiModem ∗self)

Identifier le type de démodulateur implanté dans le FPGA.

— bool LokiModem_get_demod_bypass (const LokiModem ∗self)

Lire l'état d'activation du démodulateur.

— void LokiModem_set_demod_bypass (const LokiModem ∗self, bool bypass)

Activer ou désactiver le démodulateur.

— bool LokiModem_get_test_mode (const LokiModem ∗self)

Lire l'état d'activation du mode de test.

— void LokiModem_set_test_mode (const LokiModem ∗self, bool test_mode)

Activer ou désactiver le mode de test.

— void LokiModem_restart (const LokiModem ∗self)

Redémarrer le modem.

— uint32_t LokiModem_get_adc_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons reçus sur une voie ADC en entrée du démodulateur.

— uint32_t LokiModem_get_dac_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons envoyés sur la sortie DAC du modulateur.

— uint32_t LokiModem_get_demod_sample_count (const LokiModem ∗self)

Lire le nombre d'échantillons produits sur une voie en sortie du démodulateur.

— int LokiModem_start_streaming (LokiModem ∗self, int32_t channels, bool simulate, bool raw, LokiFIFO ∗out)

Démarrer l'acquisition et le traitement des échantillons démodulés.

— void LokiModem_stop_streaming (LokiModem ∗self)

Arrêter l'acquisition.

— void LokiModem_force_stop (const LokiModem ∗self)

Arrêter l'acquisition immédiatement.

— bool LokiModem_is_streaming (const LokiModem ∗self)

Lire l'état de l'acquisition.

— unsigned LokiModem_get_dropped_packets (const LokiModem *self)

Lire le nombre de paquets perdus depuis le début de l'acquisition.

— void LokiModem_print_config (const LokiModem *self)

Afficher la configuration d'un modem sur la sortie standard.

6.47.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-modem.h.

6.48 libloki-modem.h

Aller à la documentation de ce fichier.

```
00001
00004
00015
00017
00018 #ifndef LIBLOKI_MODEM_H_
00019 #define LIBLOKI_MODEM_H_
00020
00021 #include "libloki-fifo.h"
00022
00023 #include <stdint.h>
00024 #include <stdbool.h>
00025 #include <sys/types.h>
00026
00028 typedef struct LokiModem LokiModem;
00029
00031 typedef struct {
00033     uint32_t channel;
00035     union {
00037         float x;
00039         int32_t x_raw;
00040     };
00042     union {
00044         float y;
00046         int32_t y_raw;
00047     };
00048 } LokiModemSample;
00049
00054 LokiModem *LokiModem_new(void);
00055
00060 void LokiModem_del(LokiModem *self);
00061
00067 uint32_t LokiModem_get_clk_frequency(const LokiModem *self);
00068
00074 float LokiModem_get_mod_sampling_frequency(const LokiModem *self);
00075
00084 void LokiModem_set_mod_sampling_frequency(const LokiModem *self, float freq);
00085
00093 float LokiModem_get_mod_osc1_wave_frequency_res(const LokiModem *self);
00094
00100 float LokiModem_get_mod_osc1_wave_frequency_max(const LokiModem *self);
00101
00107 float LokiModem_get_mod_osc1_wave_frequency(const LokiModem *self);
00108
00114 void LokiModem_set_mod_osc1_wave_frequency(const LokiModem *self, float freq);
```

```
00115
00123 float LokiModem_get_mod_osc1_gain_res(const LokiModem *self);
00124
00130 float LokiModem_get_mod_osc1_gain_max(const LokiModem *self);
00131
00137 float LokiModem_get_mod_osc1_gain(const LokiModem *self);
00138
00144 void LokiModem_set_mod_osc1_gain(const LokiModem *self, float gain);
00145
00153 float LokiModem_get_mod_osc2_wave_frequency_res(const LokiModem *self);
00154
00160 float LokiModem_get_mod_osc2_wave_frequency_max(const LokiModem *self);
00161
00167 float LokiModem_get_mod_osc2_wave_frequency(const LokiModem *self);
00168
00174 void LokiModem_set_mod_osc2_wave_frequency(const LokiModem *self, float freq);
00175
00183 float LokiModem_get_mod_osc2_gain_res(const LokiModem *self);
00184
00190 float LokiModem_get_mod_osc2_gain_max(const LokiModem *self);
00191
00197 float LokiModem_get_mod_osc2_gain(const LokiModem *self);
00198
00204 void LokiModem_set_mod_osc2_gain(const LokiModem *self, float gain);
00205
00213 float LokiModem_get_demod_osc_wave_frequency_res(const LokiModem *self);
00214
00220 float LokiModem_get_demod_osc_wave_frequency_max(const LokiModem *self);
00221
00227 float LokiModem_get_demod_osc_wave_frequency(const LokiModem *self);
00228
00234 void LokiModem_set_demod_osc_wave_frequency(const LokiModem *self, float freq);
00235
00241 float LokiModem_get_mod_offset_res(const LokiModem *self);
00242
00248 float LokiModem_get_mod_offset_max(const LokiModem *self);
00249
00255 float LokiModem_get_mod_offset(const LokiModem *self);
00256
00262 void LokiModem_set_mod_offset(const LokiModem *self, float offset);
00263
00269 float LokiModem_get_mod_dc_value_res(const LokiModem *self);
00270
00276 float LokiModem_get_mod_dc_value_max(const LokiModem *self);
00277
00283 float LokiModem_get_mod_dc_value(const LokiModem *self);
00284
00290 void LokiModem_set_mod_dc_value(const LokiModem *self, float dc_value);
00291
00297 uint32_t LokiModem_get_mod_pga_gain_max(const LokiModem *self);
00298
00304 uint32_t LokiModem_get_mod_pga_gain(const LokiModem *self);
00305
00311 void LokiModem_set_mod_pga_gain(const LokiModem *self, uint32_t gain);
00312
00318 uint32_t LokiModem_get_demod_channels(const LokiModem *self);
00319
00328 uint32_t LokiModem_get_demod_decimation(const LokiModem *self);
00329
00338 float LokiModem_get_demod_bandwidth(const LokiModem *self);
00339
00345 float LokiModem_get_demod_sampling_frequency(const LokiModem *self);
00346
00355 void LokiModem_set_demod_sampling_frequency(const LokiModem *self, float freq);
00356
00362 float LokiModem_get_demod_xy_sampling_frequency(const LokiModem *self);
00363
00373 void LokiModem_set_demod_xy_sampling_frequency(const LokiModem *self, float freq);
00374
00381 uint32_t LokiModem_get_demod_delay_x_max(const LokiModem *self, uint32_t channel);
00382
00390 float LokiModem_get_demod_phase_x_max(const LokiModem *self, uint32_t channel);
00391
00398 float LokiModem_get_demod_phase_x_res(const LokiModem *self, uint32_t channel);
00399
00406 uint32_t LokiModem_get_demod_delay_x(const LokiModem *self, uint32_t channel);
00407
00415 float LokiModem_get_demod_phase_x(const LokiModem *self, uint32_t channel);
00416
00423 void LokiModem_set_demod_delay_x(const LokiModem *self, uint32_t channel, uint32_t delay);
00424
00432 void LokiModem_set_demod_phase_x(const LokiModem *self, uint32_t channel, float phase);
00433
00440 uint32_t LokiModem_get_demod_delay_y_max(const LokiModem *self, uint32_t channel);
00441
00449 float LokiModem_get_demod_phase_y_max(const LokiModem *self, uint32_t channel);
00450
```

```
00457 float LokiModem_get_demod_phase_y_res(const LokiModem *self, uint32_t channel);
00458
00465 uint32_t LokiModem_get_demod_delay_y(const LokiModem *self, uint32_t channel);
00466
00474 float LokiModem_get_demod_phase_y(const LokiModem *self, uint32_t channel);
00475
00482 void LokiModem_set_demod_delay_y(const LokiModem *self, uint32_t channel, uint32_t delay);
00483
00491 void LokiModem_set_demod_phase_y(const LokiModem *self, uint32_t channel, float phase);
00492
00499 float LokiModem_get_demod_pga_gain_max(const LokiModem *self, uint32_t channel);
00500
00507 float LokiModem_get_demod_pga_gain(const LokiModem *self, uint32_t channel);
00508
00515 void LokiModem_set_demod_pga_gain(const LokiModem *self, uint32_t channel, float gain);
00516
00529 float LokiModem_get_demod_adc_min_ratio(const LokiModem *self, uint32_t channel);
00530
00543 float LokiModem_get_demod_adc_max_ratio(const LokiModem *self, uint32_t channel);
00544
00553 uint32_t LokiModem_get_demod_packet_length_max(const LokiModem *self);
00554
00563 uint32_t LokiModem_get_demod_packet_length(const LokiModem *self);
00564
00574 bool LokiModem_get_demod_mixerless(const LokiModem *self);
00575
00584 bool LokiModem_get_demod_bypass(const LokiModem *self);
00585
00594 void LokiModem_set_demod_bypass(const LokiModem *self, bool bypass);
00595
00604 bool LokiModem_get_test_mode(const LokiModem *self);
00605
00614 void LokiModem_set_test_mode(const LokiModem *self, bool test_mode);
00615
00625 void LokiModem_restart(const LokiModem *self);
00626
00634 uint32_t LokiModem_get_adc_sample_count(const LokiModem *self);
00635
00643 uint32_t LokiModem_get_dac_sample_count(const LokiModem *self);
00644
00652 uint32_t LokiModem_get_demod_sample_count(const LokiModem *self);
00653
00667 int LokiModem_start_streaming(LokiModem *self, int32_t channels, bool simulate, bool raw, LokiFIFO
      *out);
00668
00675 void LokiModem_stop_streaming(LokiModem *self);
00676
00684 void LokiModem_force_stop(const LokiModem *self);
00685
00691 bool LokiModem_is_streaming(const LokiModem *self);
00692
00701 unsigned LokiModem_get_dropped_packets(const LokiModem *self);
00702
00707 void LokiModem_print_config(const LokiModem *self);
00708
00709 #endif
00710
00713
```

6.49 Référence du fichier libloki-mseed.c

```
#include "libloki.h"
#include <stdlib.h>
#include <libmseed.h>
#include <math.h>
```

Graphe des dépendances par inclusion de libloki-mseed.c:

Structures de données

— struct LokiMSeedConfig

Configuration d'un formateur de séries temporelles miniSEED.

— struct LokiMSeedInputSample

Type des échantillons attendus par LokiMSeed_work.

— union LokiMSeedOutputSample

Type des échantillons passés à la bibliothèque `libmseed`.

— struct LokiMSeed

Représentation d'un formateur de séries temporelles miniSEED.

Macros

— #define CHANNEL_NAME_LEN 4

Le nombre maximal de caractères représentant le nom de la voie.

— #define MSEED_FILENAME_SUFFIX_LEN (CHANNEL_NAME_LEN + 7)

Le format du suffixe du nom de fichier: "_{n°de voie}.mseed".

— #define MSEED_RECLEN 512

La longueur d'un record miniSEED.

Fonctions

— LokiMSeedConfig * LokiMSeedConfig_new (bool separate_channels, const char *station, const char *location, const char *network, bool raw, bool steim2, const char *channel_names)

Construire une configuration d'enregistrement de données au format miniSEED.

— void LokiMSeedConfig_del (LokiMSeedConfig *self)

Détruire une configuration d'enregistrement de données au format miniSEED.

— LokiMSeed * LokiMSeed_new (LokiMSeedConfig *config)

Construire un formateur de séries temporelles miniSEED.

— void LokiMSeed_work (LokiMSeed *self, const char *filename, LokiTimestamp timestamp, bool file_change, void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_↩ frequency)

Enregistrer un paquet de données au format miniSEED.

— void LokiMSeed_close (LokiMSeed *self)

Fermer un formateur de séries temporelles miniSEED.

— void LokiMSeed_del (LokiMSeed *self)

Détruire un formateur de séries temporelles miniSEED.

— **LokiClass_implementation** (LokiFormatter, LokiMSeed, LokiClass_define_method(LokiMSeed, new) LokiClass_define_method(LokiMSeed, work) LokiClass_define_method(LokiMSeed, close) LokiClass_define_method(LokiMSe del))

6.49.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-mseed.c.

6.50 libloki-mseed.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki.h"
00003
00004 #include <stdlib.h>
00005 #include <libmseed.h>
00006 #include <math.h>
00007
00010
00012 struct LokiMSeedConfig {
00014     bool separate_channels;
00016     char *station;
00018     char *location;
00020     char *network;
00022     bool raw;
00024     bool steim2;
00026     char *channel_names;
00027 };
00028
00029 LokiMSeedConfig *LokiMSeedConfig_new(bool separate_channels, const char *station, const char
      *location, const char *network, bool raw, bool steim2, const char *channel_names) {
00030     LokiMSeedConfig *self = calloc(1, sizeof(LokiMSeedConfig));
00031     if (!self) {
00032         return NULL;
00033     }
00034
00035     self->separate_channels = separate_channels;
00036     self->station           = strdup(station);
00037     self->location          = strdup(location);
00038     self->network           = strdup(network);
00039     self->raw               = raw;
00040     self->steim2            = steim2;
00041     self->channel_names     = strdup(channel_names);
00042
00043     if (!self->station || !self->location || !self->network || !self->channel_names) {
00044         LokiMSeedConfig_del(self);
00045         return NULL;
00046     }
00047
00048     // Dans la liste des nom de voies, remplacer les virgules par des fins de chaîne.
00049     for (size_t i = 0; i < strlen(channel_names); i++) {
00050         if (channel_names[i] == ',') {
00051             self->channel_names[i] = 0;
00052         }
00053     }
00054
00055     return self;
00056 }
00057
00058 void LokiMSeedConfig_del(LokiMSeedConfig *self) {
00059     if (!self) {
00060         return;
00061     }
00062
00063     if (self->station) {
00064         free(self->station);
00065     }
00066
00067     if (self->location) {
00068         free(self->location);
00069     }
```

```
00070
00071     if (self->network) {
00072         free(self->network);
00073     }
00074
00075     if (self->channel_names) {
00076         free(self->channel_names);
00077     }
00078
00079     free(self);
00080 }
00081
00083 typedef struct {
00085     uint32_t channel;
00087     union {
00088         int32_t value_raw;
00089         float   value;
00090     };
00091     // D'autres données non utilisées ici.
00092 } LokiMSeedInputSample;
00093
00095 typedef union {
00097     int32_t i;
00099     float   f;
00100 } LokiMSeedOutputSample;
00101
00103 struct LokiMSeed {
00105     LokiMSeedConfig *config;
00107     LokiMSeedOutputSample *buffer;
00109     MSRecord *msr;
00110 };
00111
00112 LokiMSeed *LokiMSeed_new(LokiMSeedConfig *config) {
00113     LokiMSeed *self = calloc(1, sizeof(LokiMSeed));
00114     self->config = config;
00115
00116     // Initialiser un record MiniSEED.
00117     self->msr = msr_init(NULL);
00118     if (!self->msr) {
00119         LokiMSeed_del(self);
00120         return NULL;
00121     }
00122
00123     strncpy(self->msr->station,  config->station,  5);
00124     strncpy(self->msr->location, config->location, 2);
00125     strncpy(self->msr->network,  config->network,  2);
00126     self->msr->dataquality = 'R'; // TODO
00127     self->msr->sampletype  = config->steim2 ? 'i' : 'f';
00128
00129     return self;
00130 }
00131
00133 #define CHANNEL_NAME_LEN 4
00134
00136 #define MSEED_FILENAME_SUFFIX_LEN (CHANNEL_NAME_LEN + 7)
00137
00139 #define MSEED_RECLEN 512
00140
00141 void LokiMSeed_work(LokiMSeed *self, const char *filename, LokiTimestamp timestamp, bool file_change,
      void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float
      sampling_frequency) {
00142     // Allouer le buffer des déplacements au premier appel.
00143     if (!self->buffer) {
00144         self->buffer = calloc(length, sizeof(LokiMSeedOutputSample));
00145         if (!self->buffer) {
00146             return;
00147         }
00148         self->msr->samprate    = sampling_frequency;
00149         self->msr->datasamples = self->buffer;
00150     }
00151
00152     const bool raw               = self->config->raw;
00153     const bool steim2            = self->config->steim2;
00154     const bool separate_channels = self->config->separate_channels;
00155   bool overwrite = file_change;
00156
00157     const size_t filename_len    = strlen(filename);
00158     char *actual_filename        = calloc(filename_len + CHANNEL_NAME_LEN + 1, 1);
00159     char *filename_suffix        = actual_filename + filename_len - 6;
00160     strcpy(actual_filename, filename);
00161
00162     const flag encoding  = steim2 ? DE_STEIM2 : DE_FLOAT32;
00163     const flag byteorder = 1; // big-endian (requis pour le codage Steim2)
00164
00165     char *name = self->config->channel_names;
00166
00167     for (size_t c = 0; c < max_channels; c ++) {
```

```
00168         if (!(channels & (1 << c))) {
00169             continue;
00170         }
00171 
00172         // Collecter les données pour la voie c
00173         size_t actual_length = 0;
00174         uint8_t *ptr_end = (uint8_t*)data + length * sample_size;
00175         for (uint8_t *ptr = data; ptr != ptr_end; ptr += sample_size) {
00176             const LokiMSeedInputSample *const sample = (LokiMSeedInputSample*)ptr;
00177             if (sample->channel != c) {
00178                 continue;
00179             }
00180 
00181             if (!steim2) {
00182                 self->buffer[actual_length].f = sample->value;
00183             }
00184             else if (raw) {
00185                 self->buffer[actual_length].i = sample->value_raw;
00186             }
00187             else {
00188                 self->buffer[actual_length].i = roundf(sample->value);
00189             }
00190             actual_length ++;
00191         }
00192 
00193         strncpy(self->msr->channel, name, 3);
00194         name += strlen(name) + 1;
00195 
00196         self->msr->starttime  = timestamp / 1000;
00197         self->msr->numsamples = self->msr->samplecnt = actual_length;
00198 
00199         // Écriture de l'enregistrement dans le fichier de destination.
00200         // overwrite=file_change, reclen=512, verbose=false
00201         if (separate_channels) {
00202             snprintf(filename_suffix, MSEED_FILENAME_SUFFIX_LEN, "_%zu.mseed", c);
00203     /*PhM* correctif ecriture dans un seul fichier ( evite l'ecrasement du packet de voie 0 à n-1) */
00204             overwrite = file_change;
00205         }else{
00206         overwrite = (file_change && (c==0));
00207 
00208         }
00209 
      /*************************************************************************************************/
00210         if (msr_writemseed(self->msr, actual_filename, overwrite, MSEED_RECLEN, encoding, byteorder,
      false) < 0) {
00211             fprintf(stderr, "LokiMSeed_work: msr_writemseed failed to write to '%s'\n",
      actual_filename);
00212         }
00213     }
00214 }
00215 
00216 void LokiMSeed_close(LokiMSeed *self) {
00217     if (self->buffer) {
00218         free(self->buffer);
00219         self->buffer = NULL;
00220     }
00221 }
00222 
00223 void LokiMSeed_del(LokiMSeed *self) {
00224     if (!self) {
00225         return;
00226     }
00227 
00228     if (self->msr) {
00229         self->msr->datasamples = NULL;
00230         msr_free(&self->msr);
00231     }
00232 
00233     free(self);
00234 }
00235 
00236 LokiClass_implementation(LokiFormatter, LokiMSeed,
00237     LokiClass_define_method(LokiMSeed, new)
00238     LokiClass_define_method(LokiMSeed, work)
00239     LokiClass_define_method(LokiMSeed, close)
00240     LokiClass_define_method(LokiMSeed, del)
00241 );
00242 
```

6.51 Référence du fichier libloki-mseed.h

```
#include "libloki-formatter.h"
```

Graphe des dépendances par inclusion de libloki-mseed.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiMSeedConfig LokiMSeedConfig

Configuration d'un formateur de séries temporelles miniSEED.

— typedef struct LokiMSeed LokiMSeed

Représentation d'un formateur de séries temporelles miniSEED.

Fonctions

— LokiMSeedConfig ∗ LokiMSeedConfig_new (bool separate_channels, const char ∗station, const char ∗location, const char ∗network, bool raw, bool steim2, const char ∗channel_names)

Construire une configuration d'enregistrement de données au format miniSEED.

— void LokiMSeedConfig_del (LokiMSeedConfig ∗self)

Détruire une configuration d'enregistrement de données au format miniSEED.

— LokiMSeed ∗ LokiMSeed_new (LokiMSeedConfig ∗config)

Construire un formateur de séries temporelles miniSEED.

— void LokiMSeed_work (LokiMSeed ∗self, const char ∗filename, LokiTimestamp timestamp, bool file_change, void ∗data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_↩ frequency)

Enregistrer un paquet de données au format miniSEED.

— void LokiMSeed_close (LokiMSeed ∗self)

Fermer un formateur de séries temporelles miniSEED.

— void LokiMSeed_del (LokiMSeed ∗self)

Détruire un formateur de séries temporelles miniSEED.

— **LokiClass_declare_implementation** (LokiFormatter, LokiMSeed)

6.51.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-mseed.h.

6.52 libloki-mseed.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_MSEED_H_
00011 #define LIBLOKI_MSEED_H_
00012
00013 #include "libloki-formatter.h"
00014
00016 typedef struct LokiMSeedConfig LokiMSeedConfig;
00017
00029 LokiMSeedConfig *LokiMSeedConfig_new(bool separate_channels, const char *station, const char
      *location, const char *network, bool raw, bool steim2, const char *channel_names);
00030
00035 void LokiMSeedConfig_del(LokiMSeedConfig *self);
00036
00038 typedef struct LokiMSeed LokiMSeed;
00039
00045 LokiMSeed *LokiMSeed_new(LokiMSeedConfig *config);
00046
00073 void LokiMSeed_work(LokiMSeed *self, const char *filename, LokiTimestamp timestamp, bool file_change,
      void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float
      sampling_frequency);
00074
00082 void LokiMSeed_close(LokiMSeed *self);
00083
00088 void LokiMSeed_del(LokiMSeed *self);
00089
00090 LokiClass_declare_implementation(LokiFormatter, LokiMSeed);
00091
00092 #endif
00093
```

6.53 Référence du fichier libloki-neon.h

Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Structures de données

— struct float32x2_t

Vecteur de deux nombres flottants 32 bits.

— union float32x4_t

Vecteur de quatre nombres flottants 32 bits.

Fonctions

— static float32x2_t vadd_f32 (float32x2_t a, float32x2_t b)

Additionner deux vecteurs.

— static float vget_lane_f32 (float32x2_t a, size_t i)

Lire un élément d'un vecteur.

— static float32x4_t vmovq_n_f32 (float a)

Construire un vecteur.

— static float32x4_t vld1q_f32 (const float *a)

Lire un vecteur en mémoire.

— static float32x2_t vget_low_f32 (float32x4_t a)

Lire la première moitié d'un vecteur.

— static float32x2_t vget_high_f32 (float32x4_t a)

Lire la seconde moitié d'un vecteur.

— static float32x4_t vld1q_lane_f32 (const float *a, float32x4_t b, size_t i)

Remplacer un élément d'un vecteur avec une valeur lue en mémoire.

— static float32x4_t vmulq_f32 (float32x4_t a, float32x4_t b)

Multiplier deux vecteurs terme à terme.

— static float32x4_t vmlaq_f32 (float32x4_t a, float32x4_t b, float32x4_t c)

Calculer le résultat d'une multiplication-accumulation sur des vecteurs.

6.53.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-neon.h.

6.54 libloki-neon.h

Aller à la documentation de ce fichier.

```
00001
00013
00015
00016 #ifndef LIBLOKI_NEON_H_
00017 #define LIBLOKI_NEON_H_
00018
00019 #ifdef __ARM_NEON
00020
00021 #include <arm_neon.h>
00022
00023 #else
00024
00026 typedef struct {
00028     float val[2];
00029 } float32x2_t;
00030
00032 typedef union {
00034     float val[4];
00036     struct {
00038         float32x2_t low;
00040         float32x2_t high;
00041     };
00042 } float32x4_t;
```

```
00043
00050 static inline float32x2_t vadd_f32(float32x2_t a, float32x2_t b) {
00051     a.val[0] += b.val[0];
00052     a.val[1] += b.val[1];
00053     return a;
00054 }
00055
00062 static inline float vget_lane_f32(float32x2_t a, size_t i) {
00063     return a.val[i];
00064 }
00065
00071 static inline float32x4_t vmovq_n_f32(float a) {
00072     return (float32x4_t) {
00073         .val = {a, a, a, a}
00074     };
00075 }
00076
00082 static inline float32x4_t vld1q_f32(const float *a) {
00083     return *(float32x4_t*)a;
00084 }
00085
00091 static inline float32x2_t vget_low_f32(float32x4_t a) {
00092     return a.low;
00093 }
00094
00100 static inline float32x2_t vget_high_f32(float32x4_t a) {
00101     return a.high;
00102 }
00103
00111 static inline float32x4_t vld1q_lane_f32(const float *a, float32x4_t b, size_t i) {
00112     b.val[i] = *a;
00113     return b;
00114 }
00115
00122 static inline float32x4_t vmulq_f32(float32x4_t a, float32x4_t b) {
00123     for (size_t i = 0; i < 4; i ++) {
00124         a.val[i] *= b.val[i];
00125     }
00126     return a;
00127 }
00128
00136 static inline float32x4_t vmlaq_f32(float32x4_t a, float32x4_t b, float32x4_t c) {
00137     for (size_t i = 0; i < 4; i ++) {
00138         a.val[i] += b.val[i] * c.val[i];
00139     }
00140     return a;
00141 }
00142
00143 #endif
00144
00145 #endif
00146
```

6.55 Référence du fichier libloki-nlms.c

```
#include "libloki-nlms.h"
#include "libloki-ellipse.h"
#include <stdlib.h>
#include <string.h>
```

Graphe des dépendances par inclusion de libloki-nlms.c:

Structures de données

— struct LokiNLMS

Représentattion d'un filtre NLMS.

Macros

— #define NLMS_NU 1.0e-30f

Correction des poids du filtre pour éviter une division par zéro.

Fonctions

— LokiNLMS * LokiNLMS_new (float initial_radius, const float params[static 2])

Construire un nouveau filtre NLMS.

— LokiNLMS * LokiNLMS_clone (const LokiNLMS *other)

Construire un clone d'un filtre NLMS.

— float * LokiNLMS_get_state (LokiNLMS *self)

Retourner l'état courant du filtre.

— void LokiNLMS_work (LokiNLMS *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— void LokiNLMS_del (LokiNLMS *self)

Détruire un filtre NLMS.

— **LokiClass_implementation** (LokiEllipseEstimator, LokiNLMS, LokiClass_set_attribute(params_↩ length, 2) LokiClass_define_method(LokiNLMS, new) LokiClass_define_method(LokiNLMS, clone) LokiClass_define_method(LokiNLMS, work) LokiClass_define_method(LokiNLMS, get_state) LokiClass_define_method(LokiNL del))

6.55.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-nlms.c.

6.56 libloki-nlms.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki-nlms.h"
00003 #include "libloki-ellipse.h"
00004
00005 #include <stdlib.h>
00006 #include <string.h>
00007
00010
00012 #define NLMS_NU 1.0e-30f
00013
00015 struct LokiNLMS {
00017     float mu;
00019     float alpha;
```

```
00021     LokiEllipseEquation state;
00023     float weights[5];
00024 };
00025
00026 LokiNLMS *LokiNLMS_new(float initial_radius, const float params[static 2]) {
00027     LokiNLMS * self = calloc(1, sizeof(LokiNLMS));
00028     if (!self) {
00029         return NULL;
00030     }
00031
00032     self->mu      = params[0];
00033     self->alpha   = params[1];
00034
00035     self->state[0] = 0.5f;
00036     self->state[4] = -initial_radius * initial_radius / 2;
00037
00038     return self;
00039 }
00040
00041 LokiNLMS *LokiNLMS_clone(const LokiNLMS *other) {
00042     LokiNLMS * self = calloc(1, sizeof(LokiNLMS));
00043     if (!self) {
00044         return NULL;
00045     }
00046
00047     memcpy(self, other, sizeof(LokiNLMS));
00048
00049     return self;
00050 }
00051
00052 float *LokiNLMS_get_state(LokiNLMS *self) {
00053     return self->state;
00054 }
00055
00056 void LokiNLMS_work(LokiNLMS *self, float x, float y) {
00057     float out = -y * y;
00058     float in[] = {x * x + out, x * y, x, y, 1};
00059
00060     // Calcul de l'erreur courante :
00061     // epsilon = out - in * state
00062     float epsilon = out;
00063     for (size_t i = 0; i < LEN(in); i ++) {
00064         epsilon -= in[i] * self->state[i];
00065     }
00066
00067     // Mise à jour des facteurs de normalisation
00068     // en utilisant un filtre IIR du premier ordre.
00069     // Mise à jour des coefficients par descente de gradient.
00070     float alpha   = self->alpha;
00071     float umalpha = 1.0f - alpha;
00072     float m       = self->mu * epsilon;
00073     for (size_t i = 0; i < LEN(in); i ++) {
00074         self->weights[i] = umalpha * self->weights[i] + alpha * in[i] * in[i];
00075         self->state[i] += m * in[i] / (self->weights[i] + NLMS_NU);
00076     }
00077 }
00078
00079 void LokiNLMS_del(LokiNLMS *self) {
00080     if (!self) {
00081         return;
00082     }
00083     free(self);
00084 }
00085
00086 LokiClass_implementation(LokiEllipseEstimator, LokiNLMS,
00087     LokiClass_set_attribute(params_length, 2)
00088     LokiClass_define_method(LokiNLMS, new)
00089     LokiClass_define_method(LokiNLMS, clone)
00090     LokiClass_define_method(LokiNLMS, work)
00091     LokiClass_define_method(LokiNLMS, get_state)
00092     LokiClass_define_method(LokiNLMS, del)
00093 );
00094
```

6.57 Référence du fichier libloki-nlms.h

`#include "libloki-ellipse-estimator.h"`

Graphe des dépendances par inclusion de libloki-nlms.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiNLMS LokiNLMS

Représentattion d'un filtre NLMS.

Fonctions

— LokiNLMS * LokiNLMS_new (float initial_radius, const float params[static 2])

Construire un nouveau filtre NLMS.

— LokiNLMS * LokiNLMS_clone (const LokiNLMS *other)

Construire un clone d'un filtre NLMS.

— void LokiNLMS_work (LokiNLMS *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— float * LokiNLMS_get_state (LokiNLMS *self)

Retourner l'état courant du filtre.

— void LokiNLMS_del (LokiNLMS *self)

Détruire un filtre NLMS.

— **LokiClass_declare_implementation** (LokiEllipseEstimator, LokiNLMS)

6.57.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-nlms.h.

6.58 libloki-nlms.h

Aller à la documentation de ce fichier.

```
00001
00008
00010
00011 #ifndef LIBLOKI_NLMS_H_
00012 #define LIBLOKI_NLMS_H_
00013
00014 #include "libloki-ellipse-estimator.h"
00015
00017 typedef struct LokiNLMS LokiNLMS;
00018
00025 LokiNLMS *LokiNLMS_new(float initial_radius, const float params[static 2]);
```

```
00026
00032 LokiNLMS *LokiNLMS_clone(const LokiNLMS *other);
00033
00043 void LokiNLMS_work(LokiNLMS *self, float x, float y);
00044
00050 float *LokiNLMS_get_state(LokiNLMS *self);
00051
00056 void LokiNLMS_del(LokiNLMS *self);
00057
00058 LokiClass_declare_implementation(LokiEllipseEstimator, LokiNLMS);
00059
00060 #endif
00061
```

6.59 Référence du fichier libloki-packet.h

`#include <inttypes.h>`
Graphe des dépendances par inclusion de libloki-packet.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Structures de données

— struct LokiPacket

Structure d'un paquet d'échantillons échangés entre tâches concurrentes.

Fonctions

— static size_t LokiPacket_get_fifo_item_size (size_t packet_length, size_t sample_size)

Calculer la taille d'un paquet de données.

— static size_t LokiPacket_get_length (LokiFIFO *fifo, size_t sample_size)

Lire la longueur des paquets supportés par une FIFO.

6.59.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-packet.h.

6.60 libloki-packet.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_PACKET_H_
00011 #define LIBLOKI_PACKET_H_
00012
00013 #include <inttypes.h>
00014
00016 typedef struct {
00018     LokiTimestamp timestamp;
00020     char buffer[0];
00021     // Ne rien ajouter ici
00022 } LokiPacket;
00023
00031 static inline size_t LokiPacket_get_fifo_item_size(size_t packet_length, size_t sample_size) {
00032     return packet_length * sample_size + sizeof(LokiPacket);
00033 }
00034
00041 static inline size_t LokiPacket_get_length(LokiFIFO *fifo, size_t sample_size) {
00042     return (LokiFIFO_get_item_size(fifo) - sizeof(LokiPacket)) / sample_size;
00043 }
00044
00045 #endif
00046
```

6.61 Référence du fichier libloki-raw.c

```
#include "libloki.h"
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <errno.h>
```

Graphe des dépendances par inclusion de libloki-raw.c:

Structures de données

— struct LokiRaw

Représentation d'un formateur de données brutes.

Fonctions

— LokiRaw * LokiRaw_new (void *config)

Construire un formateur de données brutes.

— void LokiRaw_work (LokiRaw *self, const char *filename, LokiTimestamp timestamp, bool file_change, void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency)

Enregistrer un paquet de données.

— void LokiRaw_close (LokiRaw *self)

Fermer un formateur de données brutes.

— void LokiRaw_del (LokiRaw *self)

Détruire un formateur de données brutes.

— **LokiClass_implementation** (LokiFormatter, LokiRaw, LokiClass_define_method(LokiRaw, new) LokiClass_define_method(Lo
work) LokiClass_define_method(LokiRaw, close) LokiClass_define_method(LokiRaw, del))

6.61.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-raw.c.

6.62 libloki-raw.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki.h"
00003
00004 #include <stdlib.h>
00005 #include <stdio.h>
00006 #include <unistd.h>
00007 #include <fcntl.h>
00008 #include <string.h>
00009 #include <errno.h>
00010
00013
00015 struct LokiRaw {
00017     int out;
00018 };
00019
00020 LokiRaw *LokiRaw_new(void *config) {
00021     // L'objet config est ignoré
00022     LokiRaw *self = calloc(1, sizeof(LokiRaw));
00023     self->out = -1;
00024     return self;
00025 }
00026
00027 void LokiRaw_work(LokiRaw *self, const char *filename, LokiTimestamp timestamp, bool file_change, void
      *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float
      sampling_frequency) {
00028     if (file_change) {
00029         if (self->out >= 0) {
00030             close(self->out);
00031         }
00032         self->out = open(filename, O_CREAT | O_WRONLY, 0644);
00033         if (self->out < 0) {
00034             fprintf(stderr, "LokiRaw_work: failed to create file %s (%s)\n", filename,
      strerror(errno));
00035         }
00036     }
00037
00038     if (self->out < 0) {
00039         return;
00040     }
00041
00042     size_t count = sample_size * length;
00043     if (write(self->out, data, count) != count) {
00044         fprintf(stderr, "LokiRaw_work: write error (%s)\n", strerror(errno));
00045     }
00046 }
00047
00048 void LokiRaw_close(LokiRaw *self) {
00049     if (self->out >= 0) {
```

```
00050         close(self->out);
00051         self->out = -1;
00052     }
00053 }
00054
00055 void LokiRaw_del(LokiRaw *self) {
00056     if (!self) {
00057         return;
00058     }
00059
00060     LokiRaw_close(self);
00061
00062
00063     free(self);
00064 }
00065
00066 LokiClass_implementation(LokiFormatter, LokiRaw,
00067     LokiClass_define_method(LokiRaw, new)
00068     LokiClass_define_method(LokiRaw, work)
00069     LokiClass_define_method(LokiRaw, close)
00070     LokiClass_define_method(LokiRaw, del)
00071 );
00072
```

6.63 Référence du fichier libloki-raw.h

`#include "libloki-formatter.h"`
Graphe des dépendances par inclusion de libloki-raw.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiRaw LokiRaw

Représentation d'un formateur de données brutes.

Fonctions

— LokiRaw * LokiRaw_new (void *config)

Construire un formateur de données brutes.

— void LokiRaw_work (LokiRaw *self, const char *filename, LokiTimestamp timestamp, bool file_change, void *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency)

Enregistrer un paquet de données.

— void LokiRaw_close (LokiRaw *self)

Fermer un formateur de données brutes.

— void LokiRaw_del (LokiRaw *self)

Détruire un formateur de données brutes.

— **LokiClass_declare_implementation** (LokiFormatter, LokiRaw)

6.63.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-raw.h.

6.64 libloki-raw.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_RAW_H_
00011 #define LIBLOKI_RAW_H_
00012
00013 #include "libloki-formatter.h"
00014
00016 typedef struct LokiRaw LokiRaw;
00017
00023 LokiRaw *LokiRaw_new(void *config);
00024
00038 void LokiRaw_work(LokiRaw *self, const char *filename, LokiTimestamp timestamp, bool file_change, void
      *data, size_t length, size_t max_channels, int32_t channels, size_t sample_size, float
      sampling_frequency);
00039
00047 void LokiRaw_close(LokiRaw *self);
00048
00053 void LokiRaw_del(LokiRaw *self);
00054
00055 LokiClass_declare_implementation(LokiFormatter, LokiRaw);
00056
00057 #endif
00058
```

6.65 Référence du fichier libloki-recorder.c

```
#include "libloki.h"
#include <math.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <pthread.h>
#include <signal.h>
#include <inttypes.h>
```

Graphe des dépendances par inclusion de libloki-recorder.c:

Structures de données

— struct LokiRecorder

Représentation d'un enregistreur de séries temporelles.

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

— #define FILENAME_SUFFIX_DATE_LEN 16

Longueur de chaîne à allouer pour mettre en forme la date et l'heure d'un fichier ("_YYYYmmddTHHMMSS").

— #define FILENAME_SUFFIX_MSEC_LEN 6

Longueur de chaîne à allouer pour mettre en forme le nombre de millisecondes de la date d'un fichier (".mmmZ.").

— #define FILENAME_SUFFIX_LEN (FILENAME_SUFFIX_DATE_LEN + FILENAME_SUFFIX_MSEC_LEN)

Longueur de chaîne à allouer pour mettre en forme la date complète d'un fichier.

Fonctions

— static void LokiRecorder_set_formatter (LokiRecorder ∗self, void ∗config)

Affecter un formateur à un enregistreur.

— LokiRecorder ∗ LokiRecorder_new (const char ∗filename, const char ∗filetype, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency, float duration, float duration_per_file, void ∗config)

Construire un enregistreur de séries temporelles.

— char ∗ LokiRecorder_get_filename (const LokiRecorder ∗self)

Lire le nom des fichiers gérés par un enregistreur.

— size_t LokiRecorder_get_channel_count (const LokiRecorder ∗self)

Lire le nombre de voies gérées par un enregistreur.

— static void ∗ LokiRecorder_work (void ∗ptr)

Enregistrer les données.

— int LokiRecorder_start (LokiRecorder ∗self, LokiFIFO ∗in, size_t cid)

Démarrer l'enregistrement.

— bool LokiRecorder_is_recording (LokiRecorder ∗self)

Lire l'état d'un enregistreur.

— static void LokiRecorder_signal_handler (int signum)

Gestionnaire d'événements.

— void LokiRecorder_wait (LokiRecorder ∗self)

Attendre la fin de l'enregistrement.

— void LokiRecorder_stop (LokiRecorder ∗self)

Arrêter l'enregistrement.

— void LokiRecorder_del (LokiRecorder ∗self)

Détruire un enregistreur.

Variables

— static const char ∗const LokiRecorder_formatter_names [ ]

La liste des noms de formateurs supportés.

— static const LokiFormatterInterface ∗const LokiRecorder_formatter_impls [ ]

La liste des implémentations de formateurs supportées.

— static LokiRecorder ∗ LokiRecorder_waiting_instance

L'instance de LokiRecorder qui fait l'objet d'une attente par la fonction LokiRecorder_wait.

6.65.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-recorder.c.

6.65.2 Documentation des macros

6.65.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Définition à la ligne 3 du fichier libloki-recorder.c.

6.66 libloki-recorder.c

Aller à la documentation de ce fichier.

```
00001
00003 #define _GNU_SOURCE
00004
00005 #include "libloki.h"
00006
00007 #include <math.h>
00008 #include <stdlib.h>
00009 #include <stdbool.h>
00010 #include <stdio.h>
00011 #include <string.h>
00012 #include <pthread.h>
00013 #include <signal.h>
00014 #include <inttypes.h>
00015
00018
00020 struct LokiRecorder {
00022     char *filename;
00024     char *filetype;
00026     LokiFormatter *formatter;
00028     size_t max_channels;
00030     int32_t channels;
00032     size_t channel_count;
00034     float sampling_frequency;
00036     size_t sample_size;
00038     size_t sample_count;
00040     size_t sample_count_per_file;
00042     bool multifile;
00044     LokiFIFO *in;
00046     size_t cid;
00048     volatile bool stop_request;
00050     bool is_recording;
00052     pthread_mutex_t mutex;
00054     pthread_cond_t recording_done;
00056     pthread_t thread;
00057 };
00058
00060 static const char *const LokiRecorder_formatter_names[] = {
00061     "raw", "mseed"
00062 };
00063
00065 static const LokiFormatterInterface *const LokiRecorder_formatter_impls[] = {
00066     &LokiRawClass, &LokiMSeedClass
00067 };
00068
00070 #define FILENAME_SUFFIX_DATE_LEN 16
00071
00073 #define FILENAME_SUFFIX_MSEC_LEN 6
00074
00076 #define FILENAME_SUFFIX_LEN (FILENAME_SUFFIX_DATE_LEN + FILENAME_SUFFIX_MSEC_LEN)
00077
00087 static void LokiRecorder_set_formatter(LokiRecorder *self, void *config) {
00088     LokiFormatterInterface *impl = LokiRecorder_formatter_impls[0];
00089     for (size_t i = 1; i < LEN(LokiRecorder_formatter_names); i ++) {
00090         if (!strcmp(LokiRecorder_formatter_names[i], self->filetype)) {
00091             impl = LokiRecorder_formatter_impls[i];
00092             break;
00093         }
00094     }
00095     self->formatter = LokiFormatter_new_with_implementation(impl, config);
00096 }
00097
00098 LokiRecorder *LokiRecorder_new(const char *filename, const char *filetype, size_t max_channels,
      int32_t channels, size_t sample_size, float sampling_frequency, float duration, float
      duration_per_file, void *config) {
00099     LokiRecorder *self = calloc(1, sizeof(LokiRecorder));
00100     if (!self) {
00101         return NULL;
00102     }
00103
00104     self->filename = calloc(strlen(filename) + strlen(filetype) + FILENAME_SUFFIX_LEN + 1, 1);
00105     if (!self->filename) {
00106         LokiRecorder_del(self);
00107         return NULL;
00108     }
00109     strcpy(self->filename, filename);
00110     self->filetype = strdup(filetype);
00111
00112     // Compter le nombre de voies activées dans le masque channels.
00113     self->channel_count = 0;
00114     for (size_t c = 0; c < max_channels; c ++) {
00115         if (channels & (1 << c)) {
00116             self->channel_count ++;
```

```
00117         }
00118     }
00119
00120     self->max_channels          = max_channels;
00121     self->channels              = channels;
00122     self->sampling_frequency    = sampling_frequency;
00123     self->sample_size           = sample_size;
00124     self->sample_count          = self->channel_count * ceilf(duration * sampling_frequency);
00125     self->multifile             = duration_per_file > 0;
00126     self->sample_count_per_file = self->multifile ?
00127         self->channel_count * ceilf(duration_per_file * sampling_frequency) :
00128         self->sample_count;
00129
00130     fprintf(stderr, "LokiRecorder_new (%s): will record %zu samples (%zu samples per file)\n",
      self->filename, self->sample_count, self->sample_count_per_file);
00131
00132     LokiRecorder_set_formatter(self, config);
00133
00134     pthread_mutex_init(&self->mutex, NULL);
00135     pthread_cond_init(&self->recording_done, NULL);
00136
00137     return self;
00138 }
00139
00140 char *LokiRecorder_get_filename(const LokiRecorder *self) {
00141     return self->filename;
00142 }
00143
00144 size_t LokiRecorder_get_channel_count(const LokiRecorder *self) {
00145     return self->channel_count;
00146 }
00147
00161 static void *LokiRecorder_work(void *ptr) {
00162     LokiRecorder *self = ptr;
00163
00164     pthread_setname_np(self->thread, "LokiRecorder");
00165
00166     const size_t in_length  = LokiPacket_get_length(self->in,  self->sample_size);
00167
00168     uint64_t sample_index = 0;
00169     size_t sample_index_per_file = 0;
00170
00171     // Obtenir un pointeur vers la date dans le nom de fichier.
00172     char *filename_suffix_date = self->filename + strlen(self->filename);
00173     char *filename_suffix_msec = filename_suffix_date + FILENAME_SUFFIX_DATE_LEN;
00174
00175     // Ce booléen évite de terminer sans avoir écrit un seul paquet.
00176     bool data_written = false;
00177
00178     fprintf(stderr, "LokiRecorder_work (%s): starting.\n", self->filename);
00179
00180     while (!self->stop_request) {
00181         LokiPacket *in_packet = LokiFIFO_get_consumer_item(self->in, self->cid);
00182         if (!in_packet) {
00183             break;
00184         }
00185         void *in_ptr = in_packet->buffer;
00186
00187         bool file_change = sample_index_per_file == 0;
00188
00189         // Si l'échantillon courant est le premier d'un fichier,
00190         // construire un nouveau nom de fichier.
00191         if (file_change) {
00192             time_t time_sec  = LokiTime_seconds(in_packet->timestamp);
00193             time_t time_msec = LokiTime_milliseconds(in_packet->timestamp);
00194             struct tm file_time;
00195             gmtime_r(&time_sec, &file_time);
00196             strftime(filename_suffix_date, FILENAME_SUFFIX_DATE_LEN + 1, "_%Y%m%dT%H%M%S",
      &file_time);
00197             sprintf(filename_suffix_msec, ".%03luZ.%s", time_msec, self->filetype);
00198         }
00199
00200         // Écrire le paquet courant dans le fichier de destination.
00201         LokiFormatter_work(self->formatter, self->filename, in_packet->timestamp, file_change, in_ptr,
      in_length, self->max_channels, self->channels, self->sample_size, self->sampling_frequency);
00202         data_written = true;
00203
00204         // Mettre à jour le compteur d'échantillons pour le fichier courant.
00205         // Si le fichier est plein, remettre le compteur à zéro et fermer le fichier.
00206         sample_index_per_file += in_length;
00207         if (self->multifile && sample_index_per_file > self->sample_count_per_file) {
00208             sample_index_per_file = 0;
00209         }
00210
00211         LokiFIFO_pull(self->in, self->cid);
00212
00213         // Mettre à jour le compteur d'échantillons global.
```

```
00214         // Si la durée d'acquisition est atteinte, terminer.
00215         sample_index += in_length;
00216         if (data_written && self->sample_count > 0 && sample_index >= self->sample_count) {
00217             break;
00218         }
00219     }
00220
00221     fprintf(stderr, "LokiRecorder_work (%s): done.\n", self->filename);
00222
00223     LokiFormatter_close(self->formatter);
00224
00225     pthread_mutex_lock(&self->mutex);
00226     self->is_recording = false;
00227     pthread_cond_signal(&self->recording_done);
00228     pthread_mutex_unlock(&self->mutex);
00229 }
00230
00231 int LokiRecorder_start(LokiRecorder *self, LokiFIFO *in, size_t cid) {
00232     self->is_recording = true;
00233     self->in = in;
00234     self->cid = cid;
00235     self->stop_request = false;
00236     return pthread_create(&self->thread, NULL, LokiRecorder_work, self);
00237 }
00238
00239 bool LokiRecorder_is_recording(LokiRecorder *self) {
00240     return self->is_recording;
00241 }
00242
00244 static LokiRecorder *LokiRecorder_waiting_instance;
00245
00255 static void LokiRecorder_signal_handler(int signum) {
00256     signal(SIGKILL, SIG_DFL);
00257     signal(SIGTERM, SIG_DFL);
00258     signal(SIGINT,  SIG_DFL);
00259     LokiRecorder_stop(LokiRecorder_waiting_instance);
00260 }
00261
00262 void LokiRecorder_wait(LokiRecorder *self) {
00263     LokiRecorder_waiting_instance = self;
00264     signal(SIGKILL, LokiRecorder_signal_handler);
00265     signal(SIGTERM, LokiRecorder_signal_handler);
00266     signal(SIGINT,  LokiRecorder_signal_handler);
00267
00268     pthread_mutex_lock(&self->mutex);
00269     if (self->is_recording) {
00270         pthread_cond_wait(&self->recording_done, &self->mutex);
00271     }
00272     pthread_mutex_unlock(&self->mutex);
00273 }
00274
00275 void LokiRecorder_stop(LokiRecorder *self) {
00276     if (self->stop_request) {
00277         return;
00278     }
00279     self->stop_request = true;
00280     LokiFIFO_abort(self->in);
00281     pthread_join(self->thread, NULL);
00282 }
00283
00284 void LokiRecorder_del(LokiRecorder *self) {
00285     if (!self) {
00286         return;
00287     }
00288
00289     if (self->filename) {
00290         free(self->filename);
00291     }
00292
00293     if (self->filetype) {
00294         free(self->filetype);
00295     }
00296
00297     if (self->formatter) {
00298         LokiFormatter_del(self->formatter);
00299     }
00300
00301     free(self);
00302 }
00303
```

6.67 Référence du fichier libloki-recorder.h

```
#include "libloki-fifo.h"
```

Graphe des dépendances par inclusion de libloki-recorder.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiRecorder LokiRecorder

Représentation d'un enregistreur de séries temporelles.

Fonctions

— LokiRecorder * LokiRecorder_new (const char *filename, const char *filetype, size_t max_channels, int32_t channels, size_t sample_size, float sampling_frequency, float duration, float duration_per_file, void *config)

Construire un enregistreur de séries temporelles.

— char * LokiRecorder_get_filename (const LokiRecorder *self)

Lire le nom des fichiers gérés par un enregistreur.

— size_t LokiRecorder_get_channel_count (const LokiRecorder *self)

Lire le nombre de voies gérées par un enregistreur.

— int LokiRecorder_start (LokiRecorder *self, LokiFIFO *in, size_t cid)

Démarrer l'enregistrement.

— bool LokiRecorder_is_recording (LokiRecorder *self)

Lire l'état d'un enregistreur.

— void LokiRecorder_wait (LokiRecorder *self)

Attendre la fin de l'enregistrement.

— void LokiRecorder_stop (LokiRecorder *self)

Arrêter l'enregistrement.

— void LokiRecorder_del (LokiRecorder *self)

Détruire un enregistreur.

6.67.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-recorder.h.

6.68 libloki-recorder.h

Aller à la documentation de ce fichier.

```
00001
00003
00010
00012
00013 #ifndef LIBLOKI_RECORDER_H_
00014 #define LIBLOKI_RECORDER_H_
00015
00016 #include "libloki-fifo.h"
00017
00019 typedef struct LokiRecorder LokiRecorder;
00020
00034 LokiRecorder *LokiRecorder_new(const char *filename, const char *filetype, size_t max_channels,
      int32_t channels, size_t sample_size, float sampling_frequency, float duration, float
      duration_per_file, void *config);
00035
00041 char *LokiRecorder_get_filename(const LokiRecorder *self);
00042
00048 size_t LokiRecorder_get_channel_count(const LokiRecorder *self);
00049
00060 int LokiRecorder_start(LokiRecorder *self, LokiFIFO *in, size_t cid);
00061
00067 bool LokiRecorder_is_recording(LokiRecorder *self);
00068
00076 void LokiRecorder_wait(LokiRecorder *self);
00077
00084 void LokiRecorder_stop(LokiRecorder *self);
00085
00090 void LokiRecorder_del(LokiRecorder *self);
00091
00092 #endif
00093
00096
```

6.69 Référence du fichier libloki-rls.c

```
#include "libloki-rls.h"
#include "libloki-ellipse.h"
#include <stdlib.h>
#include <string.h>
```

Graphe des dépendances par inclusion de libloki-rls.c:

Structures de données

— struct LokiRLS

Représentation d'un filtre RLS.

Fonctions

— LokiRLS * LokiRLS_new (float initial_radius, const float params[static 2])

Construire un nouveau filtre RLS.

— LokiRLS * LokiRLS_clone (const LokiRLS *other)

Construire un clone d'un filtre RLS.

— float * LokiRLS_get_state (LokiRLS *self)

Retourner l'état courant du filtre.

— void LokiRLS_work (LokiRLS *self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— void LokiRLS_del (LokiRLS *self)

Détruire un filtre RLS.

— **LokiClass_implementation** (LokiEllipseEstimator, LokiRLS, LokiClass_set_attribute(params_length, 2) LokiClass_define_method(LokiRLS, new) LokiClass_define_method(LokiRLS, clone) LokiClass_define_method(LokiRLS, work) LokiClass_define_method(LokiRLS, get_state) LokiClass_define_method(LokiRLS, del))

6.69.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-rls.c.

6.70 libloki-rls.c

Aller à la documentation de ce fichier.

```
00001
00002 #include "libloki-rls.h"
00003 #include "libloki-ellipse.h"
00004
00005 #include <stdlib.h>
00006 #include <string.h>
00007
00010
00012 struct LokiRLS {
00014     float lambda;
00016     float lambda_inv;
00018     float P[5][5];
00020     LokiEllipseEquation state;
00021 };
00022
00023 LokiRLS *LokiRLS_new(float initial_radius, const float params[static 2]) {
00024     LokiRLS * self = calloc(1, sizeof(LokiRLS));
00025     if (!self) {
00026         return NULL;
00027     }
00028
00029     self->lambda     = params[0];
00030     self->lambda_inv = 1.0f / self->lambda;
00031
00032     self->state[0] = 0.5f;
00033     self->state[4] = -initial_radius * initial_radius / 2;
00034
00035     for (size_t i = 0; i < 5; i ++) {
00036         self->P[i][i] = params[1];
00037     }
00038
00039     return self;
00040 }
00041
00042 LokiRLS *LokiRLS_clone(const LokiRLS *other) {
00043     LokiRLS * self = calloc(1, sizeof(LokiRLS));
00044     if (!self) {
00045         return NULL;
```

```
00046     }
00047
00048     memcpy(self, other, sizeof(LokiRLS));
00049
00050     return self;
00051 }
00052
00053 float *LokiRLS_get_state(LokiRLS *self) {
00054     return self->state;
00055 }
00056
00057 void LokiRLS_work(LokiRLS *self, float x, float y) {
00058     const float out  = -y * y;
00059     const float in[] = {x * x + out, x * y, x, y, 1.0f};
00060
00061     // Copies locales de l'état du filtre.
00062     float (* const P)[5]   = self->P;
00063     float *const state     = self->state;
00064     const float lambda_inv = self->lambda_inv;
00065
00066     // Dans les calculs suivants, on tient compte du fait que in[4] = 1
00067
00068     // Calcul de l'erreur courante :
00069     // epsilon = out - in * state
00070     float epsilon = out - state[4];
00071     for (size_t i = 0; i < 4; i ++) {
00072         epsilon -= in[i] * state[i];
00073     }
00074
00075     // Calcul du produits G = P * in'
00076     float G[5];
00077     for (size_t i = 0; i < 5; i ++) {
00078         G[i] = P[i][4];
00079         for (size_t j = 0; j < 4; j ++) {
00080             G[i] += P[i][j] * in[j];
00081         }
00082     }
00083
00084     // Calcul du facteur de normalisation K = lambda + in * P * in'
00085     float K = self->lambda + G[4];
00086     for (size_t i = 0; i < 4; i ++) {
00087         K += in[i] * G[i];
00088     }
00089
00090     // Normalisation du vecteur G et mise à jour de l'état.
00091     float K_inv = 1.0f / K;
00092     for (size_t i = 0; i < 5; i ++) {
00093         G[i] *= K_inv;
00094         state[i] += epsilon * G[i];
00095     }
00096
00097     // Calcul de J = I - G * in
00098     float J[5][5];
00099     for (size_t i = 0; i < 5; i ++) {
00100         J[i][i] = 1.0f - G[i] * in[i];
00101         for (size_t j = i + 1; j < 5; j ++) {
00102             J[i][j] = -G[i] * in[j];
00103             J[j][i] = -G[j] * in[i];
00104         }
00105     }
00106
00107     // Mise à jour de la matrice P_next = J * P / lambda
00108     float P_next[5][5];
00109     for (size_t i = 0; i < 5; i ++) {
00110         for (size_t j = 0; j < 5; j ++) {
00111             P_next[i][j] = J[i][0] * P[0][j];
00112             for (size_t k = 1; k < 5; k ++) {
00113                 P_next[i][j] += J[i][k] * P[k][j];
00114             }
00115             P_next[i][j] *= lambda_inv;
00116         }
00117     }
00118
00119     // Cette boucle est plus rapide que memcpy(P, P_next, sizeof(P_next)).
00120     // Il est possible que le compilateur optimise le graphe flot de données
00121     // sans passer par l'allocation du tableau P_next.
00122     for (size_t i = 0; i < 5; i ++) {
00123         for (size_t j = 0; j < 5; j ++) {
00124             P[i][j] = P_next[i][j];
00125         }
00126     }
00127 }
00128
00129 void LokiRLS_del(LokiRLS *self) {
00130     if (!self) {
00131         return;
00132     }
```

```
00133     free(self);
00134 }
00135
00136 LokiClass_implementation(LokiEllipseEstimator, LokiRLS,
00137     LokiClass_set_attribute(params_length, 2)
00138     LokiClass_define_method(LokiRLS, new)
00139     LokiClass_define_method(LokiRLS, clone)
00140     LokiClass_define_method(LokiRLS, work)
00141     LokiClass_define_method(LokiRLS, get_state)
00142     LokiClass_define_method(LokiRLS, del)
00143 );
00144
```

6.71 Référence du fichier libloki-rls.h

`#include "libloki-ellipse-estimator.h"`
Graphe des dépendances par inclusion de libloki-rls.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Définitions de type

— typedef struct LokiRLS LokiRLS

Représentation d'un filtre RLS.

Fonctions

— LokiRLS ∗ LokiRLS_new (float initial_radius, const float params[static 2])

Construire un nouveau filtre RLS.

— LokiRLS ∗ LokiRLS_clone (const LokiRLS ∗other)

Construire un clone d'un filtre RLS.

— void LokiRLS_work (LokiRLS ∗self, float x, float y)

Mettre à jour les paramètres de l'ellipse pour une nouvelle paire de coordonnées.

— float ∗ LokiRLS_get_state (LokiRLS ∗self)

Retourner l'état courant du filtre.

— void LokiRLS_del (LokiRLS ∗self)

Détruire un filtre RLS.

— **LokiClass_declare_implementation** (LokiEllipseEstimator, LokiRLS)

6.71.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-rls.h.

6.72 libloki-rls.h

Aller à la documentation de ce fichier.

```
00001
00008
00010
00011 #ifndef LIBLOKI_RLS_H_
00012 #define LIBLOKI_RLS_H_
00013
00014 #include "libloki-ellipse-estimator.h"
00015
00017 typedef struct LokiRLS LokiRLS;
00018
00025 LokiRLS *LokiRLS_new(float initial_radius, const float params[static 2]);
00026
00032 LokiRLS *LokiRLS_clone(const LokiRLS *other);
00033
00043 void LokiRLS_work(LokiRLS *self, float x, float y);
00044
00050 float *LokiRLS_get_state(LokiRLS *self);
00051
00056 void LokiRLS_del(LokiRLS *self);
00057
00058 LokiClass_declare_implementation(LokiEllipseEstimator, LokiRLS);
00059
00060 #endif
00061
```

6.73 Référence du fichier libloki-socket-server.c

```
#include "libloki.h"
#include <math.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <stdio.h>
#include <pthread.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
```

Graphe des dépendances par inclusion de libloki-socket-server.c:

Structures de données

— struct LokiSocketServer

Représentation d'un serveur de données.

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Fonctions

- LokiSocketServer * LokiSocketServer_new (const char *filename)

  Construire un serveur de données.

- static void * LokiSocketServer_receive (void *ptr)

  Recevoir les données d'entrée.

- static void * LokiSocketServer_serve (void *ptr)

  Gérer les connexions et transmettre les données aux clients.

- int LokiSocketServer_start (LokiSocketServer *self, LokiFIFO *in, size_t cid)

  Démarrer un serveur.

- bool LokiSocketServer_is_streaming (LokiSocketServer *self)

  Lire l'état du serveur.

- void LokiSocketServer_stop (LokiSocketServer *self)

  Arrêter un serveur.

- void LokiSocketServer_del (LokiSocketServer *self)

  Détruire un serveur.

6.73.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-socket-server.c.

6.73.2 Documentation des macros

6.73.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Définition à la ligne 3 du fichier libloki-socket-server.c.

6.74 libloki-socket-server.c

Aller à la documentation de ce fichier.

```
00001
00003 #define _GNU_SOURCE
00004
00005 #include "libloki.h"
00006
00007 #include <math.h>
00008 #include <stdlib.h>
00009 #include <string.h>
00010 #include <stdbool.h>
00011 #include <stdio.h>
00012 #include <pthread.h>
00013 #include <errno.h>
00014 #include <sys/types.h>
00015 #include <sys/socket.h>
00016 #include <sys/un.h>
00017 #include <unistd.h>
00018
00021
00023 struct LokiSocketServer {
00025     char *filename;
00027     LokiFIFO *in;
00029     size_t cid;
00031     volatile bool stop_request;
00033     bool is_streaming;
00035     pthread_t receiver_thread;
00037     pthread_t server_thread;
00039     pthread_mutex_t mutex;
00041     void *data;
00043     size_t data_size;
00045     bool has_data;
00047     pthread_cond_t has_data_cond;
00048 };
00049
00050 LokiSocketServer *LokiSocketServer_new(const char *filename) {
00051     LokiSocketServer *self = calloc(1, sizeof(LokiSocketServer));
00052     if (!self) {
00053         return NULL;
00054     }
00055
00056     self->filename = strdup(filename);
00057     if (!self->filename) {
00058         LokiSocketServer_del(self);
00059         return NULL;
00060     }
00061
00062     pthread_mutex_init(&self->mutex, NULL);
00063     pthread_cond_init(&self->has_data_cond, NULL);
00064
00065     return self;
00066 }
00067
00077 static void *LokiSocketServer_receive(void *ptr) {
00078     LokiSocketServer *self = ptr;
00079
00080     pthread_setname_np(self->receiver_thread, "LokiSocketRecv");
00081
00082     self->data_size = LokiPacket_get_length(self->in,  1);
00083
00084     while (!self->stop_request) {
00085         // Lire un paquet de données.
00086         LokiPacket *in_packet = LokiFIFO_get_consumer_item(self->in, self->cid);
00087         if (!in_packet) {
00088             break;
00089         }
00090
00091         // Si une connexion est ouverte, indiquer la disponibilité
00092         // des données au thread serveur.
00093         pthread_mutex_lock(&self->mutex);
00094         self->data = in_packet->buffer;
00095         self->has_data = true;
00096         pthread_cond_signal(&self->has_data_cond);
00097         pthread_mutex_unlock(&self->mutex);
00098
00099         LokiFIFO_pull(self->in, self->cid);
00100     }
00101
00102     self->is_streaming = false;
00103 }
00104
00114 static void *LokiSocketServer_serve(void *ptr) {
00115     LokiSocketServer *self = ptr;
00116
```

```
00117     pthread_setname_np(self->server_thread, "LokiSocketServe");
00118
00119     // Créer une socket.
00120     int sock = socket(AF_UNIX, SOCK_STREAM | SOCK_NONBLOCK, 0);
00121     bool has_file  = false;
00122     bool listening = false;
00123     int conn[LIBLOKI_SOCKET_SERVER_CONNECTIONS];
00124     bool status[LIBLOKI_SOCKET_SERVER_CONNECTIONS] = {false};
00125     size_t accept_index = 0;
00126
00127     if (sock < 0) {
00128         fprintf(stderr, "LokiSocketServer_serve (%s): socket creation failed (%s).\n", self->filename,
      strerror(errno));
00129     }
00130     else {
00131         // Attacher cette socket à un nom de fichier.
00132         struct sockaddr_un addr;
00133         memset(&addr, 0, sizeof(addr));
00134         addr.sun_family = AF_UNIX;
00135         strncpy(addr.sun_path, self->filename, sizeof(addr.sun_path) - 1);
00136         has_file = !bind(sock, (struct sockaddr*)&addr, sizeof(addr));
00137         if (!has_file) {
00138             fprintf(stderr, "LokiSocketServer_serve (%s): failed to bind socket (%s).\n",
      self->filename, strerror(errno));
00139         }
00140         else {
00141             // En cas de succès, écouter les demandes de connexion.
00142             listening = !listen(sock, 1);
00143             if (!listening) {
00144                 fprintf(stderr, "LokiSocketServer_serve (%s): failed to start listening on socket
      (%s).\n", self->filename, strerror(errno));
00145             }
00146         }
00147     }
00148
00149     while (!self->stop_request) {
00150         pthread_mutex_lock(&self->mutex);
00151         if (!self->has_data) {
00152             pthread_cond_wait(&self->has_data_cond, &self->mutex);
00153         }
00154         if (self->stop_request) {
00155             break;
00156         }
00157         const char *data = self->data;
00158         self->has_data = false;
00159         pthread_mutex_unlock(&self->mutex);
00160
00161         if (!listening) {
00162             continue;
00163         }
00164
00165         // Vérifier si une demande de connexion est présente en file d'attente.
00166         // La socket est non bloquante.
00167         if (accept_index < LIBLOKI_SOCKET_SERVER_CONNECTIONS) {
00168             conn[accept_index] = accept(sock, NULL, NULL);
00169             if (conn[accept_index] >= 0) {
00170                 fprintf(stderr, "LokiSocketServer_serve (%s): accepting connection %zu/%zu.\n",
      self->filename, accept_index, LIBLOKI_SOCKET_SERVER_CONNECTIONS);
00171                 // Marquer la connexion comme établie et chercher un slot libre
00172                 // pour la prochaine connexion.
00173                 status[accept_index] = true;
00174                 for (accept_index = 0; accept_index < LIBLOKI_SOCKET_SERVER_CONNECTIONS &&
      status[accept_index]; accept_index ++);
00175             }
00176         }
00177
00178         // Pour chaque connexion ouverte, transmettre le paquet au client.
00179         for (size_t i = 0; i < LIBLOKI_SOCKET_SERVER_CONNECTIONS; i ++) {
00180             if (!status[i]) {
00181                 continue;
00182             }
00183
00184             const char* remaining_data = data;
00185             size_t remaining_size      = self->data_size;
00186             while (remaining_size > 0) {
00187                 ssize_t sz = write(conn[i], remaining_data, remaining_size);
00188                 if (sz >= 0) {
00189                     remaining_size -= sz;
00190                     remaining_data += sz;
00191                 }
00192                 else if (errno != EINTR) {
00193                     fprintf(stderr, "LokiSocketServer_serve (%s): write error on connection %zu
      (%s).\n", self->filename, i, strerror(errno));
00194                     status[i] = false;
00195                     accept_index = i;
00196                     break;
00197                 }
```

```
00198             }
00199         }
00200     }
00201
00202     if (sock >= 0) {
00203         close(sock);
00204     }
00205
00206     if (has_file) {
00207         unlink(self->filename);
00208     }
00209 }
00210
00211 int LokiSocketServer_start(LokiSocketServer *self, LokiFIFO *in, size_t cid) {
00212     self->is_streaming = true;
00213     self->in           = in;
00214     self->cid          = cid;
00215     self->stop_request = false;
00216     int err = pthread_create(&self->receiver_thread, NULL, LokiSocketServer_receive, self);
00217     if (err) {
00218         return err;
00219     }
00220     return pthread_create(&self->server_thread, NULL, LokiSocketServer_serve, self);
00221 }
00222
00223 bool LokiSocketServer_is_streaming(LokiSocketServer *self) {
00224     return self->is_streaming;
00225 }
00226
00227 void LokiSocketServer_stop(LokiSocketServer *self) {
00228     if (self->stop_request) {
00229         return;
00230     }
00231     self->stop_request = true;
00232
00233     // Débloquer le thread de réception.
00234     LokiFIFO_abort(self->in);
00235
00236     // Débloquer le thread serveur.
00237     pthread_mutex_lock(&self->mutex);
00238     pthread_cond_signal(&self->has_data_cond);
00239     pthread_mutex_unlock(&self->mutex);
00240
00241     // Attendre la terminaison normale des deux threads.
00242     pthread_join(self->receiver_thread, NULL);
00243     pthread_join(self->server_thread, NULL);
00244 }
00245
00246 void LokiSocketServer_del(LokiSocketServer *self) {
00247     if (!self) {
00248         return;
00249     }
00250
00251     if (self->filename) {
00252         free(self->filename);
00253     }
00254
00255     free(self);
00256 }
00257
```

6.75 Référence du fichier libloki-socket-server.h

`#include "libloki-fifo.h"`

Graphe des dépendances par inclusion de libloki-socket-server.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Macros

— #define LIBLOKI_SOCKET_SERVER_CONNECTIONS 3

Le nombre maximum de connexions simultanées acceptées par un serveur.

Définitions de type

— typedef struct LokiSocketServer LokiSocketServer

Représentation d'un serveur de données.

Fonctions

— LokiSocketServer * LokiSocketServer_new (const char *filename)

Construire un serveur de données.

— int LokiSocketServer_start (LokiSocketServer *self, LokiFIFO *in, size_t cid)

Démarrer un serveur.

— bool LokiSocketServer_is_streaming (LokiSocketServer *self)

Lire l'état du serveur.

— void LokiSocketServer_stop (LokiSocketServer *self)

Arrêter un serveur.

— void LokiSocketServer_del (LokiSocketServer *self)

Détruire un serveur.

6.75.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-socket-server.h.

6.76 libloki-socket-server.h

Aller à la documentation de ce fichier.

```
00001
00009
00010 #ifndef LIBLOKI_SOCKET_SERVER_H_
00011 #define LIBLOKI_SOCKET_SERVER_H_
00012
00013 #include "libloki-fifo.h"
00014
00016 #define LIBLOKI_SOCKET_SERVER_CONNECTIONS 3
00017
00019 typedef struct LokiSocketServer LokiSocketServer;
00020
00026 LokiSocketServer *LokiSocketServer_new(const char *filename);
00027
```

```
00040 int LokiSocketServer_start(LokiSocketServer *self, LokiFIFO *in, size_t cid);
00041
00047 bool LokiSocketServer_is_streaming(LokiSocketServer *self);
00048
00055 void LokiSocketServer_stop(LokiSocketServer *self);
00056
00061 void LokiSocketServer_del(LokiSocketServer *self);
00062
00063 #endif
00064
```

6.77 Référence du fichier libloki-system-defaults.h

```
#include "libloki.h"
#include <math.h>
```

Graphe des dépendances par inclusion de libloki-system-defaults.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Macros

— #define LOKI_SYSTEM_DEFAULT_SIMULATE false

Ne pas activer le mode de simulation logicielle des capteurs.

— #define LOKI_SYSTEM_DEFAULT_SKIP 0

Remplir la FIFO des déplacements immédiatement après la réception des premiers échantillons démodulés.

— #define LOKI_SYSTEM_DEFAULT_WAVELENGTH 1310

La longueur d'onde par défaut, 1310 nm.

— #define LOKI_SYSTEM_DEFAULT_REFLECTIONS 1

Supposer qu'il n'y a qu'une seule réflexion.

— #define LOKI_SYSTEM_DEFAULT_CAVITY_INDEX 1.0f

Supposer un indice de cavité de Fabry-Pérot égal à 1.

— #define LOKI_SYSTEM_DEFAULT_DISTANCE_UNIT 'p';

Calculer les déplacements en picomètres.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_RLS LokiRLSClass

L'implémentation du filtre RLS.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_RLS {0.999f, 0.1f}

Les paramètres par défaut du filtre RLS.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN LokiKalmanClass

L'implémentation du filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN {0.999f, 0.0f, 0.1f}

Les paramètres par défaut du filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN

Utiliser un filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN

Utiliser les paramètres par défaut du filtre de Kalman.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_THROTTLE 1

La périodicité par défaut du calcul des paramètres de l'ellipse.

— #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_INITIAL_RADIUS 1e-3

Le rayon initial par défaut des ellipses estimées.

— #define LOKI_SYSTEM_DEFAULT_FILTER_BANDWIDTH 250.0f

La bande passante par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_FILTER_ATTENUATION 60.0f

L'atténuation par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_FILTER_LENGTH 1000

Le nombre de coefficients par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_FILTER_DECIMATION 50

La facteur de décimation par défaut du filtre de suppression du déplacement fictif.

— #define LOKI_SYSTEM_DEFAULT_IIR_TAU 1.0f

La constante de temps par défaut des filtres de lissage des paramètres de l'ellipse.

6.77.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-system-defaults.h.

6.78 libloki-system-defaults.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_SYSTEM_DEFAULTS_H_
00011 #define LIBLOKI_SYSTEM_DEFAULTS_H_
00012
```

```
00013 #include "libloki.h"
00014 #include <math.h>
00015
00017 #define LOKI_SYSTEM_DEFAULT_SIMULATE                false
00018
00020 #define LOKI_SYSTEM_DEFAULT_SKIP                    0
00021
00023 #define LOKI_SYSTEM_DEFAULT_WAVELENGTH              1310
00024
00026 #define LOKI_SYSTEM_DEFAULT_REFLECTIONS             1
00027
00029 #define LOKI_SYSTEM_DEFAULT_CAVITY_INDEX            1.0f
00030
00032 #define LOKI_SYSTEM_DEFAULT_DISTANCE_UNIT           'p';
00033
00035 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_RLS      LokiRLSClass
00036
00038 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_RLS    {0.999f, 0.1f}
00039
00041 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN   LokiKalmanClass
00042
00044 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN {0.999f, 0.0f, 0.1f}
00045
00047 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL          LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL_KALMAN
00048
00050 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS        LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS_KALMAN
00051
00053 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_THROTTLE      1
00054
00056 #define LOKI_SYSTEM_DEFAULT_ESTIMATOR_INITIAL_RADIUS 1e-3
00057
00059 #define LOKI_SYSTEM_DEFAULT_FILTER_BANDWIDTH        250.0f
00060
00062 #define LOKI_SYSTEM_DEFAULT_FILTER_ATTENUATION      60.0f
00063
00065 #define LOKI_SYSTEM_DEFAULT_FILTER_LENGTH           1000
00066
00068 #define LOKI_SYSTEM_DEFAULT_FILTER_DECIMATION       50
00069
00071 #define LOKI_SYSTEM_DEFAULT_IIR_TAU                 1.0f
00072
00073 #endif
00074
```

6.79 Référence du fichier libloki-system.c

```
#include "libloki.h"
#include "libloki-system-defaults.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <pthread.h>
#include <errno.h>
```

Graphe des dépendances par inclusion de libloki-system.c:

Structures de données

— struct LokiSystemConfig

Les paramètres de configuration d'un système d'acquisition et de traitement.

— struct LokiSystem

Représentation d'un système d'acquisition et de traitement.

Macros

— #define _GNU_SOURCE

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Fonctions

— LokiSystemConfig * LokiSystemConfig_new (void)

Construire une configuration pour une instance de LokiSystem.

— void LokiSystemConfig_del (LokiSystemConfig *self)

Détruire une configuration pour une instance de LokiSystem.

— float LokiSystemConfig_get_skip (LokiSystemConfig *self)

Lire le temps d'attente au démarrage.

— void LokiSystemConfig_set_skip (LokiSystemConfig *self, float value)

Affecter un temps d'attente au démarrage.

— bool LokiSystemConfig_get_simulate (LokiSystemConfig *self)

Lire l'état d'activation de la simulation des capteurs de déplacement.

— void LokiSystemConfig_set_simulate (LokiSystemConfig *self, bool value)

Activer ou désactiver la simulation des capteurs de déplacement.

— float LokiSystemConfig_get_scale (LokiSystemConfig *self)

Lire le facteur d'échelle de déplacement.

— float LokiSystemConfig_get_wavelength (LokiSystemConfig *self)

Lire la longueur d'onde de la diode laser.

— void LokiSystemConfig_set_wavelength (LokiSystemConfig *self, float value)

Écrire la longueur d'onde de la diode laser.

— unsigned LokiSystemConfig_get_reflections (LokiSystemConfig *self)

Lire le nombre de réflexions supposé.

— void LokiSystemConfig_set_reflections (LokiSystemConfig *self, unsigned value)

Écrire le nombre de réflexions supposé.

— float LokiSystemConfig_get_cavity_index (LokiSystemConfig *self)

Lire l'indice de la cavité de Fabry-Pérot.

— void LokiSystemConfig_set_cavity_index (LokiSystemConfig *self, float value)

Écrire l'indice de la cavité de Fabry-Pérot.

— char LokiSystemConfig_get_distance_unit (LokiSystemConfig ∗self)

Lire l'unité du déplacement calculé.

— void LokiSystemConfig_set_distance_unit (LokiSystemConfig ∗self, char value)

Écrire l'unité du déplacement calculé.

— float ∗ LokiSystemConfig_get_time_constants (LokiSystemConfig ∗self)

Lire le tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— size_t LokiSystemConfig_get_time_constants_length (LokiSystemConfig ∗self)

Lire la longueur du tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— void LokiSystemConfig_set_time_constants (LokiSystemConfig ∗self, const float ∗value)

Affecter un tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— const char ∗ LokiSystemConfig_get_estimator_name (LokiSystemConfig ∗self)

Lire le nom de l'estimateur de paramètres d'ellipse.

— void LokiSystemConfig_set_estimator_name (LokiSystemConfig ∗self, const char ∗name)

Affecter un nom de l'estimateur de paramètres d'ellipse.

— unsigned LokiSystemConfig_get_estimator_throttle (LokiSystemConfig ∗self)

— void LokiSystemConfig_set_estimator_throttle (LokiSystemConfig ∗self, unsigned value)

— float LokiSystemConfig_get_estimator_initial_radius (LokiSystemConfig ∗self)

Lire le rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

— void LokiSystemConfig_set_estimator_initial_radius (LokiSystemConfig ∗self, float value)

Affecter un rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

— float ∗ LokiSystemConfig_get_estimator_params (LokiSystemConfig ∗self)

Lire les paramètres du filtre d'estimation des paramètres de l'ellipse.

— size_t LokiSystemConfig_get_estimator_params_length (LokiSystemConfig ∗self)

Lire le nombre de paramètres du filtre d'estimation des paramètres de l'ellipse.

— void LokiSystemConfig_set_estimator_params (LokiSystemConfig ∗self, const float ∗value)

Affecter des paramètres pour le filtre d'estimation des paramètres de l'ellipse.

— float LokiSystemConfig_get_filter_bandwidth (LokiSystemConfig ∗self)

Lire la bande passante du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_bandwidth (LokiSystemConfig ∗self, float value)

Affecter un bande passante pour le filtre d'élimination du déplacement fictif.

— float LokiSystemConfig_get_filter_attenuation (LokiSystemConfig ∗self)

Lire l'atténuation du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_attenuation (LokiSystemConfig ∗self, float value)

Affecter une atténuation pour le filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_filter_length (LokiSystemConfig ∗self)

Lire la longueur du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_length (LokiSystemConfig ∗self, size_t value)

Affecter une longueur au filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_filter_decimation (LokiSystemConfig ∗self)

Lire le facteur de décimation du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_decimation (LokiSystemConfig ∗self, size_t value)

Affecter un facteur de décimation au filtre d'élimination du déplacement fictif.

— LokiSystem ∗ LokiSystem_new (LokiSystemConfig ∗config, LokiModem ∗modem)

Construire un nouveau système.

— LokiSystemConfig ∗ LokiSystem_get_config (const LokiSystem ∗self)

Lire la configuration d'un système.

— LokiModem ∗ LokiSystem_get_modem (const LokiSystem ∗self)

Lire le modem utilisé par ce système.

— float LokiSystem_get_sampling_frequency (const LokiSystem ∗self)

Lire la fréquence d'échantillonnage en sortie de ce système.

— void LokiSystem_displacement_reset (LokiSystem ∗self)

Mettre à zéro le déplacement calculé.

— static void ∗ LokiSystem_work (void ∗ptr)

Calculer le déplacement à partir des données démodulées.

— int LokiSystem_start_streaming (LokiSystem ∗self, int32_t channels, LokiFIFO ∗modem_out, LokiFIFO ∗disp_out, size_t cid)

Démarrer l'acquisition et le traitement.

— void LokiSystem_stop_streaming (LokiSystem ∗self)

Arrêter l'acquisition et le traitement.

— bool LokiSystem_is_streaming (LokiSystem ∗self)

Lire l'état de l'acquisition.

— LokiEllipse ∗ LokiSystem_get_ellipse (LokiSystem ∗self, size_t channel)

Lire la dernière ellipse estimée.

— void LokiSystem_del (LokiSystem *self)

Détruire un système.

Variables

— static const char *const LokiSystemConfig_estimator_names [ ]

Les noms des filtres adaptatifs supportés pour l'estimation des paramètres de l'ellipse.

— static const LokiEllipseEstimatorInterface *const LokiSystemConfig_estimator_impls [ ]

Les implémentations des filtres adaptatifs supportés pour l'estimation des paramètres de l'ellipse.

6.79.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-system.c.

6.79.2 Documentation des macros

6.79.2.1 _GNU_SOURCE

```
#define _GNU_SOURCE
```

Permettre l'utilisation de la fonction `pthread_setname_np` pour modifier le nom d'un thread.

Définition à la ligne 3 du fichier libloki-system.c.

6.80 libloki-system.c

Aller à la documentation de ce fichier.

```
00001
00003 #define _GNU_SOURCE
00004
00005 #include "libloki.h"
00006 #include "libloki-system-defaults.h"
00007
00008 #include <stdio.h>
00009 #include <stdlib.h>
00010 #include <string.h>
00011 #include <math.h>
00012 #include <pthread.h>
00013 #include <errno.h>
00014
```

```
00017
00018 // Uncomment this line to enable malloc monitoring (see libloki-hooks)
00019 // #define DEBUG_MALLOC
00020
00022 struct LokiSystemConfig {
00024     bool simulate;
00026     float skip;
00028     float wavelength;
00030     unsigned reflections;
00032     float cavity_index;
00034     char distance_unit;
00036     LokiEllipseEstimatorInterface *estimator_impl;
00038     unsigned estimator_throttle;
00040     float estimator_initial_radius;
00042     float *estimator_params;
00044     float filter_bandwidth;
00046     float filter_attenuation;
00048     size_t filter_length;
00050     size_t filter_decimation;
00052     float time_constants[3];
00053 };
00054
00056 static const char *const LokiSystemConfig_estimator_names[] = {
00057     "rls", "nlms", "kalman"
00058 };
00059
00061 static const LokiEllipseEstimatorInterface *const LokiSystemConfig_estimator_impls[] = {
00062     &LokiRLSClass, &LokiNLMSClass, &LokiKalmanClass
00063 };
00064
00065 LokiSystemConfig * LokiSystemConfig_new(void) {
00066     LokiSystemConfig *self = calloc(1, sizeof(LokiSystemConfig));
00067     if (!self) {
00068         return NULL;
00069     }
00070
00071     self->simulate                = LOKI_SYSTEM_DEFAULT_SIMULATE;
00072     self->skip                    = LOKI_SYSTEM_DEFAULT_SKIP;
00073     self->wavelength              = LOKI_SYSTEM_DEFAULT_WAVELENGTH;
00074     self->reflections             = LOKI_SYSTEM_DEFAULT_REFLECTIONS;
00075     self->cavity_index            = LOKI_SYSTEM_DEFAULT_CAVITY_INDEX;
00076     self->distance_unit           = LOKI_SYSTEM_DEFAULT_DISTANCE_UNIT;
00077     self->estimator_impl          = &LOKI_SYSTEM_DEFAULT_ESTIMATOR_IMPL;
00078     self->estimator_throttle      = LOKI_SYSTEM_DEFAULT_ESTIMATOR_THROTTLE;
00079     self->estimator_initial_radius = LOKI_SYSTEM_DEFAULT_ESTIMATOR_INITIAL_RADIUS;
00080     self->filter_bandwidth        = LOKI_SYSTEM_DEFAULT_FILTER_BANDWIDTH;
00081     self->filter_attenuation      = LOKI_SYSTEM_DEFAULT_FILTER_ATTENUATION;
00082     self->filter_length           = LOKI_SYSTEM_DEFAULT_FILTER_LENGTH;
00083     self->filter_decimation       = LOKI_SYSTEM_DEFAULT_FILTER_DECIMATION;
00084
00085     for (size_t i = 0; i < LEN(self->time_constants); i ++) {
00086         self->time_constants[i] = LOKI_SYSTEM_DEFAULT_IIR_TAU;
00087     }
00088
00089     float estimator_params[] = LOKI_SYSTEM_DEFAULT_ESTIMATOR_PARAMS;
00090     self->estimator_params = calloc(self->estimator_impl->params_length, sizeof(float));
00091     if (!self->estimator_params) {
00092         LokiSystemConfig_del(self);
00093         return NULL;
00094     }
00095     for (size_t i = 0; i < self->estimator_impl->params_length && i < LEN(estimator_params); i ++) {
00096         self->estimator_params[i] = estimator_params[i];
00097     }
00098
00099     return self;
00100 }
00101
00102 void LokiSystemConfig_del(LokiSystemConfig *self) {
00103     if (!self) {
00104         return;
00105     }
00106     free(self);
00107 }
00108
00109 float LokiSystemConfig_get_skip(LokiSystemConfig *self) {
00110     return self->skip;
00111 }
00112
00113 void LokiSystemConfig_set_skip(LokiSystemConfig *self, float value) {
00114     self->skip = value;
00115 }
00116
00117 bool LokiSystemConfig_get_simulate(LokiSystemConfig *self) {
00118     return self->simulate;
00119 }
00120
00121 void LokiSystemConfig_set_simulate(LokiSystemConfig *self, bool value) {
```

```
00122     self->simulate = value;
00123 }
00124
00125 float LokiSystemConfig_get_scale(LokiSystemConfig *self) {
00126     float w = self->wavelength;
00127     float d = M_PI * 4 * self->reflections * self->cavity_index;
00128     switch (self->distance_unit) {
00129         case 'p': w *=    1000; break;
00130         case 'u': d *=    1000; break;
00131         case 'm': d *= 1000000; break;
00132     }
00133     return w / d;
00134 }
00135
00136 float LokiSystemConfig_get_wavelength(LokiSystemConfig *self) {
00137     return self->wavelength;
00138 }
00139
00140 void LokiSystemConfig_set_wavelength(LokiSystemConfig *self, float value) {
00141     self->wavelength = value;
00142 }
00143
00144 unsigned LokiSystemConfig_get_reflections(LokiSystemConfig *self) {
00145     return self->reflections;
00146 }
00147
00148 void LokiSystemConfig_set_reflections(LokiSystemConfig *self, unsigned value) {
00149     self->reflections = value;
00150 }
00151
00152 float LokiSystemConfig_get_cavity_index(LokiSystemConfig *self) {
00153     return self->cavity_index;
00154 }
00155
00156 void LokiSystemConfig_set_cavity_index(LokiSystemConfig *self, float value) {
00157     self->cavity_index = value;
00158 }
00159
00160 char LokiSystemConfig_get_distance_unit(LokiSystemConfig *self) {
00161     return self->distance_unit;
00162 }
00163
00164 void LokiSystemConfig_set_distance_unit(LokiSystemConfig *self, char value) {
00165     self->distance_unit = value;
00166 }
00167
00168 float *LokiSystemConfig_get_time_constants(LokiSystemConfig *self) {
00169     return self->time_constants;
00170 }
00171
00172 size_t LokiSystemConfig_get_time_constants_length(LokiSystemConfig *self) {
00173     return LEN(self->time_constants);
00174 }
00175
00176 void LokiSystemConfig_set_time_constants(LokiSystemConfig *self, const float *value) {
00177     for (size_t i = 0; i < LEN(self->time_constants); i ++) {
00178         self->time_constants[i] = value[i];
00179     }
00180 }
00181
00182 const char *LokiSystemConfig_get_estimator_name(LokiSystemConfig *self) {
00183     for (size_t i = 0; i < LEN(LokiSystemConfig_estimator_names); i ++) {
00184         if (LokiSystemConfig_estimator_impls[i] == self->estimator_impl) {
00185             return LokiSystemConfig_estimator_names[i];
00186         }
00187     }
00188     return "undefined";
00189 }
00190
00191 void LokiSystemConfig_set_estimator_name(LokiSystemConfig *self, const char *name) {
00192     for (size_t i = 0; i < LEN(LokiSystemConfig_estimator_names); i ++) {
00193         if (!strcmp(LokiSystemConfig_estimator_names[i], name)) {
00194             self->estimator_impl = LokiSystemConfig_estimator_impls[i];
00195             return;
00196         }
00197     }
00198     self->estimator_impl = NULL;
00199 }
00200
00201 unsigned LokiSystemConfig_get_estimator_throttle(LokiSystemConfig *self) {
00202     return self->estimator_throttle;
00203 }
00204
00205 void LokiSystemConfig_set_estimator_throttle(LokiSystemConfig *self, unsigned value) {
00206     self->estimator_throttle = value;
00207 }
00208
```

```
00209 float LokiSystemConfig_get_estimator_initial_radius(LokiSystemConfig *self) {
00210     return self->estimator_initial_radius;
00211 }
00212
00213 void LokiSystemConfig_set_estimator_initial_radius(LokiSystemConfig *self, float value) {
00214     self->estimator_initial_radius = value;
00215 }
00216
00217 float *LokiSystemConfig_get_estimator_params(LokiSystemConfig *self) {
00218     return self->estimator_params;
00219 }
00220
00221 size_t LokiSystemConfig_get_estimator_params_length(LokiSystemConfig *self) {
00222     return self->estimator_impl->params_length;
00223 }
00224
00225 void LokiSystemConfig_set_estimator_params(LokiSystemConfig *self, const float *value) {
00226     if (self->estimator_params) {
00227         free(self->estimator_params);
00228     }
00229     self->estimator_params = calloc(self->estimator_impl->params_length, sizeof(float));
00230     if (self->estimator_params) {
00231         memcpy(self->estimator_params, value, self->estimator_impl->params_length * sizeof(float));
00232     }
00233 }
00234
00235 float LokiSystemConfig_get_filter_bandwidth(LokiSystemConfig *self) {
00236     return self->filter_bandwidth;
00237 }
00238
00239 void LokiSystemConfig_set_filter_bandwidth(LokiSystemConfig *self, float value) {
00240     self->filter_bandwidth = value;
00241 }
00242
00243 float LokiSystemConfig_get_filter_attenuation(LokiSystemConfig *self) {
00244     return self->filter_attenuation;
00245 }
00246
00247 void LokiSystemConfig_set_filter_attenuation(LokiSystemConfig *self, float value) {
00248     self->filter_attenuation = value;
00249 }
00250
00251 size_t LokiSystemConfig_get_filter_length(LokiSystemConfig *self) {
00252     return self->filter_length;
00253 }
00254
00255 void LokiSystemConfig_set_filter_length(LokiSystemConfig *self, size_t value) {
00256     self->filter_length = value;
00257 }
00258
00259 size_t LokiSystemConfig_get_filter_decimation(LokiSystemConfig *self) {
00260     return self->filter_decimation;
00261 }
00262
00263 void LokiSystemConfig_set_filter_decimation(LokiSystemConfig *self, size_t value) {
00264     self->filter_decimation = value;
00265 }
00266
00268 struct LokiSystem {
00270     LokiSystemConfig *config;
00272     size_t channels;
00274     bool has_own_modem;
00276     LokiModem *modem;
00278     LokiDisplacement **disp;
00280     volatile bool stop_request;
00282     bool is_streaming;
00284     pthread_t thread;
00286     LokiFIFO *in;
00288     LokiFIFO *out;
00290     size_t cid;
00292     pthread_mutex_t mutex;
00293 };
00294
00295 LokiSystem *LokiSystem_new(LokiSystemConfig *config, LokiModem *modem) {
00296     #ifdef DEBUG_MALLOC
00297         fprintf(stderr, "LokiSystem_new: Starting memory allocation monitoring\n");
00298         LokiHooks_init();
00299     #endif
00300
00301     // Vérifier les paramètres de configuration.
00302     if (!config->estimator_impl) {
00303         return NULL;
00304     }
00305
00306     LokiSystem *self = calloc(1, sizeof(LokiSystem));
00307     if (!self) {
00308         return NULL;
```

```
00309     }
00310
00311     self->config = config;
00312
00313     // Initialiser et configurer le modem
00314     self->has_own_modem = modem == NULL;
00315     self->modem = self->has_own_modem ? LokiModem_new() : modem;
00316     if (!self->modem) {
00317         LokiSystem_del(self);
00318         return NULL;
00319     }
00320
00321     self->channels = LokiModem_get_demod_channels(self->modem);
00322
00323     // Initialiser la chaîne de traitement du signal logiciel.
00324     self->disp = calloc(self->channels, sizeof(LokiDisplacement*));
00325     if (!self->disp) {
00326         LokiSystem_del(self);
00327         return NULL;
00328     }
00329
00330     LokiEllipseEstimator *estimator =
      LokiEllipseEstimator_new_with_implementation(config->estimator_impl, config->estimator_initial_radius,
      config->estimator_params);
00331     LokiFIRFilter *filter =
      LokiFIRFilter_new_kaiser_bessel(LokiModem_get_demod_xy_sampling_frequency(self->modem), 0,
      config->filter_bandwidth, config->filter_attenuation, config->filter_length,
      config->filter_decimation);
00332
00333     float scale = LokiSystemConfig_get_scale(config);
00334     float sampling_frequency = LokiModem_get_demod_xy_sampling_frequency(self->modem);
00335
00336     for (size_t c = 0; c < self->channels; c ++) {
00337         self->disp[c] = c == 0 ?
00338             LokiDisplacement_new(estimator, filter, config->estimator_throttle, scale,
      config->time_constants, sampling_frequency) :
00339             LokiDisplacement_clone(self->disp[0]);
00340         if (!self->disp[c]) {
00341             LokiSystem_del(self);
00342             return NULL;
00343         }
00344     }
00345
00346     pthread_mutex_init(&self->mutex, NULL);
00347
00348     return self;
00349 }
00350
00351 LokiSystemConfig *LokiSystem_get_config(const LokiSystem *self) {
00352     return self->config;
00353 }
00354
00355 LokiModem *LokiSystem_get_modem(const LokiSystem *self) {
00356     return self->modem;
00357 }
00358
00359 float LokiSystem_get_sampling_frequency(const LokiSystem *self) {
00360     return LokiModem_get_demod_xy_sampling_frequency(self->modem) / self->config->filter_decimation;
00361 }
00362
00363 void LokiSystem_displacement_reset(LokiSystem *self) {
00364     pthread_mutex_lock(&self->mutex);
00365     for (size_t channel = 0; channel < self->channels; channel ++) {
00366         LokiDisplacement_reset(self->disp[channel]);
00367     }
00368     pthread_mutex_unlock(&self->mutex);
00369 }
00370
00380 static void *LokiSystem_work(void *ptr) {
00381     LokiSystem *self = ptr;
00382
00383     pthread_setname_np(self->thread, "LokiSystem");
00384
00385     // La taille des buffers de données dans les FIFOs d'entrée et de sortie, en échantillons.
00386     const size_t in_length  = LokiPacket_get_length(self->in,  sizeof(LokiModemSample));
00387     const size_t out_length = LokiPacket_get_length(self->out, sizeof(LokiSystemSample));
00388
00389     // Références vers le premier paquet et le premier échantillon de la FIFO des déplacements,
00390     // qui ne progresse pas à la même vitesse que la FIFO des échantillons du modem.
00391     LokiPacket *out_packet = LokiFIFO_get_producer_item(self->out);
00392     if (!out_packet) {
00393         goto LokiSystem_work_end;
00394     }
00395     LokiSystemSample *out_ptr = (LokiSystemSample*)out_packet->buffer;
00396     LokiSystemSample *out_end = out_ptr + out_length;
00397
00398     // Calculer la durée d'un paquet d'entrée, en nanosecondes.
```

```
00399     const size_t channels = self->channels;
00400     double in_period_ns = 1.0e9 * in_length / channels /
    LokiModem_get_demod_xy_sampling_frequency(self->modem);
00401
00402     // Préparer un tableau avec l'indice de l'échantillon courant pour chaque voie.
00403     uint64_t skip = self->config->skip * LokiSystem_get_sampling_frequency(self);
00404     uint64_t *sample_index = calloc(channels, sizeof(uint64_t));
00405     if (!sample_index) {
00406         goto LokiSystem_work_end;
00407     }
00408
00409     // Préparer des tableaux pour mémoriser les points de l'ellipse normalisée.
00410     const size_t decimation = self->config->filter_decimation;
00411     float *xy_norm = calloc(channels * decimation * 2, sizeof(float));
00412     if (!xy_norm) {
00413         goto LokiSystem_work_end;
00414     }
00415     float **xy_start = calloc(channels * 4, sizeof(float*));
00416     if (!xy_start) {
00417         goto LokiSystem_work_end;
00418     }
00419
00420     float **xy_end   = xy_start + channels;
00421     float **xy_read  = xy_start + channels * 2;
00422     float **xy_write = xy_start + channels * 3;
00423     for (size_t c = 0; c < channels; c ++) {
00424         xy_start[c] = xy_read[c] = xy_write[c] = xy_norm + (c * decimation * 2);
00425         xy_end[c]   = xy_start[c] + (decimation * 2);
00426     }
00427
00428     while (!self->stop_request) {
00429         LokiPacket *in_packet = LokiFIFO_get_consumer_item(self->in, self->cid);
00430         if (!in_packet) {
00431             break;
00432         }
00433         LokiModemSample *in_ptr = (LokiModemSample*)in_packet->buffer;
00434         const LokiModemSample *const in_end = in_ptr + in_length;
00435
00436         // En commençant un nouveau paquet de sortie, copier la date du paquet d'entrée.
00437         if (out_ptr == (LokiSystemSample*)out_packet->buffer) {
00438             out_packet->timestamp = in_packet->timestamp;
00439         }
00440
00441         // Calculer les déplacements, remplir les FIFO de sortie.
00442         while (in_ptr != in_end && !self->stop_request) {
00443             LokiModemSample in = *in_ptr ++;
00444
00445             if (in.channel >= channels) {
00446                 continue;
00447             }
00448
00449             pthread_mutex_lock(&self->mutex);
00450             float d;
00451             bool has_disp = LokiDisplacement_work(self->disp[in.channel], in.x, in.y,
    xy_write[in.channel], xy_write[in.channel] + 1, &d);
00452             pthread_mutex_unlock(&self->mutex);
00453
00454             xy_write[in.channel] += 2;
00455             if (xy_write[in.channel] == xy_end[in.channel]) {
00456                 xy_write[in.channel] = xy_start[in.channel];
00457             }
00458
00459             if (has_disp) {
00460                 sample_index[in.channel] ++;
00461
00462                 // Ne pas remplir le buffer de sortie tant que le nombre
00463                 // d'échantillons à omettre n'est pas atteint.
00464                 if (sample_index[in.channel] < skip) {
00465                     continue;
00466                 }
00467
00468                 // Remettre à zéro le déplacement courant avant d'enregistrer.
00469                 if (sample_index[in.channel] == skip) {
00470                     LokiDisplacement_reset(self->disp[in.channel]);
00471                     d = 0;
00472                 }
00473
00474                 *out_ptr ++ = (LokiSystemSample) {
00475                     .channel = in.channel,
00476                     .value   = d,
00477                     .x_norm  = xy_read[in.channel][0],
00478                     .y_norm  = xy_read[in.channel][1]
00479                 };
00480
00481                 xy_read[in.channel] += 2;
00482                 if (xy_read[in.channel] == xy_end[in.channel]) {
00483                     xy_read[in.channel] = xy_start[in.channel];
```

```
00484                 }
00485
00486                 if (out_ptr == out_end) {
00487                     LokiFIFO_push(self->out);
00488                     out_packet = LokiFIFO_get_producer_item(self->out);
00489                     if (!out_packet) {
00490                         goto LokiSystem_work_end;
00491                     }
00492                     if (in_ptr != in_end) {
00493                         out_packet->timestamp = in_packet->timestamp + round(in_period_ns * (in_ptr -
    (LokiModemSample*)in_packet->buffer) / in_length);
00494                     }
00495                     out_ptr = (LokiSystemSample*)out_packet->buffer;
00496                     out_end = out_ptr + out_length;
00497                 }
00498             }
00499         }
00500
00501         LokiFIFO_pull(self->in, self->cid);
00502     }
00503
00504 LokiSystem_work_end:
00505     LokiModem_stop_streaming(self->modem);
00506
00507     if (self->stop_request) {
00508         fprintf(stderr, "LokiSystem_work: complete\n");
00509     }
00510     else {
00511         fprintf(stderr, "LokiSystem_work: aborted\n");
00512     }
00513
00514     if (sample_index) {
00515         free(sample_index);
00516     }
00517     if (xy_norm) {
00518         free(xy_norm);
00519     }
00520     if (xy_start) {
00521         free(xy_start);
00522     }
00523
00524     self->is_streaming = false;
00525 }
00526
00527 int LokiSystem_start_streaming(LokiSystem *self, int32_t channels, LokiFIFO *modem_out, LokiFIFO
    *disp_out, size_t cid) {
00528     if (self->is_streaming) {
00529         return EBUSY;
00530     }
00531
00532     self->in = modem_out;
00533     self->cid = cid;
00534     self->out = disp_out;
00535     self->stop_request = false;
00536     self->is_streaming = true;
00537
00538     if (disp_out) {
00539         int err = pthread_create(&self->thread, NULL, LokiSystem_work, self);
00540         if (err) {
00541             return err;
00542         }
00543     }
00544
00545     LokiModem_restart(self->modem);
00546
00547     return LokiModem_start_streaming(self->modem, channels, self->config->simulate, false, modem_out);
00548 }
00549
00550 void LokiSystem_stop_streaming(LokiSystem *self) {
00551     if (self->stop_request) {
00552         return;
00553     }
00554     self->stop_request = true;
00555     if (self->out) {
00556         LokiFIFO_abort(self->in);
00557         LokiFIFO_abort(self->out);
00558         pthread_join(self->thread, NULL);
00559     }
00560     // Si le thread n'a pas été démarré, il faut
00561     // désactiver cet indicateur ici.
00562     self->is_streaming = false;
00563 }
00564
00565 bool LokiSystem_is_streaming(LokiSystem *self) {
00566     return self->is_streaming;
00567 }
00568
```

```
00569 LokiEllipse *LokiSystem_get_ellipse(LokiSystem *self, size_t channel) {
00570     return LokiDisplacement_get_ellipse(self->disp[channel]);
00571 }
00572
00573 void LokiSystem_del(LokiSystem *self) {
00574     if (!self) {
00575         return;
00576     }
00577
00578     if (self->has_own_modem) {
00579         LokiModem_del(self->modem);
00580     }
00581
00582     // Les instances de LokiEllipseEstimator et LokiFIRFilter créées dans le
00583     // constructeur sont détruites par LokiDisplacement_del pour c == 0.
00584     if (self->disp) {
00585         for (size_t c = 0; c < self->channels; c ++) {
00586             LokiDisplacement_del(self->disp[c]);
00587         }
00588         free(self->disp);
00589     }
00590
00591     free(self);
00592 }
00593
```

6.81 Référence du fichier libloki-system.h

```
#include "libloki-modem.h"
#include "libloki-fifo.h"
#include "libloki-ellipse-estimator.h"
```

Graphe des dépendances par inclusion de libloki-system.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Structures de données

— struct LokiSystemSample

Représentation d'un échantillon en sortie d'une instance de LokiSystem.

Définitions de type

— typedef struct LokiSystemConfig LokiSystemConfig

Paramètres de configuration d'une instance de LokiSystem.

— typedef struct LokiSystem LokiSystem

Représentation d'un système d'acquisition et de calcul de déplacement.

Fonctions

— LokiSystemConfig ∗ LokiSystemConfig_new (void)

Construire une configuration pour une instance de LokiSystem.

— void LokiSystemConfig_del (LokiSystemConfig ∗self)

Détruire une configuration pour une instance de LokiSystem.

— bool LokiSystemConfig_get_simulate (LokiSystemConfig ∗self)

Lire l'état d'activation de la simulation des capteurs de déplacement.

— void LokiSystemConfig_set_simulate (LokiSystemConfig ∗self, bool value)

Activer ou désactiver la simulation des capteurs de déplacement.

— float LokiSystemConfig_get_skip (LokiSystemConfig ∗self)

Lire le temps d'attente au démarrage.

— void LokiSystemConfig_set_skip (LokiSystemConfig ∗self, float value)

Affecter un temps d'attente au démarrage.

— float LokiSystemConfig_get_scale (LokiSystemConfig ∗self)

Lire le facteur d'échelle de déplacement.

— float LokiSystemConfig_get_wavelength (LokiSystemConfig ∗self)

Lire la longueur d'onde de la diode laser.

— void LokiSystemConfig_set_wavelength (LokiSystemConfig ∗self, float value)

Écrire la longueur d'onde de la diode laser.

— unsigned LokiSystemConfig_get_reflections (LokiSystemConfig ∗self)

Lire le nombre de réflexions supposé.

— void LokiSystemConfig_set_reflections (LokiSystemConfig ∗self, unsigned value)

Écrire le nombre de réflexions supposé.

— float LokiSystemConfig_get_cavity_index (LokiSystemConfig ∗self)

Lire l'indice de la cavité de Fabry-Pérot.

— void LokiSystemConfig_set_cavity_index (LokiSystemConfig ∗self, float value)

Écrire l'indice de la cavité de Fabry-Pérot.

— char LokiSystemConfig_get_distance_unit (LokiSystemConfig ∗self)

Lire l'unité du déplacement calculé.

— void LokiSystemConfig_set_distance_unit (LokiSystemConfig ∗self, char value)

Écrire l'unité du déplacement calculé.

— const char ∗ LokiSystemConfig_get_estimator_name (LokiSystemConfig ∗self)

Lire le nom de l'estimateur de paramètres d'ellipse.

— void LokiSystemConfig_set_estimator_name (LokiSystemConfig ∗self, const char ∗name)

Affecter un nom de l'estimateur de paramètres d'ellipse.

— float LokiSystemConfig_get_estimator_initial_radius (LokiSystemConfig ∗self)

Lire le rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

— void LokiSystemConfig_set_estimator_initial_radius (LokiSystemConfig ∗self, float value)

Affecter un rayon initial à utiliser lors de l'estimation des paramètres d'ellipse.

— size_t LokiSystemConfig_get_estimator_params_length (LokiSystemConfig ∗self)

Lire le nombre de paramètres du filtre d'estimation des paramètres de l'ellipse.

— float ∗ LokiSystemConfig_get_estimator_params (LokiSystemConfig ∗self)

Lire les paramètres du filtre d'estimation des paramètres de l'ellipse.

— void LokiSystemConfig_set_estimator_params (LokiSystemConfig ∗self, const float ∗value)

Affecter des paramètres pour le filtre d'estimation des paramètres de l'ellipse.

— float LokiSystemConfig_get_filter_bandwidth (LokiSystemConfig ∗self)

Lire la bande passante du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_bandwidth (LokiSystemConfig ∗self, float value)

Affecter un bande passante pour le filtre d'élimination du déplacement fictif.

— float LokiSystemConfig_get_filter_attenuation (LokiSystemConfig ∗self)

Lire l'atténuation du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_attenuation (LokiSystemConfig ∗self, float value)

Affecter une atténuation pour le filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_filter_length (LokiSystemConfig ∗self)

Lire la longueur du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_length (LokiSystemConfig ∗self, size_t value)

Affecter une longueur au filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_filter_decimation (LokiSystemConfig ∗self)

Lire le facteur de décimation du filtre d'élimination du déplacement fictif.

— void LokiSystemConfig_set_filter_decimation (LokiSystemConfig ∗self, size_t value)

Affecter un facteur de décimation au filtre d'élimination du déplacement fictif.

— size_t LokiSystemConfig_get_time_constants_length (LokiSystemConfig ∗self)

Lire la longueur du tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— float ∗ LokiSystemConfig_get_time_constants (LokiSystemConfig ∗self)

Lire le tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— void LokiSystemConfig_set_time_constants (LokiSystemConfig ∗self, const float ∗value)

Affecter un tableau de constantes de temps pour le filtrage des paramètres de l'ellipse.

— LokiSystem ∗ LokiSystem_new (LokiSystemConfig ∗config, LokiModem ∗modem)

Construire un nouveau système.

— void LokiSystem_del (LokiSystem ∗self)

Détruire un système.

— LokiSystemConfig ∗ LokiSystem_get_config (const LokiSystem ∗self)

Lire la configuration d'un système.

— LokiModem ∗ LokiSystem_get_modem (const LokiSystem ∗self)

Lire le modem utilisé par ce système.

— float LokiSystem_get_sampling_frequency (const LokiSystem ∗self)

Lire la fréquence d'échantillonnage en sortie de ce système.

— void LokiSystem_displacement_reset (LokiSystem ∗self)

Mettre à zéro le déplacement calculé.

— int LokiSystem_start_streaming (LokiSystem ∗self, int32_t channels, LokiFIFO ∗modem_out, LokiFIFO ∗disp_out, size_t cid)

Démarrer l'acquisition et le traitement.

— void LokiSystem_stop_streaming (LokiSystem ∗self)

Arrêter l'acquisition et le traitement.

— bool LokiSystem_is_streaming (LokiSystem ∗self)

Lire l'état de l'acquisition.

— LokiEllipse ∗ LokiSystem_get_ellipse (LokiSystem ∗self, size_t channel)

Lire la dernière ellipse estimée.

6.81.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-system.h.

6.82 libloki-system.h

Aller à la documentation de ce fichier.

```
00001
00004
00010
00012
00013 #ifndef LIBLOKI_SYSTEM_H_
00014 #define LIBLOKI_SYSTEM_H_
00015
00016 #include "libloki-modem.h"
00017 #include "libloki-fifo.h"
00018 #include "libloki-ellipse-estimator.h"
00019
00021 typedef struct {
00023     uint32_t channel;
00025     float value;
00027     float x_norm;
00029     float y_norm;
00030 } LokiSystemSample;
00031
00033 typedef struct LokiSystemConfig LokiSystemConfig;
00034
00039 LokiSystemConfig * LokiSystemConfig_new(void);
00040
00045 void LokiSystemConfig_del(LokiSystemConfig *self);
00046
00055 bool LokiSystemConfig_get_simulate(LokiSystemConfig *self);
00056
00065 void LokiSystemConfig_set_simulate(LokiSystemConfig *self, bool value);
00066
00075 float LokiSystemConfig_get_skip(LokiSystemConfig *self);
00076
00085 void LokiSystemConfig_set_skip(LokiSystemConfig *self, float value);
00086
00095 float LokiSystemConfig_get_scale(LokiSystemConfig *self);
00096
00102 float LokiSystemConfig_get_wavelength(LokiSystemConfig *self);
00103
00109 void LokiSystemConfig_set_wavelength(LokiSystemConfig *self, float value);
00110
00116 unsigned LokiSystemConfig_get_reflections(LokiSystemConfig *self);
00117
00123 void LokiSystemConfig_set_reflections(LokiSystemConfig *self, unsigned value);
00124
00130 float LokiSystemConfig_get_cavity_index(LokiSystemConfig *self);
00131
00137 void LokiSystemConfig_set_cavity_index(LokiSystemConfig *self, float value);
00138
00144 char LokiSystemConfig_get_distance_unit(LokiSystemConfig *self);
00145
00151 void LokiSystemConfig_set_distance_unit(LokiSystemConfig *self, char value);
00152
00158 const char *LokiSystemConfig_get_estimator_name(LokiSystemConfig *self);
00159
00165 void LokiSystemConfig_set_estimator_name(LokiSystemConfig *self, const char *name);
00166
00172 float LokiSystemConfig_get_estimator_initial_radius(LokiSystemConfig *self);
00173
00179 void LokiSystemConfig_set_estimator_initial_radius(LokiSystemConfig *self, float value);
00180
00186 size_t LokiSystemConfig_get_estimator_params_length(LokiSystemConfig *self);
00187
00196 float *LokiSystemConfig_get_estimator_params(LokiSystemConfig *self);
00197
00207 void LokiSystemConfig_set_estimator_params(LokiSystemConfig *self, const float *value);
00208
00214 float LokiSystemConfig_get_filter_bandwidth(LokiSystemConfig *self);
00215
00221 void LokiSystemConfig_set_filter_bandwidth(LokiSystemConfig *self, float value);
00222
00228 float LokiSystemConfig_get_filter_attenuation(LokiSystemConfig *self);
00229
00235 void LokiSystemConfig_set_filter_attenuation(LokiSystemConfig *self, float value);
00236
00242 size_t LokiSystemConfig_get_filter_length(LokiSystemConfig *self);
00243
00249 void LokiSystemConfig_set_filter_length(LokiSystemConfig *self, size_t value);
00250
00256 size_t LokiSystemConfig_get_filter_decimation(LokiSystemConfig *self);
00257
00263 void LokiSystemConfig_set_filter_decimation(LokiSystemConfig *self, size_t value);
00264
00270 size_t LokiSystemConfig_get_time_constants_length(LokiSystemConfig *self);
00271
00280 float *LokiSystemConfig_get_time_constants(LokiSystemConfig *self);
00281
00290 void LokiSystemConfig_set_time_constants(LokiSystemConfig *self, const float *value);
00291
00293 typedef struct LokiSystem LokiSystem;
```

```
00294
00301 LokiSystem *LokiSystem_new(LokiSystemConfig *config, LokiModem *modem);
00302
00307 void LokiSystem_del(LokiSystem *self);
00308
00314 LokiSystemConfig *LokiSystem_get_config(const LokiSystem *self);
00315
00321 LokiModem *LokiSystem_get_modem(const LokiSystem *self);
00322
00328 float LokiSystem_get_sampling_frequency(const LokiSystem *self);
00329
00335 void LokiSystem_displacement_reset(LokiSystem *self);
00336
00350 int LokiSystem_start_streaming(LokiSystem *self, int32_t channels, LokiFIFO *modem_out, LokiFIFO
      *disp_out, size_t cid);
00351
00358 void LokiSystem_stop_streaming(LokiSystem *self);
00359
00365 bool LokiSystem_is_streaming(LokiSystem *self);
00366
00373 LokiEllipse *LokiSystem_get_ellipse(LokiSystem *self, size_t channel);
00374
00375 #endif
00376
00379
```

6.83 Référence du fichier libloki-time.h

```
#include <inttypes.h>
#include <time.h>
```

Graphe des dépendances par inclusion de libloki-time.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

Macros

— #define TIMESTAMP_SEC 1000000000

Le nombre d'unités du type LokiTimestamp dans une seconde.

— #define TIMESTAMP_MS 1000000

Le nombre d'unités du type LokiTimestamp dans une milliseconde.

Définitions de type

— typedef uint64_t LokiTimestamp

Représentation d'une date, en nanosecondes par rapport à une date de référence.

— typedef int64_t LokiTimeDelta

Représentation d'un intervalle de temps, en nanosecondes.

Fonctions

— static LokiTimestamp LokiTime_to_timestamp (struct timespec ts)

Convertir une date en timestamp.

— static time_t LokiTime_seconds (LokiTimestamp timestamp)

Calculer le nombre total de secondes dans un timestamp.

— static time_t LokiTime_milliseconds (LokiTimestamp timestamp)

Calculer le nombre de millisecondes depuis la dernière seconde dans un timestamp.

— static LokiTimestamp LokiTime_update (LokiTimestamp ref, size_t index, float frequency)

Calculer le timestamp d'un échantillon dans un signal.

6.83.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki-time.h.

6.84 libloki-time.h

Aller à la documentation de ce fichier.

```
00001
00007
00009
00010 #ifndef LIBLOKI_TIME_H_
00011 #define LIBLOKI_TIME_H_
00012
00013 #include <inttypes.h>
00014 #include <time.h>
00015
00017 typedef uint64_t LokiTimestamp;
00018
00020 typedef int64_t  LokiTimeDelta;
00021
00023 #define TIMESTAMP_SEC 1000000000
00024
00026 #define TIMESTAMP_MS  1000000
00027
00033 static inline LokiTimestamp LokiTime_to_timestamp(struct timespec ts) {
00034     return (LokiTimestamp)ts.tv_sec * TIMESTAMP_SEC + (LokiTimestamp)ts.tv_nsec;
00035 }
00036
00042 static inline time_t LokiTime_seconds(LokiTimestamp timestamp) {
00043     return timestamp / TIMESTAMP_SEC;
00044 }
00045
00051 static inline time_t LokiTime_milliseconds(LokiTimestamp timestamp) {
00052     return (timestamp / TIMESTAMP_MS) % 1000;
00053 }
00054
00062 static inline LokiTimestamp LokiTime_update(LokiTimestamp ref, size_t index, float frequency) {
00063     return ref + (LokiTimestamp)TIMESTAMP_SEC * index / frequency;
00064 }
00065
00066 #endif
00067
```

6.85 Référence du fichier libloki.h

```
#include "libloki-hooks.h"
#include "libloki-fifo.h"
#include "libloki-time.h"
#include "libloki-packet.h"
#include "libloki-env-sensors.h"
#include "libloki-modem.h"
#include "libloki-ellipse.h"
#include "libloki-ellipse-estimator.h"
#include "libloki-nlms.h"
#include "libloki-rls.h"
#include "libloki-kalman.h"
#include "libloki-fir.h"
#include "libloki-displacement.h"
#include "libloki-system.h"
#include "libloki-formatter.h"
#include "libloki-mseed.h"
#include "libloki-raw.h"
#include "libloki-recorder.h"
#include "libloki-socket-server.h"
#include "libloki-consumer.h"
```

Graphe des dépendances par inclusion de libloki.h: Ce graphe montre quels fichiers incluent directement ou indirectement ce fichier :

6.85.1 Description détaillée

Auteur

Guillaume Savaton

Copyright

ESEO

Définition dans le fichier libloki.h.

6.86 libloki.h

Aller à la documentation de ce fichier.

```
00001
00005
00006 #include "libloki-hooks.h"
00007 #include "libloki-fifo.h"
00008 #include "libloki-time.h"
00009 #include "libloki-packet.h"
00010 #include "libloki-env-sensors.h"
00011 #include "libloki-modem.h"
00012 #include "libloki-ellipse.h"
00013 #include "libloki-ellipse-estimator.h"
00014 #include "libloki-nlms.h"
00015 #include "libloki-rls.h"
00016 #include "libloki-kalman.h"
00017 #include "libloki-fir.h"
00018 #include "libloki-displacement.h"
00019 #include "libloki-system.h"
00020 #include "libloki-formatter.h"
00021 #include "libloki-mseed.h"
00022 #include "libloki-raw.h"
00023 #include "libloki-recorder.h"
00024 #include "libloki-socket-server.h"
00025 #include "libloki-consumer.h"
```

6.87 util.py

```
00001
00002 import json
00003
00004 modem_channel_config_properties = [
00005     "demod_phase_x",
00006     "demod_phase_y",
00007     "demod_pga_gain"
00008 ]
00009
00010 # L'ordre des noms de propriétés est important :
00011 # les fréquences d'échantillonnage doivent être réglées en premier
00012 # car elles conditionnent la résolution des fréquence des oscillateurs.
00013 modem_config_properties = [
00014     "mod_sampling_frequency",
00015     "mod_osc1_wave_frequency",
00016     "mod_osc1_gain",
00017     "mod_osc2_wave_frequency",
00018     "mod_osc2_gain",
00019     "mod_offset",
00020     "mod_dc_value",
00021     "mod_pga_gain",
00022     "demod_bypass",
00023     "demod_sampling_frequency",
00024     "demod_osc_wave_frequency",
00025     "demod_xy_sampling_frequency"
00026 ]
00027
00028 def read_json(filename):
00029     try:
00030         with open(filename) as f:
00031             return json.load(f)
00032     except FileNotFoundError:
00033         print("Warning - File not found: '{}'. Using default settings.".format(filename))
00034     except json.JSONDecodeError:
00035         print("Warning - Invalid JSON file: '{}'. Using default settings.".format(filename))
00036     return None
00037
00038 def load_modem_config(modem, filename):
00039     cfgdict = read_json(filename)
00040     if cfgdict is None:
00041         return
00042
00043     # Traiter les propriétés dans l'ordre fixé par modem_config_properties.
00044     for name in modem_config_properties:
00045         if name in cfgdict:
00046             getattr(modem, "set_" + name)(cfgdict[name])
00047
00048     if "channels_config" in cfgdict:
00049         for c, channel_cfg in enumerate(cfgdict["channels_config"]):
00050             for name in modem_channel_config_properties:
00051                 if name in channel_cfg:
00052                     getattr(modem, "set_" + name)(c, channel_cfg[name])
00053
00054
00055 def save_modem_config(modem, filename):
00056     cfgdict = {
00057         "channels_config": []
00058     }
00059
00060     for name in modem_config_properties:
00061         cfgdict[name] = getattr(modem, "get_" + name)()
00062
00063     for c in range(modem.get_demod_channels()):
00064         channel_cfg = {}
00065         for name in modem_channel_config_properties:
00066             channel_cfg[name] = getattr(modem, "get_" + name)(c)
00067         cfgdict["channels_config"].append(channel_cfg)
00068
00069     with open(filename, "w") as f:
00070         json.dump(cfgdict, f, sort_keys=True, indent=4)
00071
00072
00073 system_config_properties = {
00074     "simulate",
00075     "skip",
00076     "wavelength",
00077     "reflections",
00078     "cavity_index",
00079     "distance_unit",
00080     "estimator_name",
00081     "estimator_throttle",
00082     "estimator_initial_radius",
00083     "filter_bandwidth",
00084     "filter_attenuation",
```

```
00085     "filter_length",
00086     "filter_decimation",
00087     # Managed separately:
00088     # "time_constants",
00089     # "estimator_params",
00090     # "sensors_scaling_factors"
00091 }
00092
00093 def load_system_config(syscfg, reccfg, env_sensors, filename):
00094     cfgdict = read_json(filename)
00095     if cfgdict is None:
00096         return
00097
00098     for name in reccfg.keys():
00099         if name in cfgdict:
00100             reccfg[name] = cfgdict[name]
00101
00102     for name in system_config_properties:
00103         if name in cfgdict:
00104             getattr(syscfg, "set_" + name)(cfgdict[name])
00105
00106     if "estimator_params" in cfgdict:
00107         for i, v in enumerate(cfgdict["estimator_params"]):
00108             syscfg.set_estimator_param(i, v)
00109
00110     if "time_constants" in cfgdict:
00111         for i, v in enumerate(cfgdict["time_constants"]):
00112             syscfg.set_time_constant(i, v)
00113
00114     if env_sensors is not None:
00115         c = env_sensors.get_channels()
00116         if "sensors_scaling_factors" in cfgdict:
00117             for i, v in enumerate(cfgdict["sensors_scaling_factors"]):
00118                 if i < c:
00119                     env_sensors.set_scaling_factor(i, v)
00120
00121
00122 def save_system_config(syscfg, reccfg, env_sensors, filename):
00123     cfgdict = {
00124         **reccfg,
00125         "estimator_params": [],
00126         "time_constants": [],
00127         "sensors_scaling_factors": []
00128     }
00129
00130     for name in system_config_properties:
00131         cfgdict[name] = getattr(syscfg, "get_" + name)()
00132
00133     for i in range(syscfg.get_estimator_params_length()):
00134         cfgdict["estimator_params"].append(syscfg.get_estimator_param(i))
00135
00136     for i in range(syscfg.get_time_constants_length()):
00137         cfgdict["time_constants"].append(syscfg.get_time_constant(i))
00138
00139     if env_sensors is not None:
00140         for i in range(env_sensors.get_channels()):
00141             cfgdict["sensors_scaling_factors"].append(env_sensors.get_scaling_factor(i))
00142
00143     with open(filename, "w") as f:
00144         json.dump(cfgdict, f, sort_keys=True, indent=4)
```